

DATA STRUCTURES

UNIT-I

UNIT I

Introduction to Data Structure:

- ❖ **Arrays** - Fundamentals, Representations, and Applications,
- ❖ Sparse Matrices.
- ❖ **Stacks** - Operations on stacks, Applications - Infix to postfix conversion, evaluation of postfix expressions, Tower of Hanoi.
- ❖ **Queues** - Operations and applications, Types - Circular Queue, Priority Queue.

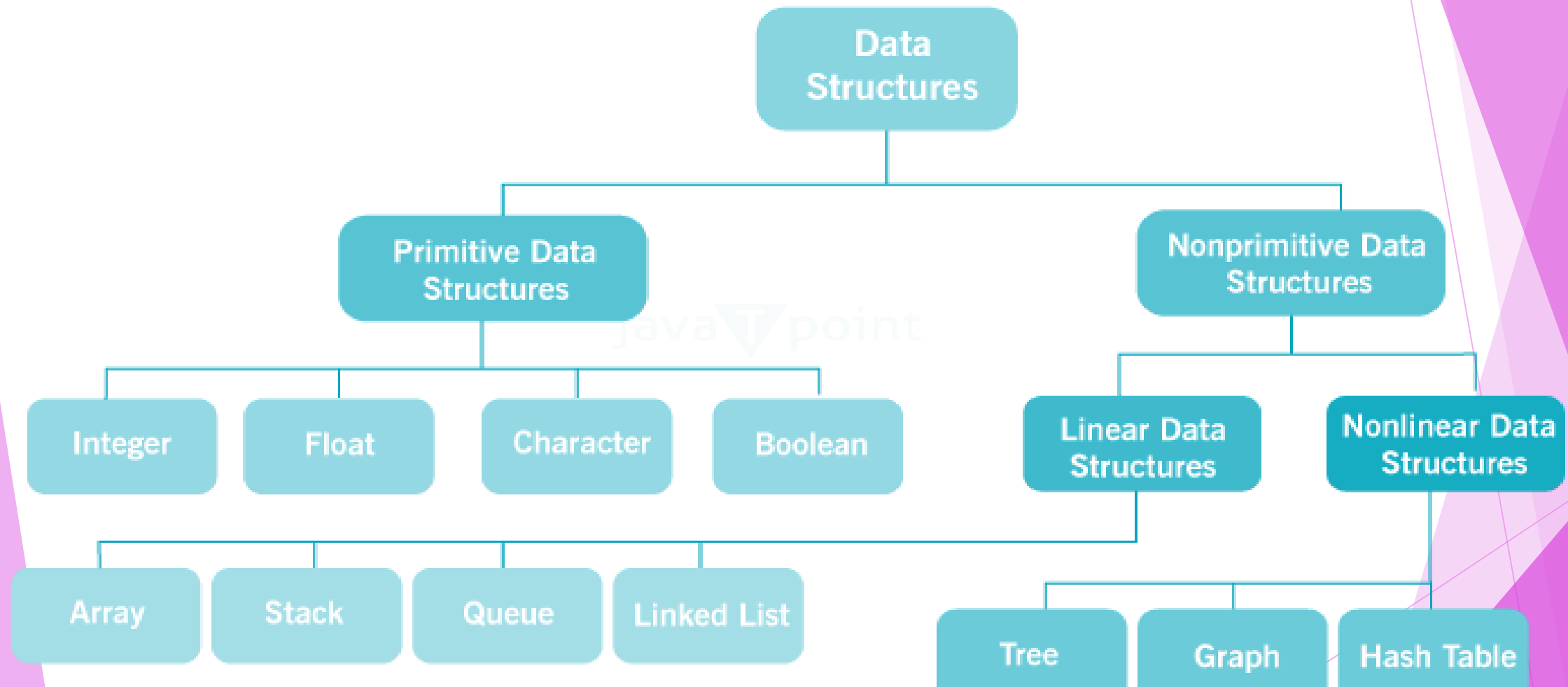
INTRODUCTION TO DATA STRUCTURES

Data Structure is a way of collecting and organizing data in such a way that we can perform operations on these data in an effective way.

Ex:- Assume student's **name** "Rohit" and **age** 20. Here "Rohit" is of **String** data type and 20 is of **integer** data type.

We can organize this data as a record like **Student** record, which will have both student's name and age in it. Now we can collect and store student's records in a file or database as a data structure.

Classifications of Data Structures



Data Structures are classified into two categories:

1. Primitive Data Structure

- ❖ These data structures consisting of the numbers and the characters that come **in-built** into programs.
- ❖ Basic data types like **Integer**, **Float**, **Character**, and **Boolean** come under the Primitive Data Structures.
- ❖ These data types are also called **Simple data types**, as they contain characters

2.Non-Primitive Data Structures

- ❖ **Non-Primitive Data Structures** are those data structures derived from Primitive Data Structures.
- ❖ The focus of these data structures is on forming a set of data elements that is either **homogeneous**(same datatype) or **heterogeneous** (different data types).
- ❖ Based on the structure and arrangement of data, we can divide these data structures into two sub-categories -

1.Linear Data Structures

2.Non-Linear Data Structures

Major Operations

The major or the common operations that can be performed on the data structures are:

- **Searching:** We can search for any element in a data structure.
- **Sorting:** We can sort the elements of a data structure either in an ascending or descending order.
- **Insertion:** We can also insert the new element in a data structure.
- **Updation:** We can also update the element, i.e., we can replace the element with another element.
- **Deletion:** We can also perform the delete operation to remove the element from the data structure

Advantages of Data structures

- Efficiency:** If the choice of a data structure for implementing a particular ADT is proper, it makes the program very efficient in terms of time and space.
- Reusability:** The data structure provides reusability means that multiple client programs can use the data structure.
- Abstraction:** The data structure specified by an ADT also provides the level of abstraction. The client cannot see the internal working of the data structure, so it does not have to worry about the implementation part. The client can only see the interface.

LINEAR DATA STRUCTURE:-

If a data structure organizes the data in sequential order, then that data structure is called a Linear Data Structure. The arrangement of the data is done linearly, where each element consists of the successors and predecessors except the first and the last data element.

Based on memory allocation, the Linear Data Structures are further classified into two types:

Static Data Structures: The data structures having a fixed size are known as Static Data Structures. The memory for these data structures is allocated at the compiler time, and their size cannot be changed

Ex:- ARRAY

Dynamic Data Structures: The data structures having a dynamic size are known as Dynamic Data Structures. The memory of these data structures is allocated at the run time

Ex:-Linked Lists, Stacks, and Queues.

Non-Linear Data Structures

- ❑ Non-Linear Data Structures are data structures where the data elements are not arranged in sequential order.
- ❑ Here, the insertion and removal of data are not feasible in a linear manner.
- ❑ There exists a hierarchical relationship between the individual data items.

Ex:- Trees, Graphs

Linear data structure

Array



Linked list



Stack



Queue

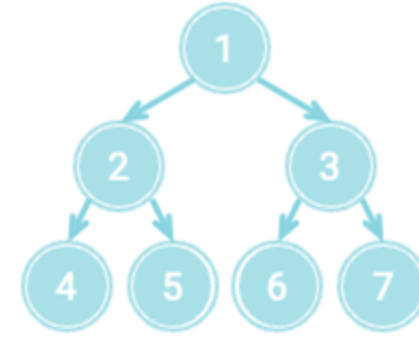


Hash table

key	a	b	c
value	1	2	3

Non-linear data structure

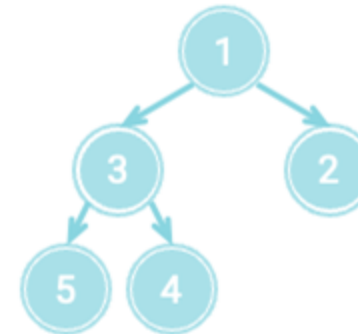
Tree



Graph



Heap



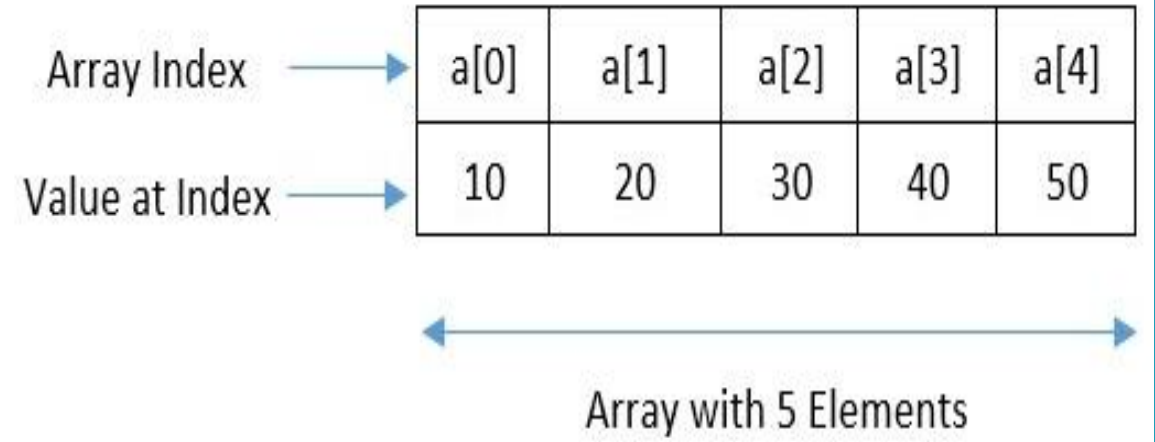


Arrays

Fundamentals, Representations, & Applications

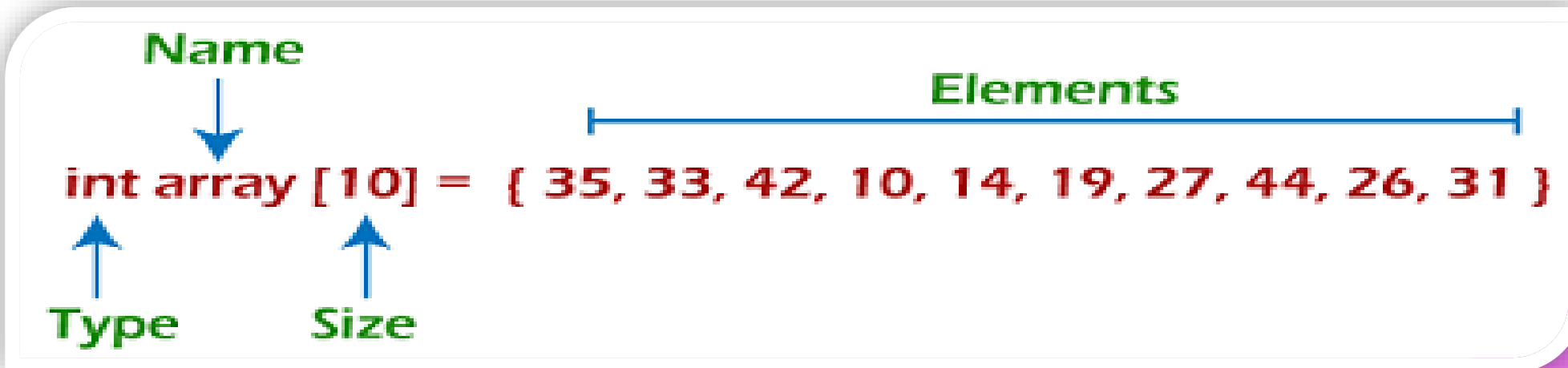
Array:

An array is defined as the collection of similar type of data items stored at contiguous memory locations.



Representation of an array

We can represent an array in various ways in different programming languages.



Declaration of Array:-

1.data_type array_name[array_size];

Ex:-int marks[5]

Initialization of Array

The simplest way to initialize an array is by using the index of each element. We can initialize each element of the array by using the index.

Consider the following example.

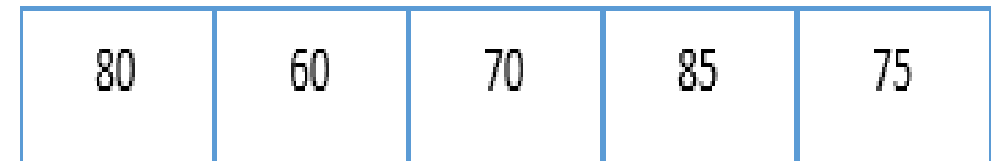
1.marks[0]=80;//initialization of array

2.marks[1]=60;

3.marks[2]=70;

4.marks[3]=85;

5.marks[4]=75;



A diagram showing an array as a horizontal row of five adjacent rectangular boxes. Each box contains a numerical value. From left to right, the values are 80, 60, 70, 85, and 75. The boxes are outlined in blue.

80	60	70	85	75
----	----	----	----	----

marks[0] marks[1] marks[2] marks[3] marks[4]

Initialization of Array

Characteristics of Arrays in Data Structures

Fixed Size: Once an array is declared, its size cannot be changed.

Homogeneous Elements: All elements in an array must be of the same data type, such as all integers, all floats, or all characters.

Indexed by Integers: Each element in an array is assigned a unique integer called an index, which identifies its position within the array. Indexing usually starts at 0.

Efficient Access: Because of the way arrays are stored in memory (contiguously), accessing any element by its index is very efficient. This direct access using the index is often referred to as "random access."

Advantages

- 1) **Code Optimization:** Less code to access the data.
- 2) **Ease of traversing:** By using the for loop, we can retrieve the elements of an array easily.
- 3) **Ease of sorting:** To sort the elements of the array, we need a few lines of code only.
- 4) **Random Access:** We can access any element randomly using the array.

Disadvantages

- 1) **Fixed Size:** Whatever size, we define at the time of declaration of the array, we can't exceed the limit.

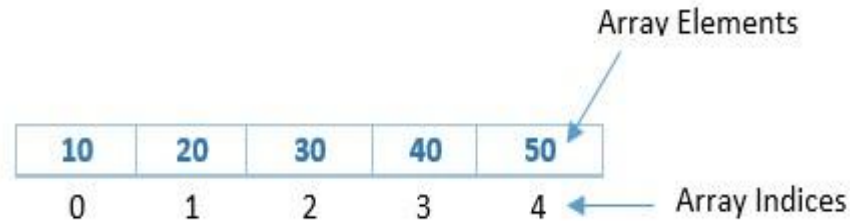
Applications of Array:

1. We can store a list of data elements belonging to the same data type.
2. Array acts as an auxiliary storage for other data structures.
3. The array also helps store data elements of a binary tree of the fixed count.
4. Array also acts as a storage of matrices.

Types of Arrays

An array in C is a collection of elements of the same data type, stored in contiguous memory locations. There are two types of arrays:

- One Dimensional Array
- Two Dimensional Array



One Dimensional Array

A one-dimensional (1D) array is a linear list of elements of the same data type, such as integers or characters. The elements are stored in contiguous memory locations and can be accessed using a single index. The index of the elements in a 1D array starts from 0.

Two Dimensional Array

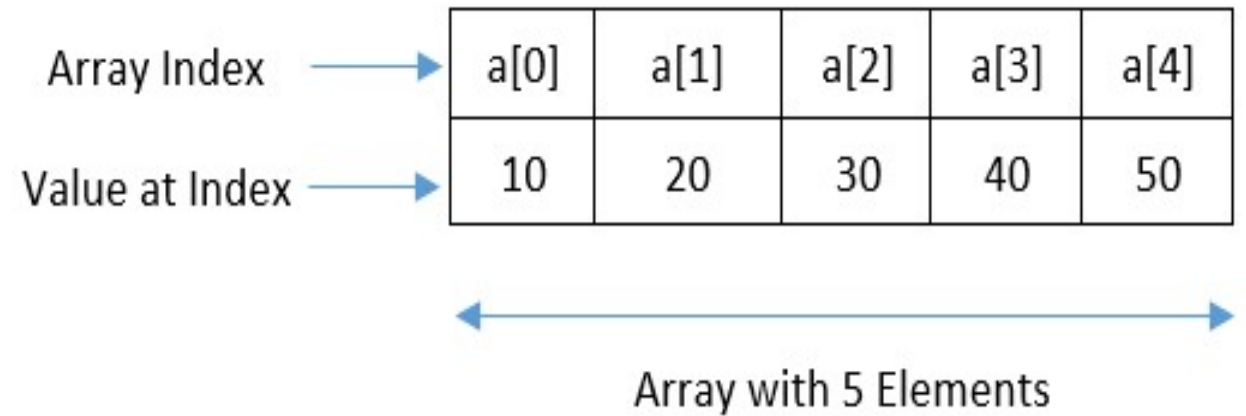
A two-dimensional (2D) array is a table-like structure that stores data in rows and columns. The elements are stored in contiguous memory locations and can be accessed using two indices, one for the row and one for the column. The index of the elements in a 2D array starts from 0.

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Memory Representation of Array

The most common way to represent an array in memory is to store the elements sequentially in a block of memory.

For example, if we have an array of integers `a` with 5 elements, the memory representation would look like this:

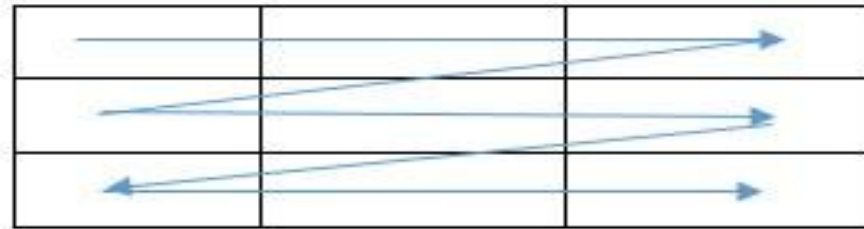


Row Major Order & Column Major Order

There are two main ways to represent an array in memory: row-major order and column-major order.

Row Major Order

In row-major order, the elements of the array are stored row by row in memory. For example, in a two-dimensional array $A[i][j]$, the elements of the first row are stored first, followed by the elements of the second row, and so on. Within each row, the elements are stored in order of increasing column index. This means that the memory location of an element $A[i][j]$ can be computed as $\&A[0][0] + i * n + j$, where n is the number of columns in the array.



ROW MAJOR ORDER

Column Major Order

In column-major order, the elements of the array are stored column by column in memory. For example, in a two-dimensional array $A[i][j]$, the elements of the first column are stored first, followed by the elements of the second column, and so on. Within each column, the elements are stored in order of increasing row index. This means that the memory location of an element $A[i][j]$ can be computed as $\&A[0][0] + j * m + i$, where m is the number of rows in the array.



COLUMN MAJOR ORDER

Address Calculation in One Dimensional Array

To access an element in an array, the computer needs to know its memory address. The memory address of an array element can be calculated using the following formula:

$$\text{address} = \text{base_address} + \text{index} * \text{size_of_element}$$

Where:

- `base_address`: the memory address of the first element in the array.
- `index`: the index or subscript of the element you want to access.
- `size_of_element`: the size (in bytes) of each element in the array.

For example, let's say you have an array of integers with 5 elements and the base address of the array is 100. The size of each integer element is 4 bytes. To access the third element `a[2]` of the array, you would use the following formula:

$$a[2] = 100 + 2 * 4 = 108$$

Address Calculation in Two Dimensional Array

To access an element in a 2D array, the computer needs to know its memory address. The memory address of an element in a 2D array can be calculated using the following formula:

$$\text{address} = \text{base_address} + (\text{row_index} * \text{num_columns} + \text{column_index}) * \text{size_of_element}$$

Where:

- `base_address`: the memory address of the first element in the 2D array.
- `row_index`: the index of the row containing the element you want to access.
- `column_index`: the index of the column containing the element you want to access.
- `num_columns`: the number of columns in the 2D array.
- `size_of_element`: the size (in bytes) of each element in the 2D array.

For example, let's say you have a 2D array of integers with 3 rows and 4 columns, and the base address of the array is 100. The size of each integer element is 4 bytes. To access the element at row 1, column 0 which is `a[1][0]` you would use the following formula:

$$a[1][0] = 100 + (1 * 4 + 0) * 4 = 116$$

Basic operations

Traversal - This operation is used to print the elements of the array.

Insertion - It is used to add an element at a particular index.

Deletion - It is used to delete an element from a particular index.

Search - It is used to search an element using the given index or by the value.

Update - It updates an element at a particular index.

Traversal Operation

Array traversal refers to the process of accessing each element of an array in a sequential order.

Algorithm for Traversing an Array

Traversal (data, LB, UB)

Here data is array name and LB is Lower Bound (start) index of the first element of an array. UB is Upper Bound (End) is the index of the last element

Step 1: Start

Step 2: [Initialize variable.] Set $i = LB$

Step 3: Repeat steps 4 and 5 While $i \leq UB$

Step 4: Apply process (Print) to $data[i]$.

Step 5: Increment $i=i+1$

Step 6: End of loop

Step 7: Stop

Insertion Operation

Insertion in an array refers to the process of adding a new element to a specific position within the array

Algorithm to Insert an element in an Array

Here size is the array size. Position (pos) is location where element to be inserted and Item is new value in the array

Step 1: Start

Step 2: [Initialize variable] Set $i = \text{size} - 1$

Step 3: Repeat Step 4 and 5 While $i \geq \text{pos} - 1$

Step 4: [Move ith element forward.] set $\text{arr}[i+1] = \text{arr}[i]$

Step 5: [Decrease counter.] Set $i = i - 1$

Step 6: [End of step 3 loop.]

Step 7: [Insert element.] Set $\text{arr}[\text{pos}-1] = \text{item}$

Step 8: Stop

Deletion Operation

Deletion in an array means removing an element from an array

Algorithm to Delete an element from an Array:

Step 1: Start

Step 2: [Initialize variable] Set $i = \text{pos} - 1$

Step 3: Repeat Step 4 and 5 for $i = \text{pos} - 1$ to $i < \text{size}$

Step 4: [Move ith element left side] set $a[i] = a[i+1]$

Step 5: [Increment] Set $i = i + 1$

Step 6: [End of step 03 loop.]

Step 7: [Update Array Size] set $\text{size} = \text{size} - 1$

Step 8: Stop

SPARSE MATRICES

Sparse Matrix:-

- ❖ A sparse matrix is a matrix that contains a large number of zero elements compared to non-zero elements.
- ❖ In such matrices, it is not efficient to use traditional representations, which store all elements, because a significant amount of memory will be wasted on storing zero elements.
- ❖ Therefore, special techniques are used to represent sparse matrices, which can save a lot of memory.

4 R x 3 C
Matrix

3	0	0
0	5	0
0	0	7
8	0	0

SPARSE MATRIX

Representation of Sparse Matirx

Sparse matrices can be represented using both arrays and linked lists.

1.Array Representation: We use Two-dimensional arrays to store the non-zero elements of the sparse matrix. In 2D array representation of sparse matrix, there are three fields used that are named as -
ROW,COLUMN,VALUE.

2.Linked List Representation: We use a linked list to store the non-zero elements of the sparse matrix. Each node of the linked list represents a non-zero element and contains three fields: the value of the element, the row index of the element, the column index of the element and next node to store the address of next node.

Array Representation of Sparse Matrix

In the array representation of a sparse matrix, we store only the non-zero elements of the matrix along with their row and column indices.

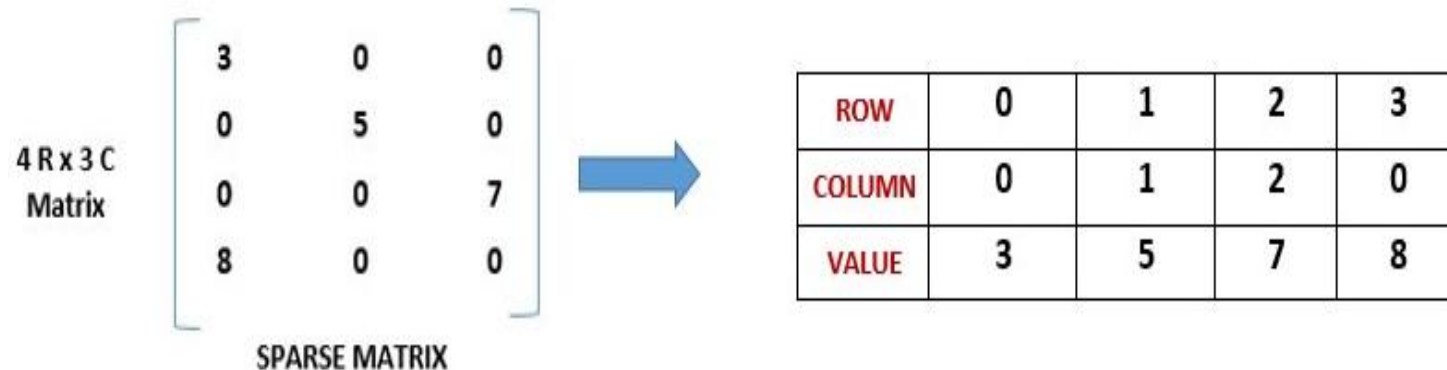
We use three arrays to represent the sparse matrix:

1.row_index: It stores the row indices of the corresponding elements in the array.

2.col_index: It stores the column indices of the corresponding elements in the array.

3.value : It stores the non-zero elements of the matrix in row-major order.

The size of these arrays depends on the number of non-zero elements in the matrix.



Program in C to implement array representation of sparse matrix

```
#include <stdio.h>

int main(){
    int sm[100][100], rows, cols, i, j, nonzero = 0;
    printf("Enter no of rows & columns of the matrix: ");
    scanf("%d %d", &rows, &cols);
    printf("Enter the matrix elements:");
    for (i = 0; i < rows; i++){
        for (j = 0; j < cols; j++){
            scanf("%d", &sm[i][j]);
            if (sm[i][j] != 0){
                nonzero++;
            }
        }
    }
    int fmatrix[3][nonzero];
    int k=0;
```

```
for(int i=0; i<rows; i++) {  
    for(int j=0; j<cols; j++) {  
        if(sm[i][j]!=0) {  
            fmatrix[0][k] = i;  
            fmatrix[1][k] = j;  
            fmatrix[2][k] = sm[i][j];  
            k++;  
        } } }  
for(int i=0 ;i<rows; i++) {  
    for(int j=0; j<cols; j++) {  
        printf("%d ", fmatrix[i][j]);  
    } } }
```

OUTPUT:-

Enter no of rows & columns of the matrix:

3

4

Enter the matrix elements:

4

0

0

3

0

2

0

0

6

1

0

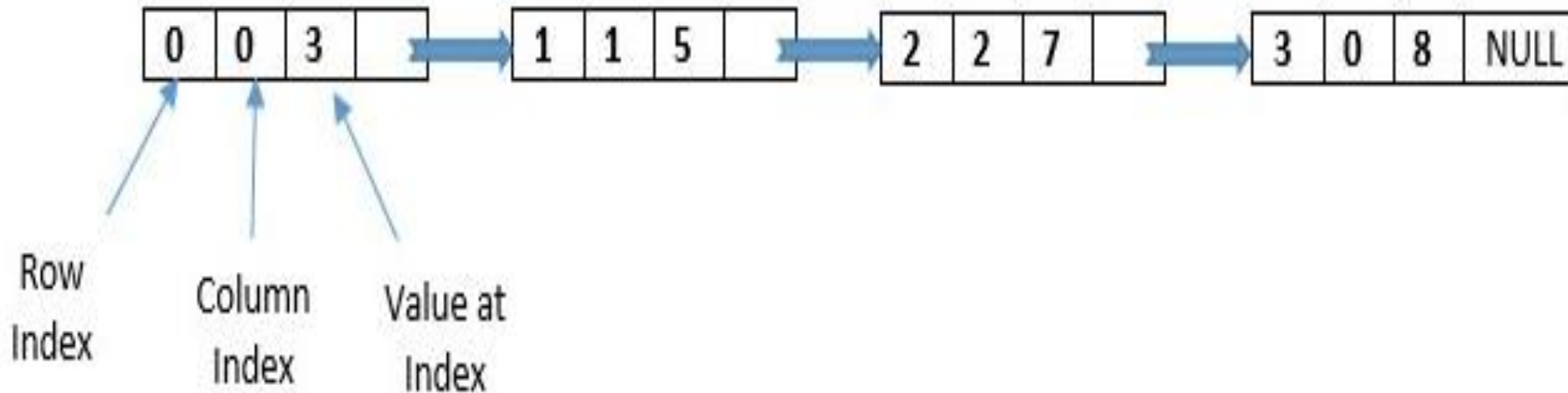
0

0 0 1 2 0 3 1 0 4 3 2 6

Linked List Representation of Sparse Matrix

- ❖ A linked list representation of a sparse matrix is a way of storing a matrix that has a large number of zero values efficiently by using linked lists.
- ❖ Each node in the linked list contains the value of the non-zero element, its column index, row index, and a pointer to the next node in the list.
- ❖ This representation reduces the amount of memory required to store the matrix and allows for efficient manipulation of the non-zero elements.
- ❖ The benefit of using a linked list instead of an array to represent the sparse matrix is that it is simpler to add or remove nodes from a linked list than from an array.

- ❖ **Row** - Row where the non-zero element is located.
- ❖ **Column** - Column where the non-zero element is located.
- ❖ **Value** - It is the value of the non-zero element that is located at the index (row, column).
- ❖ **Next node** - Address of the next node.



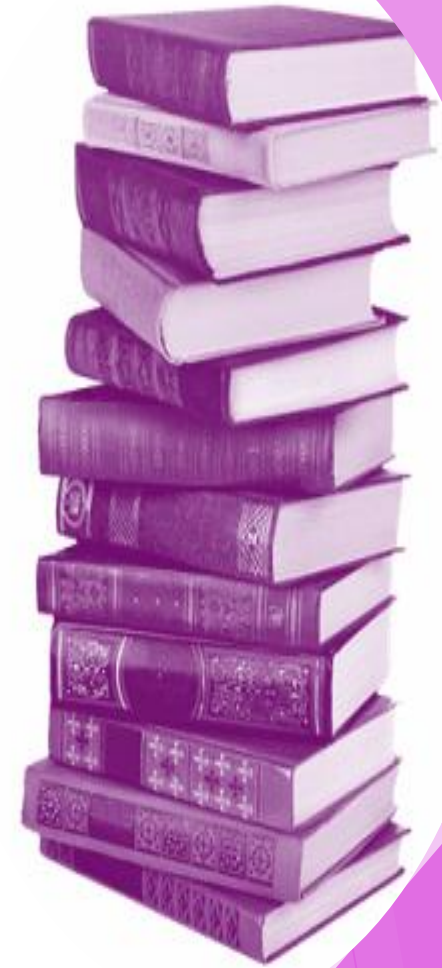
STACKS

Operations on stacks,

Applications - Infix to postfix conversion,
evaluation of postfix expressions

Stack

- ❖ A stack is a linear data structure that follows the **Last-In-First-Out (LIFO)** principle, where the element that is inserted last will be the first to be removed.
- ❖ A stack has one end, typically called the top. The top of the stack is the element that was most recently added, and the bottom of the stack is the element that was added first or the earliest.
- ❖ The top of the stack is represented using a pointer, called the **top pointer**, which points to the last element inserted. When a new element is added to the stack, it is placed on the top of the stack, and the top pointer is updated to point to the new element.

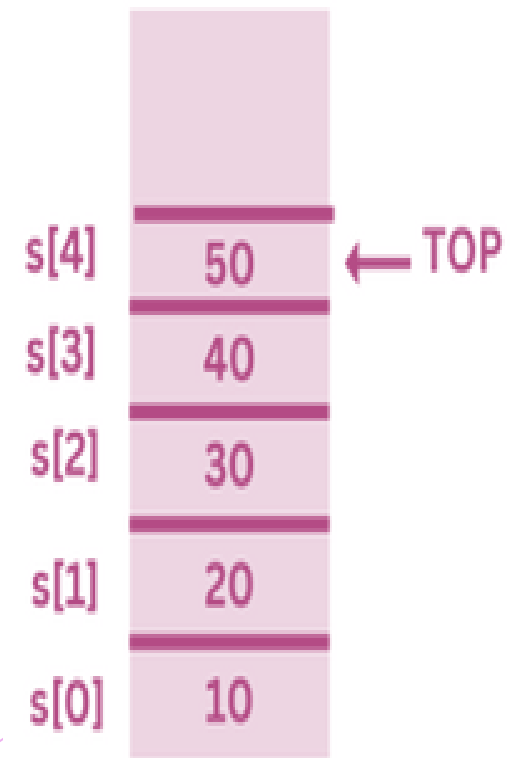


Memory Representation of Stack

A stack can be implemented using an array or a linked list. In both cases, memory is used to store the stack elements and manage the stack operations.

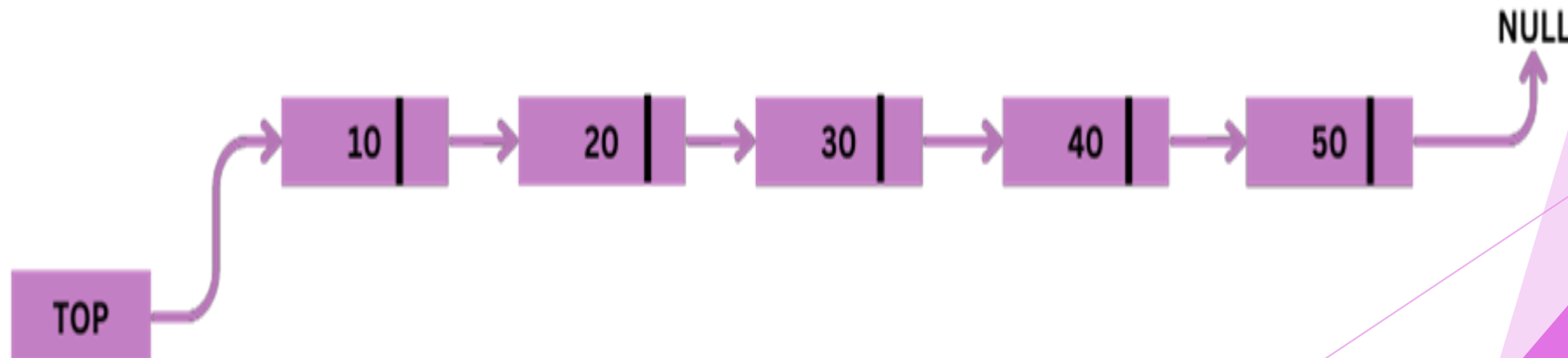
Memory Representation of Stack using

Arrays: In an array-based stack, the stack elements are stored in a contiguous block of memory, typically an array. The stack pointer points to the topmost element of the stack, and the stack operations modify the stack pointer and the elements in the array. When the stack size is fixed, the array-based stack can overflow or underflow, causing memory access errors.



Memory Representation of Stack using Linked List:

- ❖ In a linked list-based stack, the stack elements are stored in individual nodes, each with a pointer to the next node.
- ❖ The stack pointer points to the head node of the linked list, and the stack operations modify the stack pointer and the links between the nodes.
- ❖ When the stack size is dynamic, the linked list-based stack can allocate or deallocate memory as needed, avoiding overflow or underflow errors.



BASIC STACK OPERATIONS

Push Operation: Inserts a new element 'item' onto the top of the stack. However, attempting a push will result in an overflow error condition if the stack is already at its maximum capacity.

1. Check if the stack is full ($\text{top} == \text{size} - 1$)
2. If full, return an error (stack overflow)
3. If not full, increment 'top' and insert the new element at the new 'top' index

Pop Operation: Removes and returns the topmost element from the stack. But if the stack is empty, attempting a pop will trigger an underflow error, as there are no elements to remove.

1. Check if the stack is empty ($\text{top} == -1$)
2. If empty, return an error (stack underflow)
3. If not empty, retrieve the element at the 'top' index
4. Decrement 'top' to remove the topmost element

Peek Operation: Retrieves the value of the topmost element without removing it from the stack. If the stack is empty, it indicates an error state.

1. Check if the stack is empty ($\text{top} == -1$)
2. If empty, return an error
3. If not empty, return the element at the 'top' index (without removing it)

isEmpty Operation: Checks if the stack is empty by verifying if the top pointer points to a sentinel value (e.g., -1). Returns true if the stack is empty; otherwise, it is false.

1. Check if 'top' is equal to -1
2. If yes, return true (stack is empty)
3. If no, return false (stack is not empty)

isFull Operation:isFull(): Determines if the stack has reached its maximum capacity by checking if the top pointer is at the last valid index. Returns true if full, false if not.

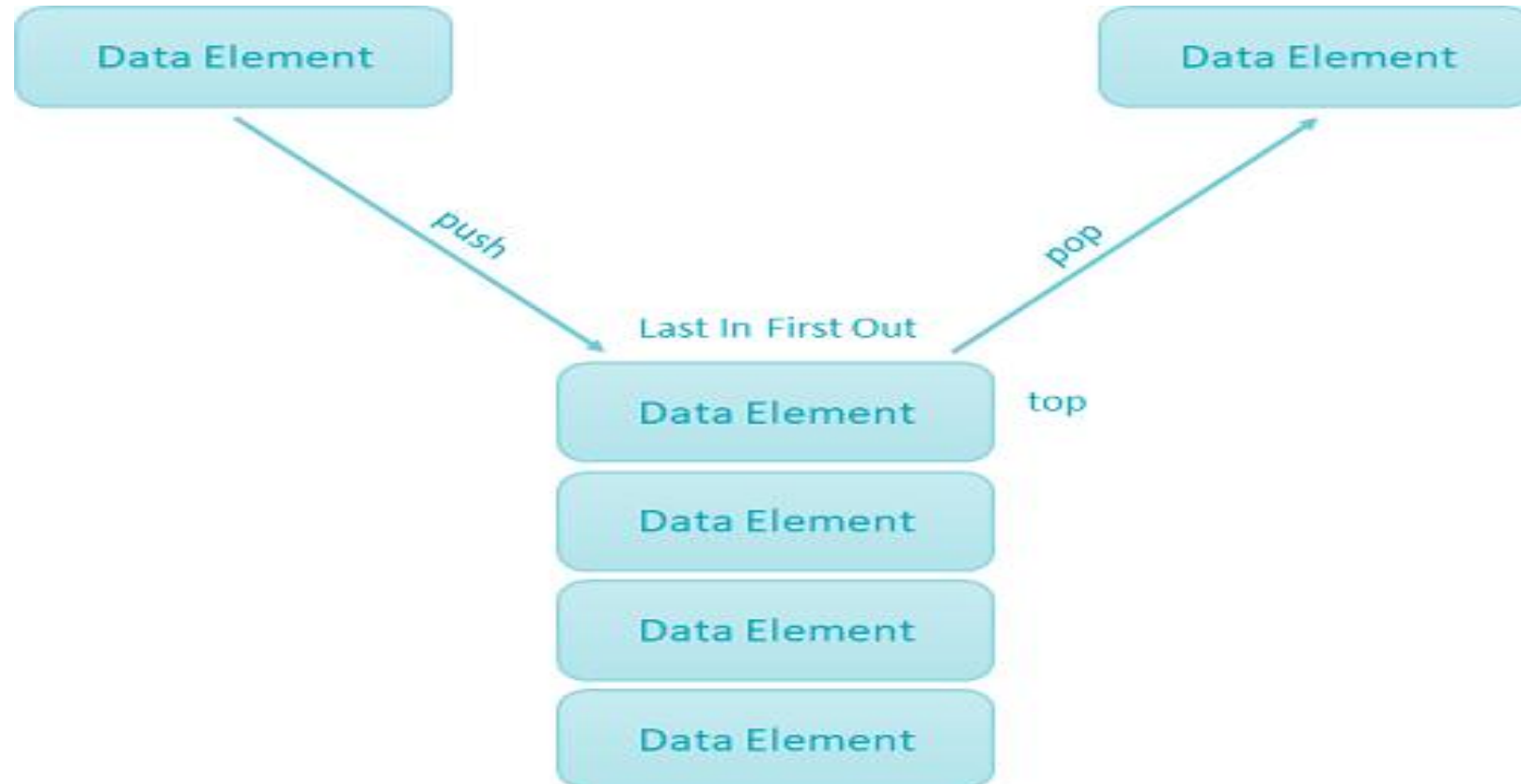
1. Check if 'top' is equal to size - 1
2. If yes, return true (stack is full)
3. If no, return false (stack is not full)

Size: The size operation returns the number of elements in the stack. For example, if we have a stack containing [10, 20, 30], and we perform the size operation, it will return 3.

Search: The search operation allows you to search for a specific element in the stack and returns its position (counting from the top of the stack). If the element is not found in the stack, it returns -1. For example, if we have a stack containing [10, 20, 30], and we perform the search operation with the value 20, it will return 2 (since 20 is the second element counting from the top of the stack).

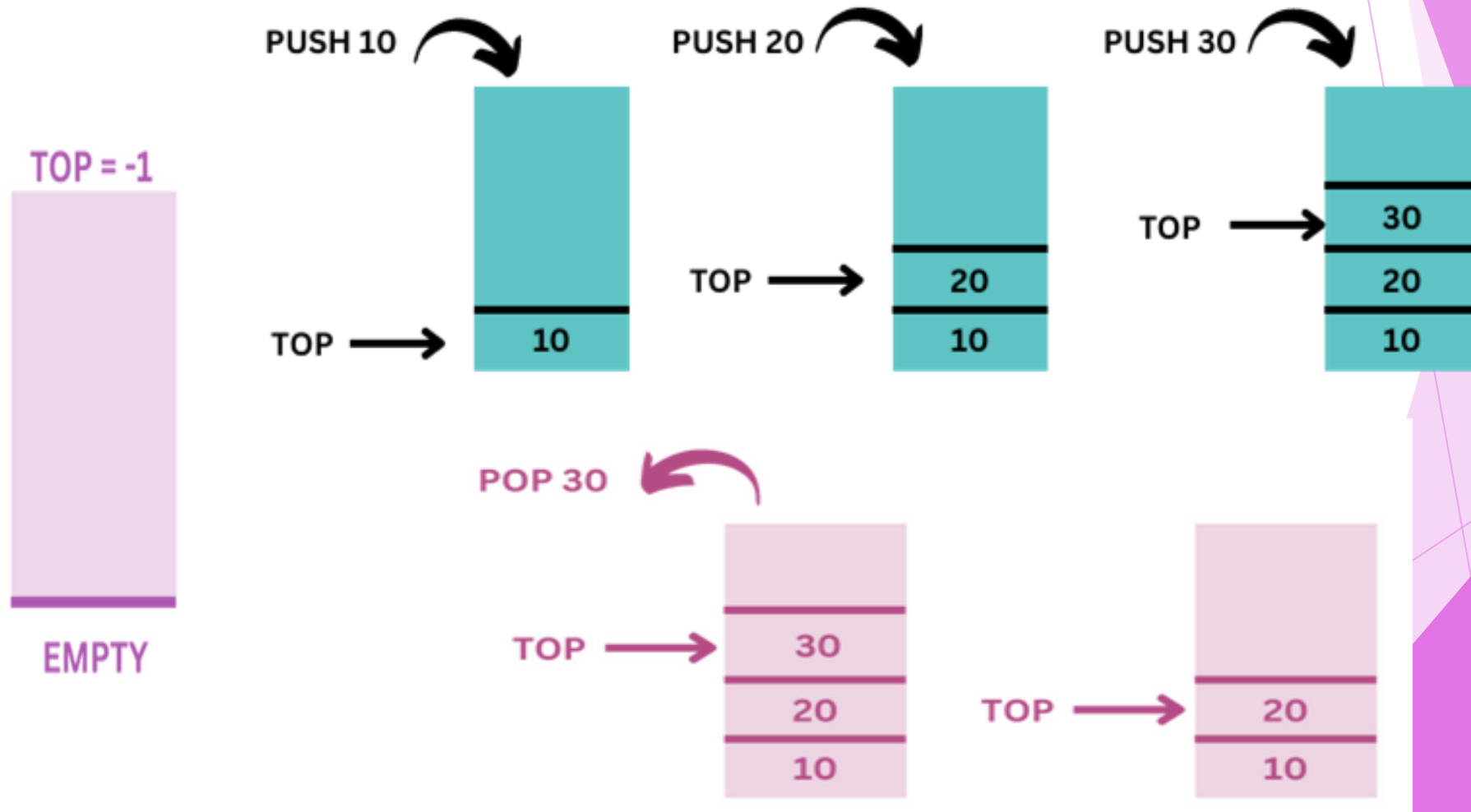
The primary operations in the Stack are as follows:

- 1.Push:** Operation to insert a new element in the Stack is termed as Push Operation.
- 2.Pop:** Operation to remove or delete elements from the Stack is termed as Pop Operation.



Working of Stack

A stack is a data structure that stores elements with a Last-In-First-Out (LIFO) ordering. It works by using a stack pointer to manage the insertion and removal of elements from the top of the stack.



PUSH operation in Stack

The push operation is an operation in the stack data structure that adds a new element to the top of the stack. When a new element is pushed onto the stack, it becomes the new top element, and the size of the stack is increased by 1.

1. Check if the stack is full (if the stack has a fixed size and is already at its maximum capacity). If the stack is full, the push operation cannot be performed.
2. Increment the top pointer of the stack to move it to the next empty position (i.e., the position above the current top element).
3. Add the new element to the position pointed by the top pointer.
4. Update the size of the stack to reflect the addition of the new element.

Algorithm of PUSH Operation in Stack

PUSH(S,TOP,N,X)

Here S is Stack, TOP is Stack Pointer, N is the Maximum Size of the Stack, X is new element to be inserted

Step 1: If $TOP == N$

Write "OVERFLOW"

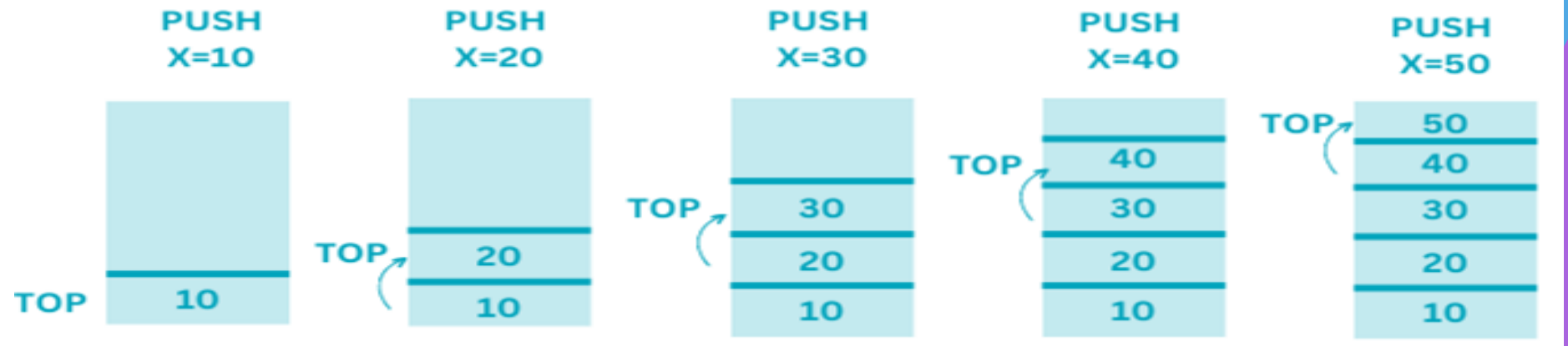
Return

(End of IF Statement)

Step 2: $TOP = TOP + 1$ (Increment the TOP)

Step 3: $S[TOP] = X$ (Insert New Element)

Step 4 END



Program in C to perform PUSH Operation in Stack using Array.

```
#include <stdio.h>
#define MAX_SIZE 100
int stack[MAX_SIZE];
int top = -1;
void push(int);
int main()
{
    push(10);
    push(20);
    push(30);
    push(40);
    push(50);
    printf("Elements in the Stack\n");
    for(int i=top; i>=0;i--)
```

```
{
    printf("|%d|",stack[i]);
}
return 0;
}
void push(int element)
{
    if (top >= MAX_SIZE - 1)
    {
        printf("Stack Overflow");
    }
else
    {
        top=top+1;
        stack[top] = element;
        printf("%d pushed to stack\n", element);
    }
}
```

OUTPUT:-

10 pushed to stack

20 pushed to stack

30 pushed to stack

40 pushed to stack

50 pushed to stack

Elements in the Stack

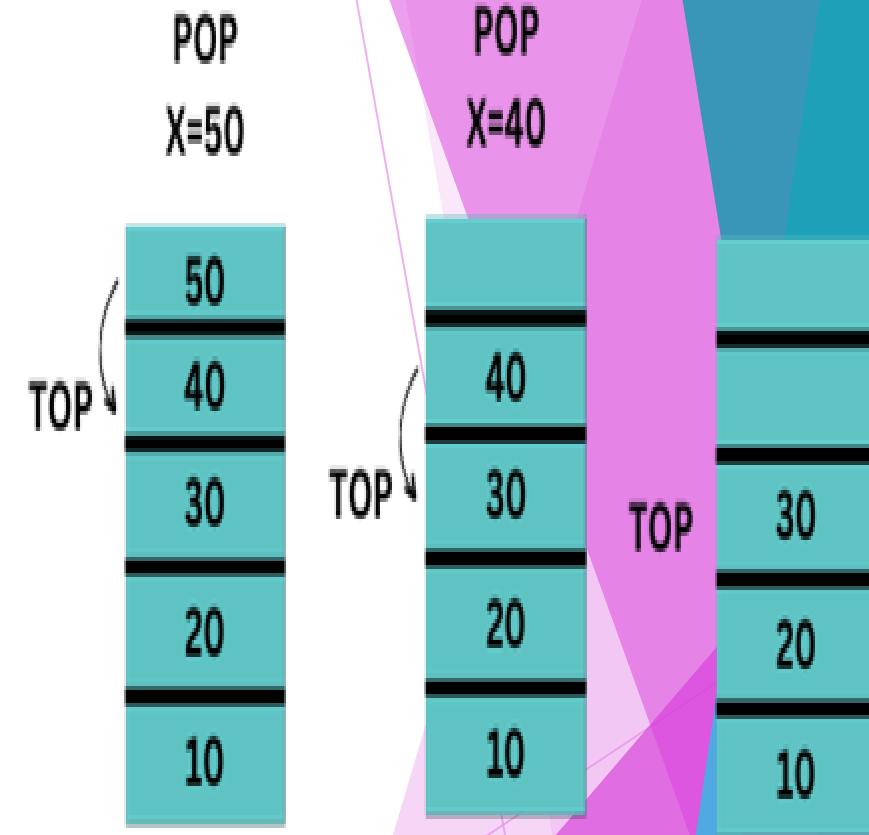
|50||40||30||20||10|

POP operation in a stack:

The POP operation in a stack is typically used in conjunction with the "PUSH" operation, which adds an element to the top of the stack.

Together, these operations allow you to implement a Last-In-First-Out (LIFO) data structure.

- 1. Check if the stack is empty. If the stack is empty, return an error, as popping from an empty stack is not allowed.**
- 2. Otherwise, retrieve the topmost element from the stack.**
- 3. Decrement the stack pointer to point to the next element in the stack.**
- 4. Remove the topmost element from the stack.**
- 5. Return the removed element if necessary.**



Algorithm of POP Operation in Stack

POP(S,TOP,N,X)

Here S is Stack,

TOP is Stack Pointer,

N is the Maximum Size of the Stack,

X is element to be deleted

Step 1: If $TOP == -1$ (Stack Empty)

Write "Underflow"

Return

(End of IF Statement)

Step 2: $X = S[TOP]$ (Delete Top Element)

Step 3: $TOP = TOP - 1$ (Decrement the Top)

Step 4 END

//Program in C to perform POP operation in Stack using Array

```
#include <stdio.h>
#define MAX_SIZE 100
int stack[MAX_SIZE];
int top = -1;
void push(int);
void pop(int);
int main()
{
    push(10);
    push(20);
    push(30);
    push(40);
    push(50);
    printf("Elements Available in the Stack\n");
    for(int i=top; i>=0;i--)
```

```
{
    printf("|%d|",stack[i]);
}
pop(50);
pop(40);
printf("Stack After Pop Operation\n");
for(int i=top; i>=0;i--)
{
    printf("|%d|",stack[i]);
}
return 0;
}
void push(int element)
{
    if (top >= MAX_SIZE - 1)
    {
        printf("Stack Overflow");
    }
}
```

```
else
{
    top=top+1;
    stack[top] = element;
    printf("%d pushed to stack\n", element);
} }

void pop(int element)
{
    if (top == - 1)
    {
        printf("Stack Underflow");
    }
    else
    {
        element= stack[top];
        top=top-1;
        printf("%d element deleted from Stack\n", element);
    } }
```

OUTPUT:-

10 pushed to stack

20 pushed to stack

30 pushed to stack

40 pushed to stack

50 pushed to stack

Elements Available in the Stack

|50||40||30||20||10|50 element

deleted from Stack

40 element deleted from Stack

Stack After Pop Operation

|30||20||10|

Applications of stacks :

Parenthesis Matching: In programming languages, parentheses are used to group expressions and indicate the order of evaluation.

- Parenthesis matching is a common problem where we need to check if a given expression contains matching opening and closing parentheses. This can be easily solved using a stack.
- For Example $X = a + (b * c)$, is a valid expression, it has same number of opening brackets and closing brackets.
- If the expression do not have same number of opening and closing brackets, that expression will be invalid.

Arithmetic Expression Evaluation: Stacks are often used in programming languages to evaluate arithmetic expressions. Operators and operands are added to the stack and the expression is evaluated according to the rules of precedence. For example, consider the expression $3 * (5 + 2)$. In this case output will be 21.

Function Call/Return: Stacks are used by programming languages to keep track of function calls and returns. When a function is called, its return address and local variables are pushed onto the stack. When the function returns, these values are popped off the stack. For example, consider a function that calls another function. The calling function's return address and local variables are pushed onto the stack, the called function is executed, and then the return address and variables are popped off the stack.

Undo/Redo Operations: Stacks are commonly used in software applications to keep track of undo/redo operations. Each time an action is performed, it is added to the stack.

When undo is requested, the most recent action is popped off the stack and reversed. When redo is requested, the previously undone action is popped off the redo stack and re-executed.

Browser History: Stacks are used in web browsers to keep track of visited web pages. Each time a new page is visited, its URL is added to the stack. When the back button is clicked, the most recent URL is popped off the stack and the browser navigates to the previous page.

Compiler Design: Stacks are used in compiler design to check the validity of a program's syntax. The stack is used to keep track of opening and closing brackets, braces, and parentheses.

For example, if a program contains an opening brace {, it is pushed onto the stack. When a closing brace } is encountered, the stack is popped to check if the opening brace matches the closing brace.

Depth First Search: Stacks are used in graph algorithms such as Depth First Search (DFS). The DFS algorithm visits all vertices in a graph by exploring as far as possible along each branch before backtracking. The stack is used to keep track of vertices that have been visited but not fully explored. Each time a vertex is visited, it is pushed onto the stack. When all adjacent vertices have been visited, the vertex is popped off the stack and the algorithm backtracks.

• **Expression conversion:** Stack can also be used for expression conversion. This is one of the most important applications of stack.

• The list of the expression conversion is given below:

- *Infix to prefix*
- *Infix to postfix*
- *Prefix to infix*
- *Prefix to postfix*
- *Postfix to infix*

1) Arithmetic expressions

Infix Notation:-

The expression which is in the normal format i.e the operator lies in between the operand

Ex: $a+b$, $a*b$

Syntax: $\langle \text{Operand} \rangle \langle \text{Operator} \rangle \langle \text{Operand} \rangle$

Prefix notation (or) Polish Notation:-

The operator comes first followed by the operands

Ex: $+ab$, $*ab$

Syntax: $\langle \text{Operator} \rangle \langle \text{Operand} \rangle \langle \text{Operand} \rangle$

Postfix Notation (or) Reverse Polish Notation:-

The operand comes first followed by the operator

Ex: $ab+$, $ab*$

Syntax: $\langle \text{Operand} \rangle \langle \text{Operator} \rangle \langle \text{Operand} \rangle$

Conversion of Infix to Postfix Expression

Rules:-

1. Priority of Operators

$\wedge$:- Highest Priority

$*, /, \%$:- next Priority

$+, -$:- Lowest Priority

2. No two operators of same priority can stay together in a stack column. If Stored POP() has to be done

3. Lowest Priority cannot be Placed before Highest Priority

If the operator is closed within the paranthesis then it has to be POP()

CONVERSION OF INFIX EXPRESSION TO POST FIX (A+B/C*(D+E)-F)

SYMBOL	STACK	POSTFIX
(	(	
A	(	A
+	(+	A
B	(+	B
/	(+ /	AB
C	(+ /	ABC
*	(+ *	ABC /
(	(+ * (	ABC /
D	(+ * (	ABC / D
+	(+ * (+	ABC / D
E	(+ * (+	ABC / DE
)	(+ * (+	ABC / DE +
-	(-	ABC / DE + * +
F	(-	ABC / DE + * + F
)	(-	ABC / DE + * + F -

FINAL EXPRESSION
ABC/DE+*+F-

Convert the INFIX Expression $A*B-C/D+E$ to Prefix

STEP 1:- $(A*B)-(C/D)+E$

STEP 2:- $[*AB]-[/CD]+E$

STEP 3:- $(T1-T2)+E$

// $*AB=T1, /CD=T2$

STEP 4:- $[-T1T2]+E$

// $-T1T2=T3$

STEP 5:- $T3+E$

STEP 6:- $+T3E$

STEP 7:- $+T3E$

STEP 8:- $+T3E$

Evaluating expression in postfix notation

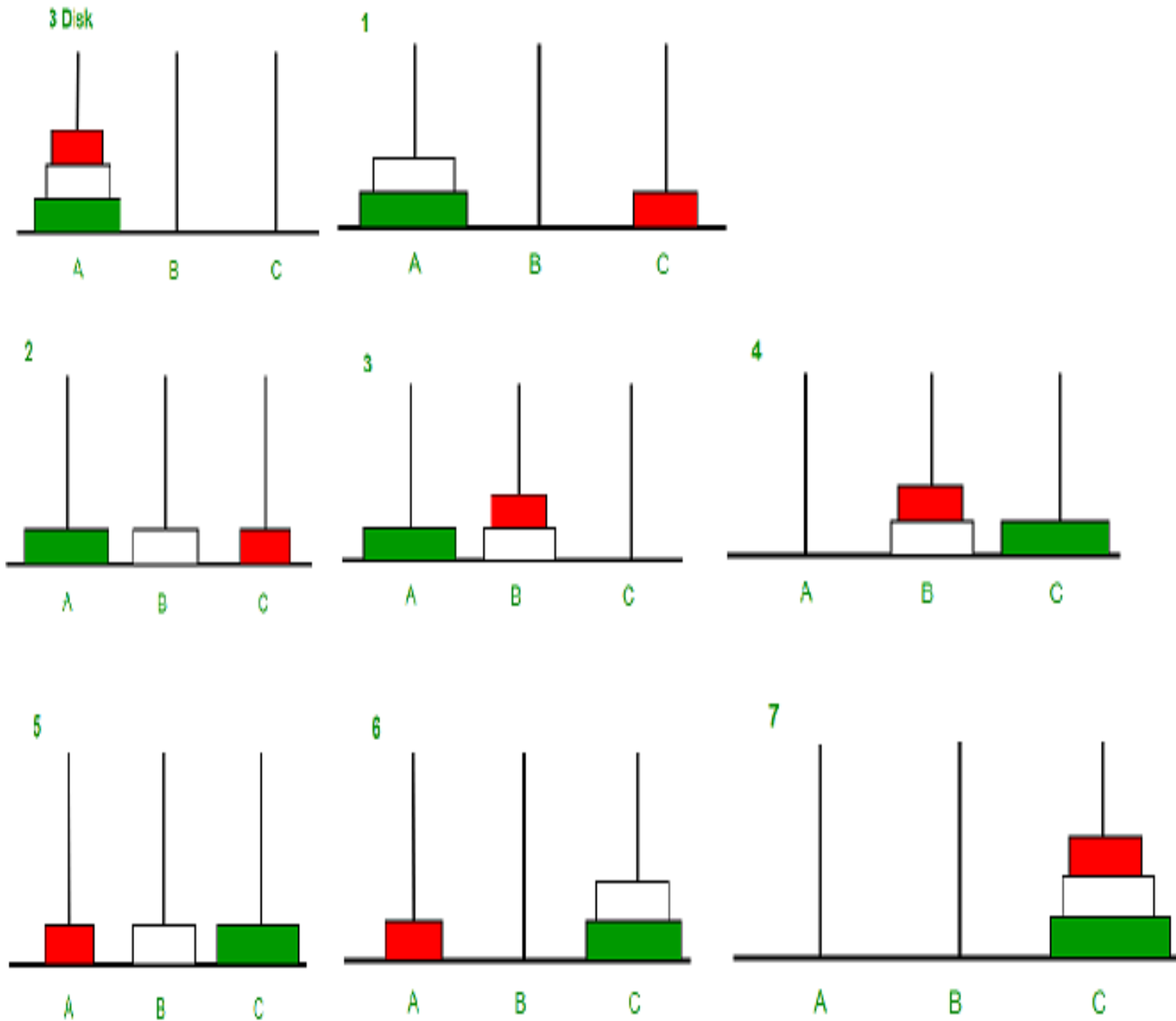
Consider the following postfix expression p:

$7\ 5 - 9\ 2 / *$

Character Scanned	Stack
7	7
5	7 5
-	2
9	2 9
2	2 9 2
/	2 4.5
*	9
End of expression	9

Introduction to Tower of Hanoi

- The Tower of Hanoi is a mathematical puzzle also known as **Lucas' Tower**.
- It is a game which consists of 3 rods/pegs/stack and n number of disks. These disks are of different sizes.
- These disks are arranged in an ascending order on a given peg.
- The objective of the game is to move the entire lot of disks of size n to another peg, following certain simple constraints which are:
 - *Only a single disk can be moved from one rod to another rod at a given time.*
 - *Only the topmost disk of a peg can be moved to the other.*
 - *No larger disk can be placed on a smaller disk.*
 - *We can use Auxiliary tower for temporary storage of disks*



STEPS:

1. Move the disk from S to D
2. Move the disk from S to A
3. Move the disk from D to A
4. Move the disk from S to D
5. Move the disk from A to S
6. Move the disk from A to D
7. Move the disk from S to D

Algorithm for Tower of Hanoi

procedure TOH(disk, Source, Dest, Aux):

 if disk == 1:

 move disk from Source to Dest

 else:

 TOH(disk - 1, Source, Aux, Dest)

 move disk from Source to Dest

 TOH(disk - 1, Aux, Dest, Source)

 END if

END TOH

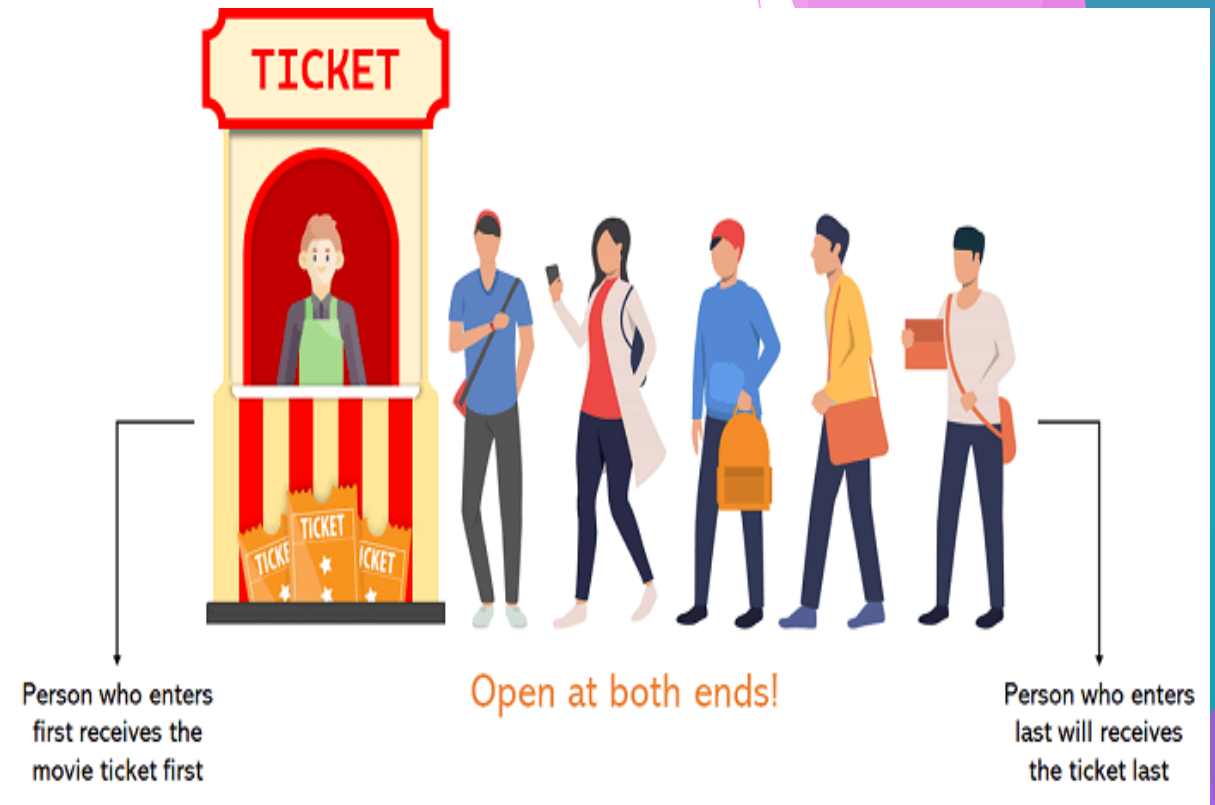
```
#include<stdio.h>
void towers(int,char,char,char);
int main(){
int n;
char src='S',aux='A',dest='D';
printf("enter no of disks\n");
scanf("%d",&n);
towers(n,src,aux,dest); }
void towers(int n, char src,char aux,char dest)
{
if(n==1)
printf("\n move disk from %c to %c",src,dest);
else
{
towers(n-1,src,dest,aux);
towers(1,src,aux,dest);
towers(n-1,aux,src,dest);
}}
```

Advantages of STACK	Disadvantages of STACK
Simple to Implement:- Stacks are straightforward to implement using either arrays or linked lists, facilitating easy management of data with basic operations.	Limited Access: Access is restricted to the top element, making it unsuitable for applications needing direct access to other elements.
Efficient Operations:- Operations such as push, pop, and peek are generally efficient, typically running in $O(1)$ time complexity.	Fixed Size :- If implemented using static arrays, the stack has a fixed size, which can lead to stack overflow if not managed correctly.
Memory Saving:- Only a pointer to the top is required, saving memory compared to other data structures that may require more complex management overheads.	Underutilization of Memory:- If the allocated memory is not fully utilized, it can lead to memory wastage, particularly with static array implementations.
Supports Recursion:- Stacks are essential for managing recursion in functions, making them integral to handling recursive algorithms effectively.	Potential for Stack Overflow/Underflow:- Improper handling of push and pop operations can lead to stack overflow (when full) or underflow (when empty), causing runtime errors.
Helps with Backtracking:- Useful in various backtracking algorithms, allowing easy reversal to previous states.	Not Scalable:- For dynamic implementations, resizing (e.g., using linked lists or vectors) can be costly in terms of time and memory.

QUEUE:- Operations and applications, Types - Circular Queue, Priority Queue.

Queue

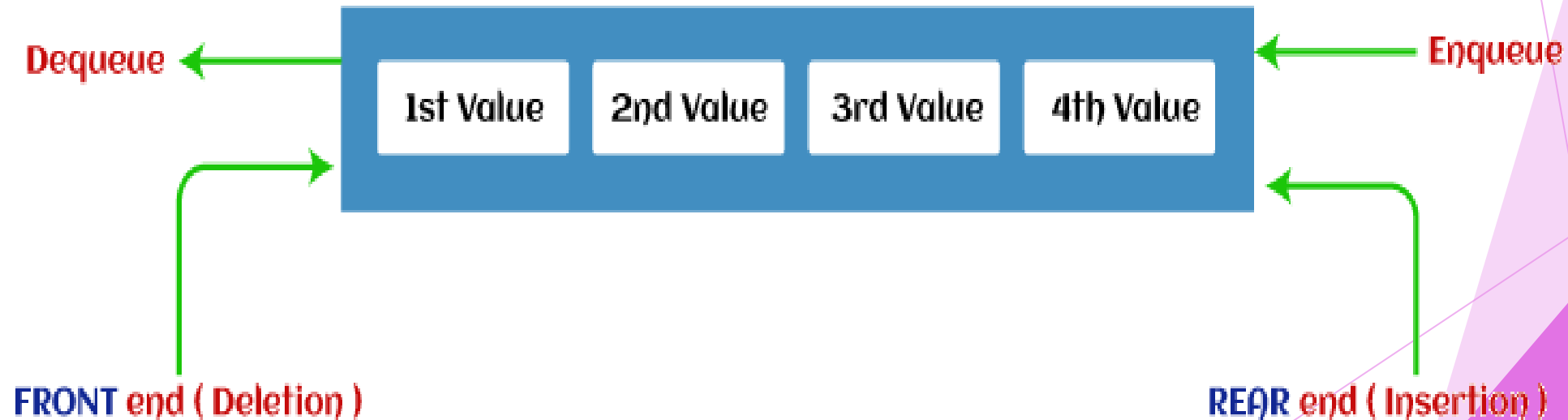
- ❖ A queue can be defined as an ordered list which enables insert operations to be performed at one end called **REAR** and delete operations to be performed at another end called **FRONT**.
- ❖ Queue is referred to be as First In First Out list.
- ❖ The principle is simple - the first item that enters a queue is the first one to be removed from it.



The primary operations of the Queue:

1.Enqueue: The insertion or Addition of some data elements to the Queue is called Enqueue. The element insertion is always done with the help of the rear pointer.

2.Dequeue: Deleting or removing data elements from the Queue is termed Dequeue. The deletion of the element is always done with the help of the front pointer.



Operations :

- Enqueue:** The Enqueue operation is used to insert the element at the rear end of the queue. It returns void.
- Dequeue:** It performs the deletion from the front-end of the queue. It also returns the element which has been removed from the front-end. It returns an integer value.
- Peek:** This is the third operation that returns the element, which is pointed by the front pointer in the queue but does not delete it.
- Queue overflow (isfull):** It shows the overflow condition when the queue is completely full.
- Queue underflow (isempty):** It shows the underflow condition when the Queue is empty, i.e., no elements are in the Queue.

Memory Representation of Queue

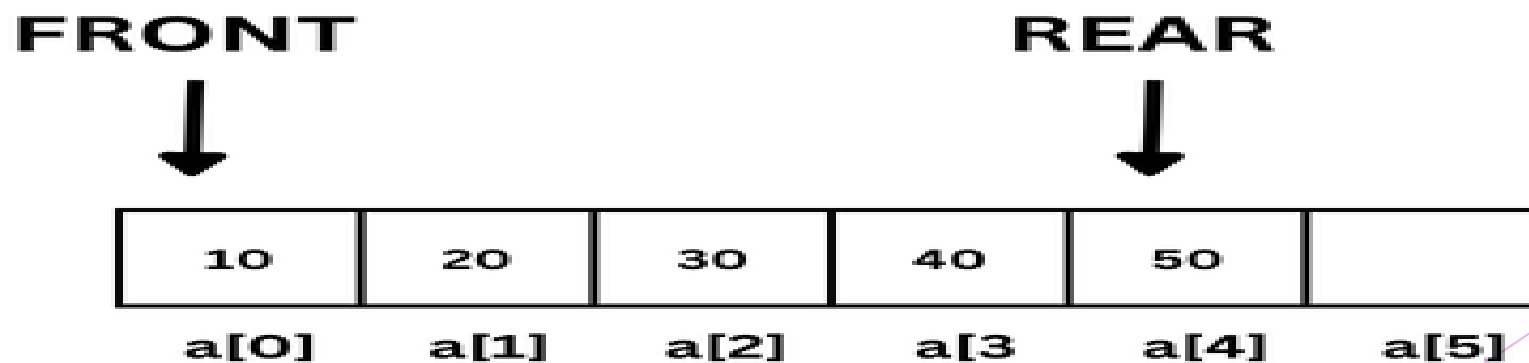
Array representation of a queue:

A queue is implemented using a fixed-size array.

The front and rear of the queue are represented by two pointers, front and rear respectively.

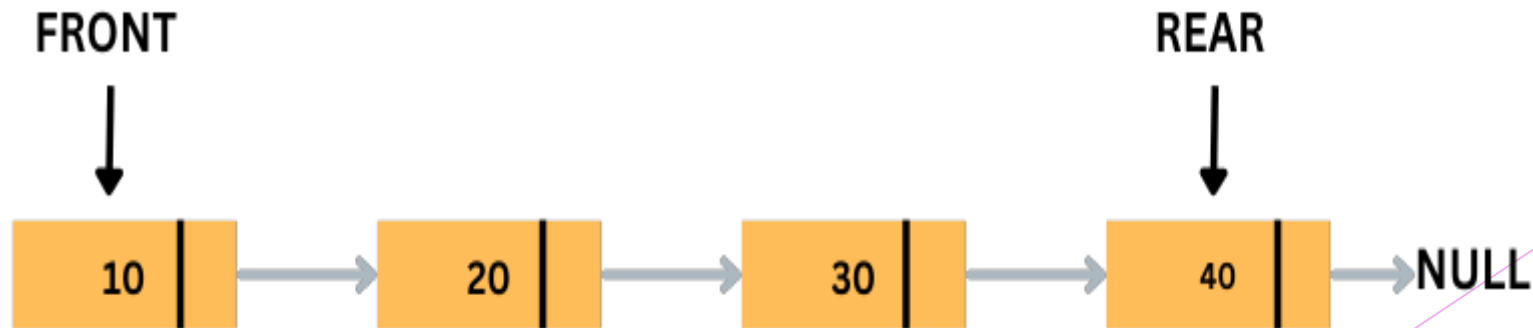
When a new element is added to the queue, it is inserted at the rear end, and the rear pointer is incremented.

Similarly, when an element is removed from the queue, it is removed from the front end, and the front pointer is incremented.



Linked list representation of a queue:-

- ✓ In this representation, a queue is implemented using a linked list.
- ✓ Each node in the linked list contains an element and a pointer to the next node.
- ✓ The front and rear of the queue are represented by two pointers, front and rear respectively, which point to the first and last nodes of the linked list.
- ✓ When a new element is added to the queue, it is added to the rear end of the linked list, and the rear pointer is updated to point to the new node.
- ✓ Similarly, when an element is removed from the queue, it is removed from the front end of the linked list, and the front pointer is updated to point to the next node.



Algorithm to Insert a New Element in a Queue (ENQUEUE Operation)

Here QUEUE is the name of the queue and MAX is Maximum Size of the queue and FRONT pointer variable , REAR pointer variable, ITEM is the element to be inserted.

Step 1: If $\text{REAR} = \text{MAX} - 1$ then

Write "OVERFLOW"

End of If Statement

Step 2: IF $\text{FRONT} = -1$ (IS QUEUE EMPTY)

Set $\text{FRONT} = 0$

End of If Statement

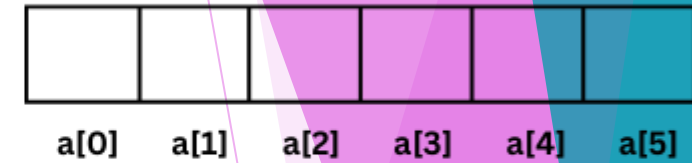
Step 3: $\text{REAR} = \text{REAR} + 1$ (Increment REAR)

Step 4: $\text{QUEUE}[\text{REAR}] = \text{ITEM}$

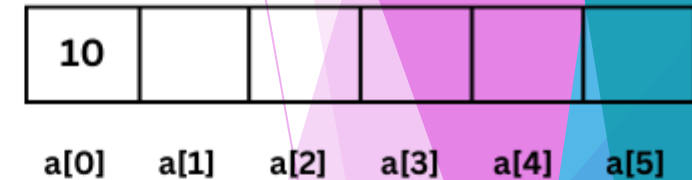
Step 5: END

QUEUE is EMPTY

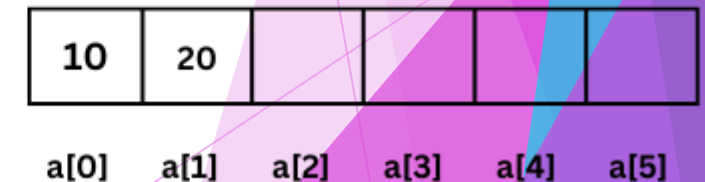
FRONT and REAR = -1



FRONT REAR
↓ ↓



FRONT REAR
↓ ↓



Insert ITEM =10 in QUEUE

Insert ITEM =20 in QUEUE

```
#include <stdio.h>
#define MAX_SIZE 10
void enqueue(int);
int queue[MAX_SIZE];
int front = -1, rear = -1;
int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);
    printf("Printing QUEUE\n");
    for(int i=front;i<=rear;i++)
    {
        printf("|%d|",queue[i]);
    }
    return 0; }
```

```
void enqueue(int value)
{
    if (rear == MAX_SIZE - 1)
    {
        printf("Queue overflow!");
        return;
    }
    if (front == -1)
    {
        front = 0;
    }
    rear=rear+1;
    queue[rear] = value;
    printf("%d enqueued to queue\n.", value); }
```

OUTPUT:-

Algorithm to Delete an Element from a Queue (DEQUEUE Operation)

QUEUE is the name, MAX is Size
,FRONT , REAR pointer variable, ITEM
element to be inserted. If FRONT and
REAR = -1 then Queue is Empty

Step 1: If FRONT = -1 then

Write "UNDERFLOW"

End of If Statement

Step 2: ITEM = QUEUE[FRONT]

Step 3: IF FRONT = REAR (QUEUE
has only 1 Element)

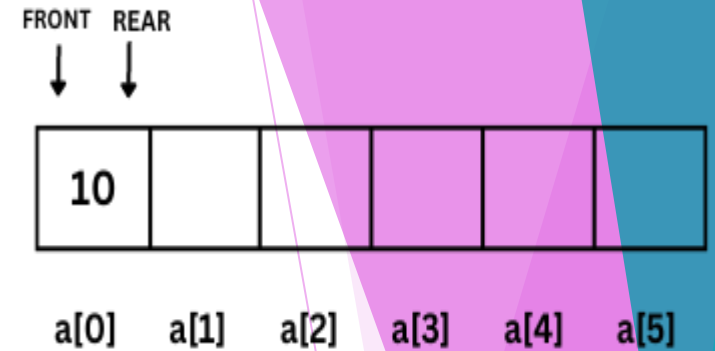
Set FRONT = -1 and REAR = -1

ELSE

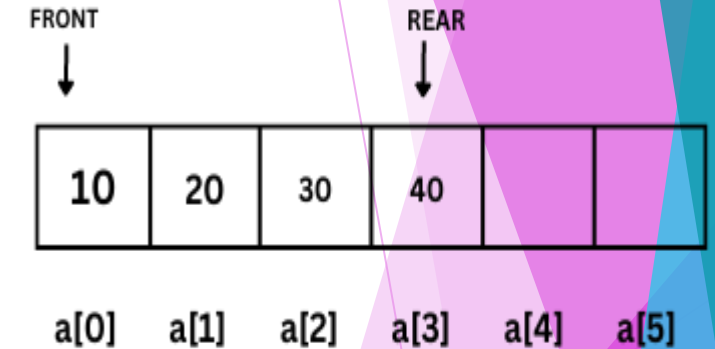
Step 4: FRONT = FRONT+1(Increment
FRONT)

End of If Statement

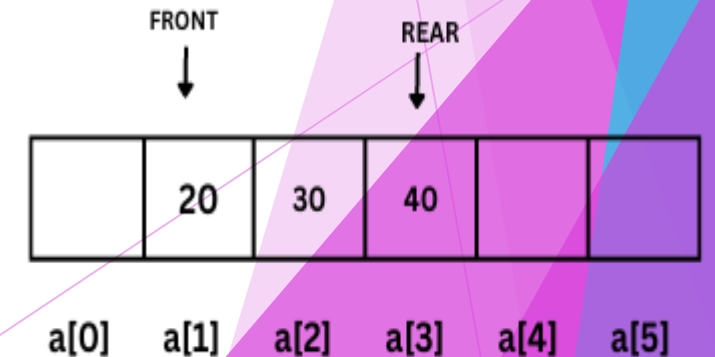
Case 1: Queue has only One Element



Case 2: Queue has Multiple Elements



Delete Element From Queue



```
#include <stdio.h>
#define MAX_SIZE 10
void enqueue(int);
void dequeue();
int queue[MAX_SIZE];
int front = -1, rear = -1;
int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    dequeue();
    dequeue();
    dequeue();
    dequeue();
    return 0;}
void enqueue(int value)
{
    if (rear == MAX_SIZE - 1) {
        printf("Queue overflow!");
        return; }
    if (front == -1)
    { front = 0; }
    rear++;
```

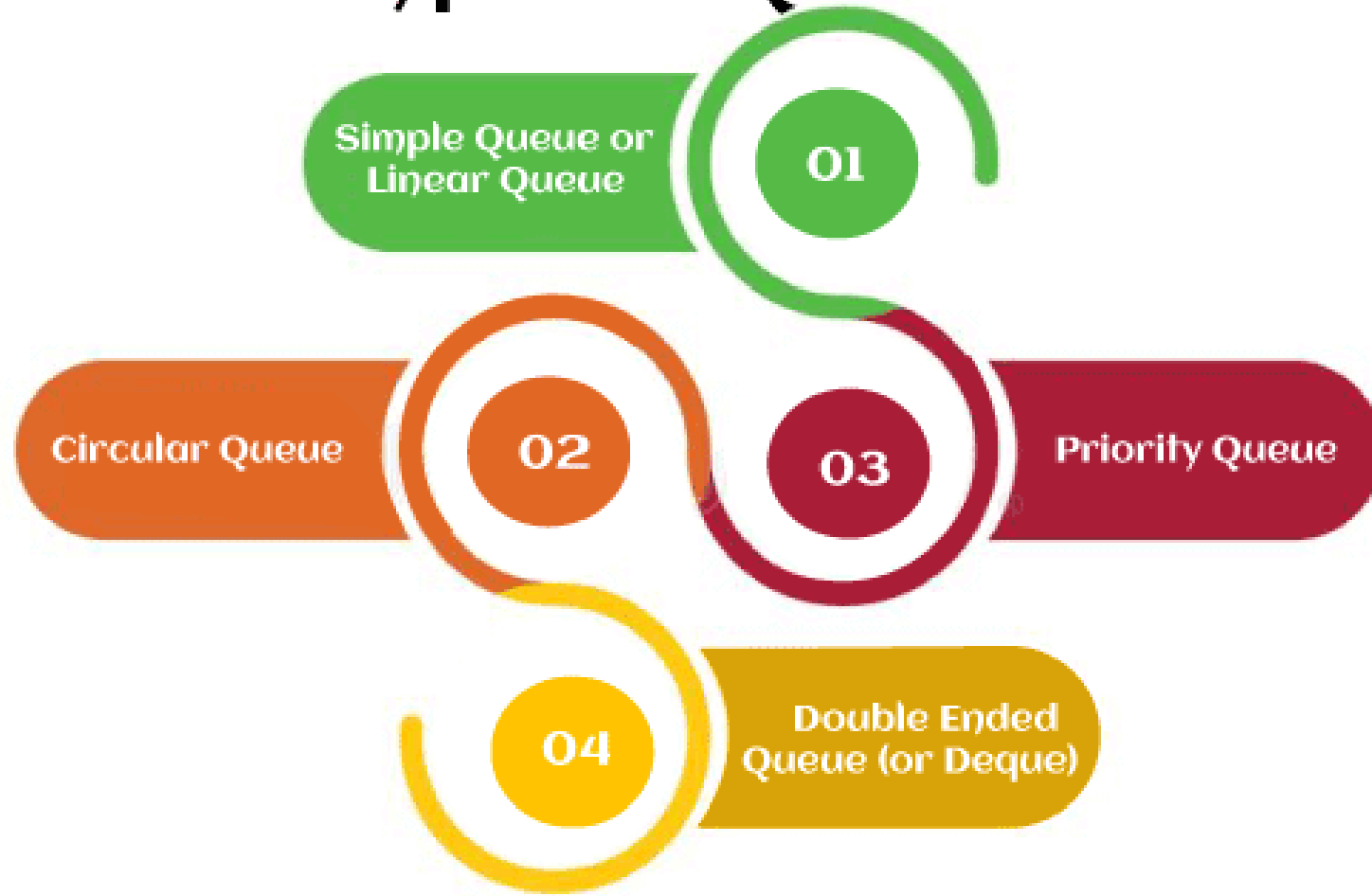
```
    queue[rear] = value;
    printf("%d enqueued to queue\n", value);}
void dequeue()
{
    if (front == -1 )
    {
        printf("Queue underflow!");
        return;
    }
    if (front==rear)
    {
        front = -1;
        rear = -1;
    }
    else
    {
        printf("%d dequeued from queue\n", queue[front]);
        front++;
    }
}
```

Applications of Queue

Some of the applications of queues include:

- 1. Operating systems:** Operating systems use queues to manage resources such as CPU time, memory, and input/output devices.
- 2. Network routing:** In computer networks, packets of data are sent from one device to another. Queues are used to manage the flow of these packets, ensuring that they are delivered in the correct order and without delay.
- 3. Customer service:** Many businesses use queues to manage customer service requests.
- 4. Print spooling:** When multiple users want to print a document, they are placed in a queue and processed in the order they were received.
- 5. Call centers:** When customers call a call center, their calls are placed in a queue and handled by an available agent. This ensures that all customers are served in the order they called and that no calls are lost or ignored.
- 6. Task scheduling:** In software development, queues are used to schedule tasks such as database updates, report generation, and file processing.

Types of Queues



Simple Queue or Linear Queue

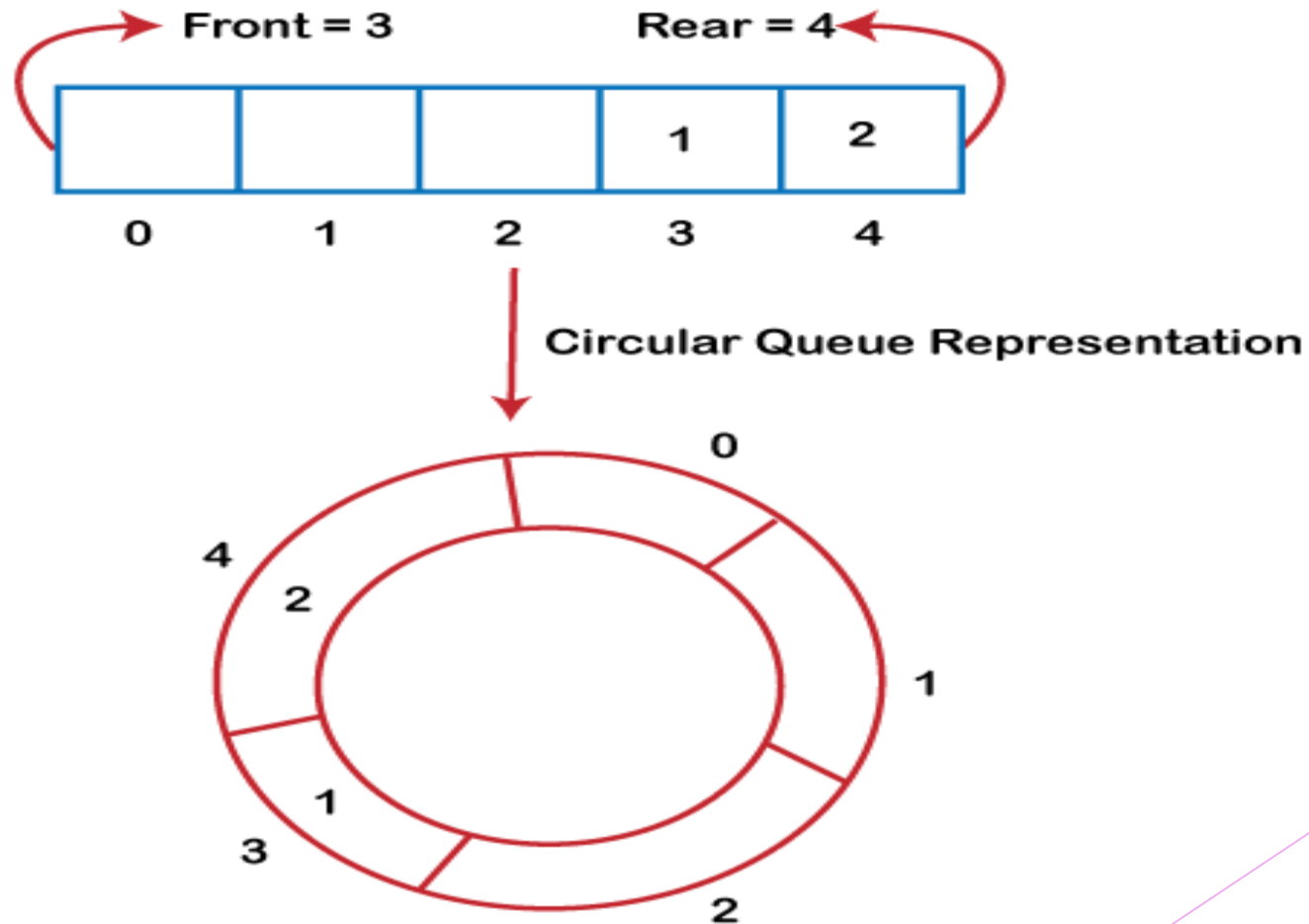
In Linear Queue, an insertion takes place from one end while the deletion occurs from another end. The end at which the insertion takes place is known as the rear end, and the end at which the deletion takes place is known as front end. It strictly follows the FIFO rule.

The major drawback of using a linear Queue is that insertion is done only from the rear end. If the first three elements are deleted from the Queue, we cannot insert more elements even though the space is available in a Linear Queue.



Circular Queue

In Circular Queue, all the nodes are represented as circular. It is similar to the linear Queue except that the last element of the queue is connected to the first element. It is also known as Ring Buffer, as all the ends are connected to another end.



Operations on Circular Queue

The following are the operations that can be performed on a circular queue:

- Front:** It is used to get the front element from the Queue.
- Rear:** It is used to get the rear element from the Queue.
- enQueue(value):** This function is used to insert the new value in the Queue. The new element is always inserted from the rear end.
- deQueue():** This function deletes an element from the Queue. The deletion in a Queue always takes place from the front end.

Algorithm to insert an element in a circular queue

Step 1: IF $(\text{REAR} + 1) \% \text{MAX} = \text{FRONT}$

Write " OVERFLOW "

Goto step 4

[End OF IF]

Step 2: IF $\text{FRONT} = -1$ and $\text{REAR} = -1$

SET $\text{FRONT} = \text{REAR} = 0$

ELSE IF $\text{REAR} = \text{MAX} - 1$ and $\text{FRONT} \neq 0$

SET $\text{REAR} = 0$

ELSE

SET $\text{REAR} = (\text{REAR} + 1) \% \text{MAX}$

[END OF IF]

Step 3: SET $\text{QUEUE}[\text{REAR}] = \text{VAL}$

Step 4: EXIT

Algorithm to delete an element from the circular queue

Step 1: IF FRONT = -1

Write " UNDERFLOW "

Goto Step 4

[END of IF]

Step 2: SET VAL = QUEUE[FRONT]

Step 3: IF FRONT = REAR

SET FRONT = REAR = -1

ELSE

IF FRONT = MAX -1

SET FRONT = 0

ELSE

SET FRONT = FRONT + 1

[END of IF]

[END OF IF]



0 1 2 3 4

Front = -1

Rear = -1



0 1 2 3 4

Front = 0

Rear = 0



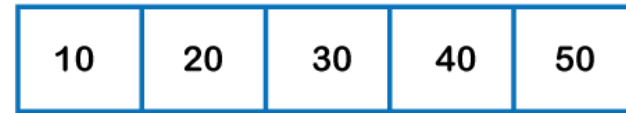
Front = 0 Rear = 2



0 1 2 3 4

Front = 0

Rear = 3



0 1 2 3 4

Front = 0

Rear = 4



0 1 2 3 4
dequeue

Front = 2

Rear = 4



0 1 2 3 4

Rear

Front



0 1 2 3 4

Rear

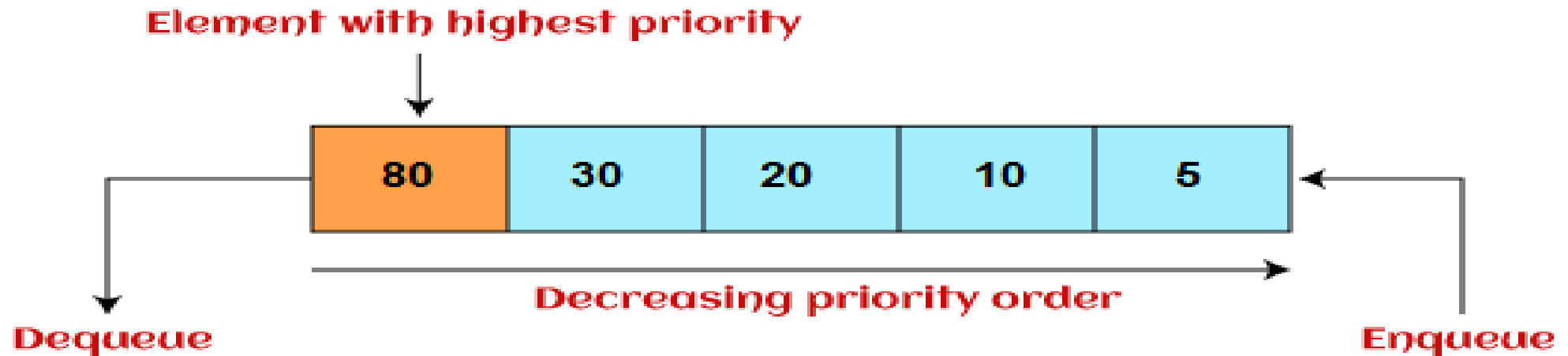
Front

Implementation of circular queue using Array

https://onlinegdb.com/_jIUnzofuH

Priority Queue:-

It is a special type of queue in which the elements are arranged based on the priority. It is a special type of queue data structure in which every element has a priority associated with it. Suppose some elements occur with the same priority, they will be arranged according to the FIFO principle. Insertion in priority queue takes place based on the arrival, while deletion in the priority queue occurs based on the priority. Priority queue is mainly used to implement the CPU scheduling algorithms.



Insertion in priority queue takes place based on the arrival, while deletion in the priority queue occurs based on the priority. Priority queue is mainly used to implement the CPU scheduling algorithms.

There are two types of priority queues

- **Ascending priority queue** - In ascending priority queue, elements can be inserted in arbitrary order, but only smallest can be deleted first. Suppose an array with elements 7, 5, and 3 in the same order, so, insertion can be done with the same sequence, but the order of deleting the elements is 3, 5, 7.

- **Descending priority queue** - In descending priority queue, elements can be inserted in arbitrary order, but only the largest element can be deleted first. Suppose an array with elements 7, 3, and 5 in the same order, so, insertion can be done with the same sequence, but the order of deleting the elements is 7, 5, 3.

Advantages of Queue Data Structure

Simplicity and Predictability: Queues operate on a simple and predictable First In, First Out (FIFO) principle, which is easy to implement and understand.

Fairness: By processing elements in the order they arrive, queues ensure fairness in systems like scheduling processes in an operating system or managing customers in service industries.

Resource Sharing: Queues help manage resources efficiently in various applications, such as print spooling and task scheduling, by maintaining a queue of tasks that await processing.

Asynchronous Data Handling: Queues are excellent for handling data from asynchronous processes, such as buffering inputs in I/O operations, where data can arrive at any time and needs to be processed in the correct order.

Load Balancing: In distributed computing environments, queues can help in load balancing by queuing tasks and distributing them evenly amongst multiple servers.

Disadvantages of Queue Data Structure

Fixed Size: In static queue implementations (using arrays), the size of the queue is fixed, leading to potential overflow if the queue gets filled. This requires careful management and can lead to inefficient use of memory.

Time-consuming Operations: Operations such as searching for an element or accessing elements other than the front can be time-consuming as they require traversal of the queue.

One-way Access: The queue structure only allows access from one end for addition and the other for removal, which limits flexibility compared to other data structures like deques (double-ended queues) that allow insertion and removal from both ends.

UNIT-II

UNIT II

Linked Lists: Singly/Linear Linked Lists,
Operations - Insertion and deletion from a list, Linked List
Implementation of Stacks and Queues,
Doubly and Circular Linked Lists, Applications.

Linked Lists: Singly/Linear Linked Lists

Linked List:-

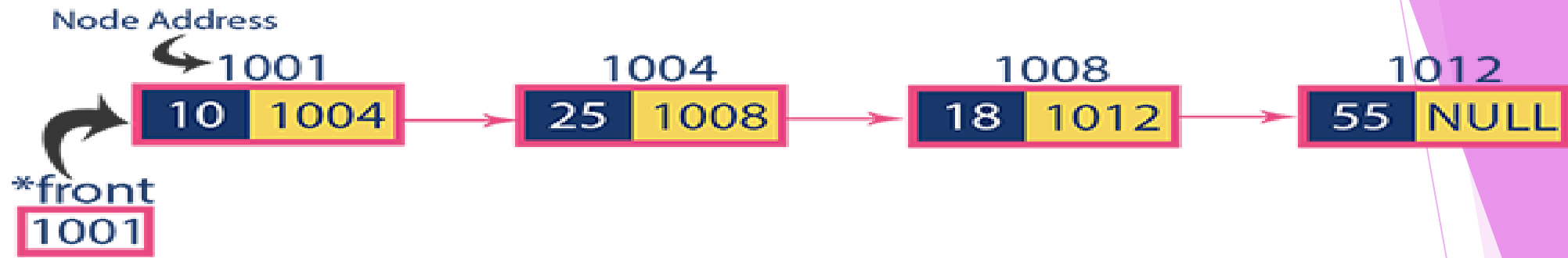
The linked list is a linear data structure that contains a sequence of elements such that each element links to its next element in the sequence. Each element in a linked list is called "Node".



Single Linked List:-

Single linked list is a sequence of elements in which every element has link to its next element in the sequence.

In any single linked list, the individual element is called as "Node". Every "Node" contains two fields, data field, and the next field. The data field is used to store actual value of the node and next field is used to store the address of next node in the sequence



Operations on Single Linked List

The following operations are performed on a Single Linked List

- Insertion
- Deletion
- Display

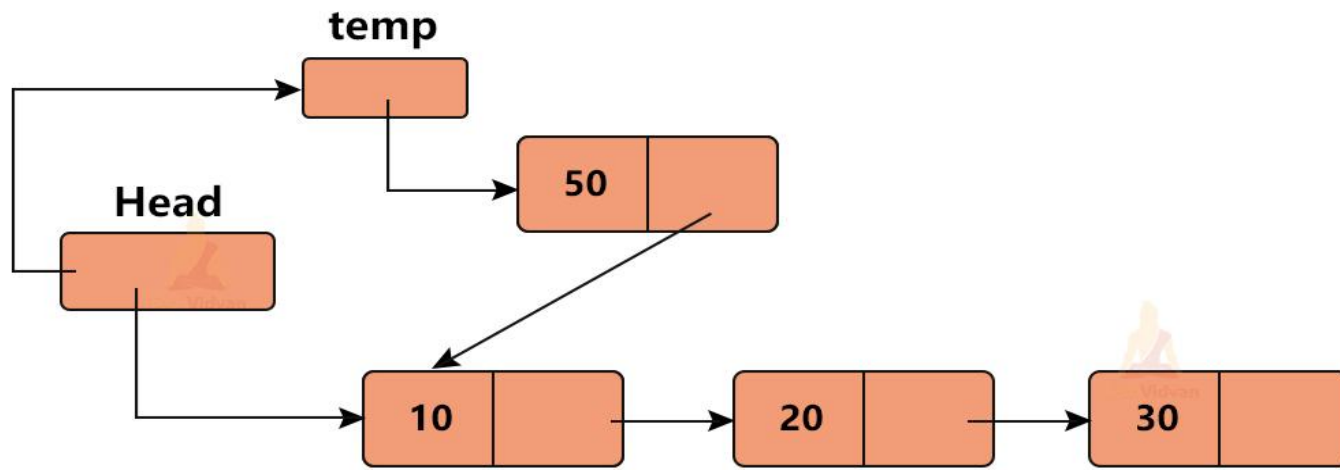
Insertion

In a single linked list, the insertion operation can be performed in three ways.

- 1.Inserting At Beginning of the list
- 2.Inserting At End of the list
- 3.Inserting At Specific location in the list

Inserting At Beginning of the list

- Step 1** - Create a **newNode** with given value.
- Step 2** - Check whether list is **Empty** (**head == NULL**)
- Step 3** - If it is **Empty** then, set **newNode→next = NULL** and **head = newNode**.
- Step 4** - If it is **Not Empty** then, set **newNode→next = head** and **head = newNode**.



The above diagram shows that we have a linked list of the following elements:
10→20→30.

Once we insert a new node at the beginning, the list will be 50→10→20→30.

```
Void Insert_at_beginning(int key)
{
    struct Node *temp; //temp is a
    pointer pointing to the node to be
    inserted.
    temp = (struct Node*)
    malloc(sizeof(struct Node));
    if(temp != NULL)
    {
        temp→data = key; //key is the
        data value to be inserted
        temp→next = Head; //this step
        will make the incoming node the first
        node.
    }
}
```

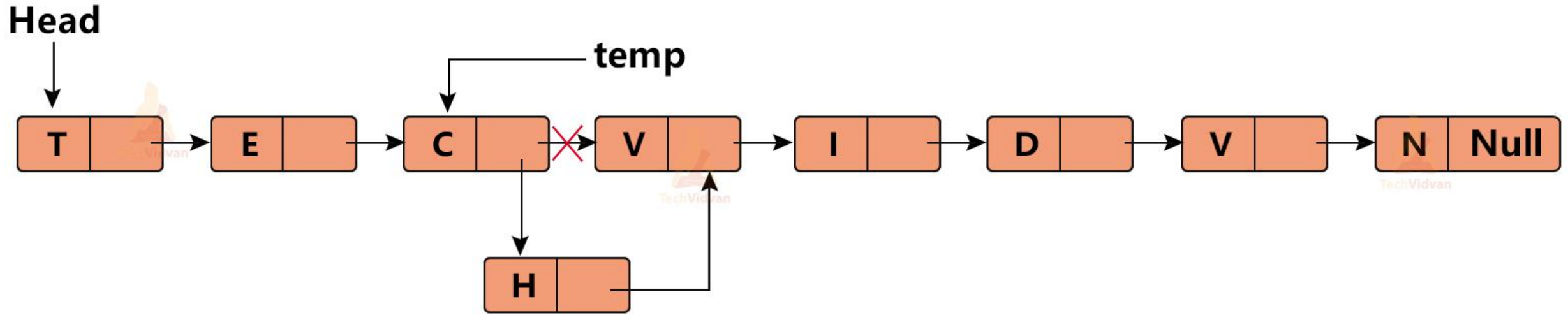
Inserting At End of the list

- **Step 1** - Create a **newNode** with given value and **newNode** $\rightarrow$ **next** as **NULL**.
- **Step 2** - Check whether list is **Empty** (**head** == **NULL**).
- **Step 3** - If it is **Empty** then, set **head** = **newNode**.
- **Step 4** - If it is **Not Empty** then, define a node pointer **temp** and initialize with **head**.
- **Step 5** - Keep moving the **temp** to its next node until it reaches to the last node in the list (until **temp** $\rightarrow$ **next** is equal to **NULL**).
- **Step 6** - Set **temp** $\rightarrow$ **next** = **newNode**.

```
void Insert_at_end(int key)
{
    struct Node *temp, *Ptr;
    temp = (struct Node*) malloc(sizeof(struct Node));
    if(temp != NULL)
    {
        temp->data = key;
        temp->Next = NULL;
        if(Head == NULL) //This will be true if the list is initially empty
        {
            Head = temp;
            return;
        }
        Ptr = Head;
        while(Ptr->Next != NULL)
            Ptr->Next = temp;
    }
}
```

Inserting At Specific location in the list (After a Node)

- Step 1** - Create a **newNode** with given value.
- Step 2** - Check whether list is **Empty** (**head == NULL**)
- Step 3** - If it is **Empty** then, set **newNode** $\rightarrow$ **next** = **NULL** and **head** = **newNode**.
- Step 4** - If it is **Not Empty** then, define a node pointer **temp** and initialize with **head**.
- Step 5** - Keep moving the **temp** to its next node until it reaches to the node after which we want to insert the newNode (until **temp1** $\rightarrow$ **data** is equal to **location**, here location is the node value after which we want to insert the newNode).
- Step 6** - Every time check whether **temp** is reached to last node or not. If it is reached to last node then display '**Given node is not found in the list!!! Insertion not possible!!!**' and terminate the function. Otherwise move the **temp** to next node.
- Step 7** - Finally, Set '**newNode** $\rightarrow$ **next** = **temp** $\rightarrow$ **next**' and '**temp** $\rightarrow$ **next** = **newNode**'



SYNTAX:-

$\text{temp} \rightarrow \text{Next} = \text{Ptr} \rightarrow \text{Next};$

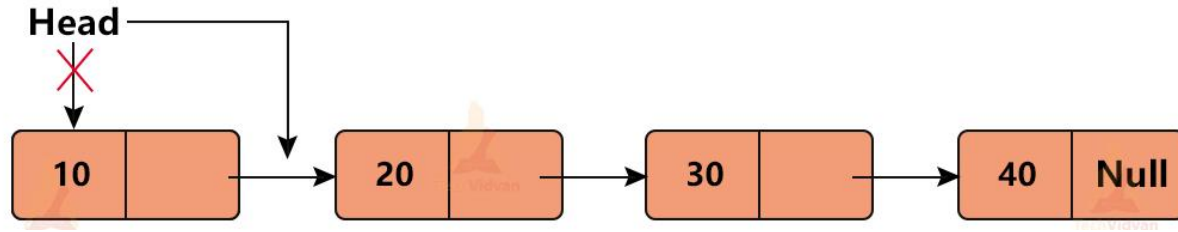
$\text{Ptr} \rightarrow \text{Next} = \text{temp};$

Deletion:- In a single linked list, the deletion operation can be performed in three ways.

1. Deleting from Beginning of the list
2. Deleting from End of the list
3. Deleting a Specific Node

Deleting from Beginning of the list

- **Step 1** - Check whether list is **Empty** (**head == NULL**)
- **Step 2** - If it is **Empty** then, display '**List is Empty!!! Deletion is not possible**' and terminate the function.
- **Step 3** - If it is **Not Empty** then, define a Node pointer '**temp**' and initialize with **head**.
- **Step 4** - Check whether list is having only one node (**temp → next == NULL**)
- **Step 5** - If it is **TRUE** then set **head = NULL** and delete **temp** (Setting **Empty** list conditions)
- **Step 6** - If it is **FALSE** then set **head = temp → next**, and delete **temp**.



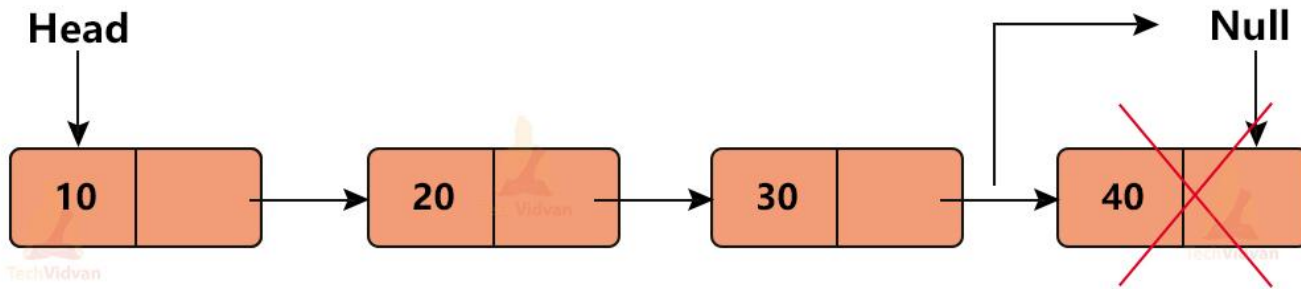
Suppose we initially had the linked list elements as $10 \rightarrow 20 \rightarrow 30 \rightarrow 40$.

If we perform deletion of one node at the beginning, the linked list will be:
 $20 \rightarrow 30 \rightarrow 40$.

```
void Delete_from_beginning(int key)
{
    struct Node *Ptr = Head;
    if(Head == NULL) //i.e. If the list is
empty
    {
        return;
    }
    Head = Head->Next; //linking Head with
the 2nd node of the list
    free(Ptr);
}
```


Deleting from End of the list

- **Step 1** - Check whether list is **Empty** (**head == NULL**)
- **Step 2** - If it is **Empty** then, display '**List is Empty!!! Deletion is not possible**' and terminate the function.
- **Step 3** - If it is **Not Empty** then, define two Node pointers '**temp1**' and '**temp2**' and initialize '**temp1**' with **head**.
- **Step 4** - Check whether list has only one Node (**temp1 → next == NULL**)
- **Step 5** - If it is **TRUE**. Then, set **head = NULL** and delete **temp1**. And terminate the function. (Setting **Empty** list condition)
- **Step 6** - If it is **FALSE**. Then, set '**temp2 = temp1** ' and move **temp1** to its next node. Repeat the same until it reaches to the last node in the list. (until **temp1 → next == NULL**)
- **Step 7** - Finally, Set **temp2 → next = NULL** and delete **temp1**.

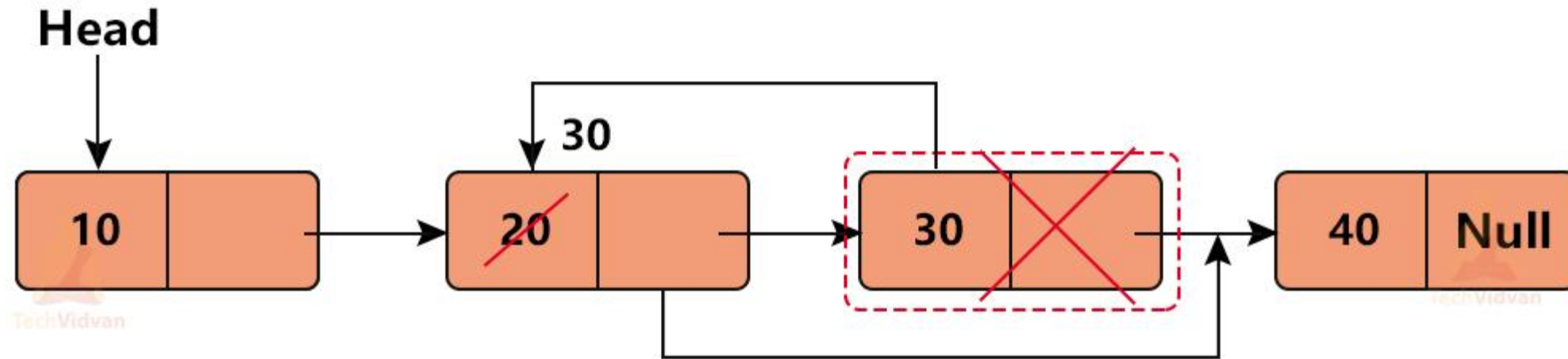


```
void Delete_at_end(int key)
```

```
{
    struct Node *Ptr = Head;
    struct Node *Prev = NULL;
    if(Head == NULL) //If the list is empty
    {
        return;
    }
    if(Head->Next == NULL) //If the list has only one
element
    {
        Head = NULL;
        free(Ptr);
    }
    else
    {
        while(Ptr->Next != NULL)
        {
            Prev = Ptr;
            Ptr = Ptr->Next;
        }
        free(Ptr);
    }
}
```

Deleting a Specific Node from the list:- We can use the following steps to delete a specific node from the single linked list...

- Step 1** - Check whether list is **Empty** (**head == NULL**)
- Step 2** - If it is **Empty** then, display '**List is Empty!!! Deletion is not possible**' and terminate the function.
- Step 3** - If it is **Not Empty** then, define two Node pointers '**temp1**' and '**temp2**' and initialize '**temp1**' with **head**.
- Step 4** - Keep moving the **temp1** until it reaches to the exact node to be deleted or to the last node. And every time set '**temp2 = temp1**' before moving the '**temp1**' to its next node.
- Step 5** - If it is reached to the last node then display '**Given node not found in the list! Deletion not possible!!!**'. And terminate the function.
- Step 6** - If it is reached to the exact node which we want to delete, then check whether list is having only one node or not
- Step 7** - If list has only one node and that is the node to be deleted, then set **head = NULL** and delete **temp1** (**free(temp1)**).
- Step 8** - If list contains multiple nodes, then check whether **temp1** is the first node in the list (**temp1 == head**).
- Step 9** - If **temp1** is the first node then move the **head** to the next node (**head = head → next**) and delete **temp1**.
- Step 10** - If **temp1** is not first node then check whether it is last node in the list (**temp1 → next == NULL**).
- Step 11** - If **temp1** is last node then set **temp2 → next = NULL** and delete **temp1** (**free(temp1)**).
- Step 12** - If **temp1** is not first node and not last node then set **temp2 → next = temp1 → next** and delete **temp1** (**free(temp1)**).



SYNTAX:-

```
Ptr->data = Ptr->Next->data;  
Ptr = Ptr->Next;  
Ptr->Next = Ptr->Next->Next;  
free(Ptr);
```

Displaying a Single Linked List

- Step 1** - Check whether list is **Empty** (**head == NULL**)
- Step 2** - If it is **Empty** then, display '**List is Empty!!!**' and terminate the function.
- Step 3** - If it is **Not Empty** then, define a Node pointer '**temp**' and initialize with **head**.
- Step 4** - Keep displaying **temp** → **data** with an arrow (--->) until **temp** reaches to the last node
- Step 5** - Finally display **temp** → **data** with arrow pointing to **NULL** (**temp** → **data** ---> **NULL**).

Advantages of Singly Linked Lists

- ✓ Singly linked lists in data structures can grow and shrink during runtime as needed without a predefined size.
- ✓ They only allocate memory for nodes that are actually in use, reducing memory waste compared to pre-allocated data structures like **arrays**.
- ✓ Adding or removing nodes doesn't require shifting elements, which can be a costly operation in arrays. This is particularly beneficial at the beginning of the list.
- ✓ Unlike arrays, **linked lists** don't reserve memory in advance, which can be more efficient for certain types of applications where the size of the data structure fluctuates.

Disadvantages of Singly Linked Lists

- ❖ Accessing an element in a singly linked list requires traversal from the head to the point of interest, which can be time-consuming.
- ❖ Each node in a singly linked list requires additional memory for the pointer alongside the actual data.
- ❖ Implementing a singly linked list is more complex than using an array, particularly when it comes to handling pointers, which can introduce bugs like memory leaks.

Linked List Implementation of Stacks & Queues

Linked List vs Array

ARRAY

Arrays are contiguously stored in the memory.

The array size should be declared inside the program in advance.

It is difficult to change the size of the array afterwards.

Arrays allow random access of elements.

Arrays have slower insertion or deletion time as compared to linked lists.

Arrays require less memory per element.

LINKED LIST

Linked lists are not necessarily contiguous inside the memory.

We need not declare the size of the list in advance.

We can expand or shrink a linked list any time during the execution of the program.

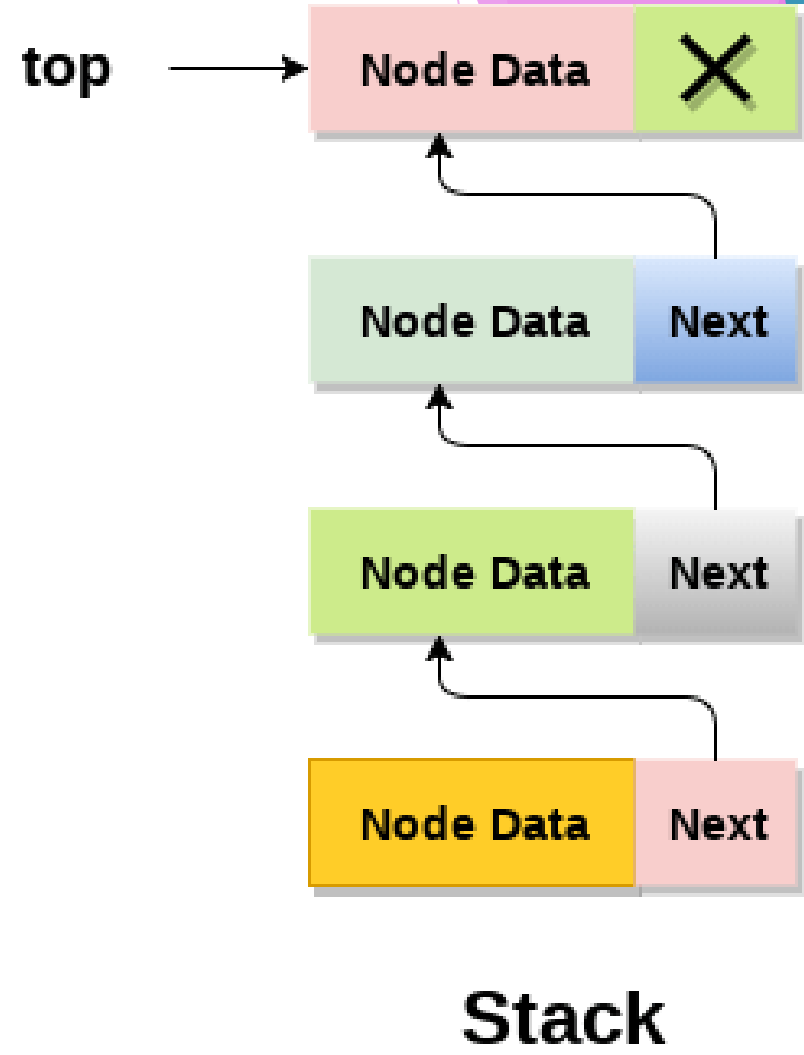
In a linked list, we cannot randomly access any element, we need to traverse the list from the beginning.

Linked lists have a slower search time because of the absence of random access.

Lists require more memory because we need to maintain extra pointers.

Linked list implementation of stack

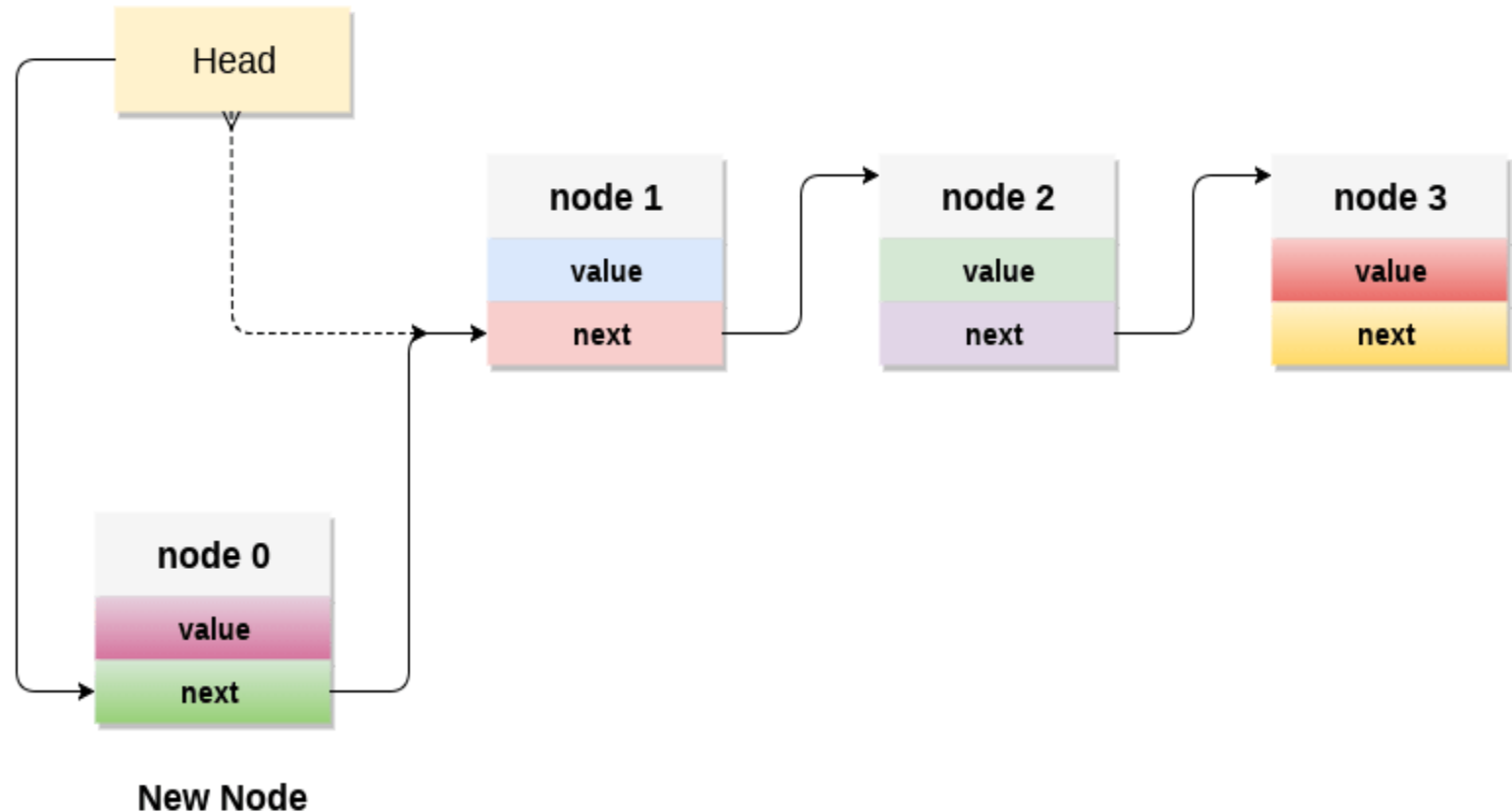
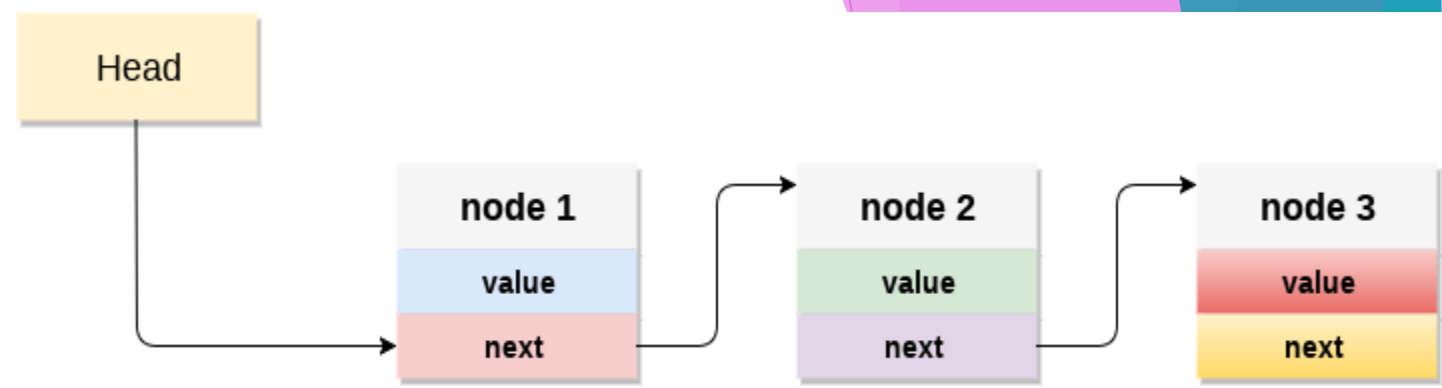
- ✓ Instead of using array, we can also use linked list to implement stack.
- ✓ Linked list allocates the memory dynamically, whereas time complexity in both the scenario is same for all the operations i.e. push, pop and peek.
- ✓ In linked list implementation of stack, the nodes are maintained non-contiguously in the memory.
- ✓ Each node contains a pointer to its immediate successor node in the stack.
- ✓ The top most node in the stack always contains null in its address field.



Adding a node to the stack (Push operation)

Adding a node to the stack is referred to as **push** operation. Pushing an element to a stack in linked list implementation is different from that of an array implementation.

1. Create a node first and allocate memory to it.
2. If the list is empty then the item is to be pushed as the start node of the list.
3. If there are some nodes in the list already, then we have to add the new element in the beginning of the list.



```
void push () {
    int val;
    struct node *ptr =(struct node*)malloc(sizeof(struct node));
    if(ptr == NULL) {
        printf("not able to push the element");
    }
    else {
        printf("Enter the value");
        scanf("%d",&val);
        if(head==NULL) {
            ptr->val = val;
            ptr -> next = NULL;
            head=ptr;
        }
        else {
            ptr->val = val;
            ptr->next = head;
            head=ptr;
        }
        printf("Item pushed");    } }
```

Deleting a node from the stack (POP operation)

Deleting a node from the top of stack is referred to as **pop** operation. Deleting a node from the linked list implementation of stack is different from that in the array implementation.

Check for the underflow condition: The underflow condition occurs when we try to pop from an already empty stack. The stack will be empty if the head pointer of the list points to null.

Adjust the head pointer accordingly: In stack, the elements are popped only from one end, therefore, the value stored in the head pointer must be deleted and the node must be freed. The next node of the head node now becomes the head node.

```
void pop()
{
    int item;
    struct node *ptr;
    if (head == NULL)
    {
        printf("Underflow");
    }
    else
    {
        item = head->val;
        ptr = head;
        head = head->next;
        free(ptr);
        printf("Item popped");
    }
}
```

Display the nodes (Traversing)

Displaying all the nodes of a stack needs traversing all the nodes of the linked list organized in the form of stack.

- ❑ Copy the head pointer into a temporary pointer.
- ❑ Move the temporary pointer through all the nodes of the list and print the value field attached to every node.

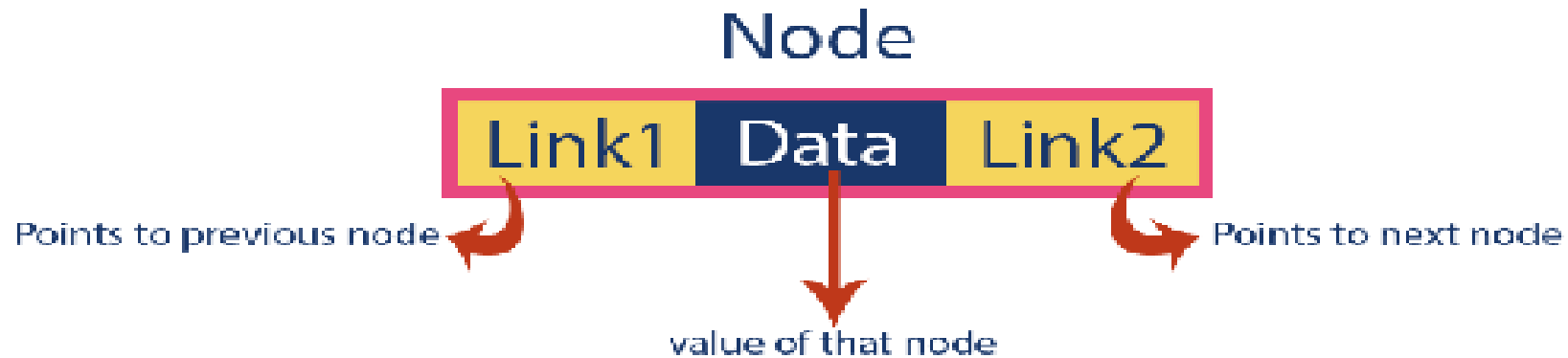
```
void display()
{
    int i;
    struct node *ptr;
    ptr=head;
    if(ptr == NULL)
    {
        printf("Stack is empty\n");
    }
    else
    {
        printf("Printing Stack elements \n");
        while(ptr!=NULL)
        {
            printf("%d\n",ptr->val);
            ptr = ptr->next;
        }
    }
}
```

Doubly and Circular Linked Lists, Applications.

Double Linked List:-

Double linked list is a sequence of elements in which every element has links to its previous element and next element in the sequence.

In a double linked list, every node has a link to its previous node and next node. So, we can traverse forward by using the next field and can traverse backward by using the previous field. Every node in a double linked list contains three fields



Here, '**link1**' field is used to store the address of the previous node in the sequence, '**link2**' field is used to store the address of the next node in the sequence and '**data**' field is used to store the actual value of that node.

Operations on Double Linked List

In a double linked list, we perform the following operations...

- 1.Insertion
- 2.Deletion
- 3.Display

Insertion

- 1.Inserting At Beginning of the list
- 2.Inserting At End of the list
- 3.Inserting At Specific location in the list

Inserting At Beginning of the list

- Step 1** - Create a **newNode** with given value and **newNode** → **previous** as **NULL**.
- Step 2** - Check whether list is **Empty** (**head == NULL**)
- Step 3** - If it is **Empty** then, assign **NULL** to **newNode** → **next** and **newNode** to **head**.
- Step 4** - If it is **not Empty** then, assign **head** to **newNode** → **next** and **newNode** to **head**.

Inserting At End of the list

We can use the following steps to insert a new node at end of the double linked list...

- Step 1** - Create a **newNode** with given value and **newNode** → **next** as **NULL**.
- Step 2** - Check whether list is **Empty** (**head == NULL**)
- Step 3** - If it is **Empty**, then assign **NULL** to **newNode** → **previous** and **newNode** to **head**.
- Step 4** - If it is **not Empty**, then, define a node pointer **temp** and initialize with **head**.
- Step 5** - Keep moving the **temp** to its next node until it reaches to the last node in the list (until **temp** → **next** is equal to **NULL**).
- Step 6** - Assign **newNode** to **temp** → **next** and **temp** to **newNode** → **previous**.

Inserting At Specific location in the list (After a Node)

- Step 1** - Create a **newNode** with given value.
- Step 2** - Check whether list is **Empty** (**head == NULL**)
- Step 3** - If it is **Empty** then, assign **NULL** to both **newNode** → **previous** & **newNode** → **next** and set **newNode** to **head**.
- Step 4** - If it is **not Empty** then, define two node pointers **temp1** & **temp2** and initialize **temp1** with **head**.
- Step 5** - Keep moving the **temp1** to its next node until it reaches to the node after which we want to insert the **newNode** (until **temp1** → **data** is equal to **location**, here location is the node value after which we want to insert the **newNode**).
- Step 6** - Every time check whether **temp1** is reached to the last node. If it is reached to the last node then display '**Given node is not found in the list!!! Insertion not possible!!!**' and terminate the function. Otherwise move the **temp1** to next node.
- Step 7** - Assign **temp1** → **next** to **temp2**, **newNode** to **temp1** → **next**, **temp1** to **newNode** → **previous**, **temp2** to **newNode** → **next** and **newNode** to **temp2** → **previous**.

Deletion

1. Deleting from Beginning of the list
2. Deleting from End of the list
3. Deleting a Specific Node

Deleting from Beginning of the list

- **Step 1** - Check whether list is **Empty** (**head == NULL**)
- **Step 2** - If it is **Empty** then, display '**List is Empty!!! Deletion is not possible**' and terminate the function.
- **Step 3** - If it is not Empty then, define a Node pointer '**temp**' and initialize with **head**.
- **Step 4** - Check whether list is having only one node (**temp** → **previous** is equal to **temp** → **next**)
- **Step 5** - If it is **TRUE**, then set **head** to **NULL** and delete **temp** (Setting **Empty** list conditions)
- **Step 6** - If it is **FALSE**, then assign **temp** → **next** to **head**, **NULL** to **head** → **previous** and delete **temp**.

Deleting from End of the list

- Step 1** - Check whether list is **Empty** (**head == NULL**)
- Step 2** - If it is **Empty**, then display '**List is Empty!!! Deletion is not possible**' and terminate the function.
- Step 3** - If it is not Empty then, define a Node pointer '**temp**' and initialize with **head**.
- Step 4** - Check whether list has only one Node (**temp** → **previous** and **temp** → **next** both are **NULL**)
- Step 5** - If it is **TRUE**, then assign **NULL** to **head** and delete **temp**. And terminate from the function. (Setting **Empty** list condition)
- Step 6** - If it is **FALSE**, then keep moving **temp** until it reaches to the last node in the list. (until **temp** → **next** is equal to **NULL**)
- Step 7** - Assign **NULL** to **temp** → **previous** → **next** and delete **temp**.

- **Step 1** - Check whether list is **Empty** (**head == NULL**)
- **Step 2** - If it is **Empty** then, display '**List is Empty!!! Deletion is not possible**' and terminate the function.
- **Step 3** - If it is not Empty, then define a Node pointer '**temp**' and initialize with **head**.
- **Step 4** - Keep moving the **temp** until it reaches to the exact node to be deleted or to the last node.
- **Step 5** - If it is reached to the last node, then display '**Given node not found in the list! Deletion not possible!!!**' and terminate the function.
- **Step 6** - If it is reached to the exact node which we want to delete, then check whether list is having only one node or not
- **Step 7** - If list has only one node and that is the node which is to be deleted then set **head** to **NULL** and delete **temp** (**free(temp)**).
- **Step 8** - If list contains multiple nodes, then check whether **temp** is the first node in the list (**temp == head**).
- **Step 9** - If **temp** is the first node, then move the **head** to the next node (**head = head → next**), set **head** of **previous** to **NULL** (**head → previous = NULL**) and delete **temp**.
- **Step 10** - If **temp** is not the first node, then check whether it is the last node in the list (**temp → next == NULL**).
- **Step 11** - If **temp** is the last node then set **temp** of **previous** of **next** to **NULL** (**temp → previous → next = NULL**) and delete **temp** (**free(temp)**).
- **Step 12** - If **temp** is not the first node and not the last node, then set **temp** of **previous** of **next** to **temp** of **next** (**temp → previous → next = temp → next**), **temp** of **next** of **previous** to **temp** of **previous** (**temp → next → previous = temp → previous**) and delete **temp** (**free(temp)**).

Displaying a Double Linked List

- **Step 1** - Check whether list is **Empty** (**head == NULL**)
- **Step 2** - If it is **Empty**, then display '**List is Empty!!!**' and terminate the function.
- **Step 3** - If it is not Empty, then define a Node pointer '**temp**' and initialize with **head**.
- **Step 4** - Display '**NULL <---** '.
- **Step 5** - Keep displaying **temp** → **data** with an arrow (**<===>**) until **temp** reaches to the last node
- **Step 6** - Finally, display **temp** → **data** with arrow pointing to **NULL** (**temp** → **data ---> NULL**).

Advantages of Doubly Linked Lists

Bidirectional Navigation: Allows traversal in both forward and backward directions, facilitating easier and more flexible data manipulation.

Easier Insertion and Deletion: Nodes can be added or removed from both ends and the middle of the list without needing to traverse the entire list, especially if the tail pointer is maintained.

Efficient Operations at Both Ends: Adding or removing elements at the beginning and the end is efficient because both head and tail pointers provide direct access to the endpoints of the list.

Dynamic Size: The size of the list can increase or decrease dynamically, which is efficient for memory usage since it allocates space only as needed.

Disadvantages of Doubly Linked Lists

Increased Memory Usage: Each node requires extra memory for an additional pointer (previous pointer), which can be significant in memory-constrained environments.

Complexity: Managing two pointers (next and prev) per node increases the complexity of the operations, making the code more prone to errors such as memory leaks and pointer corruption.

Slower Individual Operations: The overhead of maintaining an extra pointer can slightly slow down operations compared to **singly linked lists**, as more pointer operations are required.

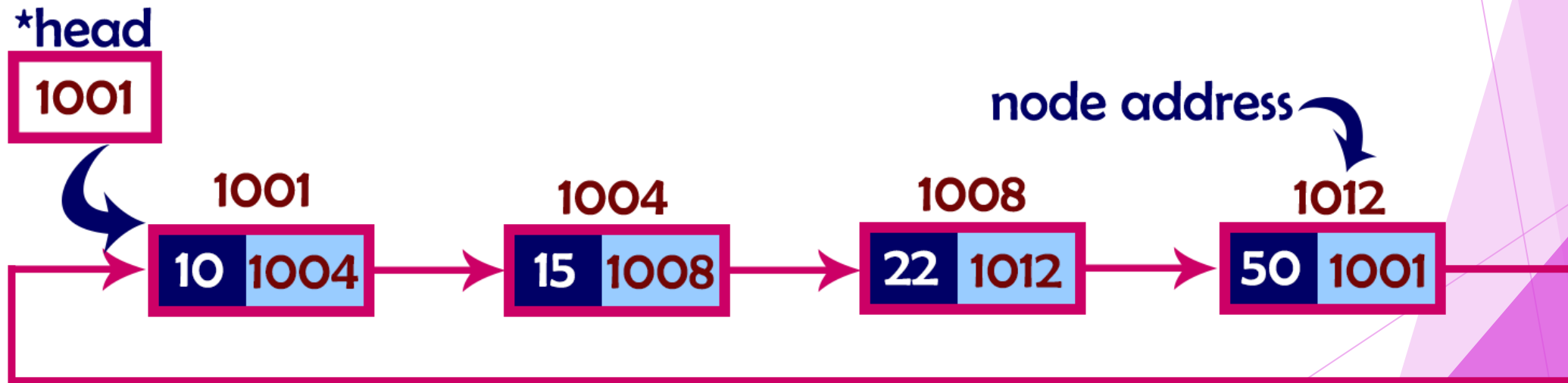
Overhead in Memory Management: More complex memory management is needed, especially in languages that do not handle garbage collection automatically, due to the additional pointers that need to be correctly handled during insertions and deletions.

Difference Between Singly and Doubly Linked List

Feature	Singly Linked List	Doubly Linked List
Direction of Traversal	Only forwards. You can traverse from the head to the end of the list.	Both forwards and backwards. You can traverse from the head to the tail and vice versa.
Memory Usage	Less memory per node (one pointer per node).	More memory per node (two pointers per node).
Insertions/Deletions	Efficient at the beginning; requires traversal from the head for other positions.	Efficient at both the beginning and the end; easier insertions and deletions in the middle without full traversal.
Complexity	Simpler and requires less code to manage compared to doubly linked lists.	More complex due to additional pointers, requiring more management and care in code.
Use Case	Suitable when memory is a concern and only forward traversal is needed.	Preferred when frequent operations require elements to be accessed from both ends.
Operations	Typically slower for operations that involve elements at the end of the list.	Faster for operations involving the end of the list due to the tail pointer.
Head and Tail Management	Only head pointer is used.	Both head and tail pointers are used, providing immediate access to both ends of the list.

Circular Linked List

Circular Linked List:- A circular linked list is a sequence of elements in which every element has a link to its next element in the sequence and the last element has a link to the first element. That means circular linked list is similar to the single linked list except that the last node points to the first node in the list



Operations

1.Insertion

2.Deletion

3.Display

Before we implement actual operations, first we need to setup empty list. First perform the following steps before implementing actual operations.

- Step 1** - Include all the **header files** which are used in the program.
- Step 2** - Declare all the **user defined** functions.
- Step 3** - Define a **Node** structure with two members **data** and **next**
- Step 4** - Define a Node pointer '**head**' and set it to **NULL**.
- Step 5** - Implement the **main** method by displaying operations menu and make suitable function calls in the main method to perform user selected operation.

Insertion

- 1.Inserting At Beginning of the list
- 2.Inserting At End of the list
- 3.Inserting At Specific location in the list

Inserting At Beginning of the list

- Step 1** - Create a **newNode** with given value.
- Step 2** - Check whether list is **Empty** (**head == NULL**)
- Step 3** - If it is **Empty** then, set **head = newNode** and **newNode→next = head** .
- Step 4** - If it is **Not Empty** then, define a Node pointer '**temp**' and initialize with '**head**'.
- Step 5** - Keep moving the '**temp**' to its next node until it reaches to the last node (until '**temp → next == head**').
- Step 6** - Set '**newNode → next = head**', '**head = newNode**' and '**temp → next = head**'.

Inserting At End of the list

- **Step 1** - Create a **newNode** with given value.
- **Step 2** - Check whether list is **Empty** (**head == NULL**).
- **Step 3** - If it is **Empty** then, set **head = newNode** and **newNode** → **next = head**.
- **Step 4** - If it is **Not Empty** then, define a node pointer **temp** and initialize with **head**.
- **Step 5** - Keep moving the **temp** to its next node until it reaches to the last node in the list (until **temp** → **next == head**).
- **Step 6** - Set **temp** → **next = newNode** and **newNode** → **next = head**.

Inserting At Specific location in the list (After a Node)

- Step 1** - Create a **newNode** with given value.
- Step 2** - Check whether list is **Empty** (**head == NULL**)
- Step 3** - If it is **Empty** then, set **head = newNode** and **newNode → next = head**.
- Step 4** - If it is **Not Empty** then, define a node pointer **temp** and initialize with **head**.
- Step 5** - Keep moving the **temp** to its next node until it reaches to the node after which we want to insert the newNode (until **temp1 → data** is equal to **location**, here location is the node value after which we want to insert the newNode).
- Step 6** - Every time check whether **temp** is reached to the last node or not. If it is reached to last node then display '**Given node is not found in the list!!! Insertion not possible!!!**' and terminate the function. Otherwise move the **temp** to next node.
- Step 7** - If **temp** is reached to the exact node after which we want to insert the newNode then check whether it is last node (**temp → next == head**).
- Step 8** - If **temp** is last node then set **temp → next = newNode** and **newNode → next = head**.
- Step 8** - If **temp** is not last node then set **newNode → next = temp → next** and **temp → next = newNode**.

Deletion

1. Deleting from Beginning of the list
2. Deleting from End of the list
3. Deleting a Specific Node

Deleting from Beginning of the list

- **Step 1** - Check whether list is **Empty** (**head == NULL**)
- **Step 2** - If it is **Empty** then, display '**List is Empty!!! Deletion is not possible**' and terminate the function.
- **Step 3** - If it is **Not Empty** then, define two Node pointers '**temp1**' and '**temp2**' and initialize both '**temp1**' and '**temp2**' with **head**.
- **Step 4** - Check whether list is having only one node (**temp1 → next == head**)
- **Step 5** - If it is **TRUE** then set **head = NULL** and delete **temp1** (Setting **Empty** list conditions)
- **Step 6** - If it is **FALSE** move the **temp1** until it reaches to the last node. (until **temp1 → next == head**)
- **Step 7** - Then set **head = temp2 → next**, **temp1 → next = head** and delete **temp2**.

Deleting from End of the list

- Step 1** - Check whether list is **Empty** (**head == NULL**)
- Step 2** - If it is **Empty** then, display '**List is Empty!!! Deletion is not possible**' and terminate the function.
- Step 3** - If it is **Not Empty** then, define two Node pointers '**temp1**' and '**temp2**' and initialize '**temp1**' with **head**.
- Step 4** - Check whether list has only one Node (**temp1 → next == head**)
- Step 5** - If it is **TRUE**. Then, set **head = NULL** and delete **temp1**. And terminate from the function. (Setting **Empty** list condition)
- Step 6** - If it is **FALSE**. Then, set '**temp2 = temp1** ' and move **temp1** to its next node. Repeat the same until **temp1** reaches to the last node in the list. (until **temp1 → next == head**)
- Step 7** - Set **temp2 → next = head** and delete **temp1**.

Deleting a Specific Node from the list

- **Step 1** - Check whether list is **Empty** (**head == NULL**)
- **Step 2** - If it is **Empty** then, display '**List is Empty!!! Deletion is not possible**' and terminate the function.
- **Step 3** - If it is **Not Empty** then, define two Node pointers '**temp1**' and '**temp2**' and initialize '**temp1**' with **head**.
- **Step 4** - Keep moving the **temp1** until it reaches to the exact node to be deleted or to the last node. And every time set '**temp2 = temp1**' before moving the '**temp1**' to its next node.
- **Step 5** - If it is reached to the last node then display '**Given node not found in the list! Deletion not possible!!!**'. And terminate the function.
- **Step 6** - If it is reached to the exact node which we want to delete, then check whether list is having only one node (**temp1 → next == head**)
- **Step 7** - If list has only one node and that is the node to be deleted then set **head = NULL** and delete **temp1** (**free(temp1)**).
- **Step 8** - If list contains multiple nodes then check whether **temp1** is the first node in the list (**temp1 == head**).
- **Step 9** - If **temp1** is the first node then set **temp2 = head** and keep moving **temp2** to its next node until **temp2** reaches to the last node. Then set **head = head → next**, **temp2 → next = head** and delete **temp1**.
- **Step 10** - If **temp1** is not first node then check whether it is last node in the list (**temp1 → next == head**).
- **Step 11** - If **temp1** is last node then set **temp2 → next = head** and delete **temp1** (**free(temp1)**).
- **Step 12** - If **temp1** is not first node and not last node then set **temp2 → next = temp1 → next** and delete **temp1** (**free(temp1)**).

Displaying a circular Linked List

- Step 1** - Check whether list is **Empty** (**head == NULL**)
- Step 2** - If it is **Empty**, then display '**List is Empty!!!**' and terminate the function.
- Step 3** - If it is **Not Empty** then, define a Node pointer '**temp**' and initialize with **head**.
- Step 4** - Keep displaying **temp** → **data** with an arrow (--->) until **temp** reaches to the last node
- Step 5** - Finally display **temp** → **data** with arrow pointing to **head** → **data**.

Advantages of Circular Linked Lists

Continuous Traversal: The circular nature allows for continuous traversal of the list, which is useful for applications that require looping over the same data repeatedly.

No Null Values: There are no null references in the nodes (for the next/previous pointers), which can simplify certain list operations by eliminating the need to check for null as a termination condition.

Efficient Queue Operations: Especially in a doubly circular linked list, operations such as enqueue and dequeue can be made more efficient because both the front and rear of the queue are accessible.

Resource Sharing: Useful in scenarios like round-robin scheduling where a circular list can manage the allocation and cycling through resources or tasks in a repeating pattern.

Disadvantages (Limitations) of Circular Linked Lists

Complexity in Implementation: The implementation of circular linked lists can be more complex than simple linked lists due to the need to handle circular connections properly, which increases the chance of errors such as infinite loops.

Overhead in Traversing: Care must be taken to stop traversing the list properly; otherwise, it could lead to infinite loop errors. Keeping track of the starting point is essential to avoid such issues.

Additional Logic for Insertion and Deletion: Inserting or deleting nodes requires more logic to correctly adjust the links, particularly in a doubly circular linked list where both next and previous links must be accurately maintained.

Memory Usage: Each node in a **doubly circular linked list** uses additional memory for the backward link, which can be a concern in memory-constrained environments.

UNIT -III

UNIT III

Trees: Terminology and Representations,

Types -Binary Trees,

Binary Search Tree,

Heap,

B-Trees,

N-ary Trees,

B+ Trees,

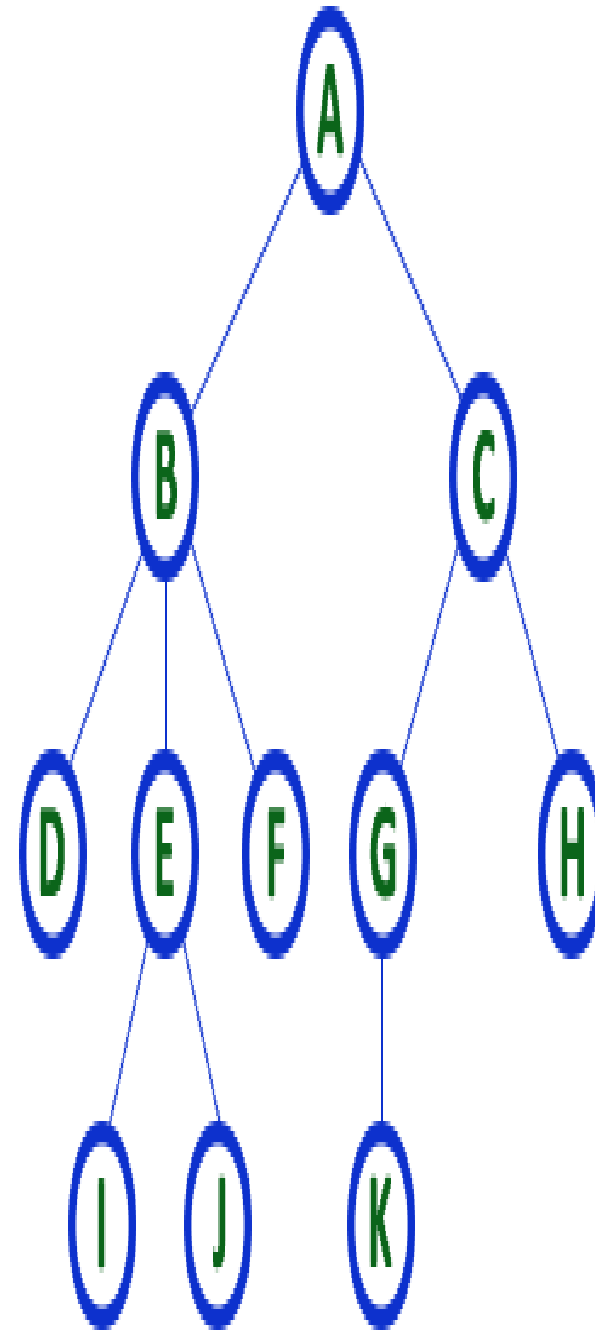
Tree Traversals – Preorder and in-order construction,

Applications of Trees, Hashing.

Trees: Terminology & Representations

Tree:-

- ❖ A tree is a very popular non-linear data structure used in a wide range of applications.
- ❖ **Tree is a non-linear data structure which organizes data in hierarchical structure**
- ❖ In tree data structure, every individual element is called as **Node**.
- ❖ Node in a tree data structure stores the actual data of that particular element and link to next element
- ❖ In a tree data structure, if we have **N** number of nodes then we can have a maximum of **N-1** number of links

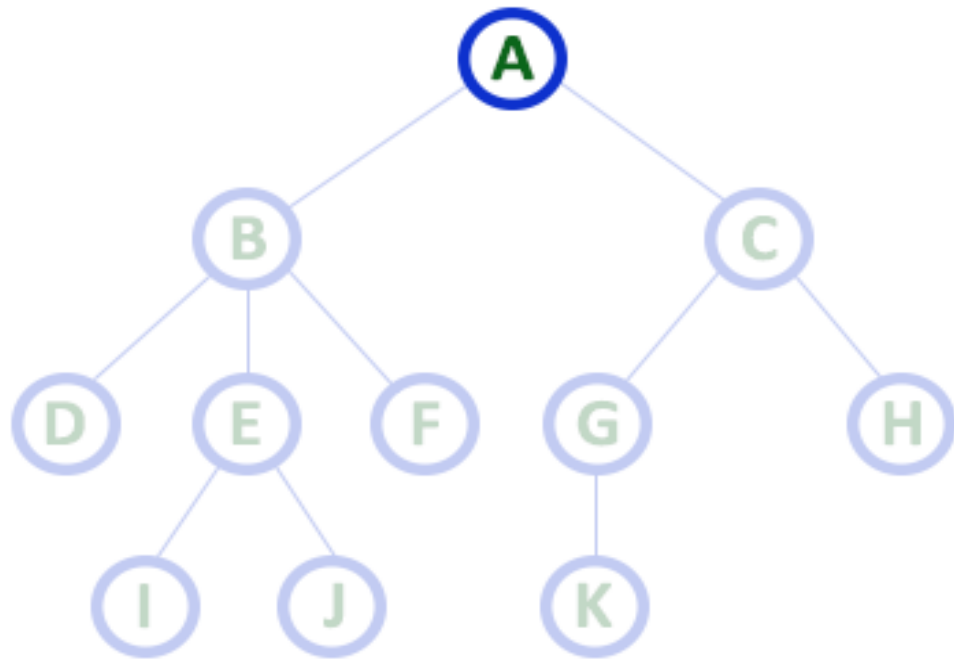


TREE with 11 nodes and 10 edges

- In any tree with '**N**' nodes there will be maximum of '**N-1**' edges
- In a tree every individual element is called as '**NODE**'

TREE Terminology

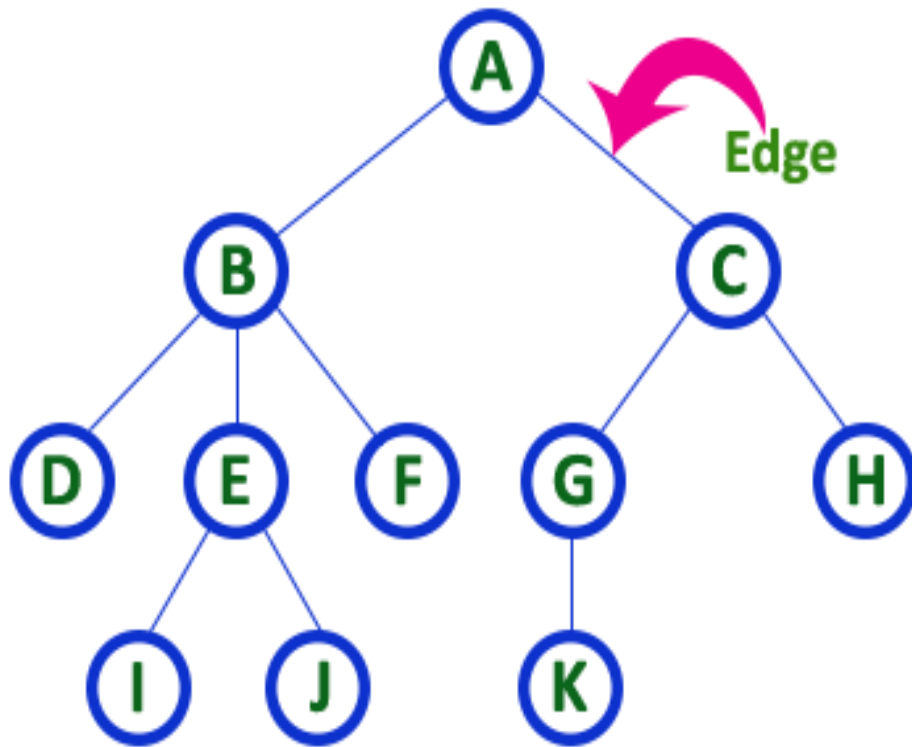
Root:- In a tree data structure, the first node is called as **Root Node**. Every tree must have a root node. We can say that the root node is the origin of the tree data structure. In any tree, there must be only one root node. We never have multiple root nodes in a tree.



Here 'A' is the 'root' node

- In any tree the first node is called as ROOT node

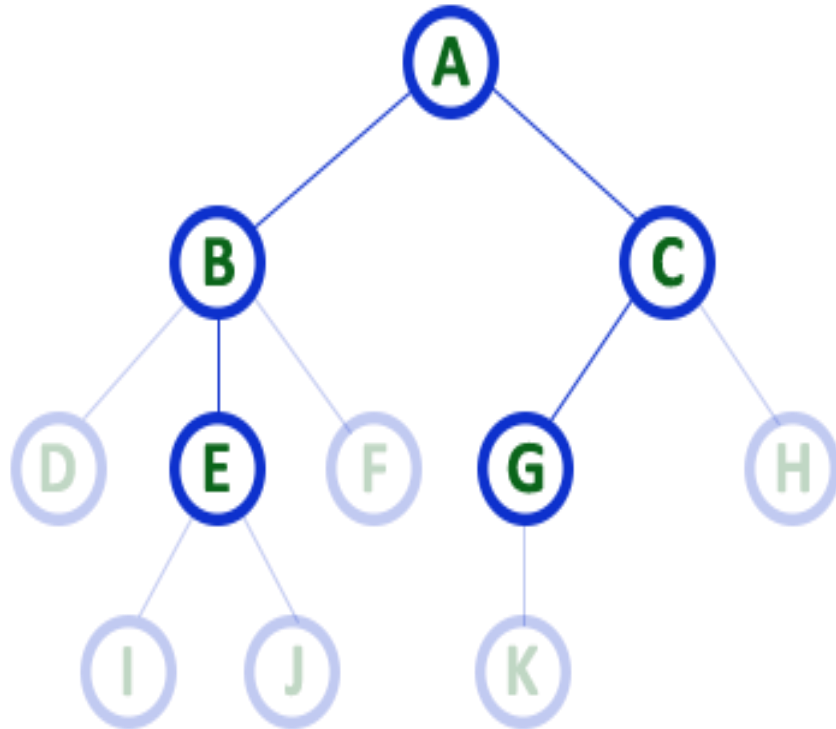
Edge:- In a tree data structure, the connecting link between any two nodes is called as **EDGE**. In a tree with '**N**' number of nodes there will be a maximum of '**N-1**' number of edges.



- In any tree, '**Edge**' is a connecting link between two nodes.

Parent

In a tree data structure, the node which is a predecessor of any node is called as **PARENT NODE**. In simple words, the node which has a branch from it to any other node is called a parent node. Parent node can also be defined as "**The node which has child / children**".

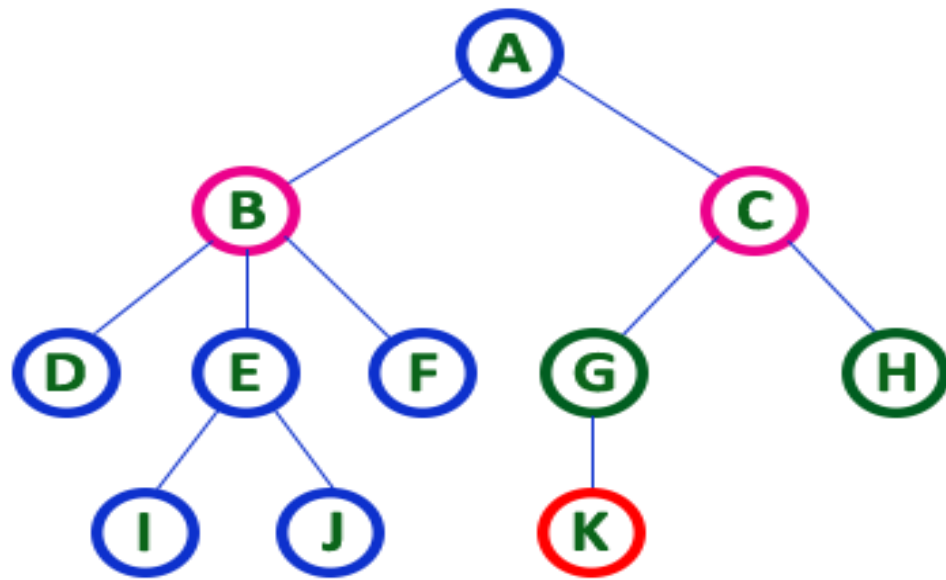


Here A, B, C, E & G are **Parent** nodes

- In any tree the node which has child / children is called '**Parent**'
- A node which is predecessor of any other node is called '**Parent**'

Child

In a tree data structure, the node which is descendant of any node is called as **CHILD Node**. In simple words, the node which has a link from its parent node is called as child node. In a tree, any parent node can have any number of child nodes. In a tree, all the nodes except root are child nodes.



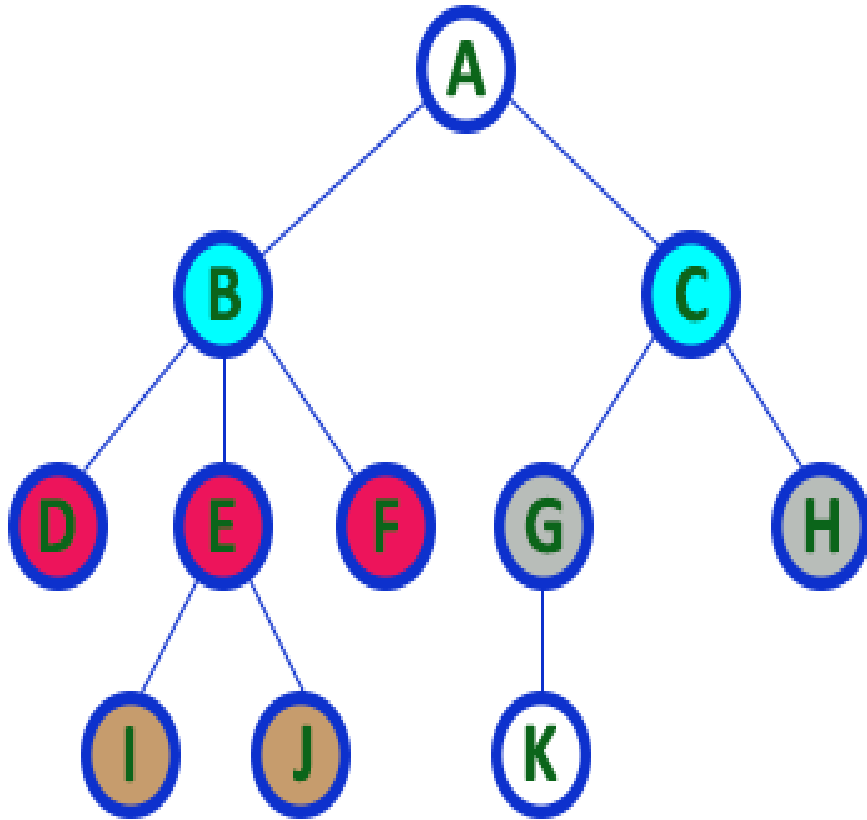
Here **B & C** are **Children** of **A**

Here **G & H** are **Children** of **C**

Here **K** is **Child** of **G**

- descendant of any node is called as **CHILD Node**

Siblings:-In a tree data structure, nodes which belong to same Parent are called as **SIBLINGS NODES**.



Here **B & C** are **Siblings**

Here **D E & F** are **Siblings**

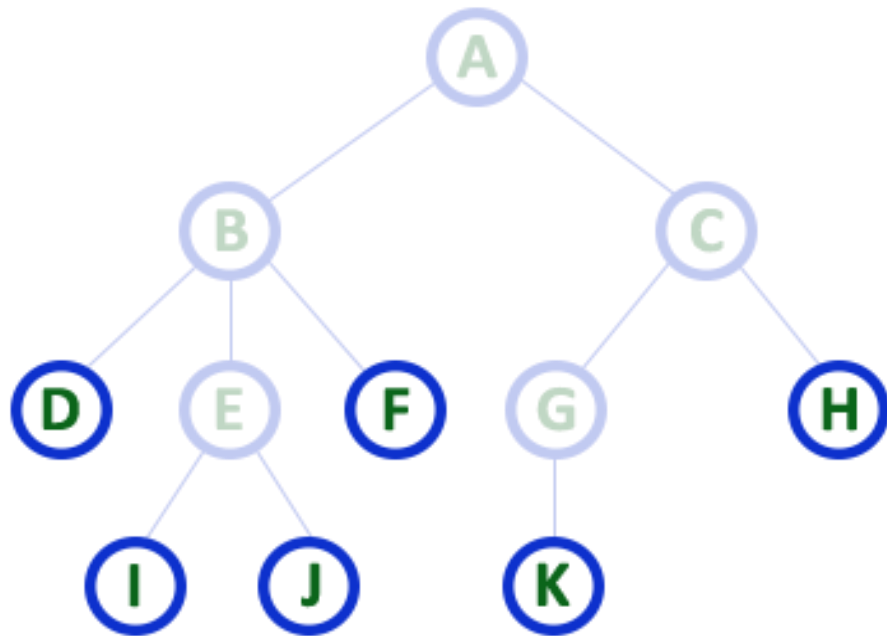
Here **G & H** are **Siblings**

Here **I & J** are **Siblings**

- In any tree the nodes which has same Parent are called '**Siblings**'

- The children of a Parent are called '**Siblings**'

Leaf:- In a tree data structure, the node which does not have a child is called as **LEAF Node**. In simple words, a leaf is a node with no child. In a tree data structure, the leaf nodes are also called as **External Nodes**. External node is also a node with no child. In a tree, leaf node is also called as 'Terminal' node.

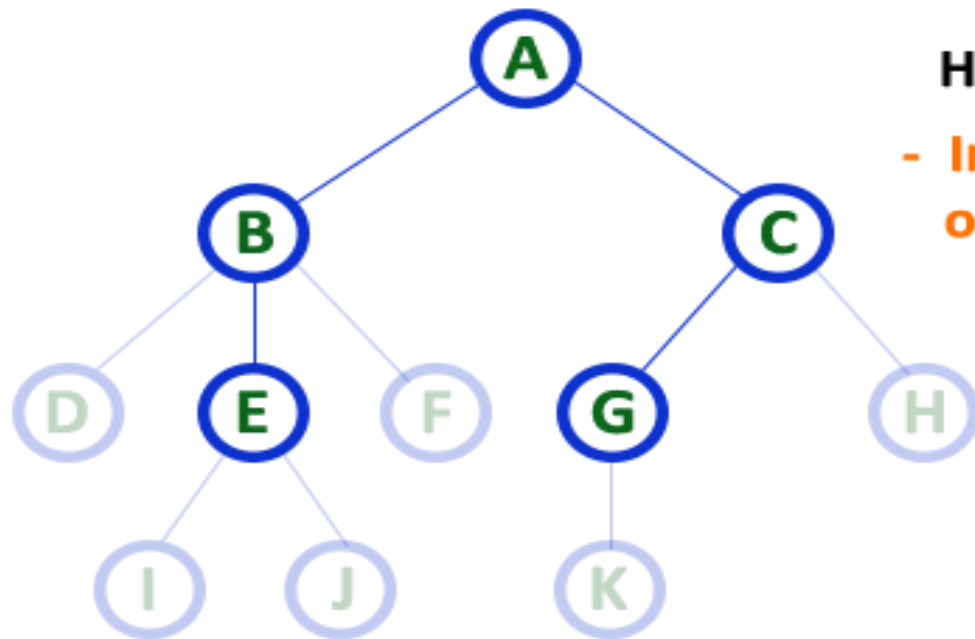


Here D, I, J, F, K & H are **Leaf** nodes

- In any tree the node which does not have children is called '**Leaf**'
- A node without successors is called a '**leaf**' node

Internal Nodes:- In a tree data structure, the node which has atleast one child is called as **INTERNAL Node**. In simple words, an internal node is a node with atleast one child.

In a tree data structure, nodes other than leaf nodes are called as **Internal Nodes**. The root node is also said to be Internal Node if the tree has more than one node. Internal nodes are also called as 'Non-Terminal' nodes.

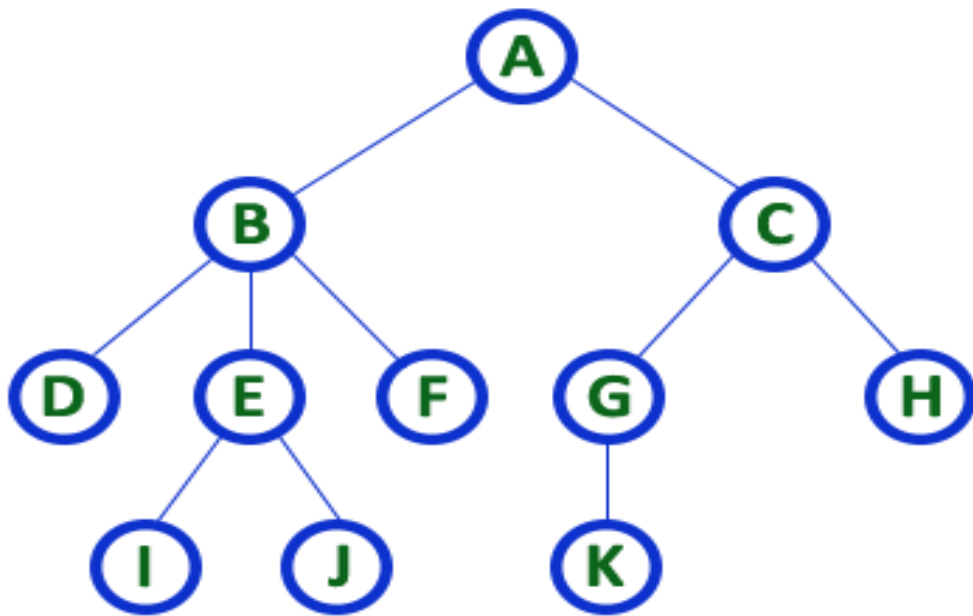


Here A, B, C, E & G are **Internal** nodes

- In any tree the node which has atleast one child is called '**Internal**' node

- Every non-leaf node is called as '**Internal**' node

Degree:- In a tree data structure, the total number of children of a node is called as **DEGREE** of that Node. In simple words, the Degree of a node is total number of children it has. The highest degree of a node among all the nodes in a tree is called as '**Degree of Tree**'



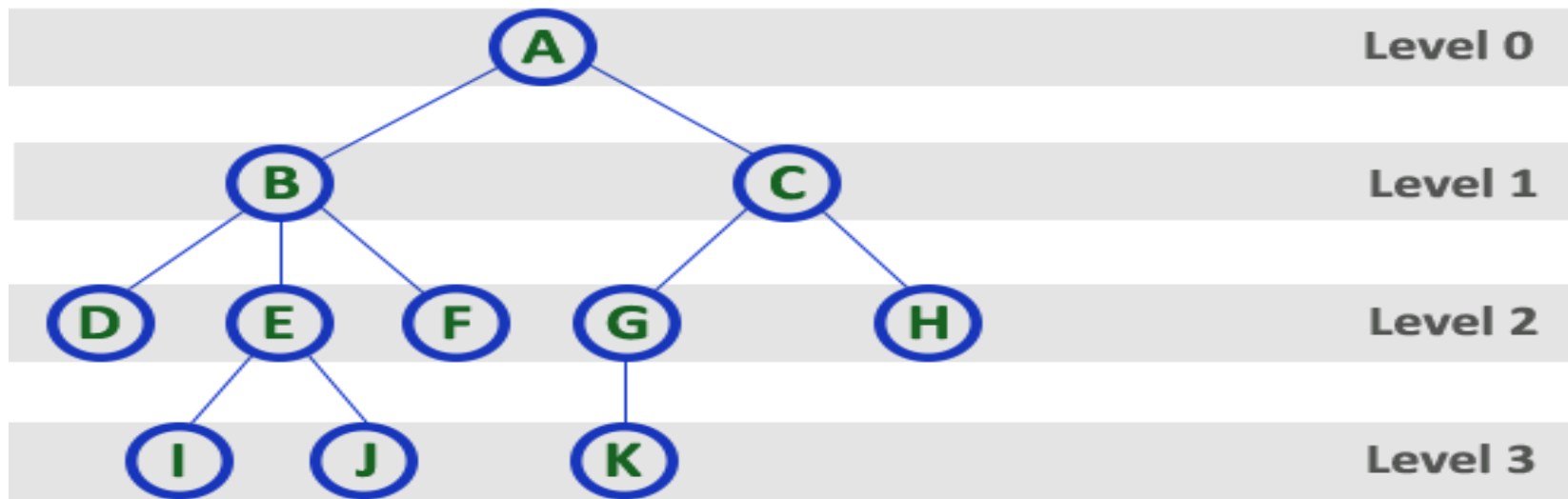
Here **Degree** of B is 3

Here **Degree** of A is 2

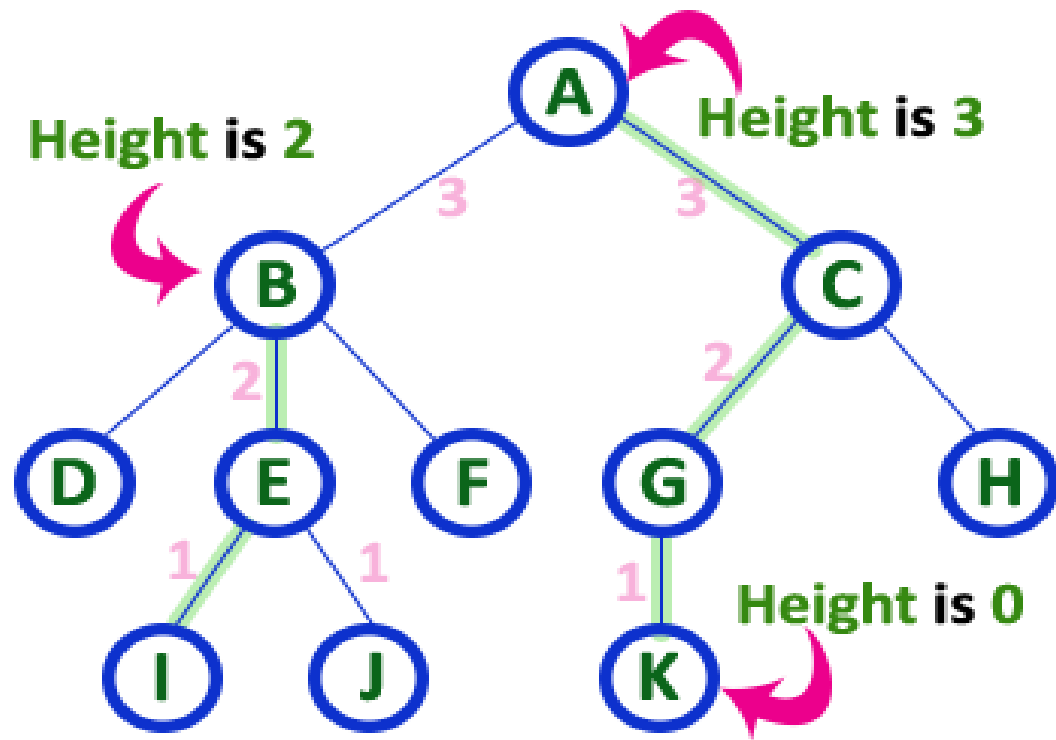
Here **Degree** of F is 0

- In any tree, '**Degree**' of a node is total number of children it has.

Level:- In a tree data structure, the root node is said to be at Level 0 and the children of root node are at Level 1 and the children of the nodes which are at Level 1 will be at Level 2 and so on... In simple words, in a tree each step from top to bottom is called as a Level and the Level count starts with '0' and incremented by one at each level (Step).



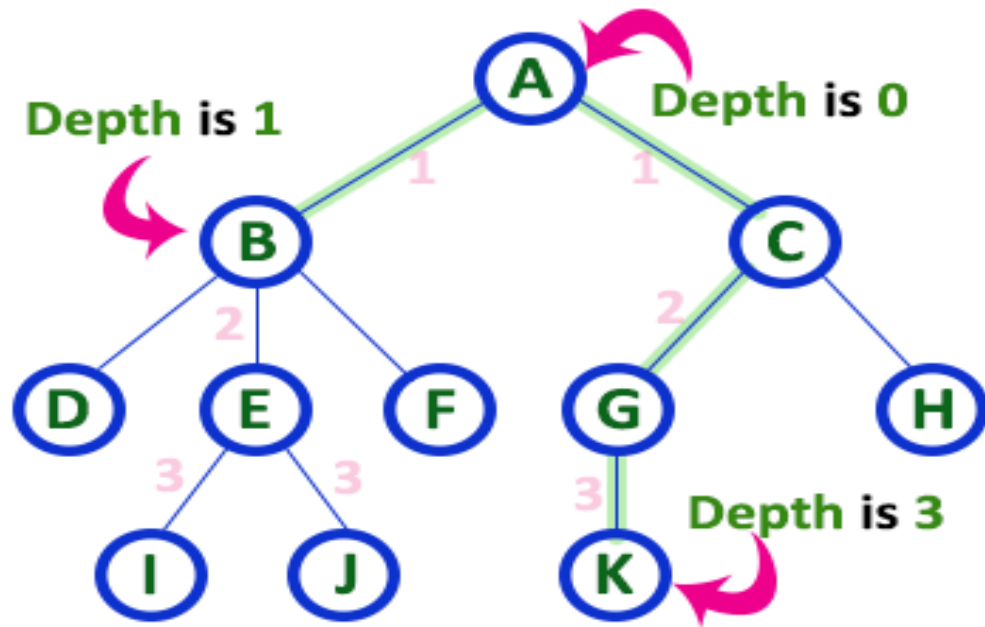
Height:- In a tree data structure, the total number of edges from leaf node to a particular node in the longest path is called as **HEIGHT** of that Node. In a tree, height of the root node is said to be **height of the tree**. In a tree, **height of all leaf nodes is '0'**.



Here Height of tree is 3

- In any tree, 'Height of Node' is total number of Edges from leaf to that node in longest path.
- In any tree, 'Height of Tree' is the height of the root node.

Depth:- In a tree data structure, the total number of edges from root node to a particular node is called as **DEPTH** of that Node. In a tree, the total number of edges from root node to a leaf node in the longest path is said to be **Depth of the tree**. In simple words, the highest depth of any leaf node in a tree is said to be depth of that tree. In a tree, **depth of the root node is '0'.**

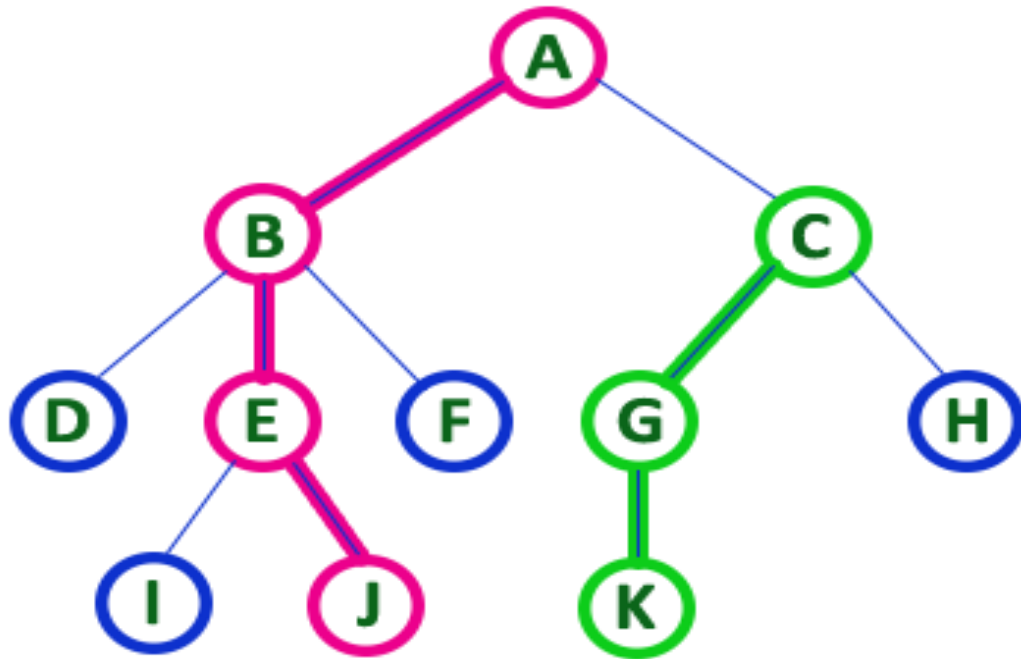


Here Depth of tree is 3

- In any tree, 'Depth of Node' is total number of Edges from root to that node.
- In any tree, 'Depth of Tree' is total number of edges from root to leaf in the longest path.

Path:- In a tree data structure, the sequence of Nodes and Edges from one node to another node is called as **PATH** between that two Nodes. **Length of a Path** is total number of nodes in that path.

In below example **the path A - B - E - J has length 4.**



- In any tree, 'Path' is a sequence of nodes and edges between two nodes.

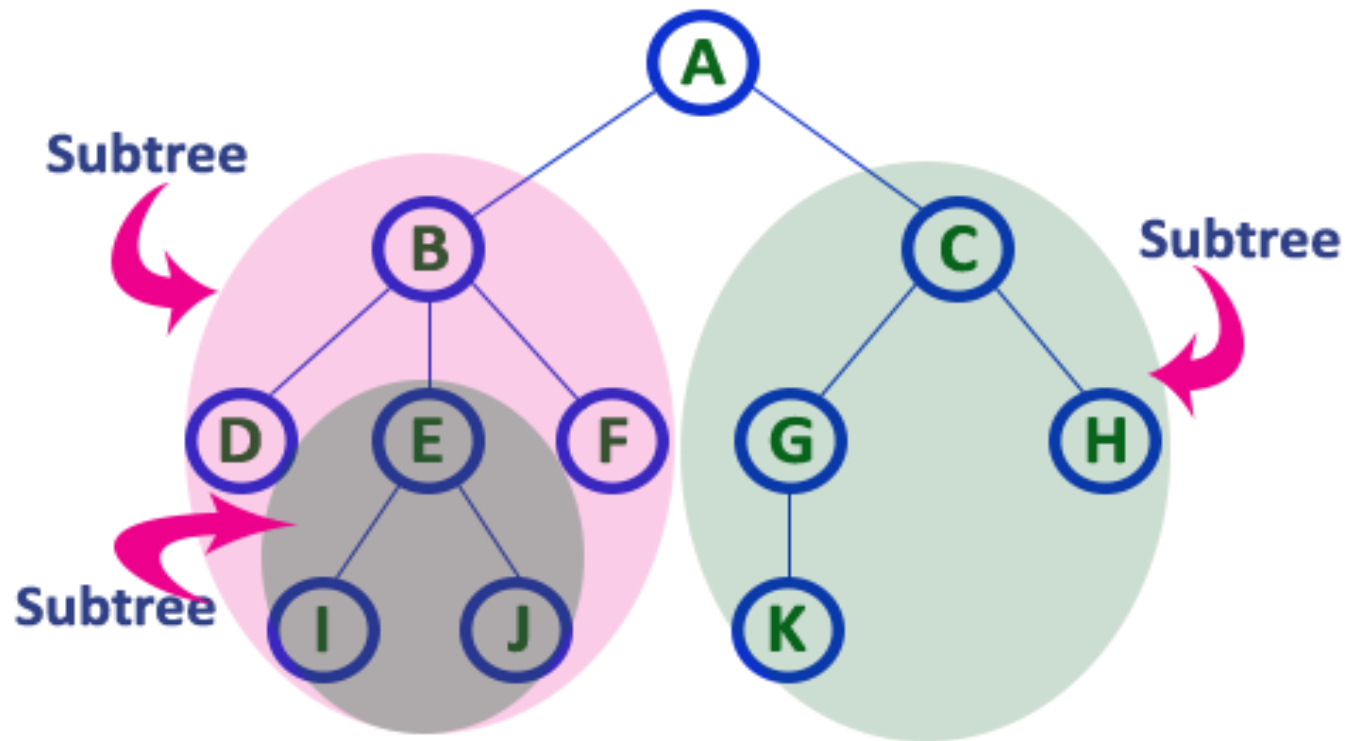
Here, 'Path' between A & J is

A - B - E - J

Here, 'Path' between C & K is

C - G - K

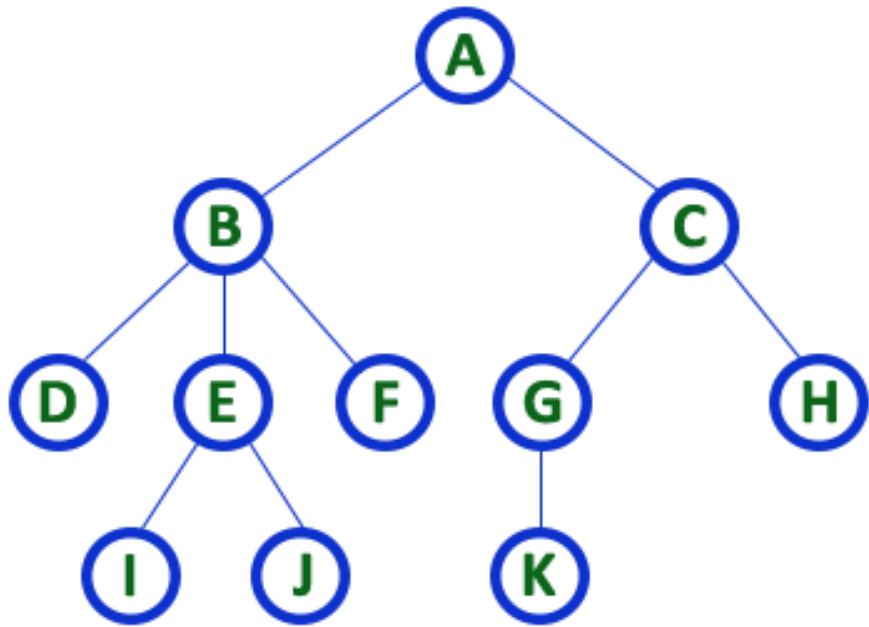
Sub Tree:- In a tree data structure, each child from a node forms a subtree recursively. Every child node will form a subtree on its parent node.



Tree Representation

A tree data structure can be represented in two methods.

- ❖ List Representation
- ❖ Left Child - Right Sibling Representation

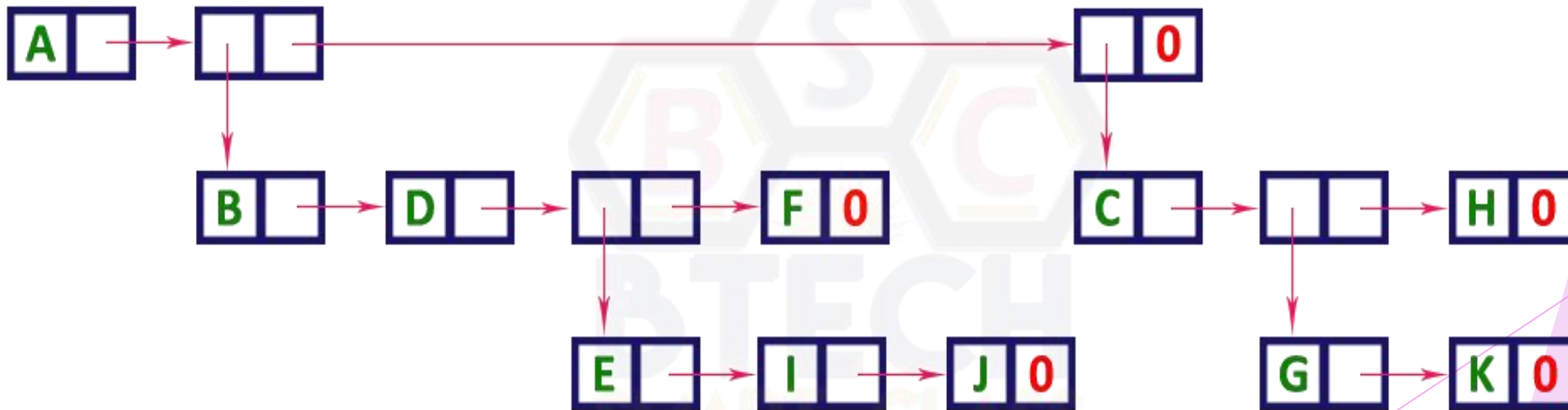


TREE with 11 nodes and 10 edges

- In any tree with '**N**' nodes there will be maximum of '**N-1**' edges
- In a tree every individual element is called as '**NODE**'

List Representation

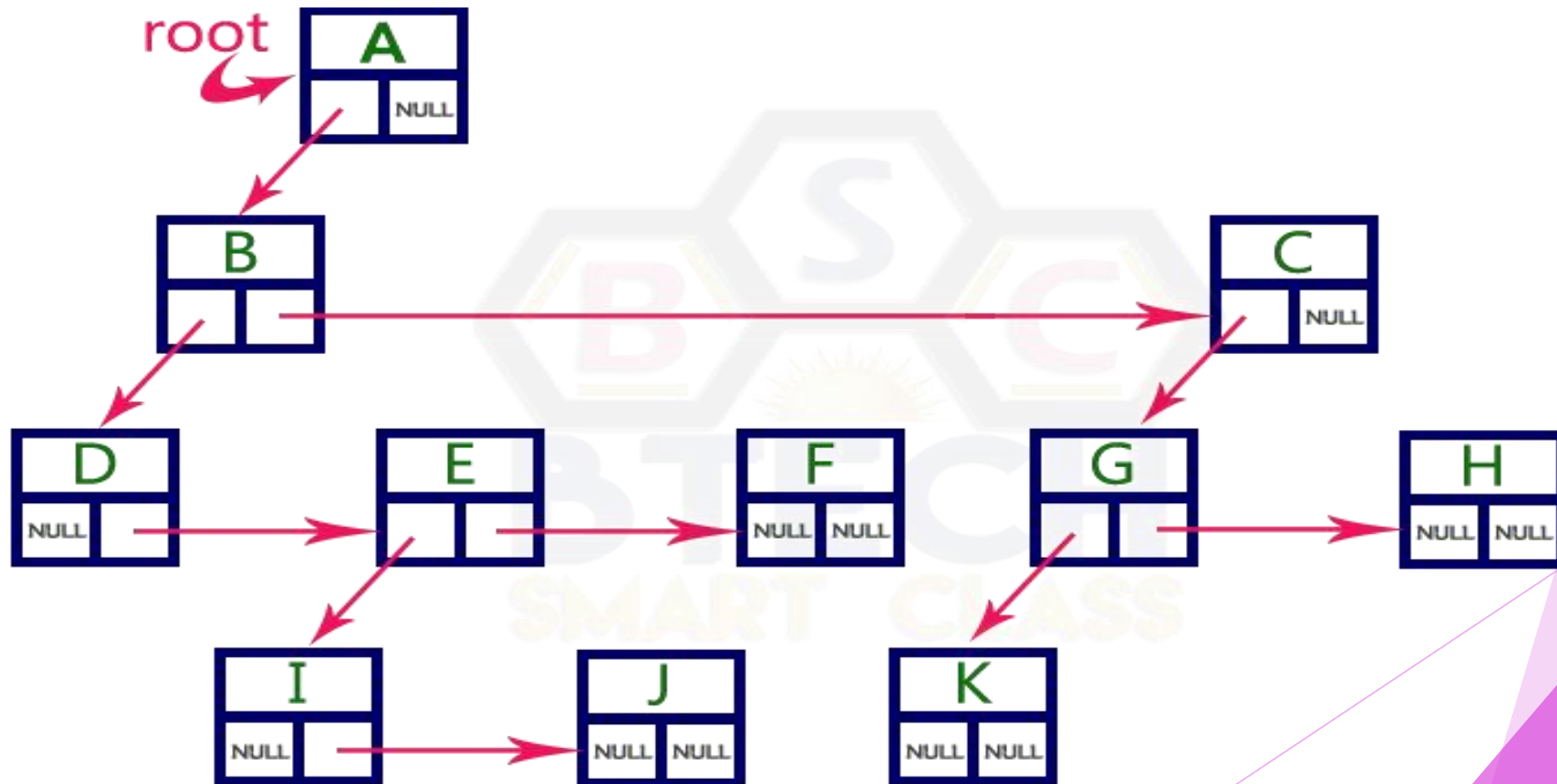
- ❖ In this representation, we use two types of nodes one for representing the node with data called 'data node' and another for representing only references called 'reference node'.
- ❖ We start with a 'data node' from the root node in the tree. Then it is linked to an internal node through a 'reference node' which is further linked to any other node directly. This process repeats for all the nodes in the tree.



Left Child - Right Sibling Representation

- ❖ In this representation, we use a list with one type of node which consists of three fields namely Data field, Left child reference field and Right sibling reference field.
- ❖ Data field stores the actual value of a node, left reference field stores the address of the left child and right reference field stores the address of the right sibling node.
- ❖ In this representation, every node's data field stores the actual value of that node.
- ❖ If that node has left a child, then left reference field stores the address of that left child node otherwise stores NULL.
- ❖ If that node has the right sibling, then right reference field stores the address of right sibling node otherwise stores NULL.

Data	
Left Child	Right Sibling



Applications of trees:-

- Storing naturally hierarchical data:** Trees are used to store the data in the hierarchical structure.
- Organize data:** It is used to organize data for efficient insertion, deletion and searching.
- Trie:** It is a special kind of tree that is used to store the dictionary. It is a fast and efficient way for dynamic spell checking.
- Heap:** It is also a tree data structure implemented using arrays. It is used to implement priority queues.
- B-Tree and B+Tree:** B-Tree and B+Tree are the tree data structures used to implement indexing in databases.
- Routing table:** The tree data structure is also used to store the data in routing tables in the routers.
- Parse Trees:** The tree data structure useful in creating parse tree for evaluating arithmetic expressions.

TREE TRAVERSALS

Tree traversals:-

The term 'tree traversal' means traversing or visiting each node of a tree. There is a single way to traverse the linear data structure such as linked list, queue, and stack. Whereas, there are multiple ways to traverse a tree that are listed as follows

-

- Preorder traversal
- Inorder traversal
- Postorder traversal

Preorder traversal

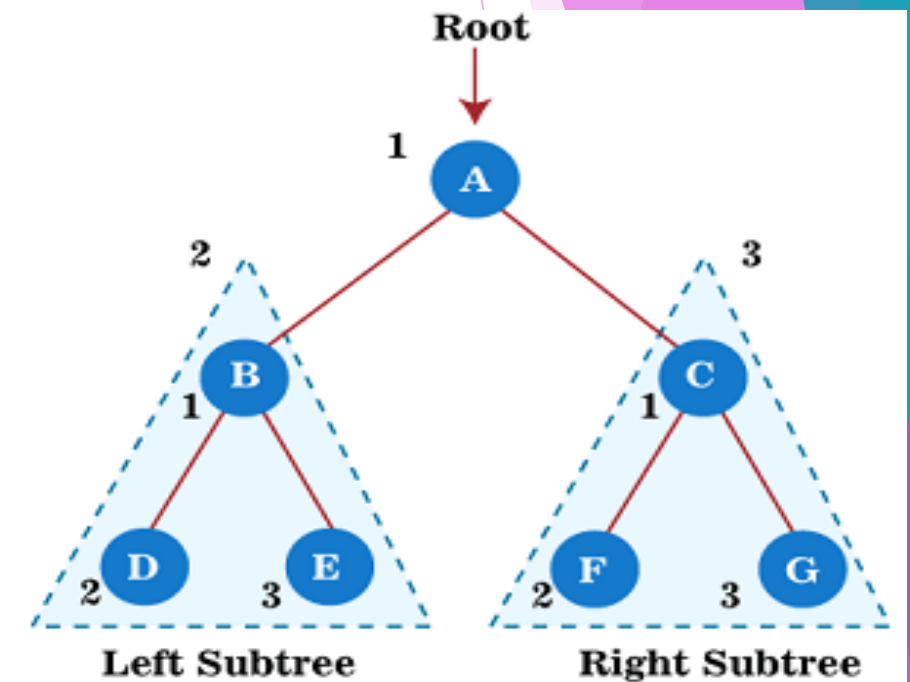
- ❑ This technique follows the 'root left right' policy.
- ❑ It means that, first root node is visited after that the left subtree is traversed recursively, and finally, right subtree is recursively traversed.
- ❑ As the root node is traversed before (or pre) the left and right subtree, it is called preorder traversal.
- ❑ So, in a preorder traversal, each node is visited before both of its subtrees.

The **applications of preorder traversal** include -

- It is used to create a copy of the tree.
- It can also be used to get the prefix expression of an expression tree.

Algorithm for preorder traversal

1. Until all nodes of the tree are not visited
2. Step 1 - Visit the root node
3. Step 2 - Traverse the left subtree recursively.
4. Step 3 - Traverse the right subtree recursively



- ❖ First, we traverse the root node **A**;
- ❖ after that, move to its left subtree **B**,
- ❖ So, for left subtree B, first, the root node **B** is traversed itself;
- ❖ after that, its left subtree **D** is traversed.
- ❖ Since node **D** does not have any children, move to right subtree **E**.
- ❖ As node E also does not have any children, the traversal of the left subtree of root node A is completed.
- ❖ Now, move towards the right subtree of root node A that is C.
- ❖ So, for right subtree C, first the root node **C** has traversed itself; after that, its left subtree **F** is traversed.
- ❖ Since node F does not have any children, move to the right subtree **G**.
- ❖ As node G also does not have any children, traversal of the right subtree of root node A is completed.
- ❖ Therefore, all the nodes of the tree are traversed.
- ❖ So, the output of the preorder traversal of the above tree is -
A → B → D → E → C → F → G

Postorder traversal

This technique follows the 'left-right root' policy.

It means that the first left subtree of the root node is traversed, after that recursively traverses the right subtree, and finally, the root node is traversed.

As the root node is traversed after the left and right subtree, it is called postorder traversal.

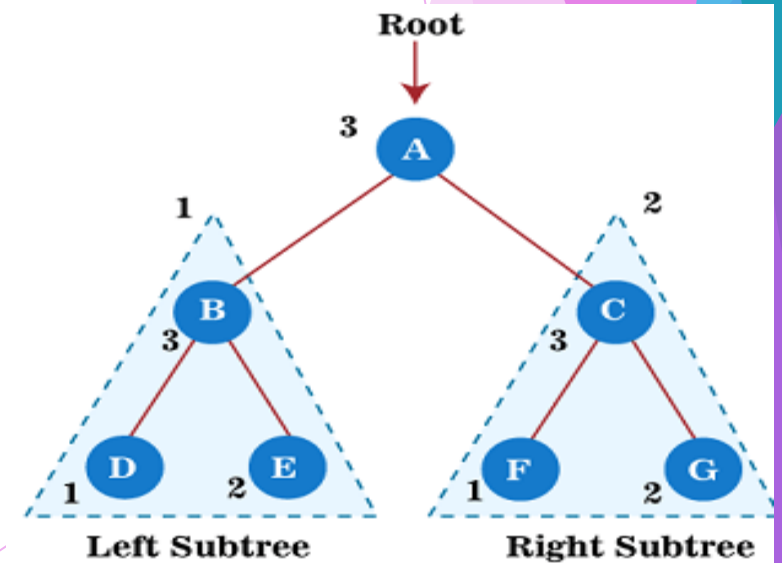
So, in a postorder traversal, each node is visited after both of its subtrees.

Applications of postorder traversal :-

- It is used to delete the tree.
- It can also be used to get the postfix expression of an expression tree.

Algorithm:- Until all nodes of the tree are not visited

- 1.Step 1 - Traverse the left subtree recursively.
- 2.Step 2 - Traverse the right subtree recursively.
- 3.Step 3 - Visit the root node.



- ❖ First, we traverse the left subtree **B** that will be traversed in postorder.
- ❖ After that, we will traverse the right subtree **C** in postorder. And finally, the root node of the above tree, i.e., **A**, is traversed.
- ❖ So, for left subtree **B**, first, its left subtree **D** is traversed.
- ❖ Since node **D** does not have any children, traverse the right subtree **E**.
- ❖ As node **E** also does not have any children, move to the root node **B**.
- ❖ After traversing node **B**, the traversal of the left subtree of root node **A** is completed.
- ❖ Now, move towards the right subtree of root node **A** that is **C**.
- ❖ So, for right subtree **C**, first its left subtree **F** is traversed.
- ❖ Since node **F** does not have any children, traverse the right subtree **G**.
- ❖ As node **G** also does not have any children, therefore, finally, the root node of the right subtree, i.e., **C**, is traversed.
- ❖ The traversal of the right subtree of root node **A** is completed.
- ❖ At last, traverse the root node of a given tree, i.e., **A**.
- ❖ After traversing the root node, the postorder traversal of the given tree is completed.
- ❖ Therefore, all the nodes of the tree are traversed.
- ❖ So, the output of the postorder traversal of the above tree is -
- ❖ **D → E → B → F → G → C → A**

Inorder traversal:- This technique follows the 'left root right' policy. It means that first left subtree is visited after that root node is traversed, and finally, the right subtree is traversed.

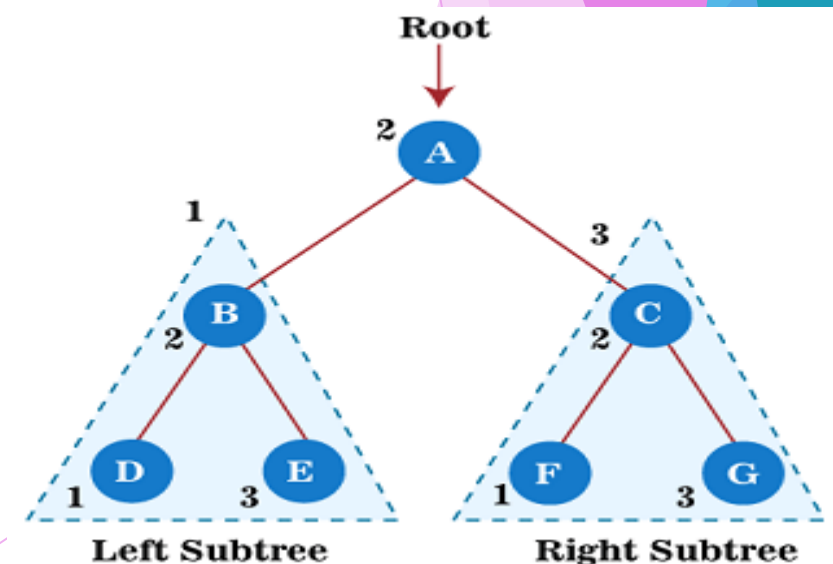
As the root node is traversed between the left and right subtree, it is named inorder traversal. So, in the inorder traversal, each node is visited in between of its subtrees.

Applications of Inorder traversal -

- It is used to get the BST nodes in increasing order.
- It can also be used to get the prefix expression of an expression tree.

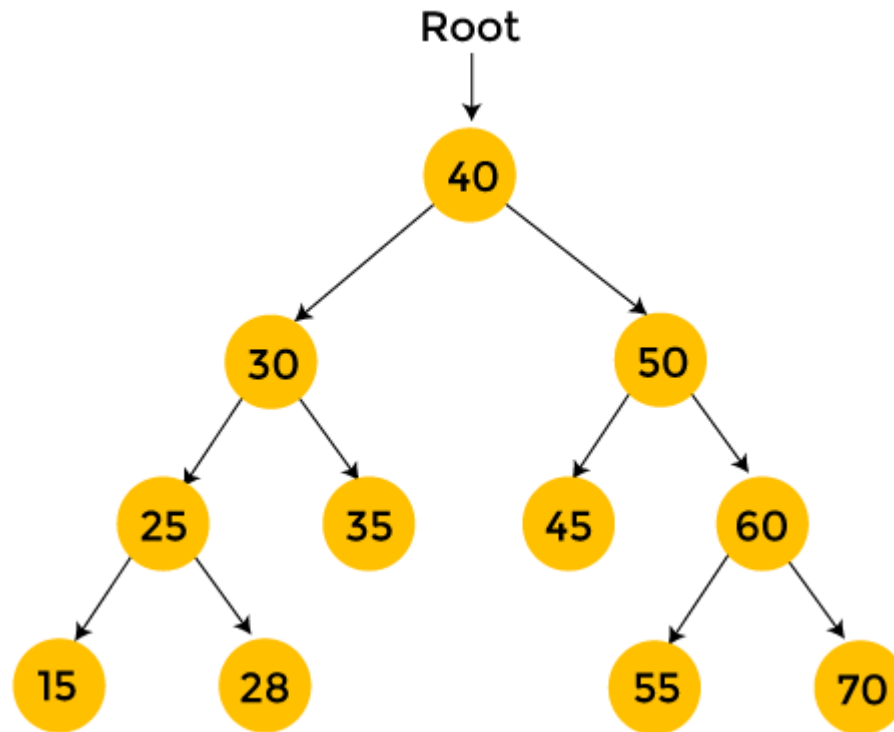
Algorithm:- Until all nodes of the tree are not visited

- 1.Step **1** - Traverse the left subtree recursively.
- 2.Step **2** - Visit the root node.
- 3.Step **3** - Traverse the right subtree recursively.



First, we traverse the left subtree **B** that will be traversed in inorder. After that, we will traverse the root node **A**. And finally, the right subtree **C** is traversed in inorder. So, for left subtree **B**, first, its left subtree **D** is traversed. Since node **D** does not have any children, so after traversing it, node **B** will be traversed, and at last, right subtree of node **B**, that is **E**, is traversed. Node **E** also does not have any children; therefore, the traversal of the left subtree of root node **A** is completed. After that, traverse the root node of a given tree, i.e., **A**. At last, move towards the right subtree of root node **A** that is **C**. So, for right subtree **C**; first, its left subtree **F** is traversed. Since node **F** does not have any children, node **C** will be traversed, and at last, a right subtree of node **C**, that is, **G**, is traversed. Node **G** also does not have any children; therefore, the traversal of the right subtree of root node **A** is completed. As all the nodes of the tree are traversed, the inorder traversal of the given tree is completed. The output of the inorder traversal of the above tree is -

PREORDER TRAVERSAL, INORDER TRAVERSAL, POST ORDER TRAVERSAL

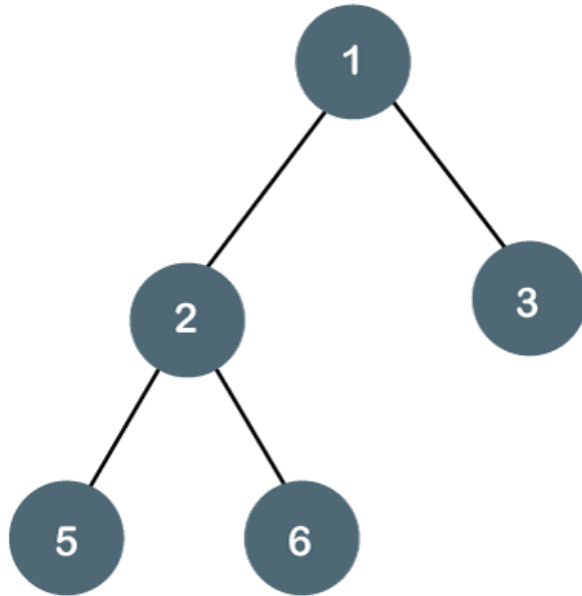


TYPES OF TREES

Binary Tree

The Binary tree means that the node can have maximum two children.
These two children are usually called:

- **Left child**
- **Right child**



Properties of Binary Tree

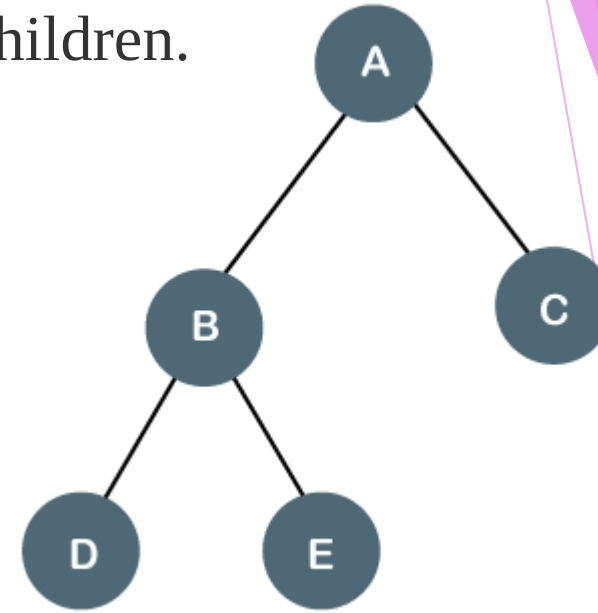
- ✓ At each level of i , the maximum number of nodes is 2^i .
- ✓ The height of the tree is defined as the longest path from the root node to the leaf node.
In general, the maximum number of nodes possible at height h is $(2^0 + 2^1 + 2^2 + \dots + 2^h) = 2^{h+1} - 1$.
- ✓ The minimum number of nodes possible at height h is equal to $h+1$.
- ✓ The minimum height can be computed as: $h = \log_2(n+1) - 1$
- ✓ The maximum height can be computed as: $h = n-1$

Types of Binary Tree

- Full/ proper/ strict Binary tree
- Complete Binary tree
- Perfect Binary tree
- Degenerate Binary tree
- Balanced Binary tree

Full/ proper/ strict Binary tree

The full binary tree is also known as a strict binary tree. The tree can only be considered as the full binary tree if each node must contain either 0 or 2 children.

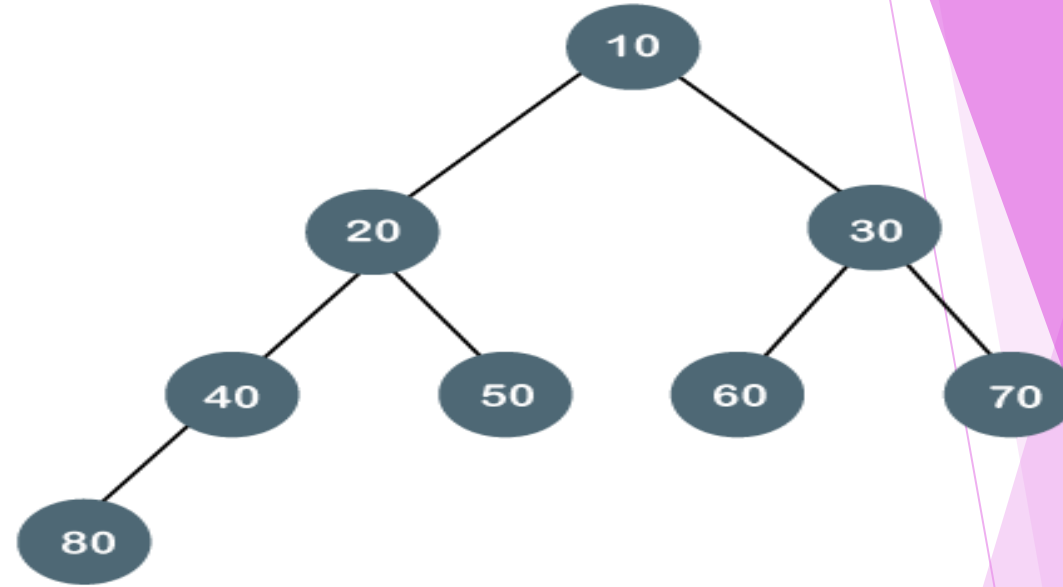


Properties of Full Binary Tree

- The number of leaf nodes is equal to the number of internal nodes plus 1.
- The maximum number of nodes is the same as the number of nodes in the binary tree, i.e., $2^{h+1} - 1$.
- The minimum number of nodes in the full binary tree is $2 * h - 1$.
- The minimum height of the full binary tree is $\log_2(n+1) - 1$.
- The maximum height of the full binary tree can be computed as: $h = \frac{n+1}{2}$

Complete Binary Tree

The complete binary tree is a tree in which all the nodes are completely filled except the last level. In the last level, all the nodes must be as left as possible. In a complete binary tree, the nodes should be added from the left.

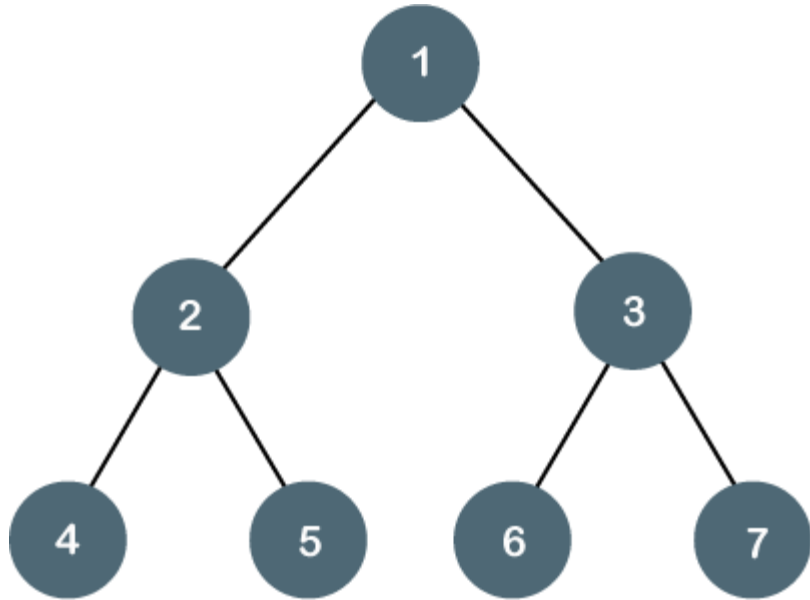


Properties of Complete Binary Tree

- The maximum number of nodes in complete binary tree is $2^{h+1} - 1$.
- The minimum number of nodes in complete binary tree is 2^h .
- The minimum height of a complete binary tree is $\log_2(n+1) - 1$.
- The maximum height of a complete binary tree is

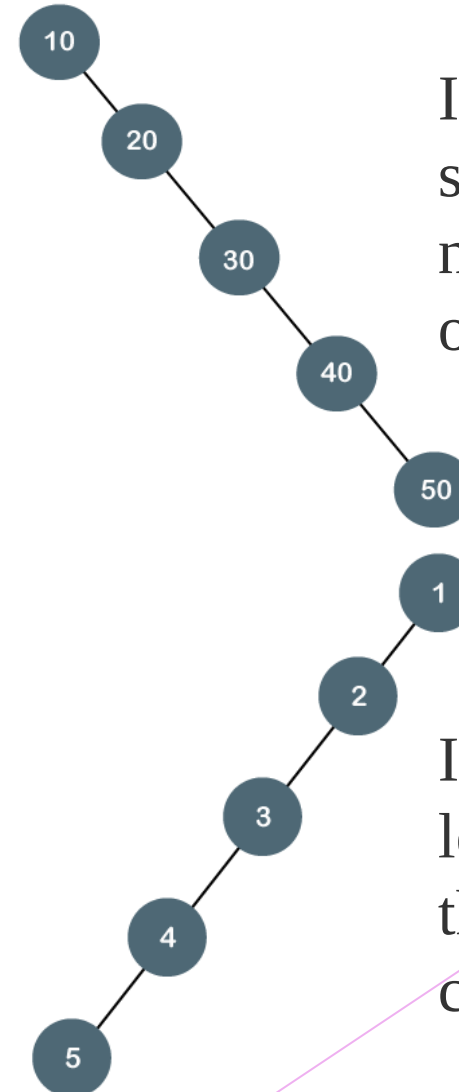
Perfect Binary Tree

A tree is a perfect binary tree if all the internal nodes have 2 children, and all the leaf nodes are at the same level.



Degenerate Binary Tree

The degenerate binary tree is a tree in which all the internal nodes have only one children.

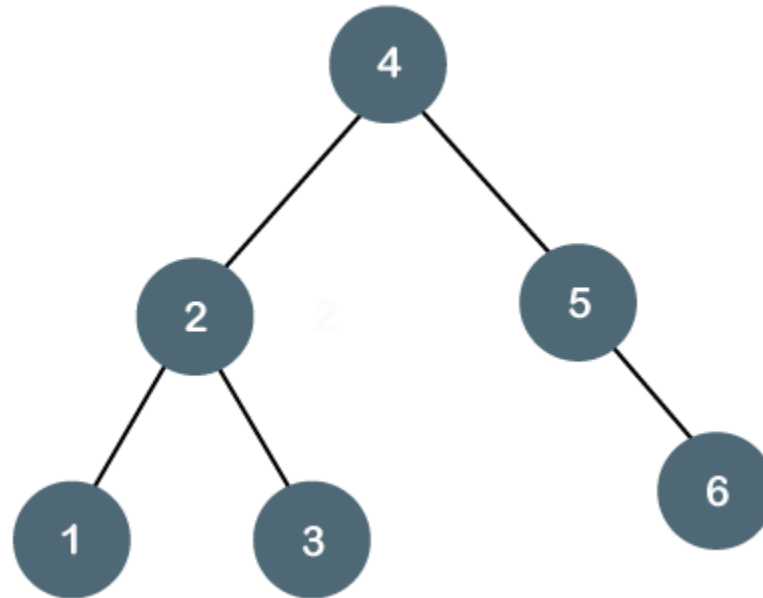


It is also known as a right-skewed tree as all the nodes have a right child only.

It is also known as a left-skewed tree as all the nodes have a left child only.

Balanced Binary Tree

The balanced binary tree is a tree in which both the left and right trees differ by at most 1.



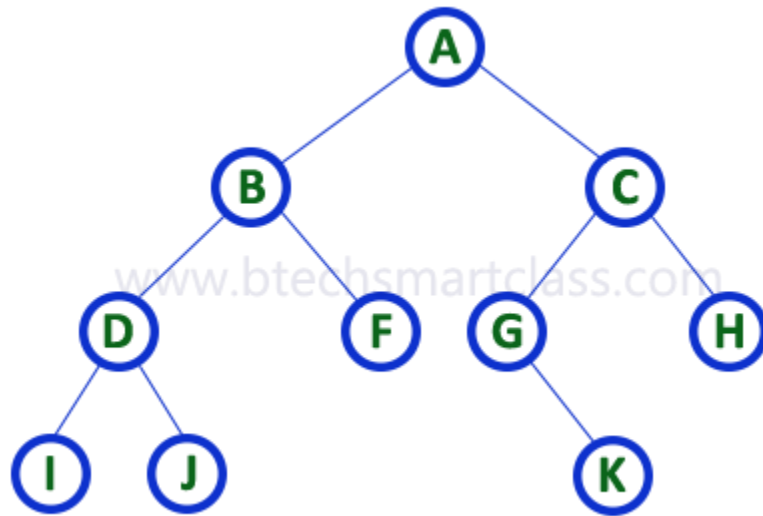
Time and Space Complexity of Binary Tree

Operation	Worst Case <u>Time Complexity</u>	<u>Space Complexity</u>
Insertion	$O(n)$	$O(n)$
Deletion	$O(n)$	$O(n)$
Search	$O(n)$	$O(n)$
Traversal (In-order, Pre-order, Post-order)	$O(n)$	$O(h)$ (where h is the height of the tree)

Binary Tree Representations

A binary tree data structure is represented using two methods.

1. Array Representation
2. Linked List Representation



Array Representation of Binary Tree

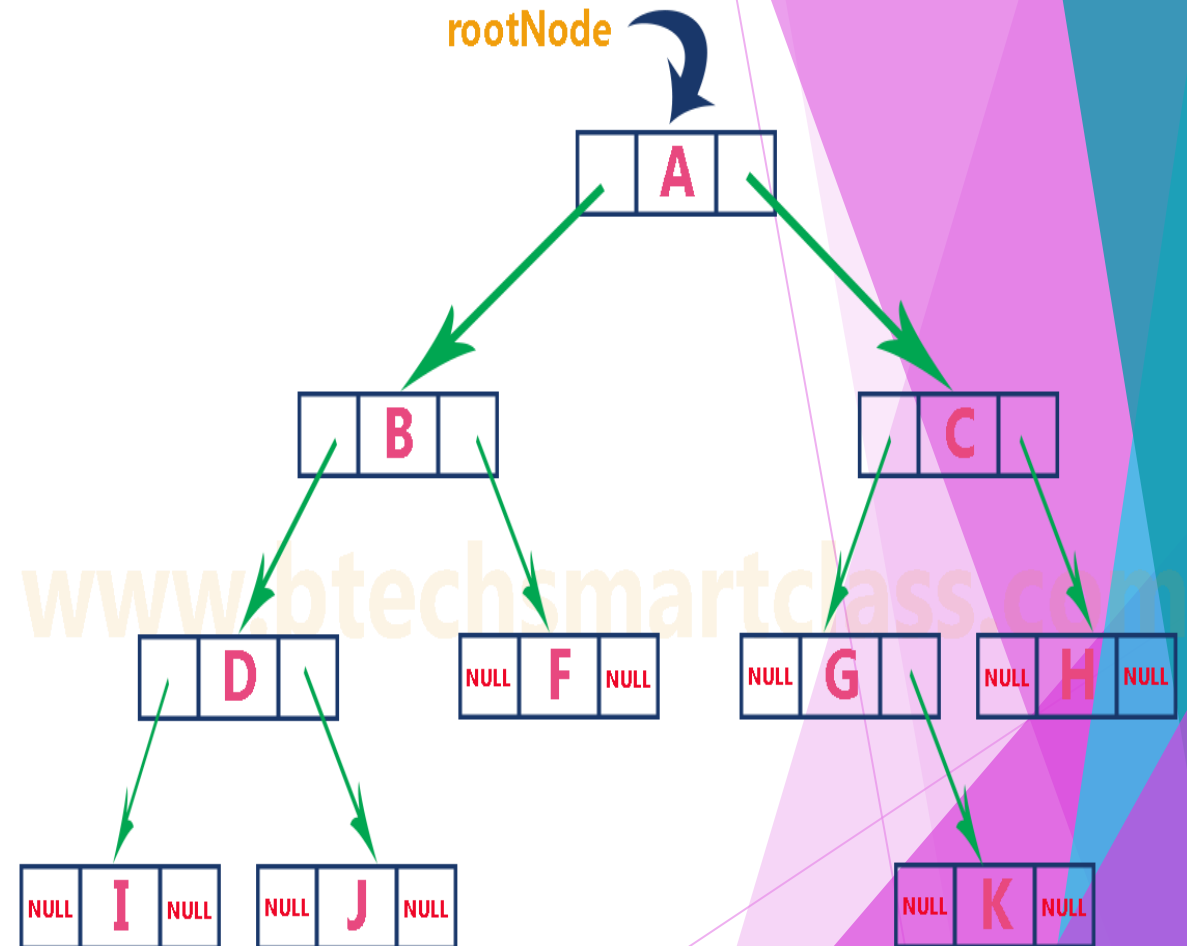
In array representation of a binary tree, we use one-dimensional array (1-D Array) to represent a binary tree.

To represent a binary tree of depth '**n**' using array representation, we need one dimensional array with a maximum size of **$2n + 1$** .



Linked List Representation of Binary Tree

We use a double linked list to represent a binary tree. In a double linked list, every node consists of three fields. First field for storing left child address, second for storing actual data and third for storing right child address.

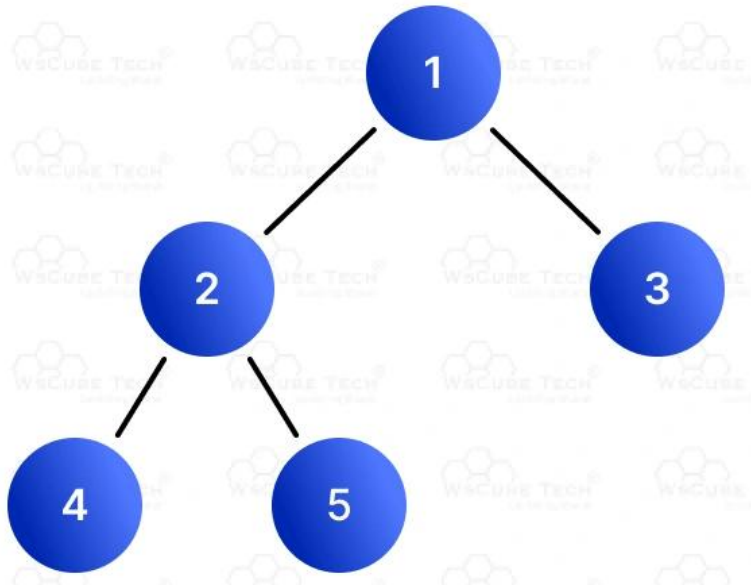


Binary Tree Operations

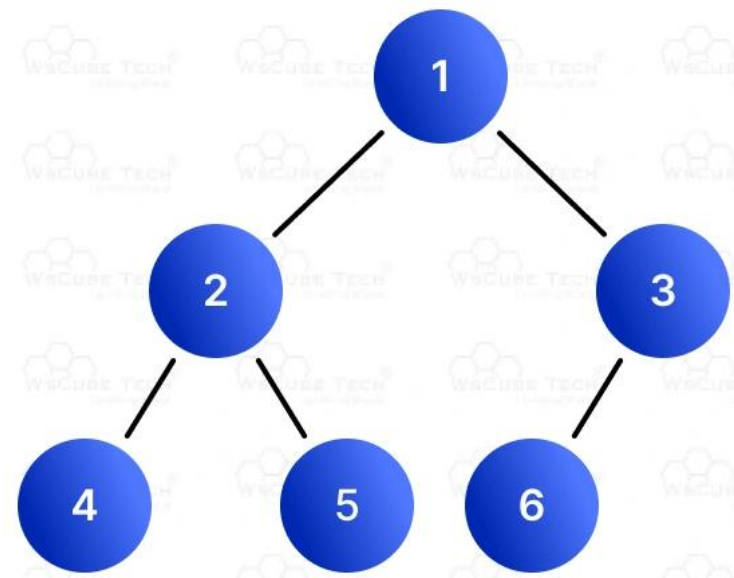
Binary trees support several fundamental operations, including insertion, deletion, searching, and traversals

1. Insertion

Insertion involves adding a new node to the binary tree. In a binary tree, a new node is usually inserted at the first available position in level order to maintain the completeness of the tree.

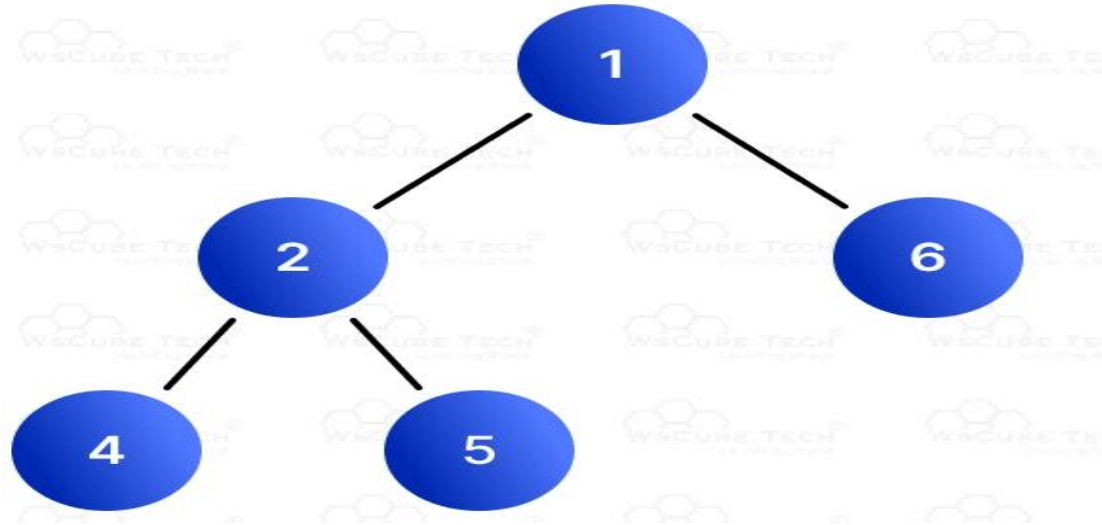


After inserting a node 6,



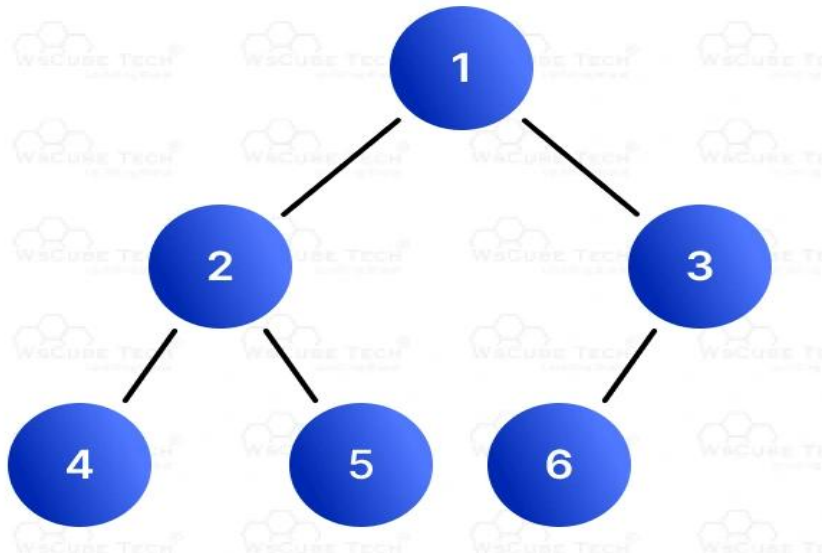
Deletion

Deletion involves removing a node from the binary tree. In a binary tree, the node to be deleted is replaced by the deepest and rightmost node to maintain the tree's structure.



Search

Searching involves finding a node with a given value in the binary tree. The search operation can be implemented using any traversal method (in-order, pre-order, post-order, or level-order).



search for the value 5 in the following binary tree:

Using level-order traversal:

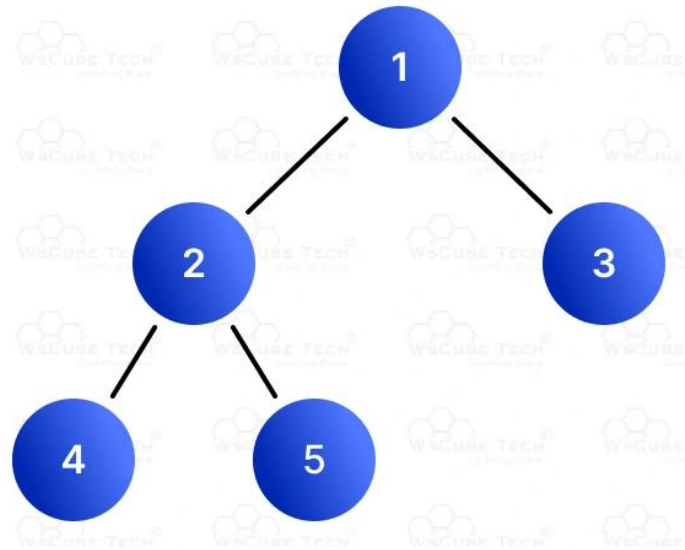
- Visit node 1
- Visit node 2
- Visit node 3
- Visit node 4
- Visit node 5 (found)

Traversal

Traversal involves visiting all the nodes in the binary tree in a specific order.

The main traversal methods are in-order, pre-order, post-order, and level-order.

- In-order Traversal (Left, Root, Right): 4, 2, 5, 1, 3
- Pre-order Traversal (Root, Left, Right): 1, 2, 4, 5, 3
- Post-order Traversal (Left, Right, Root): 4, 5, 2, 3, 1
- Level-order Traversal (Breadth-First): 1, 2, 3, 4, 5



BINARY TREE TRAVERSALS:

The process of visiting the nodes is known as tree traversal. There are three types traversals used to visit a node:

- Inorder traversal
- Preorder traversal
- Postorder traversal

Inorder Traversal (Left, Root, Right)

```
void inorder(struct Node* root) {  
    if (root == NULL) return;  
    inorder(root->left);  
    printf("%d ", root->data);  
    inorder(root->right);  
}
```

Preorder Traversal (Root, Left, Right)

```
void preorder(struct Node* root) {  
    if (root == NULL) return;  
    printf("%d ", root->data);  
    preorder(root->left);  
    preorder(root->right);  
}
```

Postorder Traversal (Left, Right, Root)

```
void postorder(struct Node* root) {  
    if (root == NULL) return;  
  
    postorder(root->left);  
    postorder(root->right);  
    printf("%d ", root->data);  
}
```

Advantages of Binary Trees

- ✓ Efficient search, insertion, and deletion in balanced trees (**BSTs, AVL trees**).
- ✓ Natural representation of hierarchical data.
- ✓ Flexible with multiple variants for specific use cases.
- ✓ Supports various traversal methods for different operations.
- ✓ Self-balancing trees ensure consistent performance.
- ✓ Efficient priority queue implementation (heaps).
- ✓ Memory-efficient linked representation.

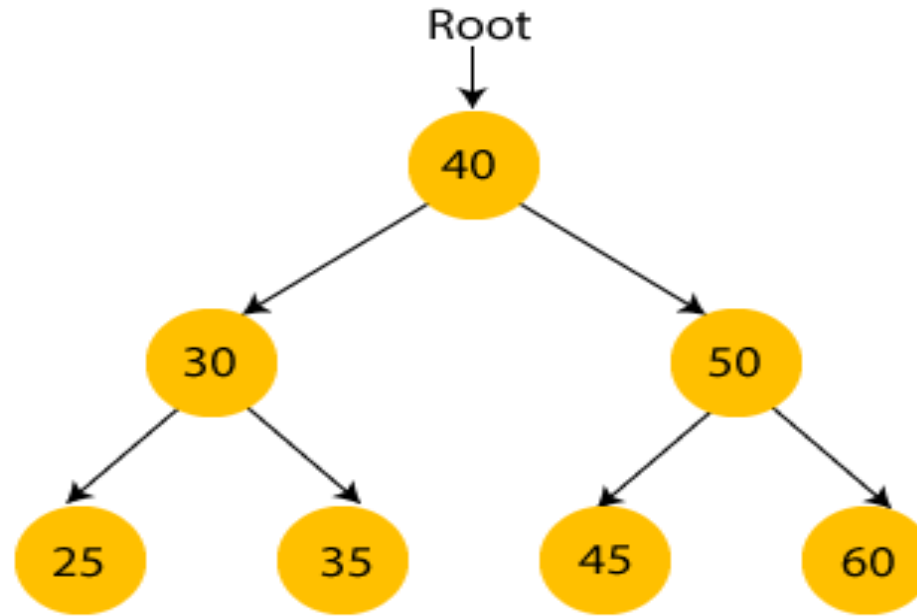
Disadvantages of Binary Trees

- ✓ Unbalanced trees can degrade to **linked lists** with $O(n)$ operations.
- ✓ Complex implementation of balancing algorithms.
- ✓ Memory overhead due to pointer storage.
- ✓ Additional computational overhead for specific traversals.
- ✓ Rebalancing costs in self-balancing trees.
- ✓ Fixed hierarchical structure may not suit all datasets.

BINARY SEARCH TREE

Binary Search tree:-

In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node. This rule is applied recursively to the left and right subtrees of the root.



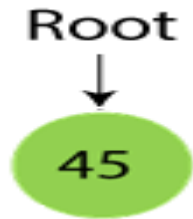
Advantages of Binary search tree

- Searching an element in the Binary search tree is easy as we always have a hint that which subtree has the desired element.
- As compared to array and linked lists, insertion and deletion operations are faster in BST.

Creation of Binary Search tree with the data elements 45, 15, 79, 90, 10, 55, 12, 20, 50

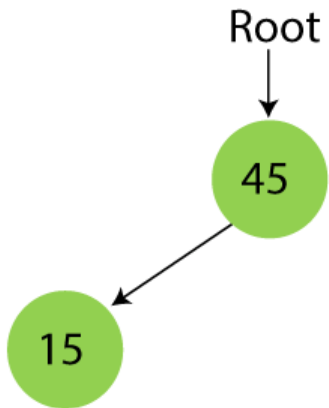
First, we have to insert 45 into the tree as the root of the tree.

Step 1 - Insert 45



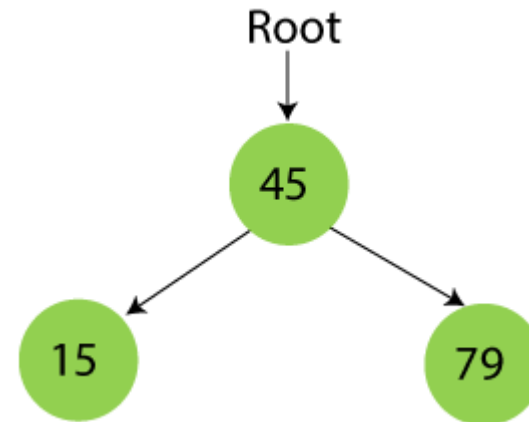
Step 2 - Insert 15.

As 15 is smaller than 45, so insert it as the root node of the left subtree.



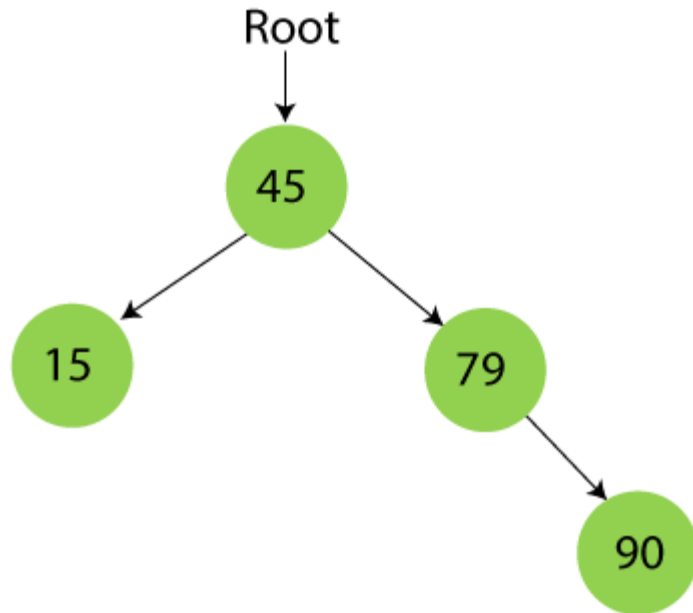
Step 3 - Insert 79.

As 79 is greater than 45, so insert it as the root node of the right subtree.



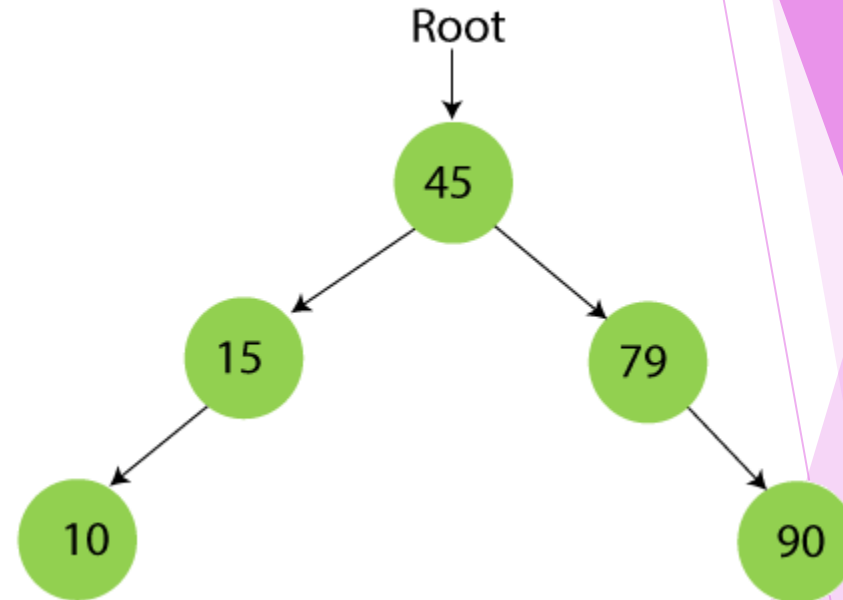
Step 4 - Insert 90.

90 is greater than 45 and 79, so it will be inserted as the right subtree of 79.



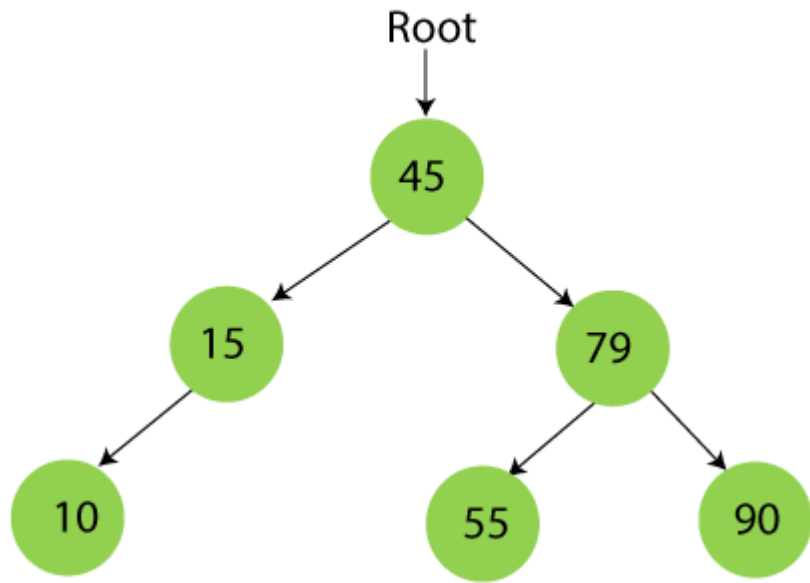
Step 5 - Insert 10.

10 is smaller than 45 and 15, so it will be inserted as a left subtree of 15.



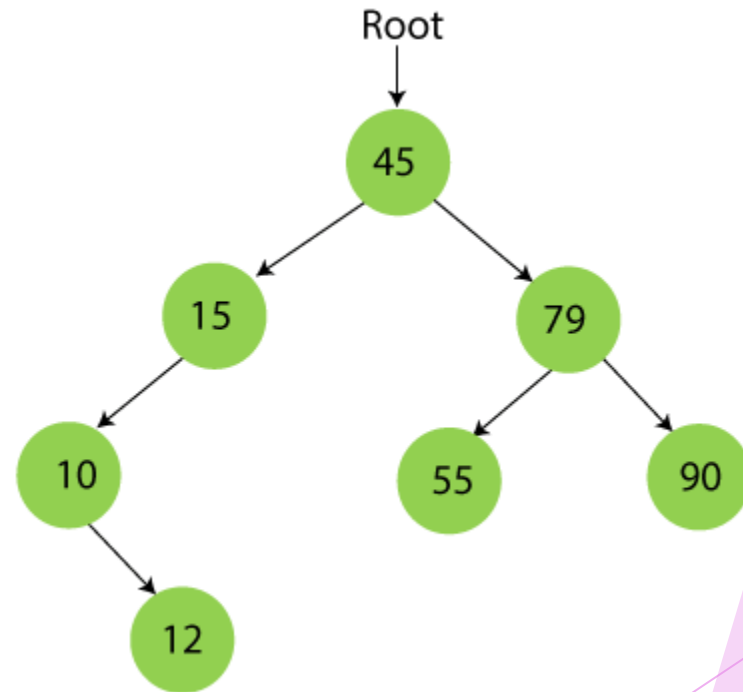
Step 6 - Insert 55.

55 is larger than 45 and smaller than 79, so it will be inserted as the left subtree of 79.



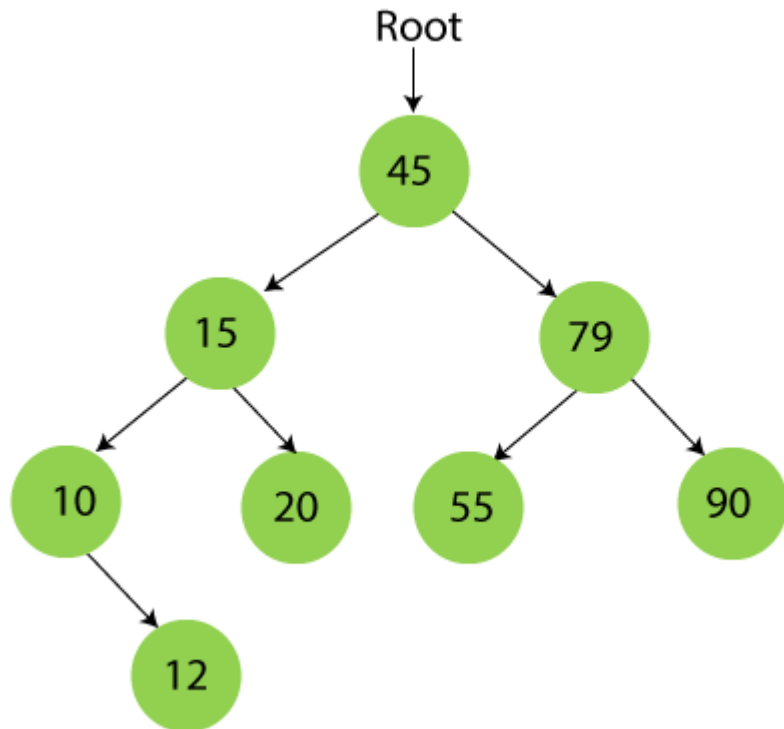
Step 7 - Insert 12.

12 is smaller than 45 and 15 but greater than 10, so it will be inserted as the right subtree of 10.



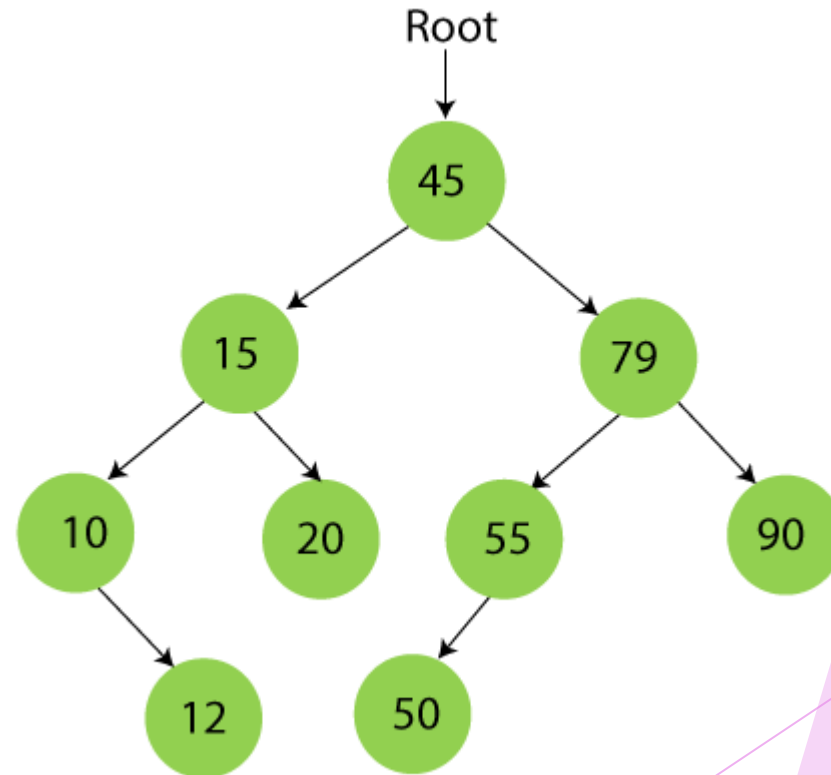
Step 8 - Insert 20.

20 is smaller than 45 but greater than 15, so it will be inserted as the right subtree of 15.



Step 9 - Insert 50.

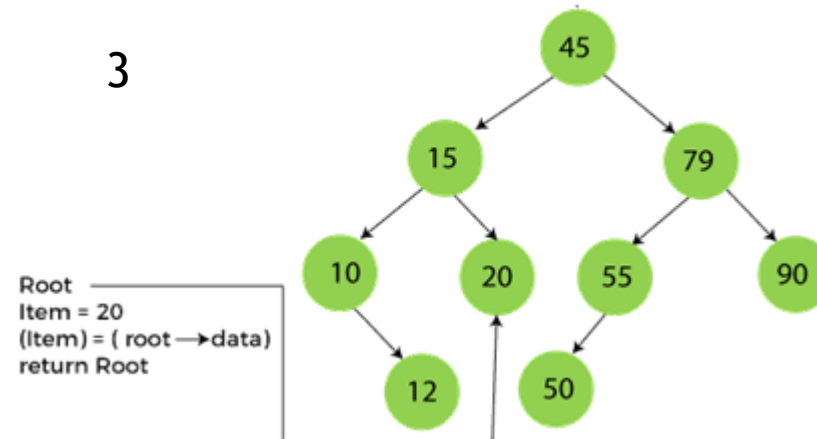
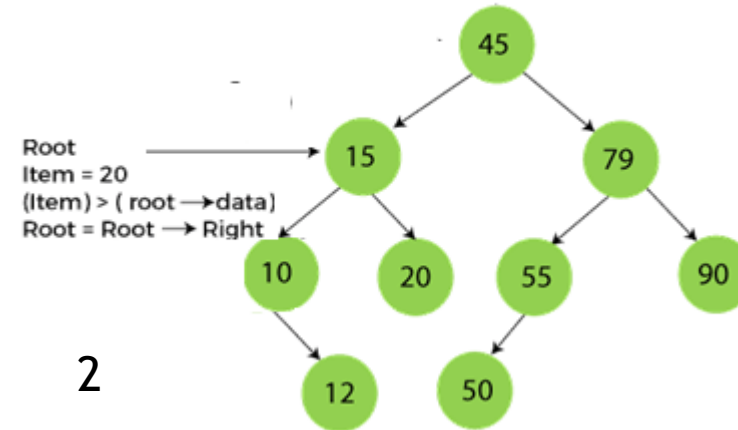
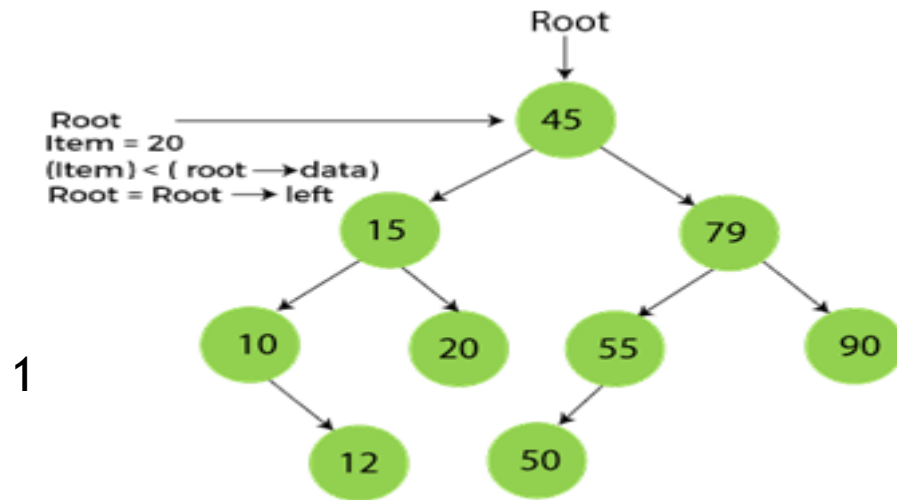
50 is greater than 45 but smaller than 79 and 55. So, it will be inserted as a left subtree of 55.



Searching in Binary search tree

- 1.First, compare the element to be searched with the root element of the tree.
- 2.If root is matched with the target element, then return the node's location.
- 3.If it is not matched, then check whether the item is less than the root element, if it is smaller than the root element, then move to the left subtree.
- 4.If it is larger than the root element, then move to the right subtree.
- 5.Repeat the above procedure recursively until the match is found.
- 6.If the element is not found or not present in the tree, then return NULL.

Suppose we have to find node 20 from the below tree.



Algorithm to search an element in Binary search tree

Search (root, item)

Step 1 - if (item = root $\rightarrow$ data) or (root = NULL)

return root

else if (item < root $\rightarrow$ data)

return Search(root $\rightarrow$ left, item)

else

return Search(root $\rightarrow$ right, item)

END if

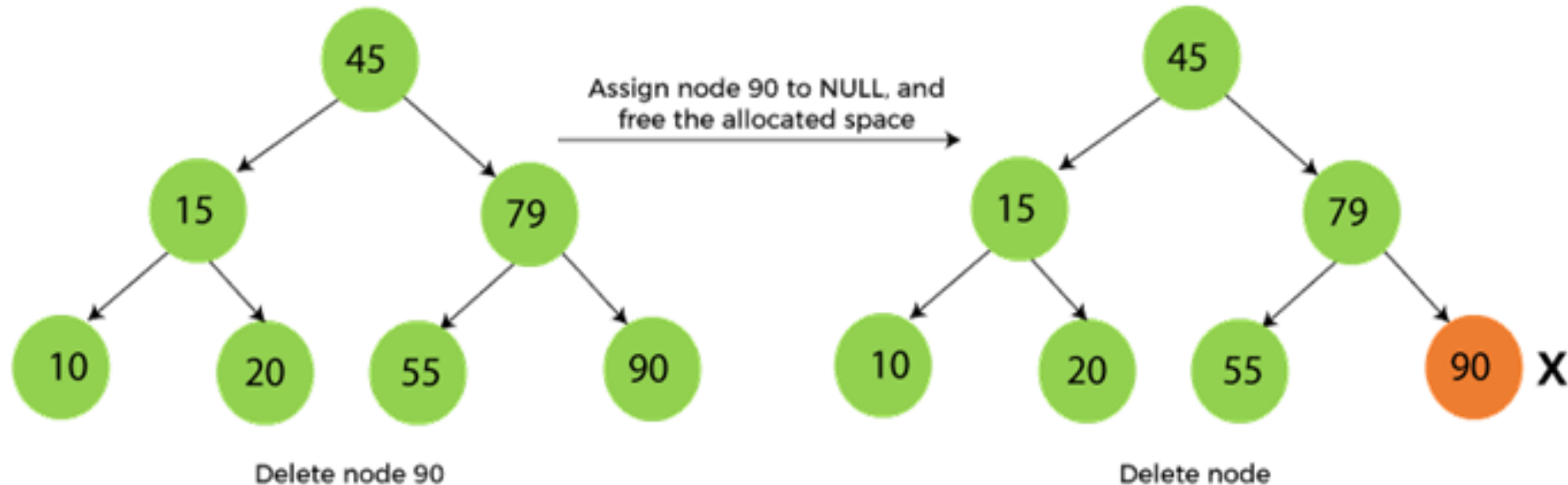
Step 2 - END

Deletion in Binary Search tree

- The node to be deleted is the leaf node, or,
- The node to be deleted has only one child, and,
- The node to be deleted has two children

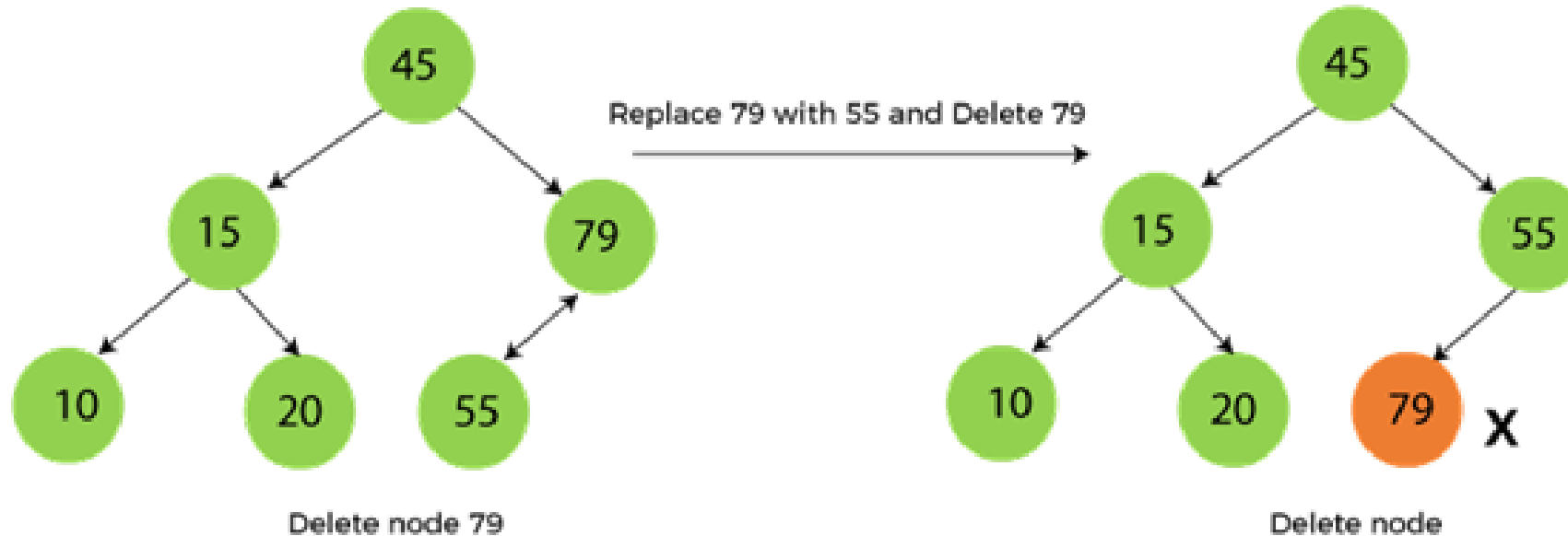
When the node to be deleted is the leaf node

It is the simplest case to delete a node in BST. Here, we have to replace the leaf node with NULL and simply free the allocated space.



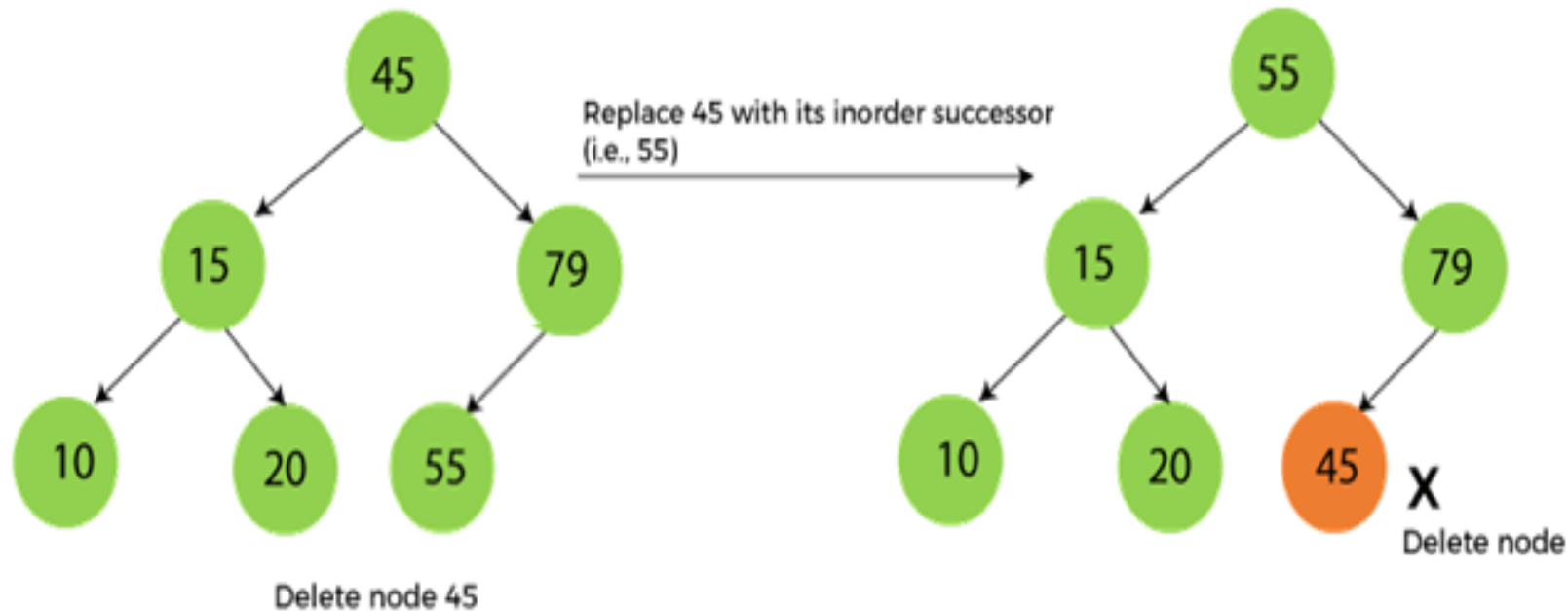
When the node to be deleted has only one child

In this case, we have to replace the target node with its child, and then delete the child node. It means that after replacing the target node with its child node, the child node will now contain the value to be deleted. So, we simply have to replace the child node with NULL and free up the allocated space.



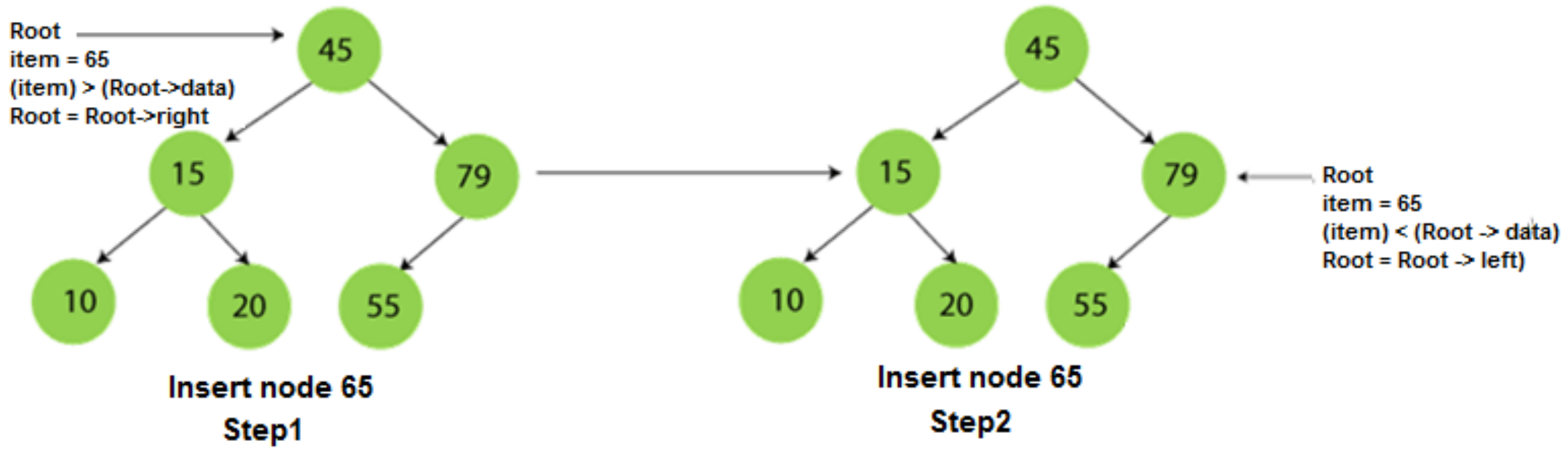
When the node to be deleted has two children

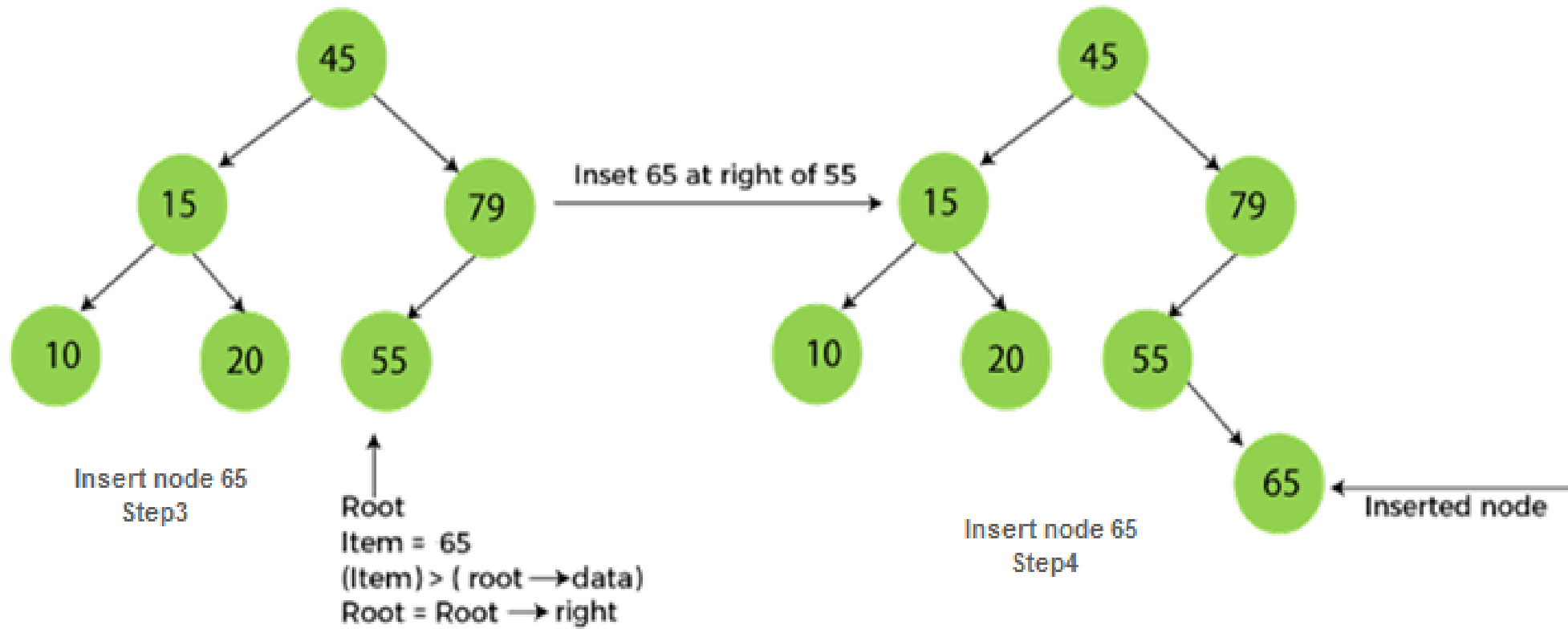
- First, find the inorder successor of the node to be deleted.
- After that, replace that node with the inorder successor until the target node is placed at the leaf of tree.
- And at last, replace the node with NULL and free up the allocated space.



Insertion in Binary Search tree

To insert an element in BST, we have to start searching from the root node; if the node to be inserted is less than the root node, then search for an empty location in the left subtree. Else, search for the empty location in the right subtree and insert the data. Insert in BST is similar to searching, as we always have to maintain the rule that the left subtree is smaller than the root, and right subtree is larger than the root.





Advantages and Disadvantages of Binary Search Tree (BST)

Advantages	Disadvantages
Efficient Operations: $O(\log n)$ for balanced trees	Unbalanced Trees: Can degrade to $O(n)$ operations in worst case
Ordered Structure: Natural sorting via in-order traversal	Complexity of Balancing: Additional overhead for maintaining balance
Dynamic Data: Flexible and can grow/shrink dynamically	Space Overhead: Extra memory required for pointers
Hierarchical Organization: Useful for hierarchical data	Traversal Overhead: Recursive traversal can lead to stack overflow
Memory Efficiency: Efficient use of memory for sparse datasets	Deletion Complexity: Deleting nodes with two children is complex
Balanced Trees: Ensures $O(\log n)$ operations	Inefficiency for Uniform Access: Performance depends on tree height

HEAP

Heap

A heap is a complete binary tree, and the binary tree is a tree in which the node can have at most two children.

There are two types of the heap:

- Min Heap
- Max heap

Min Heap: The value of the parent node should be less than or equal to both children.

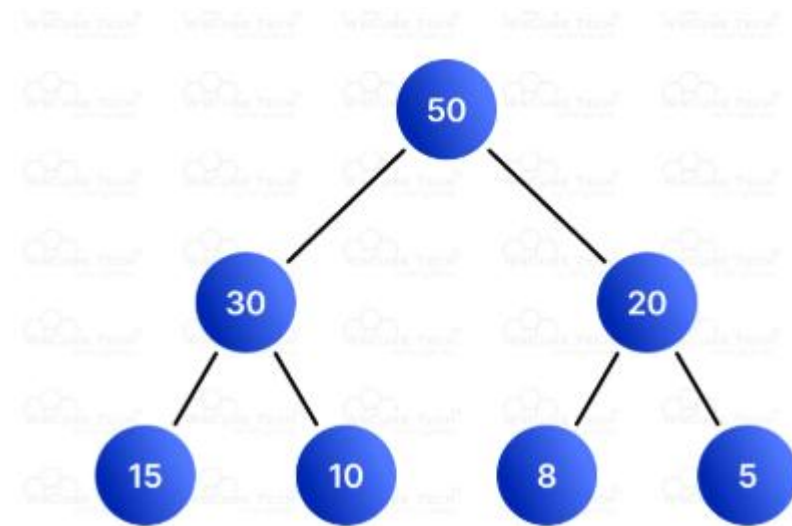
Max Heap: The value of the parent node is greater than or equal to its children.

Max-Heap

In a max-heap, each parent node is greater than or equal to its child nodes. This ensures that the largest value is always at the root of the tree.

Ex:-

- The root node (50) is greater than its children (30 and 20).
- Each parent node (30, 20) is greater than its children (15, 10 for 30 and 8, 5 for 20).

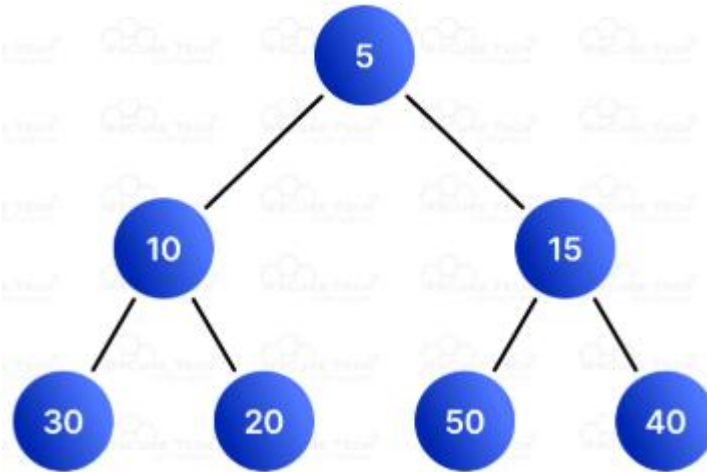


Min-Heap

In a min-heap, each parent node is less than or equal to its child nodes. This ensures that the smallest value is always at the root of the tree.

Ex:-

- The root node (5) is smaller than its children (10 and 15).
- Each parent node (10, 15) is smaller than its children (30, 20 for 10 and 50, 40 for 15).



Properties of Heap Data Structure

1. Complete Binary Tree

A heap is always a complete binary tree, meaning:

All levels are fully filled except possibly the last level.

The last level is filled from left to right.

This property ensures that the tree remains balanced, which is crucial for maintaining the efficiency of heap operations.

2. Heap Order Property

The heap must satisfy the heap order property, which differs for max-heaps and min-heaps:

Max-Heap Order Property: In a max-heap, each parent node is greater than or equal to its child nodes. This ensures that the largest element is always at the root.

Min-Heap Order Property: In a min-heap, each parent node is less than or equal to its child nodes. This ensures that the smallest element is always at the root.

Heapify Operations

Heapify operations are crucial for maintaining the heap property after insertion or deletion. The two main heapify operations are Up-Heapify (Bubble-Up) and Down-Heapify (Bubble-Down).

1. Up-Heapify (Bubble-Up)

Up-Heapify is used after inserting a new element to restore the heap property by moving the element up the tree.

Steps:

Insert the new element at the end of the heap.

Compare the inserted element with its parent.

If the heap property is violated (the new element is greater than the parent in a max-heap or smaller than the parent in a min-heap), swap the new element with its parent.

Repeat this process until the heap property is restored or the element becomes the root.

Down-Heapify (Bubble-Down)

Down-Heapify is used after deleting the root element to restore the heap property by moving the new root element down the tree.

Steps:

Replace the root element with the last element in the heap.

Remove the last element from the heap.

Compare the new root with its children.

If the heap property is violated (the new root is smaller than a child in a max-heap or larger than a child in a min-heap), swap the new root with the larger (in max-heap) or smaller (in min-heap) child.

Repeat this process until the heap property is restored or the element reaches a leaf node.

Insertion in the Max Heap tree 44, 33, 77, 11, 55, 88, 66

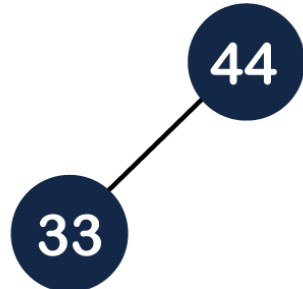
To create the max heap tree, we need to consider the following two cases:

- First, we have to insert the element in such a way that the property of the complete binary tree must be maintained.
- Secondly, the value of the parent node should be greater than the either of its child.

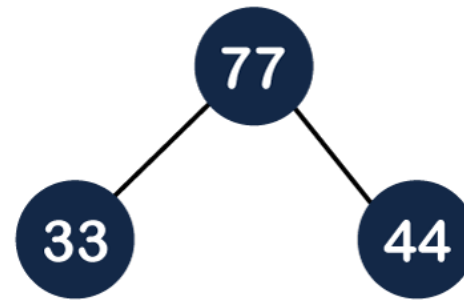
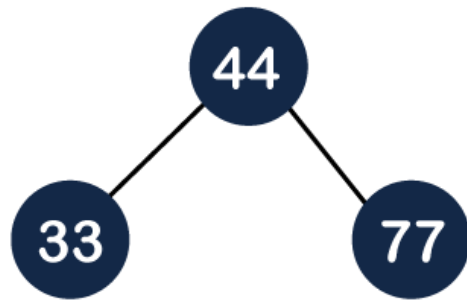
Step 1: First we add the 44 element in the tree



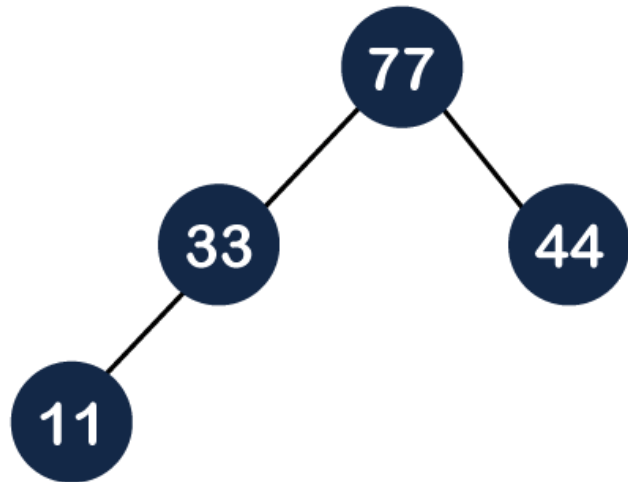
Step 2: The next element is 33. As we know that insertion in the binary tree always starts from the left side so 44 will be added at the left of 33



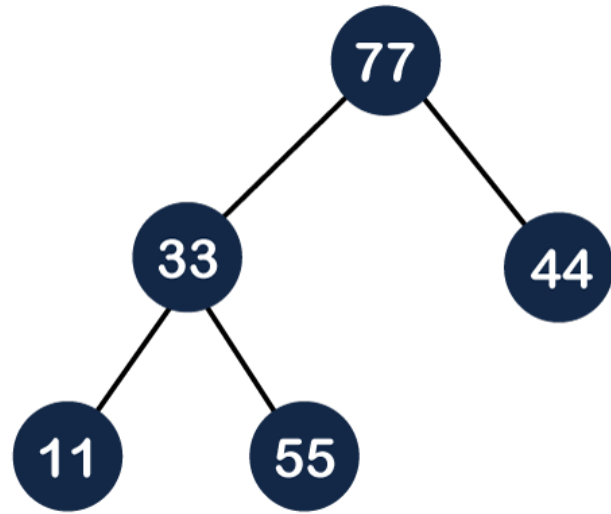
Step 3: The next element is 77 and it will be added to the right of the 44. As we can observe in the above tree that it does not satisfy the max heap property, i.e., parent node 44 is less than the child 77. So, we will swap these two values



Step 4: The next element is 11. The node 11 is added to the left of 33



Step 5: The next element is 55. To make it a complete binary tree, we will add the node 55 to the right of 33



As we can observe in the above figure that it does not satisfy the property of the max heap because $33 < 55$, so we will swap these two values

Step 6: The next element is 88. The left subtree is completed so we will add 88 to the left of 44

As we can observe in the above figure that it does not satisfy the property of the max heap because $44 < 88$, so we will swap these two values

Step 7: As we can observe in the above figure that $88 > 77$, so swap these values

Step 8:- The next element is 66. To make a complete binary tree, we will add the 66 element to the right side of 77

Advantages of Heap Tree in Data Structure

- ✓ Efficient priority queue operations ($O(\log n)$ for insertion and deletion)
- ✓ Memory-efficient with contiguous storage
- ✓ Suitable for heap sort ($O(n \log n)$ time complexity)
- ✓ Effective for dynamic datasets
- ✓ Supports quick access to minimum/maximum elements

Disadvantages of Heap Tree in Data Structure

- ✓ Inefficient search operations ($O(n)$ **time complexity**)
- ✓ Elements are not stored in a sorted order
- ✓ More complex implementation and maintenance
- ✓ Fixed structure can lead to inefficient memory use
- ✓ Balancing overhead during heapify operations

B TREE

B-Tree(Height balanced M way search Tree)

A **B-Tree** is a **self-balancing search tree** in which multiple **nodes can have more than two children**.

Properties of a B-Tree (of order m)

All leaf nodes must be at same level.

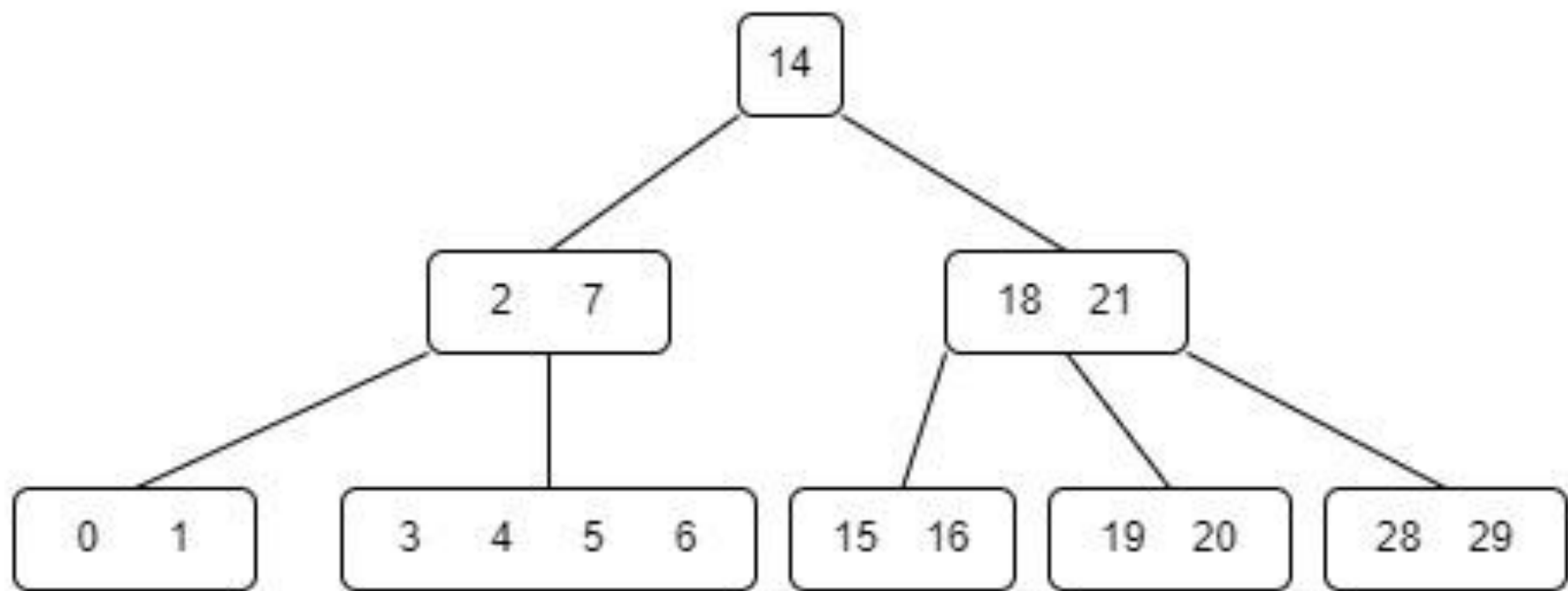
All nodes except root must have at least $\lceil m/2 \rceil - 1$ keys and maximum of $m - 1$ keys

All non leaf nodes except root must have at least $m/2$ children

If the root node is a non leaf node, then it must have at least 2 children

A non leaf node with $n - 1$ keys must have n number of children

All the key values in a node must be in ascending order.



OPERATIONS IN B TREE

INSERTION

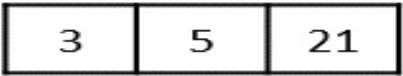
- ❖ In a B-tree ,a new element must be added only at the leaf node.
- ❖ Check whether the tree is empty
- ❖ If tree is empty, then find the suitable leaf node to which the new key value is added using binary search tree logic.
- ❖ If that leaf node has empty position add the new key value to that leaf node ascending order of the key value within the node
- ❖ If that leaf node is already full, split that leaf node by sending middle value to its parent node. Repeat the same until the sending value is fixed into a node
- ❖ If the splitting is performed at root node then the middle value becomes new root node for the tree and the height of the tree is increased by one

Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree

- Order (m) = 4
- Maximum Keys ($m - 1$) = 3
- Minimum Keys ($\lceil \frac{m}{2} \rceil$) - 1 = 1
- Maximum Children = 4
- Minimum Children ($\lceil \frac{m}{2} \rceil$) = 2

The data is inserted into the tree using the binary search insertion and once the keys reach the maximum number, the node is split into half and the median key becomes the internal node while the left and right keys become its children.

Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree



Adding 9 will cause overflow in the node; hence it must be split.

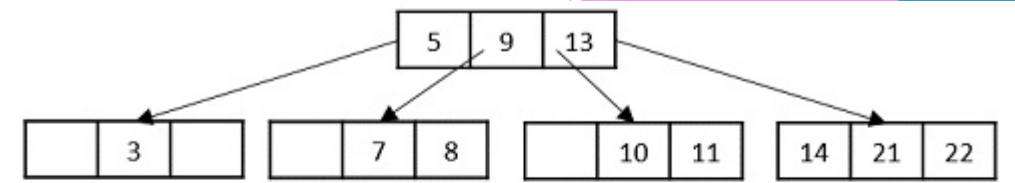
All the leaf nodes must be on the same level.

Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree

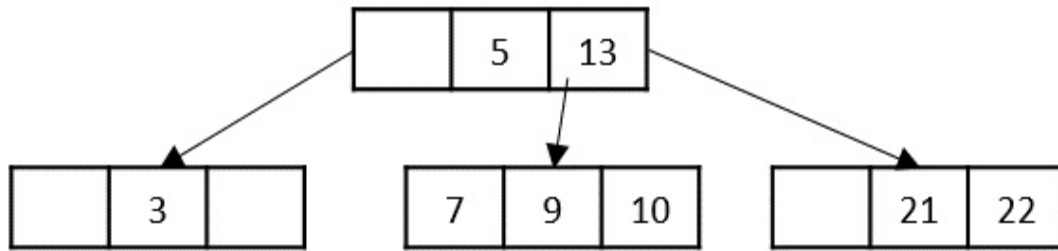


Adding 22 will cause overflow in the node; hence it must be split.

The keys, 5, 3, 21, 9, 13 are all added into the node according to the binary search property but if we add the key 22, it will violate the maximum key property. Hence, the node is split in half, the median key is shifted to the parent node and the insertion is then continued.

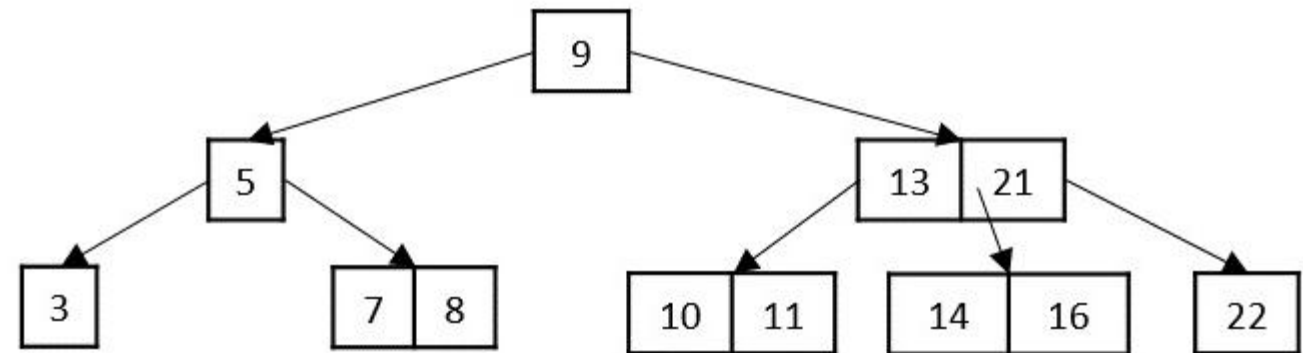


Adding 16 will cause overflow in the node; hence it must be split.



Adding 11 will cause overflow in the node; hence it must be split.

Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree

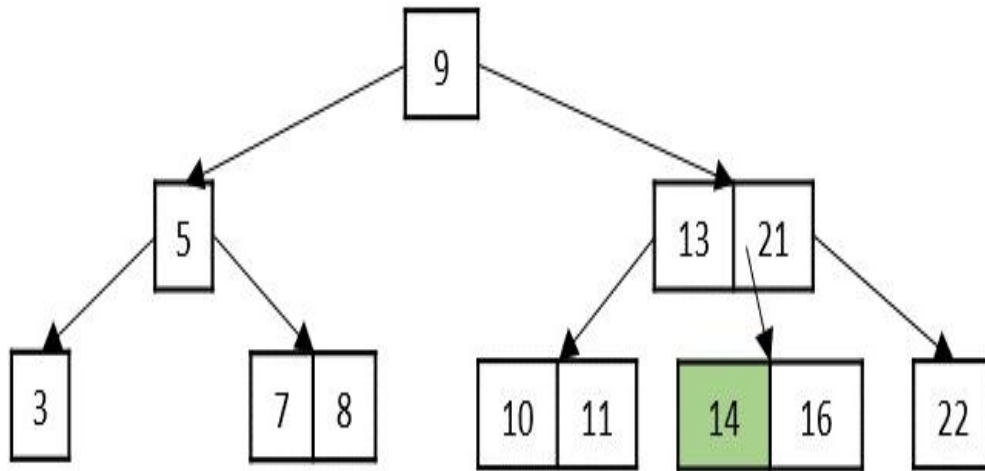


Deletion operation

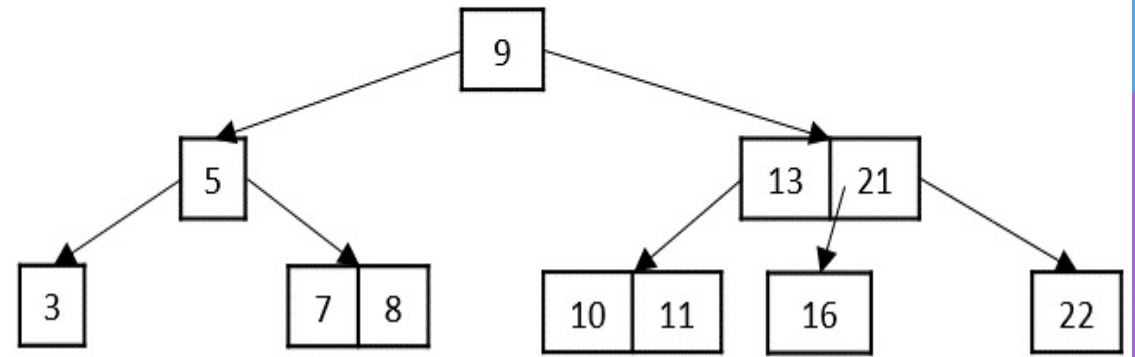
The deletion operation in a B tree is slightly different from the deletion operation of a Binary Search Tree.

Case 1 – If the key to be deleted is in a leaf node and the deletion does not violate the minimum key property, just delete the node.

Delete key 14

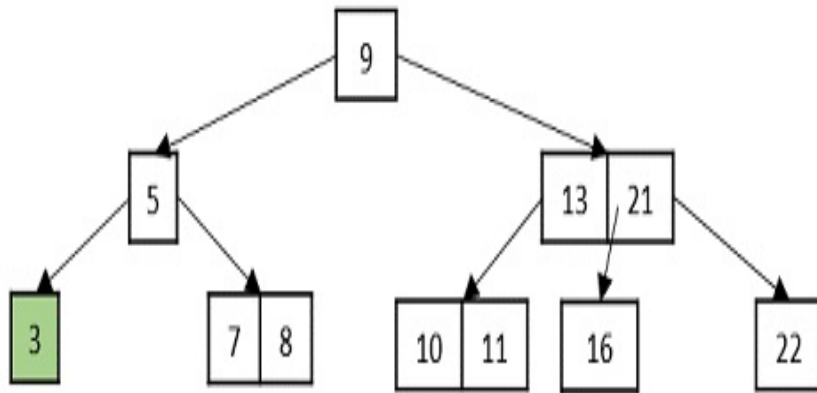


Delete key 14

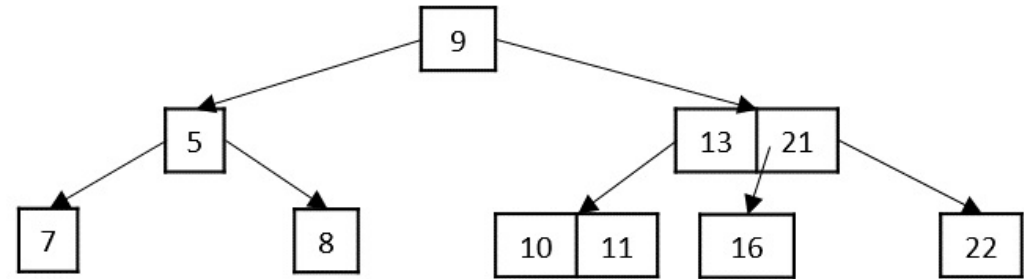


Case 2 – If the key to be deleted is in a leaf node but the deletion violates the minimum key property, borrow a key from either its left sibling or right sibling. In case if both siblings have exact minimum number of keys, merge the node in either of them.

Delete key 3

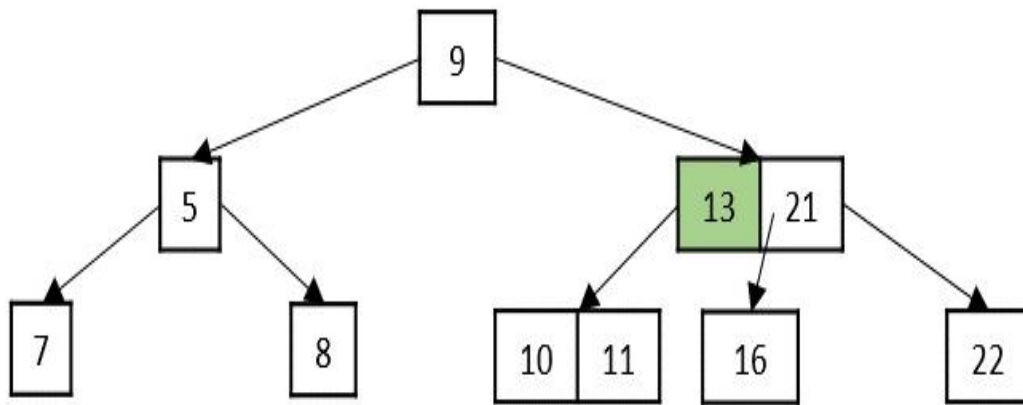


Delete key 3

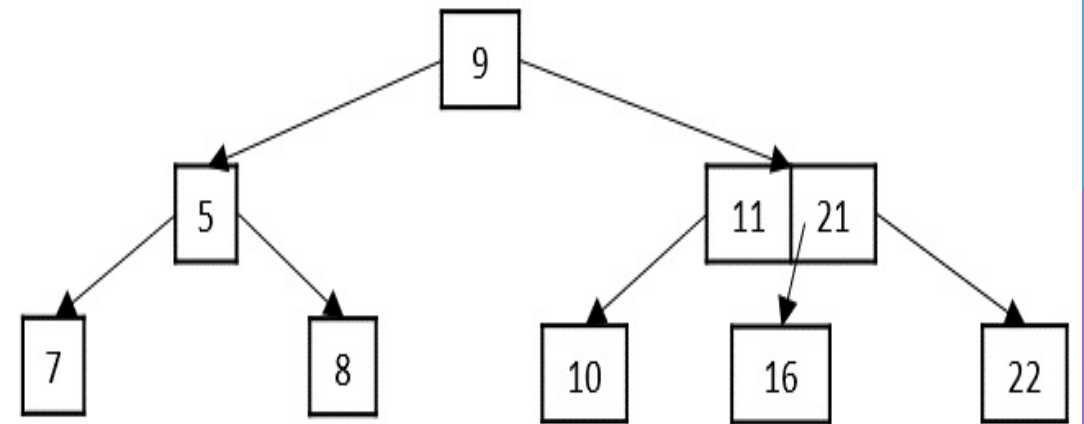


Case 3 – If the key to be deleted is in an internal node, it is replaced by a key in either left child or right child based on which child has more keys. But if both child nodes have minimum number of keys, they're merged together.

Delete key 13

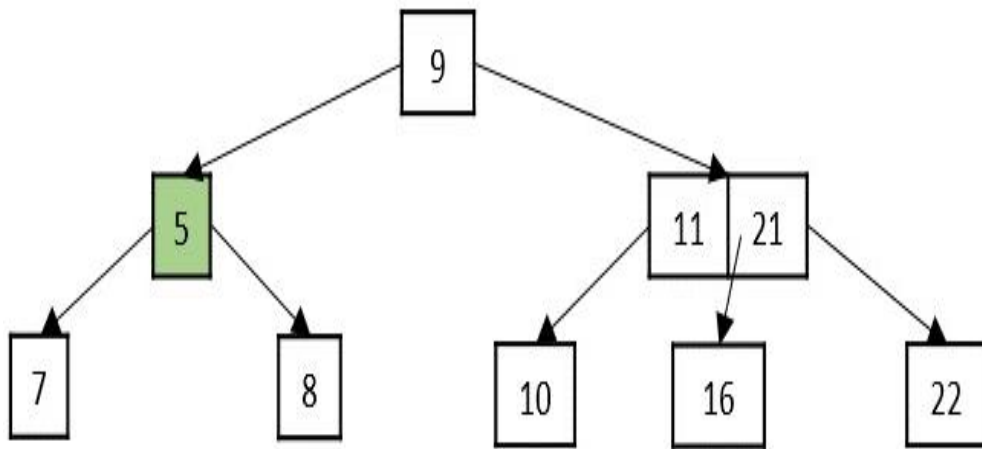


Delete key 13

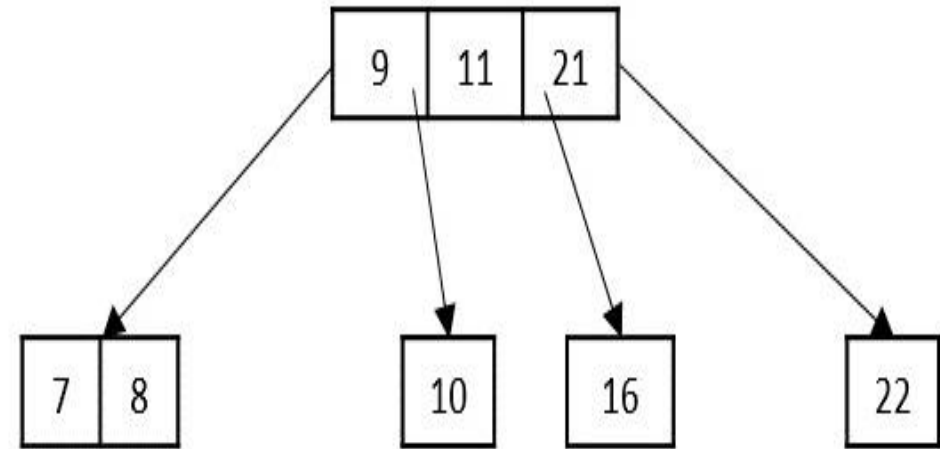


Case 4 – If the key to be deleted is in an internal node violating the minimum keys property, and both its children and sibling have minimum number of keys, merge the children. Then merge its sibling with its parent.

Delete key 5



Delete key 5



Advantages and Disadvantages of B-Tree

Advantages	Disadvantages
Efficient disk read/write operations	Complex implementation
Balanced tree structure ($O(\log n)$ time)	Higher memory overhead
Scalable with large datasets	Performance degradation with small datasets
Flexibility in node size	Complex balancing operations
Low latency for insertions and deletions	Inefficient for point queries

Applications of B-Tree Data Structure

B-Trees are versatile data structures widely used in various applications due to their efficiency in managing large datasets and minimizing disk I/O operations:

1. Database Indexing

B-Trees are extensively used in database management systems (DBMS) to implement indexes.

2. File Systems

B-Trees are used in file systems to manage files and directories efficiently.

3. Databases and Indexing for Embedded Systems

B-Trees are used in embedded systems for efficient data storage and retrieval.

4. Multi-Level Indexing

B-Trees are used in multi-level indexing schemes to improve search performance.

5. Caching Systems

B-Trees are used in caching systems to manage cache entries efficiently.

6. Geographic Information Systems (GIS)

B-Trees are used in Geographic Information Systems (GIS) for spatial data indexing.

7. Operating System Schedulers

B-Trees are used in operating system schedulers to manage process scheduling.

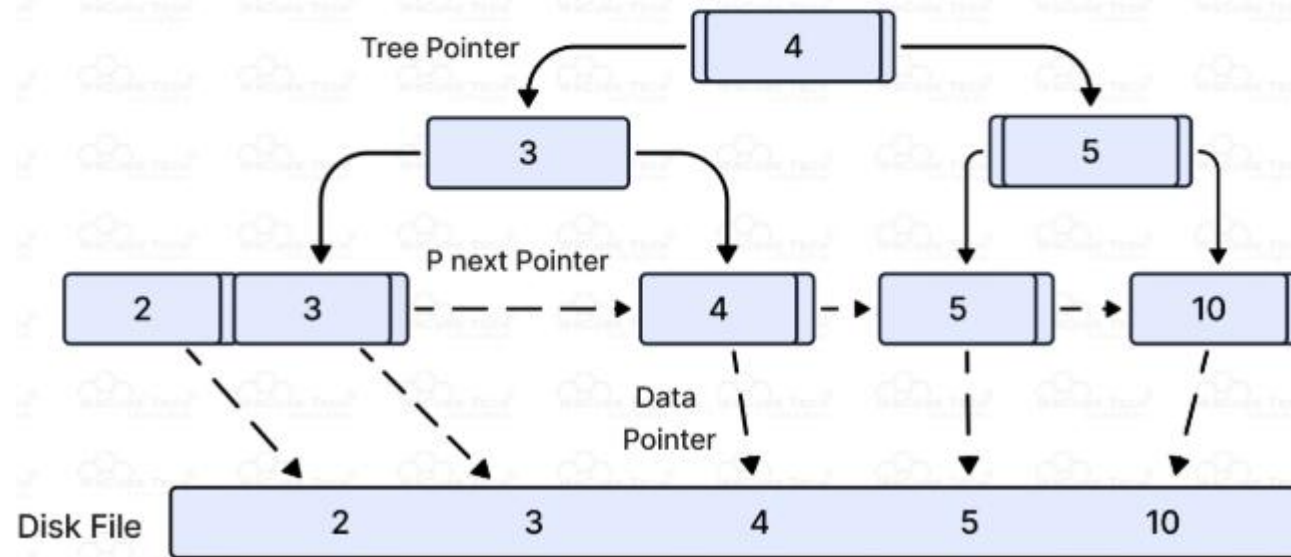
B+ Tree

B+ Tree:-

A **B+ Tree** is a **balanced tree data structure** that maintains **sorted data** and allows **searches, sequential access, insertions, and deletions** . It is a type of **self-balancing tree** and a variant of the **B-Tree**.

The properties of B+ trees are similar to the properties of B trees, except that the B trees can store keys and records in all internal nodes and leaf nodes while B+ trees store records in leaf nodes and keys in internal nodes.

B+ Tree



Properties of B+ Tree

Balanced Tree Structure: B+ Trees are balanced, meaning all leaf nodes are at the same level.

Leaf Nodes Store All Values: All actual data values are stored in the leaf nodes.

Internal Nodes for Indexing: Internal nodes contain only keys and pointers to child nodes.

Linked List of Leaf Nodes: Leaf nodes are linked together in a linked list.

Order of the Tree: The order (m) of a B+ Tree defines the maximum number of children each internal node can have.

Minimum Degree: The minimum degree (t) of a B+ Tree is defined as $\lceil m/2 \rceil$.

Efficient Disk Read/Write: B+ Trees are designed to minimize disk I/O operations.

Order of B+ Tree

The order of a B+ Tree, often denoted as m , is a fundamental property that determines the tree's structure and balance. The order of a B+ Tree specifies the maximum number of children that any node in the tree can have.

Order m :

The order of a B+ Tree is the maximum number of children that any internal node can have.

For a B+ Tree of order m :

Each internal node can have at most m children.

Each internal node (except the root) must have at least $(m/2)$ children.

Each internal node must have between $(m/2) - 1$ and $m - 1$ keys.

Each leaf node must have between $(m/2)$ and m values.

B+ Tree Traversal Methods

Traversal methods for B+ Trees involve visiting all the nodes in the tree in a specific order. The main traversal methods are in-order traversal and range queries. Each method serves different purposes and has its own **algorithm**.

1. In-order Traversal

In-order traversal in a B+ Tree involves visiting all the leaf nodes in sorted order. This method is particularly efficient because B+ Trees store all values in the leaf nodes, and these nodes are linked together in a **linked list**.

Algorithm:

Begin the traversal from the leftmost leaf node.

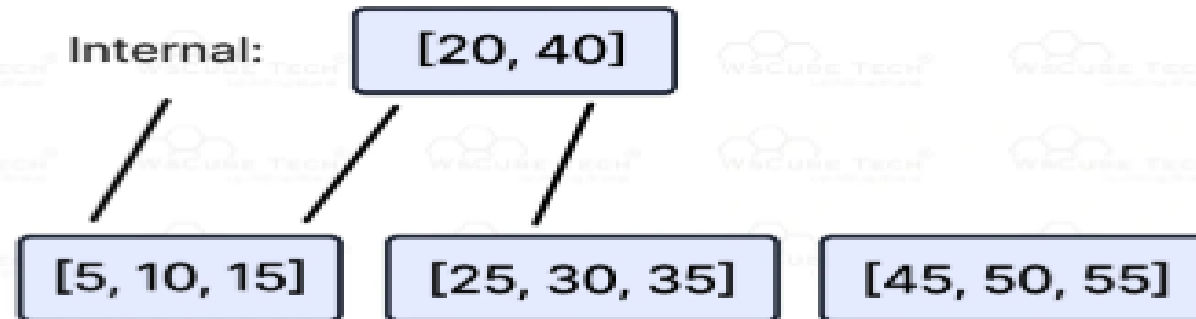
Visit each key in the current leaf node.

Follow the pointer to the next leaf node and repeat the process.

Continue until you reach the end of the leaf nodes.

Example:

Consider the following B+ Tree:



In-order Traversal Steps:

1. Start at the leftmost leaf node: [5, 10, 15]
2. Visit keys in the current leaf node: 5, 10, 15
3. Move to the next leaf node: [25, 30, 35]
4. Visit keys: 25, 30, 35
5. Move to the next leaf node: [45, 50, 55]
6. Visit keys: 45, 50, 55

In-order Traversal Result: 5, 10, 15, 25, 30, 35, 45, 50, 55

Range Queries

Range queries in a B+ Tree involve finding all the keys within a specified range. This method takes advantage of the linked list structure of the leaf nodes, making it efficient for range searches.

Algorithm:

Start from the root and find the leaf node containing the starting key of the range.

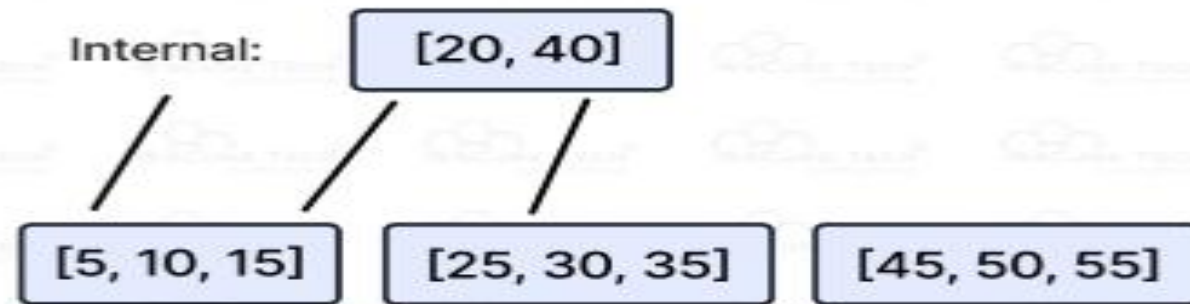
Visit each key in the current leaf node that falls within the range.

Follow the pointer to the next leaf node and repeat the process until all keys in the range are found.

Continue until you reach the end of the range.

Example:

Consider the following B+ Tree and a range query for keys between 15 and 35:



Range Query Steps:

1. Find the starting leaf node: [5, 10, 15]
2. Visit keys in the current leaf node: 15
3. Move to the next leaf node: [25, 30, 35]
4. Visit keys: 25, 30, 35
5. Range query result: 15, 25, 30, 35

Insertion operation

The insertion to a B+ tree starts at a leaf node.

Step 1 – Calculate the maximum and minimum number of keys to be added onto the B+ tree node.

Insert 1, 2, 3, 4, 5, 6, 7 into a B+ Tree with order 4

Order = 4

Maximum Children (m) = 4

Minimum Children ($\lceil \frac{m}{2} \rceil$) = 2

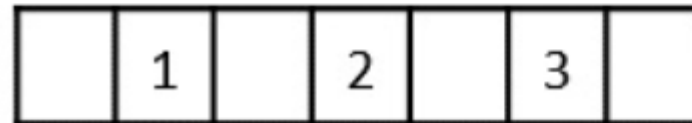
Maximum Keys ($m - 1$) = 3

Minimum Keys ($\lceil \frac{m-1}{2} \rceil$) = 1

Step 2 – Insert the elements one by one accordingly into a leaf node until it exceeds the maximum key number.

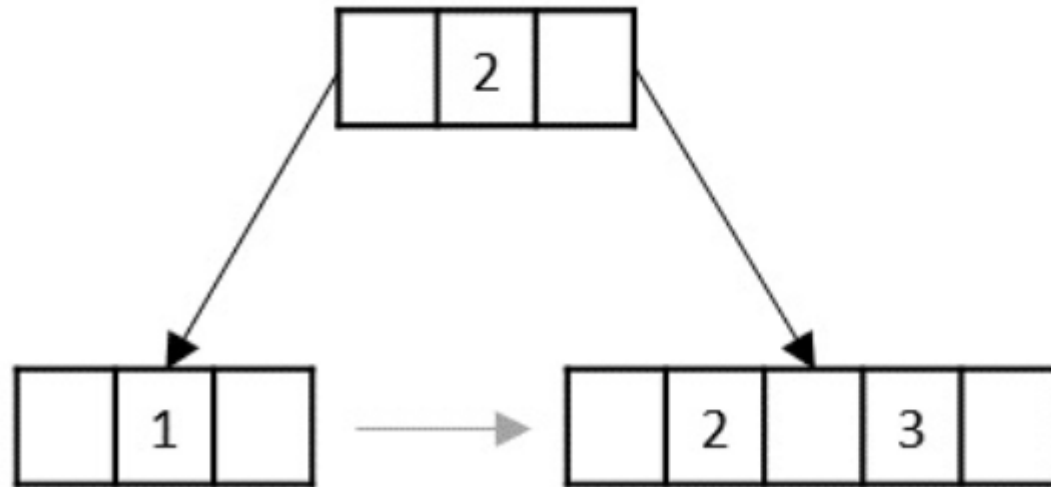
Insert 1, 2, 3, 4, 5, 6, 7 into a B+ Tree with order 4

Adding 4 into
this node will
lead to an
overflow



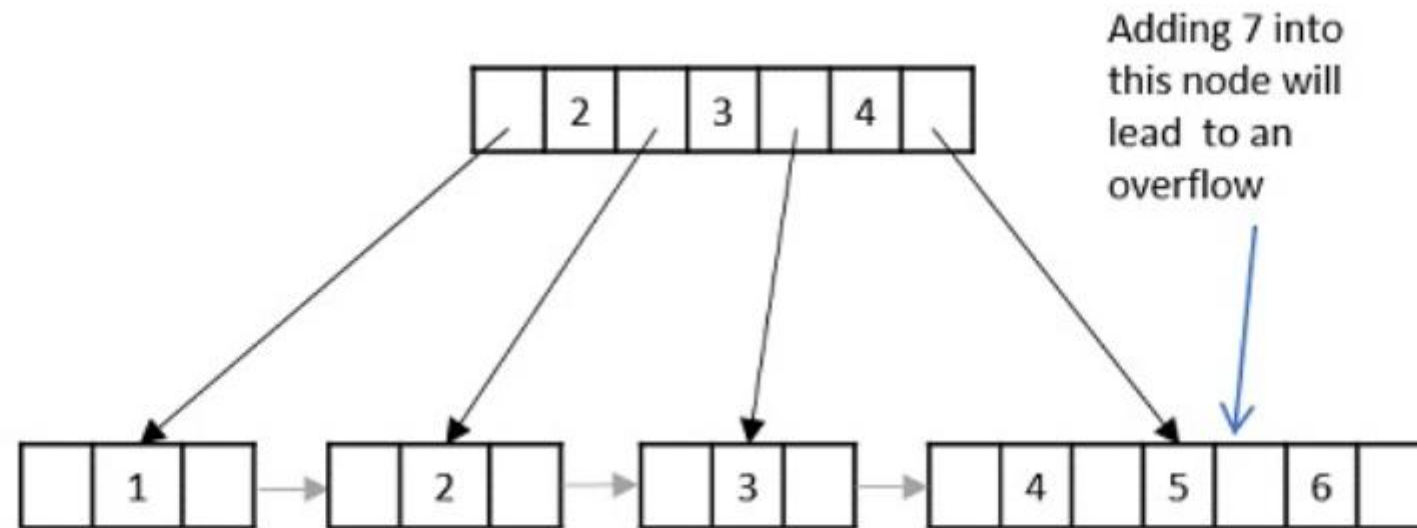
Step 3 – The node is split into half where the left child consists of minimum number of keys and the remaining keys are stored in the right child.

Insert 1, 2, 3, 4, 5, 6, 7 into a B+ Tree with order 4



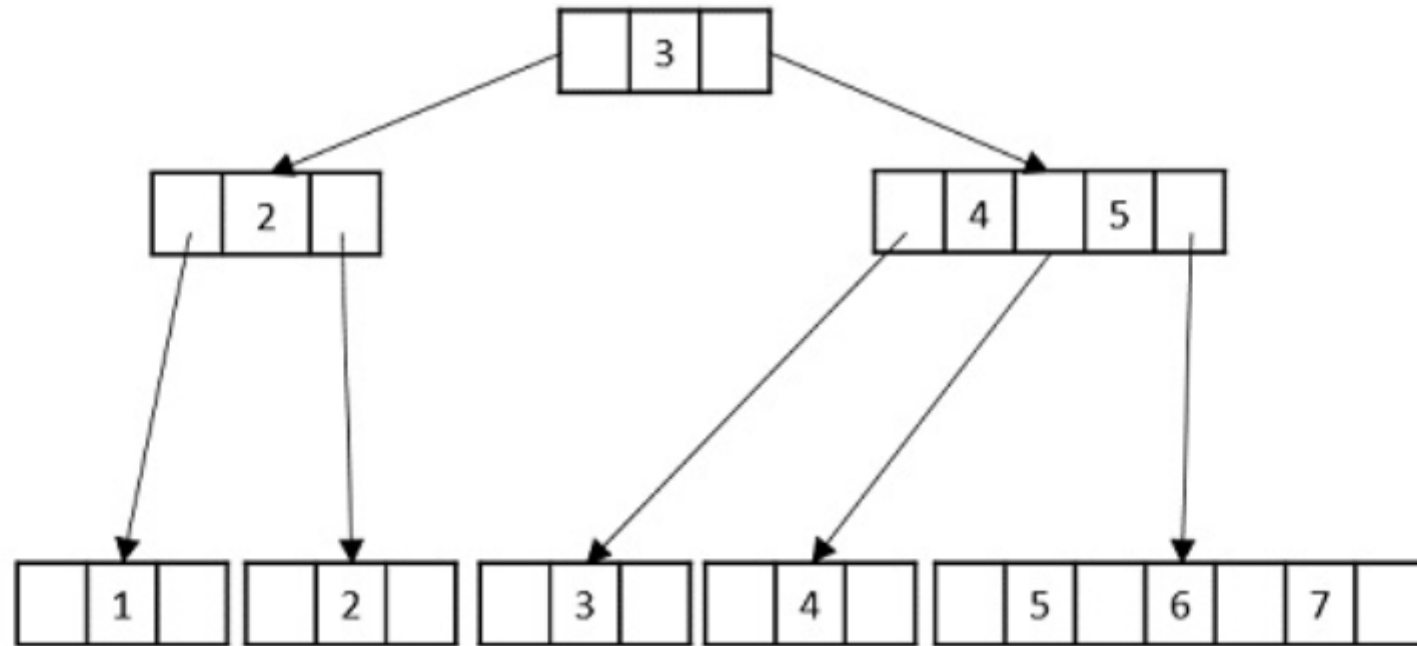
Step 4 – But if the internal node also exceeds the maximum key property, the node is split in half where the left child consists of the minimum keys and remaining keys are stored in the right child. However, the smallest number in the right child is made the parent.

Insert 1, 2, 3, 4, 5, 6, 7 into a B+ Tree with order 4



Step 5 – If both the leaf node and internal node have the maximum keys, both of them are split in the similar manner and the smallest key in the right child is added to the parent node.

Insert 1, 2, 3, 4, 5, 6, 7 into a B+ Tree with order 4



Advantages and Disadvantages of B+ Tree

Advantages	Disadvantages
Efficient Disk Usage: Minimizes disk I/O	Complex Implementation: Difficult to implement
Support for Range Queries: Efficient range searching	Higher Memory Overhead: More pointers and structures
Balanced Tree Structure: Consistent performance	Maintenance Overhead: Costly balance maintenance
Higher Fanout: Shallower tree structure	Not Always Necessary: Overkill for some applications
Separation of Keys and Data: Efficient memory use	Cache Unfriendly for Internal Nodes: Can lead to cache misses
Bulk Loading: Efficiently load large datasets	

Applications of B+ Tree

B Plus Tree is widely used in various applications due to their efficient data storage, retrieval, and range query capabilities:

1. Database Indexing

B+ Trees are extensively used in database management systems to implement indexing.

2. File Systems

B+ Trees are used in file systems to manage files and directories.

3. Metadata Indexing

B+ Trees are used to index metadata in various systems.

4. Caching Systems

B+ Trees are used in caching systems to manage cache entries.

5. Transaction Processing

B+ Trees are used in transaction processing systems to manage transaction logs.

6. Memory Management

B+ Trees are used in memory management systems to track free memory blocks.

7. Geographic Information Systems (GIS)

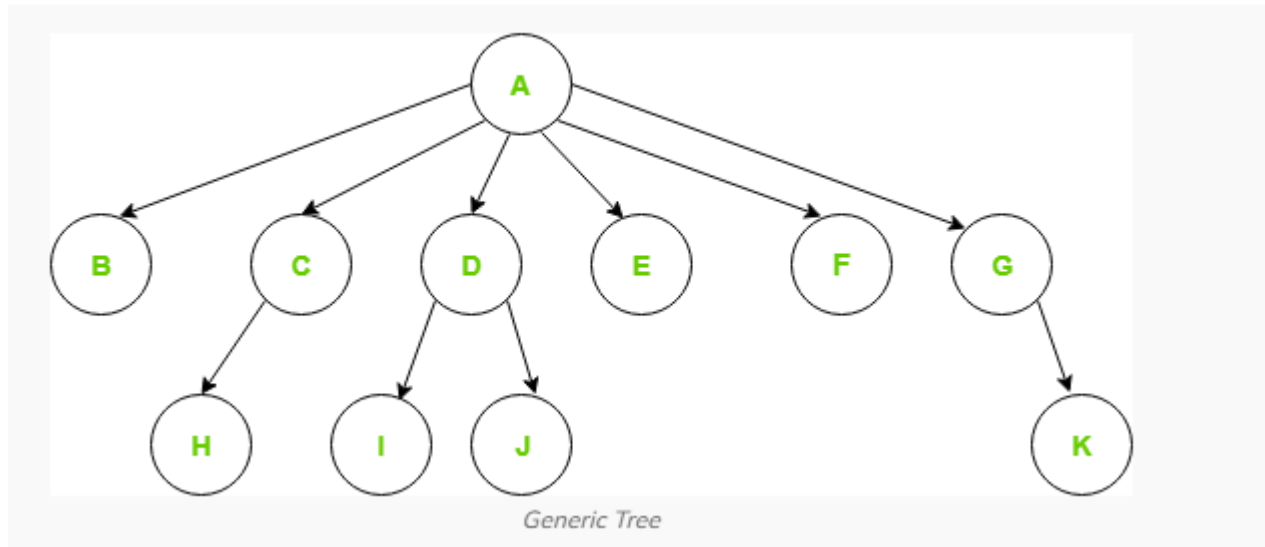
B+ Trees are used in GIS to index spatial data.

N-ARY TREE



N-ary Tree

The N-ary tree is a tree that allows us to have **n** number of children of a particular node, hence the name **N-ary**, making it **slightly complex than the very common binary trees** that allow us to have at most 2 children of a particular node.



To represent the above tree, we have to consider the worst case, that is the node with maximum children (in above example, 6 children) and allocate that many pointers for each node.

- ✓ Generic trees are a collection of nodes where each node is a data structure that consists of records and a list of references to its children(duplicate references are not allowed).
- ✓ Unlike the linked list, each node stores the address of multiple nodes. Every node stores address of its children and the very first node's address will be stored in a separate pointer called root.
- ✓ The Generic trees are the N-ary trees which have the following properties:
 1. Many children at every node.
 2. The number of nodes for each node is not known in advance.

Disadvantages of the representation are:

Memory Wastage – All the pointers are not required in all the cases. Hence, there is lot of memory wastage.

Unknown number of children – The number of children for each node is not known in advance.

Simple Approach:

For storing the address of children in a node we can use an array or linked list. But we will face some issues with both of them.

In **Linked list**, we can not randomly access any child's address. So it will be expensive.

In **array**, we can randomly access the address of any child, but we can store only fixed number of children's addresses in it.

Better Approach:

We can use Dynamic Arrays for storing the address of children. We can randomly access any child's address and the size of the vector is also not fixed.

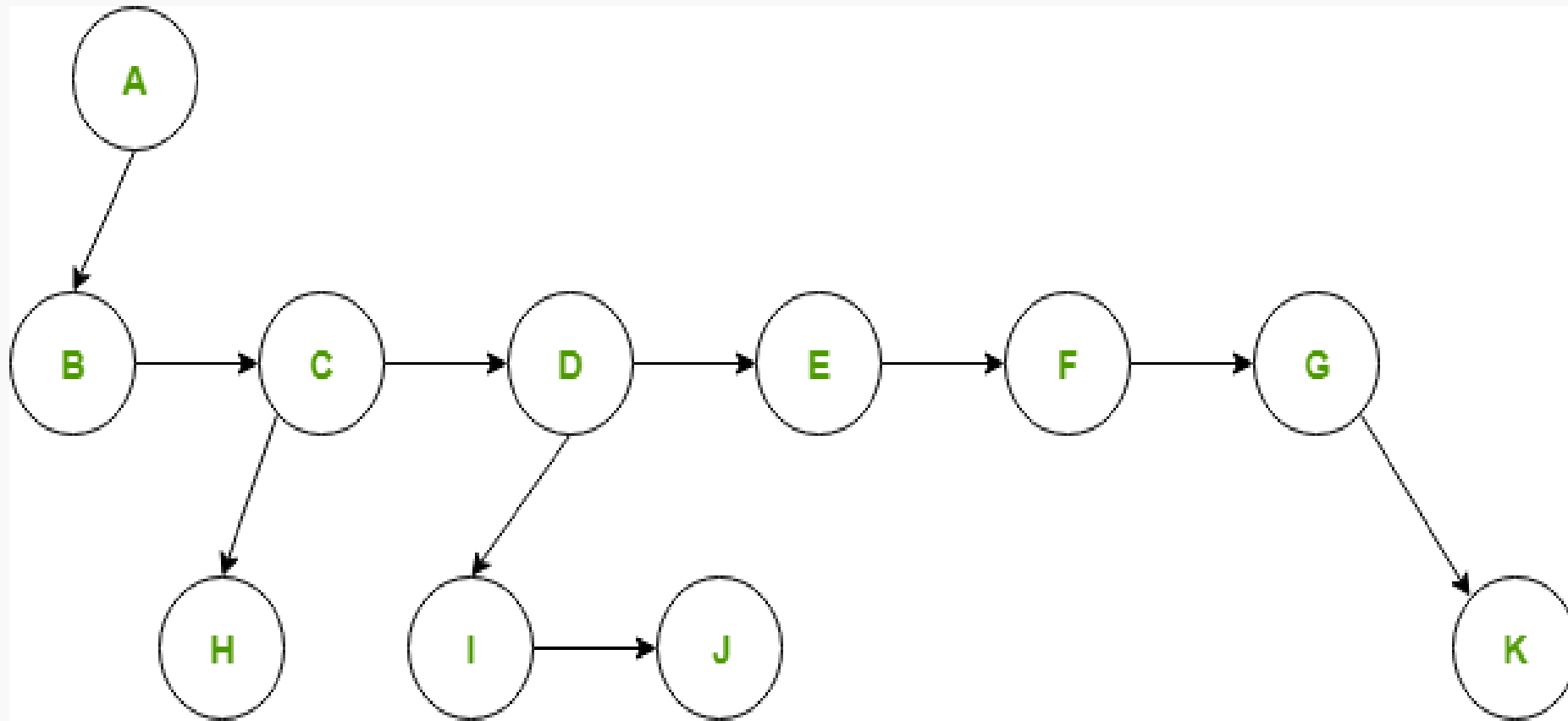
Efficient Approach:

First child / Next sibling representation

In the first child/next sibling representation, the steps taken are:

At each node-link the children of the same parent(siblings) from left to right.

Remove the links from parent to all children except the first child.



HASHING

Hashing :

Hashing is a popular technique in computer science that involves mapping large data sets to fixed-length values. It is a process of converting a data set of variable size into a data set of a fixed size.

Hashing algorithm:

A hashing algorithm is used to convert an input into a fixed-size output (referred to as a hash code or hash value). The data is then stored and retrieved using this hash value as an index in an array or hash table.

Hash Key:-

Hash key (also known as a hash value or hash code) is a fixed-size numerical or alphanumeric representation generated by a hashing algorithm. It is derived from the input data, such as a text string or a file, through a process known as hashing.

UNIT IV



Graphs: Types, Terminology, and Representations.

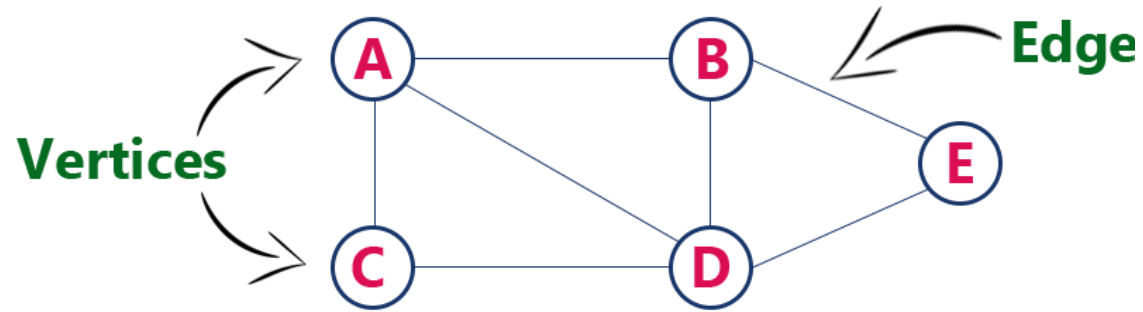
Graph Traversals - BFS and DFS.

Applications of Graphs –

Minimum Spanning Tree, Shortest Path Algorithm.

Graphs

A **graph** is an **advanced data structure** that is used to organize items in an **interconnected network**. Each item in a graph is known as a **node**(or **vertex**) and these nodes are connected by **edges**.



Generally, a graph **G** is represented as $G = (V, E)$, where **V** is set of **vertices** and **E** is set of **edges**.

Example

The following is a graph with 5 vertices and 7 edges.

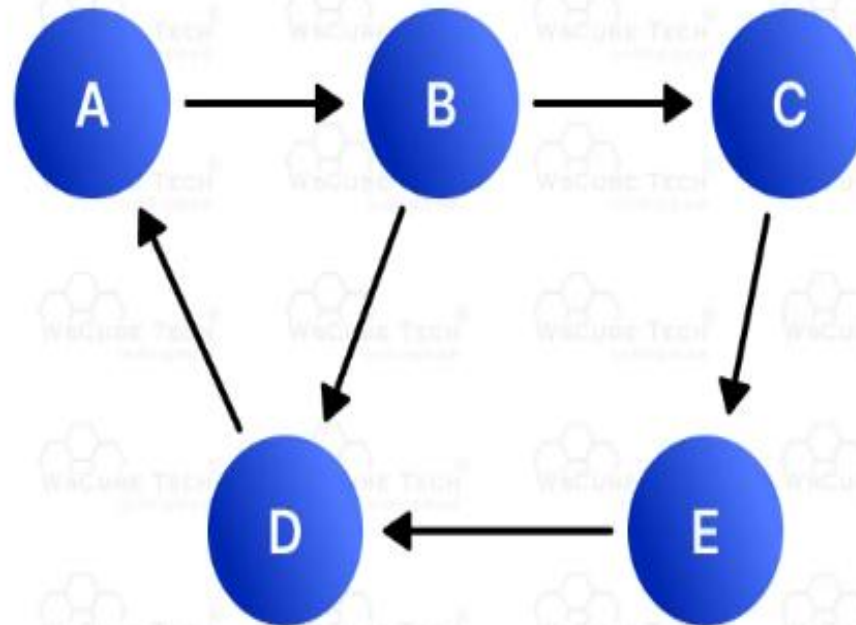
This graph **G** can be defined as $G = (V, E)$

Where $V = \{A, B, C, D, E\}$ and $E = \{(A, B), (A, C), (A, D), (B, D), (C, D), (B, E), (E, D)\}$

1. Directed Graph

In a directed graph, edges have a direction, meaning they go from one node to another in a specific way.

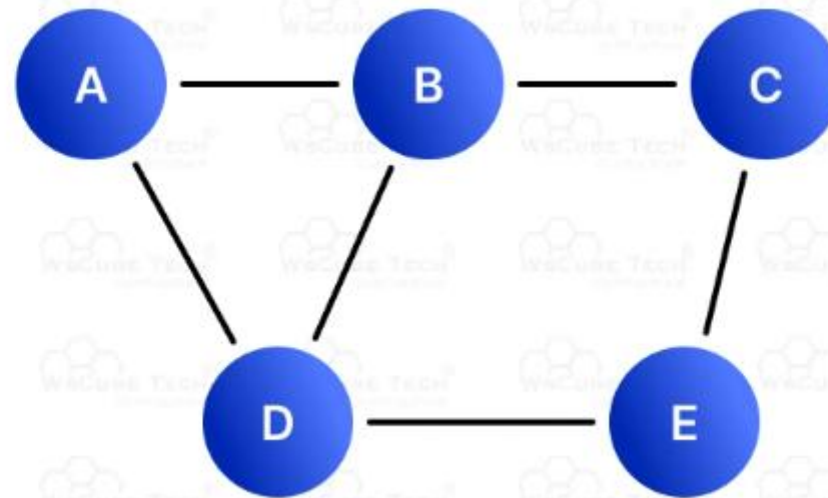
Directed Graph



2. Undirected Graph

In an undirected graph, edges do not have a direction. They simply connect two nodes without any particular order.

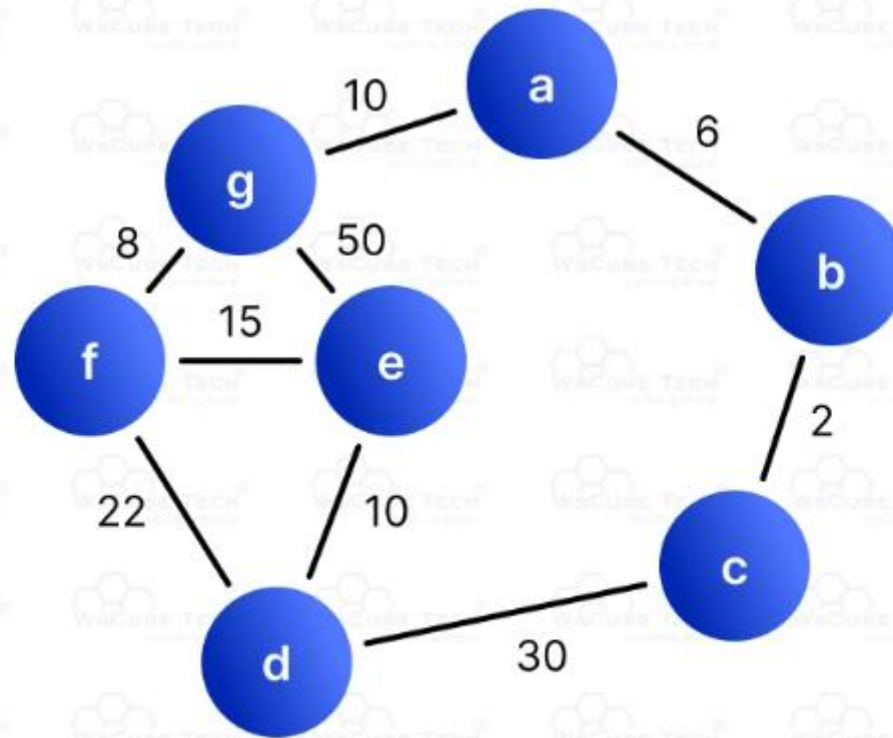
Undirected Graph



3. Weighted Graph

In a weighted graph, edges have weights or costs associated with them. These weights can represent distances, costs, or any other metric.

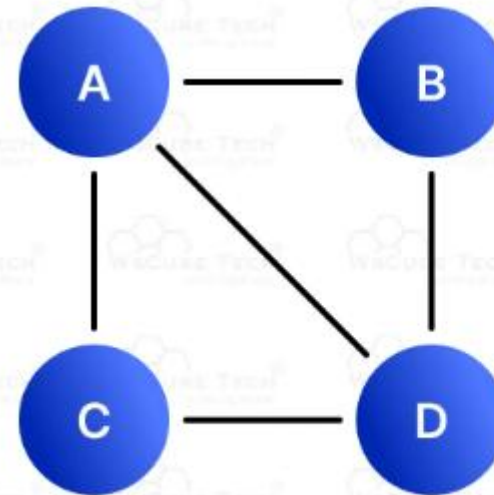
Weighted Graph



4. Unweighted Graph

In an unweighted graph, all edges have the same weight, typically considered as 1.

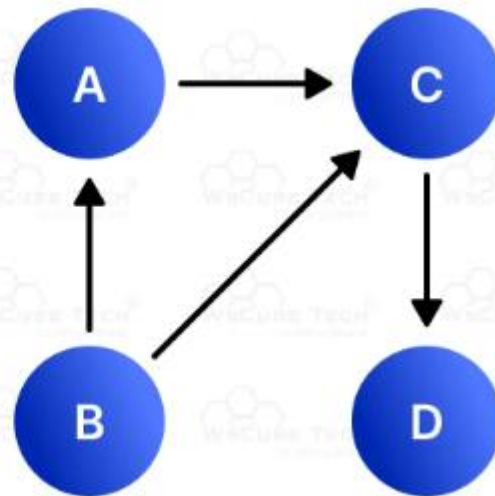
Unweighted Graph



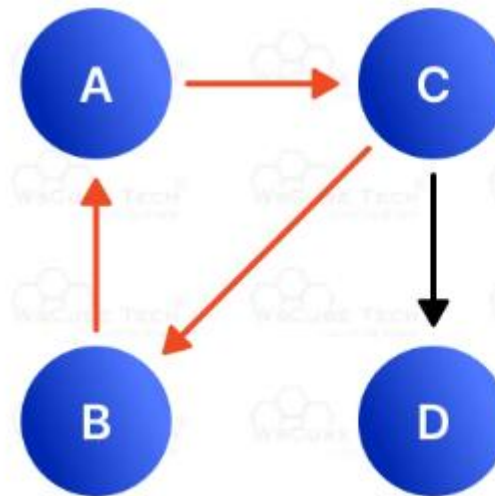
5. Cyclic Graph

A cyclic graph contains at least one cycle, meaning you can start at a node and follow a path that leads back to the same node

Acyclic



Cyclic



6. Acyclic Graph

An acyclic graph does not contain any cycles.

Example:

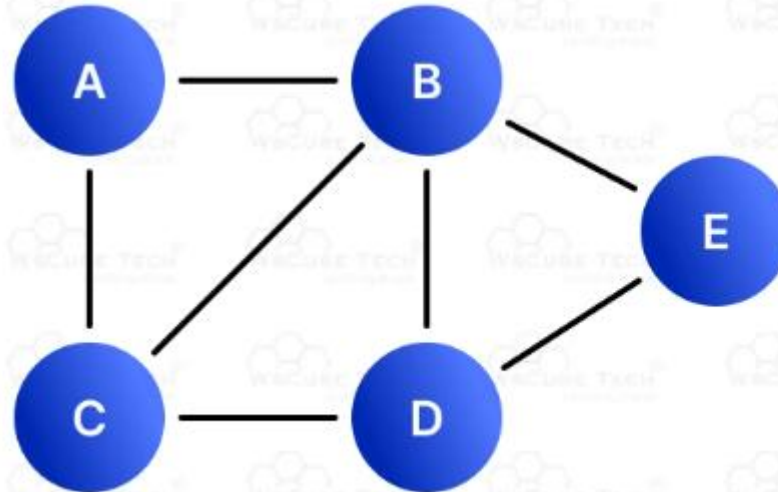
A family tree where no child node points back to its ancestors.



7. Connected Graph

A connected graph has a path between every pair of nodes.

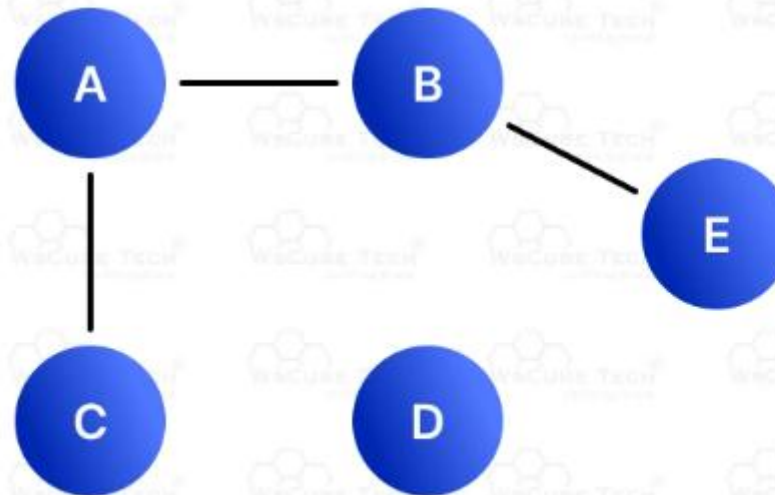
Connected Graph



8. Disconnected Graph

A disconnected graph has at least one pair of nodes with no path between them.

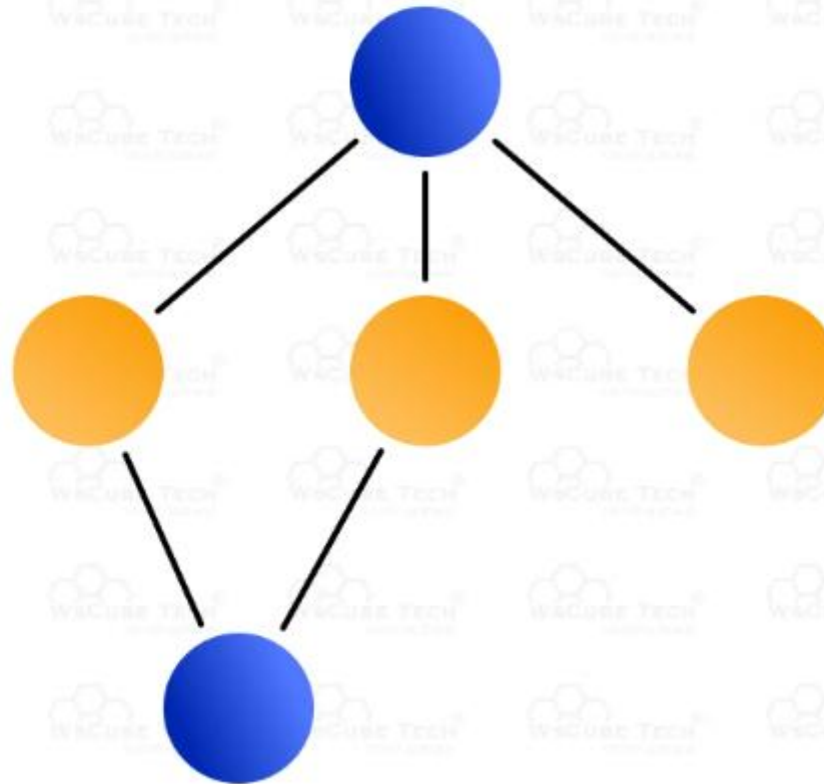
Disconnected Graph



9. Bipartite Graph

A bipartite graph has its nodes divided into two sets such that no two nodes within the same set are adjacent.

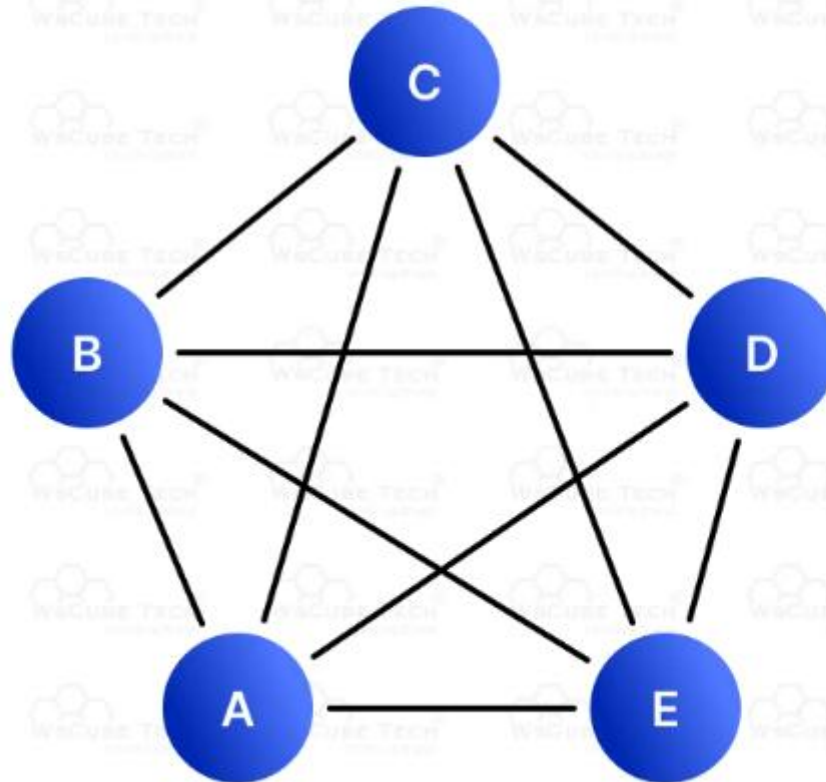
Example Of Bipartite Graph



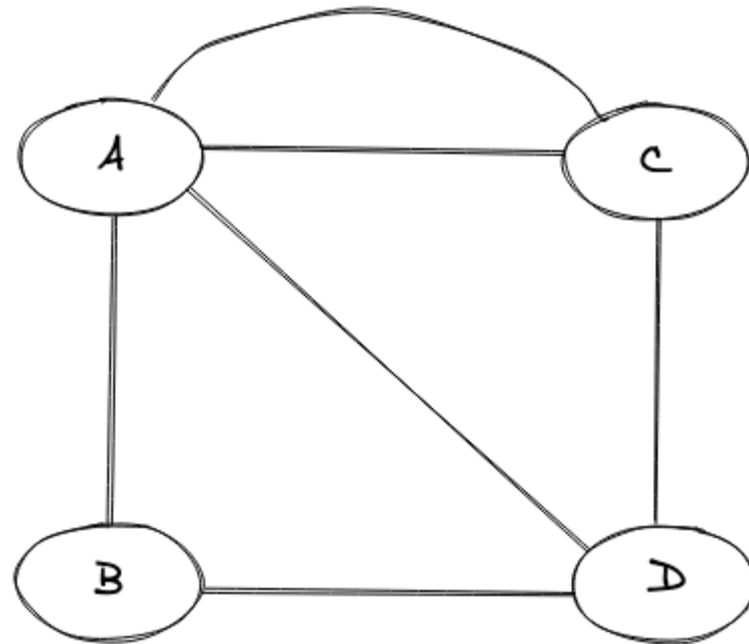
10. Complete Graph

In a complete graph, there is an edge between every pair of nodes.

Complete Graph



Multigraph:- A graph is said to be a multigraph if there exist two or more than two edges between any pair of nodes in the graph.



Graph Terminologies

Path - A sequence of alternating nodes and edges such that each of the successive nodes are connected by the edge.

Cycle - A path where the starting and the ending node is the same.

Simple Path - A path where we do not encounter a vertex again.

Bridge - An edge whose removal will simply make the graph disconnected.

Forest - A forest is a graph without cycles.

Tree - A connected graph that doesn't have any cycles.

Degree - The degree in a graph is the number of edges that are incident on a particular node.

Neighbour - We say vertex "A" and "B" are neighbours if there exists an edge between them.

Applications of Graphs in Data Structure

Graph data structures have a variety of applications.

- It helps to define the flow of computation of software programs.
- Used in Google Maps for building transportation systems. In Google Maps, the intersection of two or more roads represents the node while the road connecting two nodes represents an edge. Google Maps algorithm uses graphs to calculate the shortest distance between two vertices.
- Used in social networks such as Facebook and LinkedIn.
- Operating Systems use a Resource Allocation Graph where every process and resource acts as a node. While we draw edges from resources to the allocated process.
- Used in the World Wide Web where the web pages represent the nodes.
- Blockchains also use graphs. The nodes store many transactions while the edges connect subsequent blocks.
- Used in modelling data.

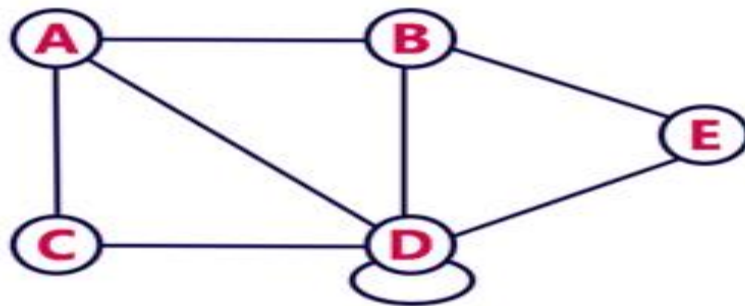
Graph Representations

There are different ways to optimally represent a graph,

Adjacency Matrix

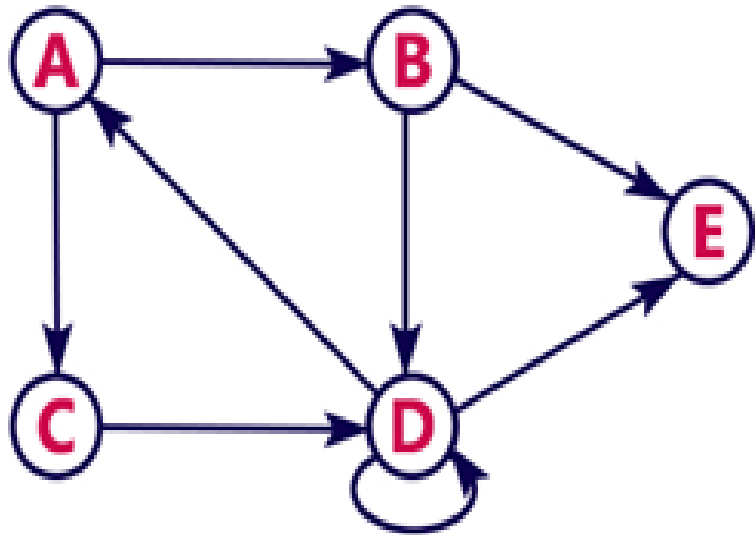
- Adjacency matrix is a sequential representation.
- It is used to represent which nodes are adjacent to each other. i.e. is there any edge connecting nodes to a graph.
- In this representation, we have to construct a $n \times n$ matrix A . If there is any edge from a vertex i to vertex j , then the corresponding element of A , $a^{i,j} = 1$, otherwise $a^{i,j} = 0$.

Consider the following **undirected graph representation**:



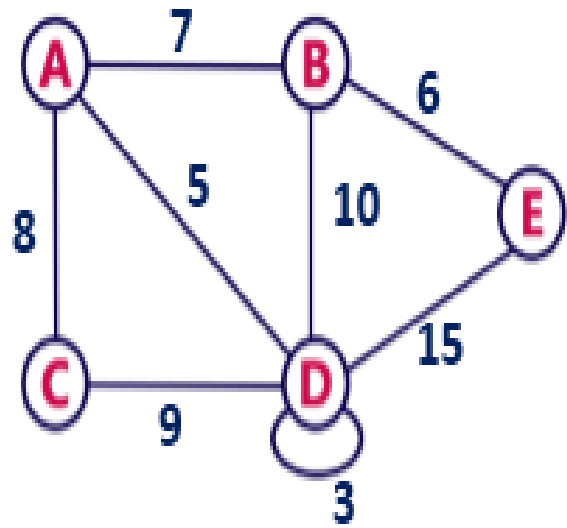
	A	B	C	D	E
A	0	1	1	1	0
B	1	0	0	1	1
C	1	0	0	1	0
D	1	1	1	1	1
E	0	1	0	1	0

Directed graph representation



	A	B	C	D	E
A	0	1	1	0	0
B	0	0	0	1	1
C	0	0	0	1	0
D	1	0	0	1	1
E	0	0	0	0	0

Undirected weighted graph representation



→

	A	B	C	D	E
A	0	7	8	5	0
B	7	0	0	10	6
C	8	0	0	9	0
D	5	10	9	3	15
E	0	6	0	15	0

Incidence Matrix

In **Incidence matrix representation**, graph can be represented using a matrix of size:

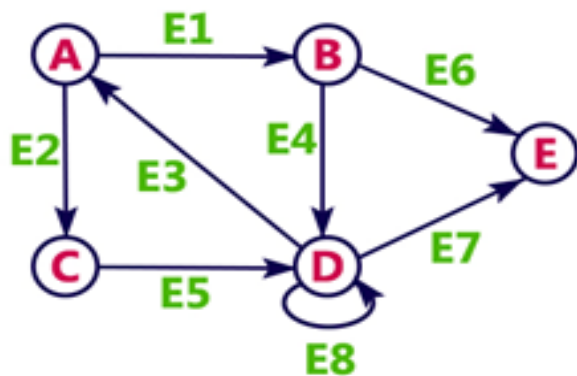
Total number of vertices by total number of edges.

It means if a graph has 5 vertices and 8 edges, then it can be represented using a matrix of 5X8 class. In this matrix, columns represent edges and rows represent vertices.

This matrix is filled with either **0** or **1** or **-1**. Where,

- 0 is used to represent row edge which is not connected to column vertex.
- 1 is used to represent row edge which is connected as outgoing edge to column vertex.
- 1 is used to represent row edge which is connected as incoming edge to column vertex.

directed graph representation.

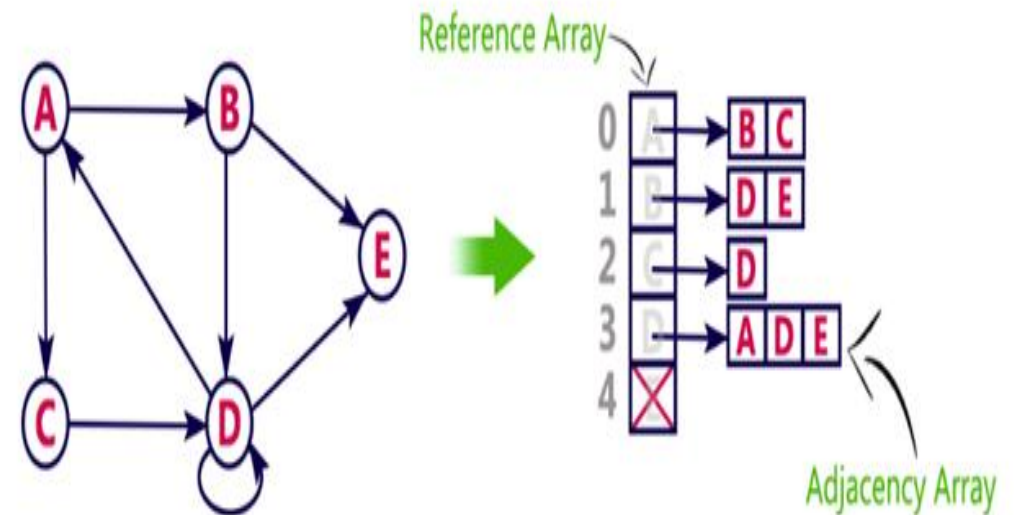
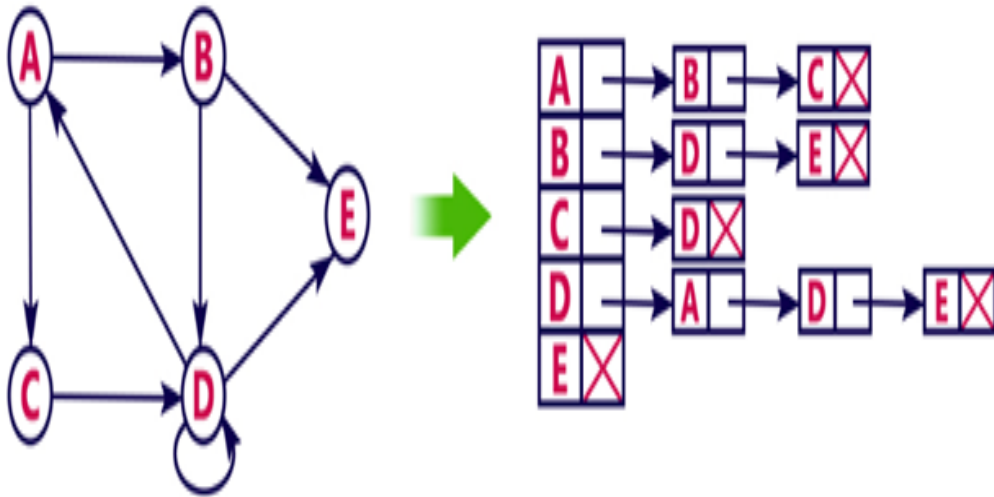


	E1	E2	E3	E4	E5	E6	E7	E8
A	1	1	-1	0	0	0	0	0
B	-1	0	0	1	0	1	0	0
C	0	-1	0	0	1	0	0	0
D	0	0	1	-1	-1	0	1	1
E	0	0	0	0	0	-1	-1	0

Adjacency List

- Adjacency list is a linked representation.
- In this representation, for each vertex in the graph, we maintain the list of its neighbors. It means, every vertex of the graph contains list of its adjacent vertices.
- We have an array of vertices which is indexed by the vertex number and for each vertex v , the corresponding array element points to a **singly linked list** of neighbors of v .

directed graph representation implemented using linked list & array







GRAPH TRAVERSALS

Graph Traversal

- ✓ Graph traversal is a technique used for searching a vertex in a graph.
- ✓ The graph traversal is also used to decide the order of vertices visited in the search process.
- ✓ A graph traversal finds the edges to be used in the search process without creating loops.
- ✓ That means using graph traversal we visit all the vertices of the graph without getting into a looping path.

There are two graph traversal techniques and they are as follows...

DFS (Depth First Search)

BFS (Breadth First Search)

DFS

DFS (Depth First Search)

DFS traversal of a graph produces a **spanning tree** as final result. **Spanning Tree** is a graph without loops. We use **Stack data structure** with maximum size of total number of vertices in the graph to implement DFS traversal.

Algorithm steps to implement DFS traversal...

Step 1 - Define a Stack of size total number of vertices in the graph.

Step 2 - Select any vertex as **starting point** for traversal. Visit that vertex and push it on to the Stack.

Step 3 - Visit any one of the non-visited **adjacent** vertices of a vertex which is at the top of stack and push it on to the stack.

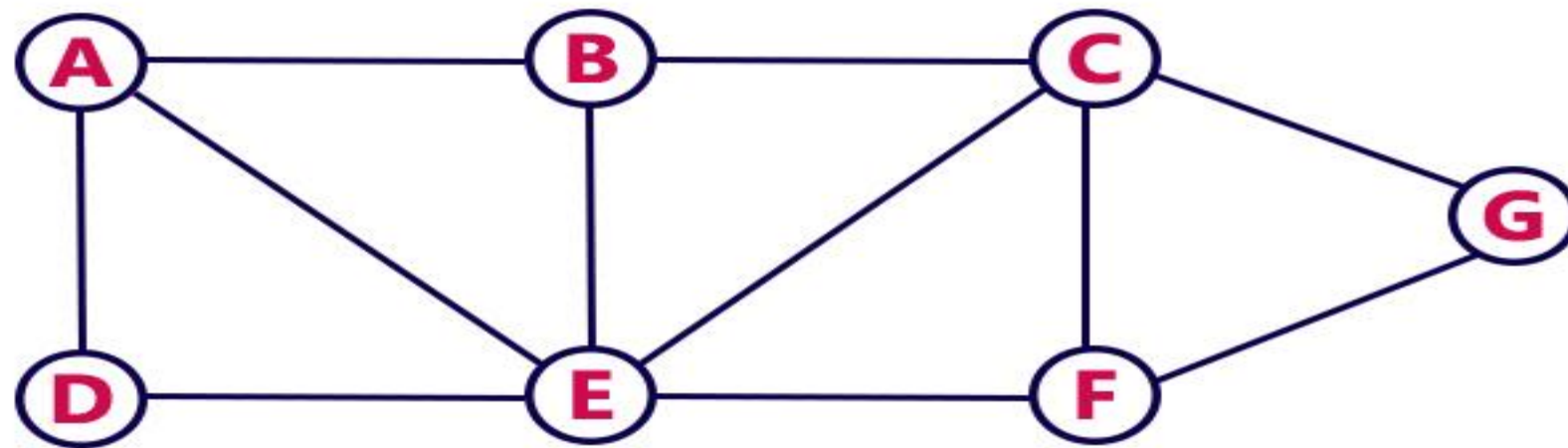
Step 4 - Repeat step 3 until there is no new vertex to be visited from the vertex which is at the top of the stack.

Step 5 - When there is no new vertex to visit then use **back tracking** and pop one vertex from the stack.

Step 6 - Repeat steps 3, 4 and 5 until stack becomes Empty.

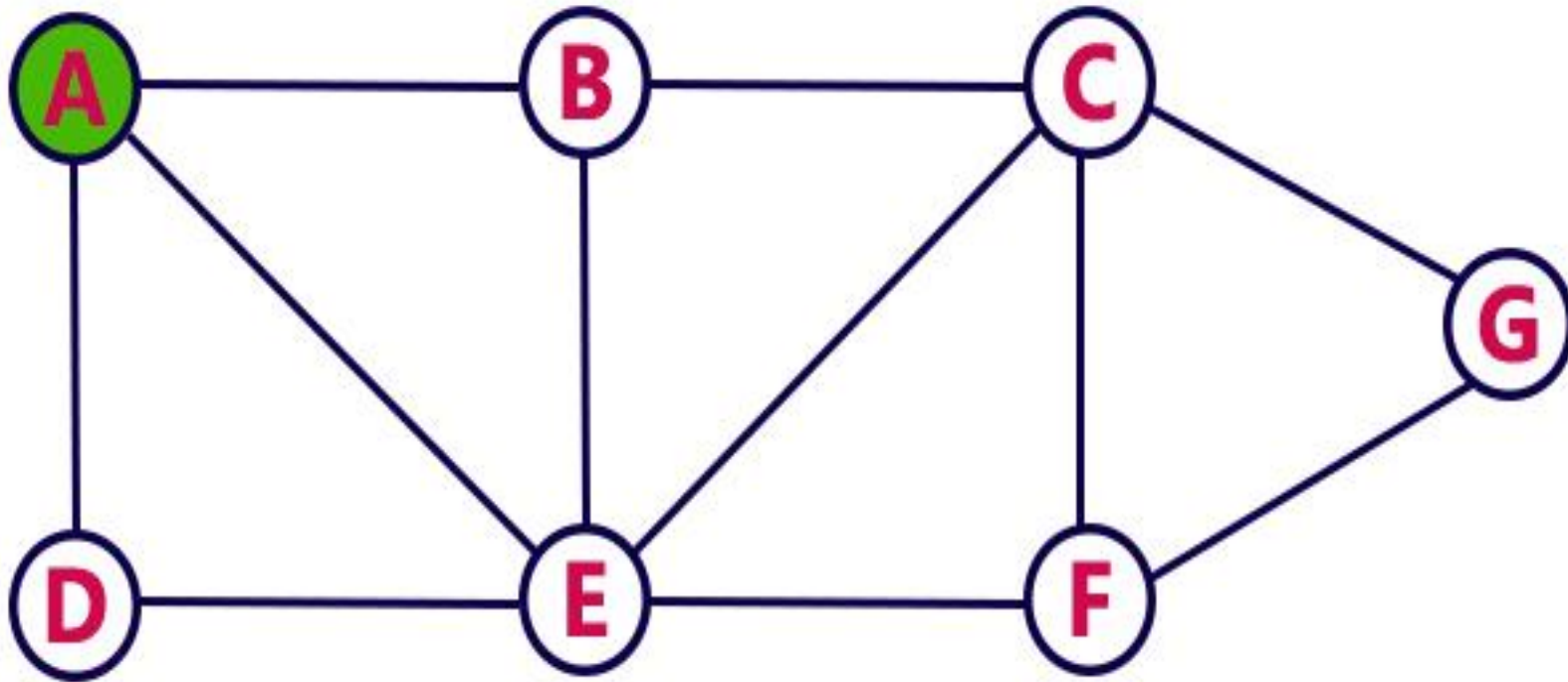
Step 7 - When stack becomes Empty, then produce final spanning tree by removing unused edges from the graph

Consider the following example graph to perform DFS traversal



Step 1:

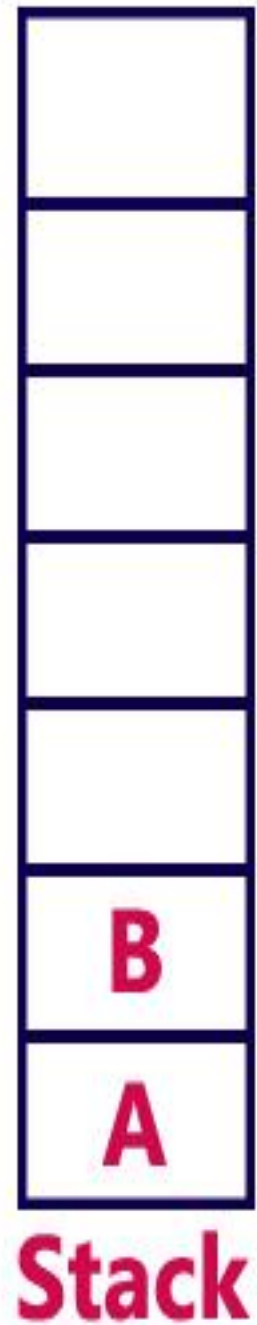
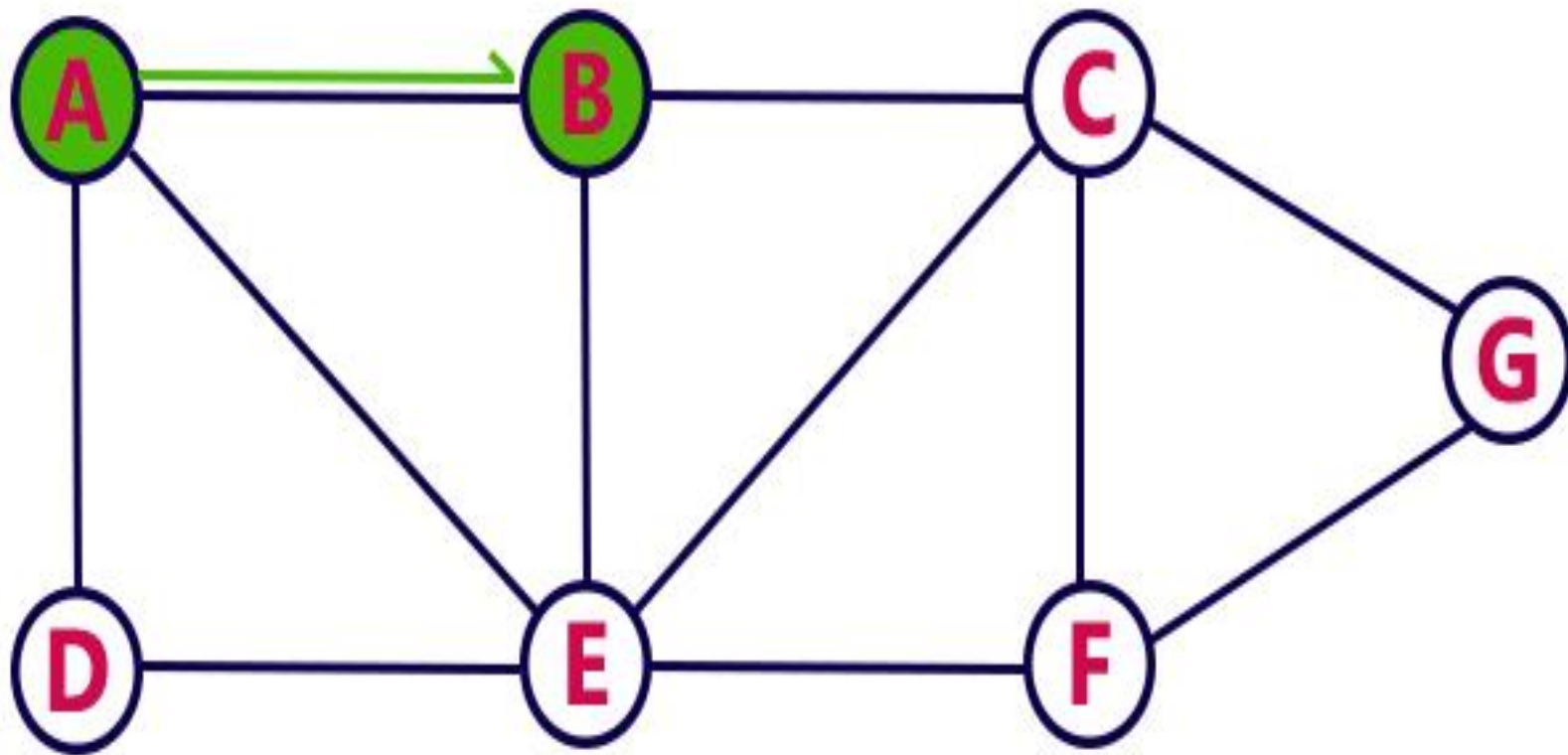
- Select the vertex **A** as starting point (visit **A**).
- Push **A** on to the Stack.



Stack

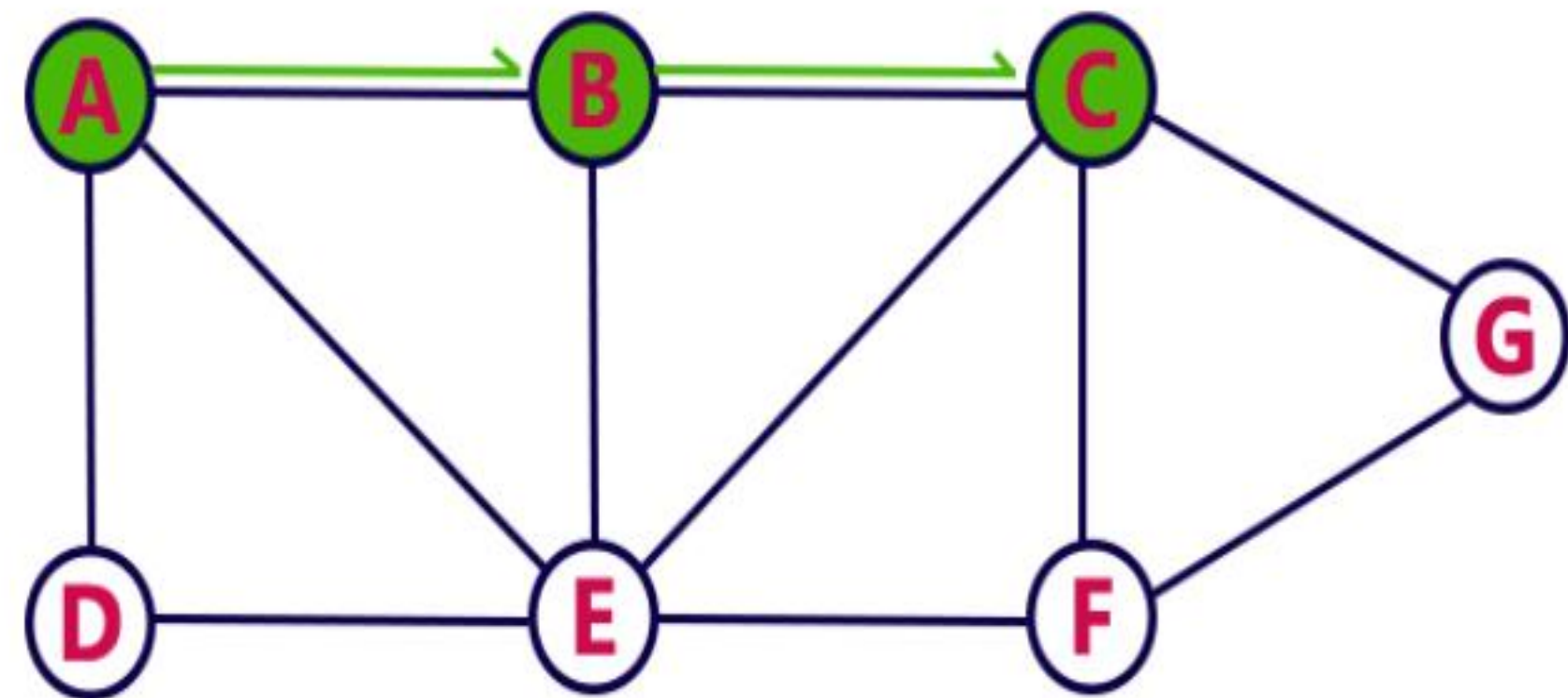
Step 2:

- Visit any adjacent vertex of **A** which is not visited (**B**).
- Push newly visited vertex B on to the Stack.



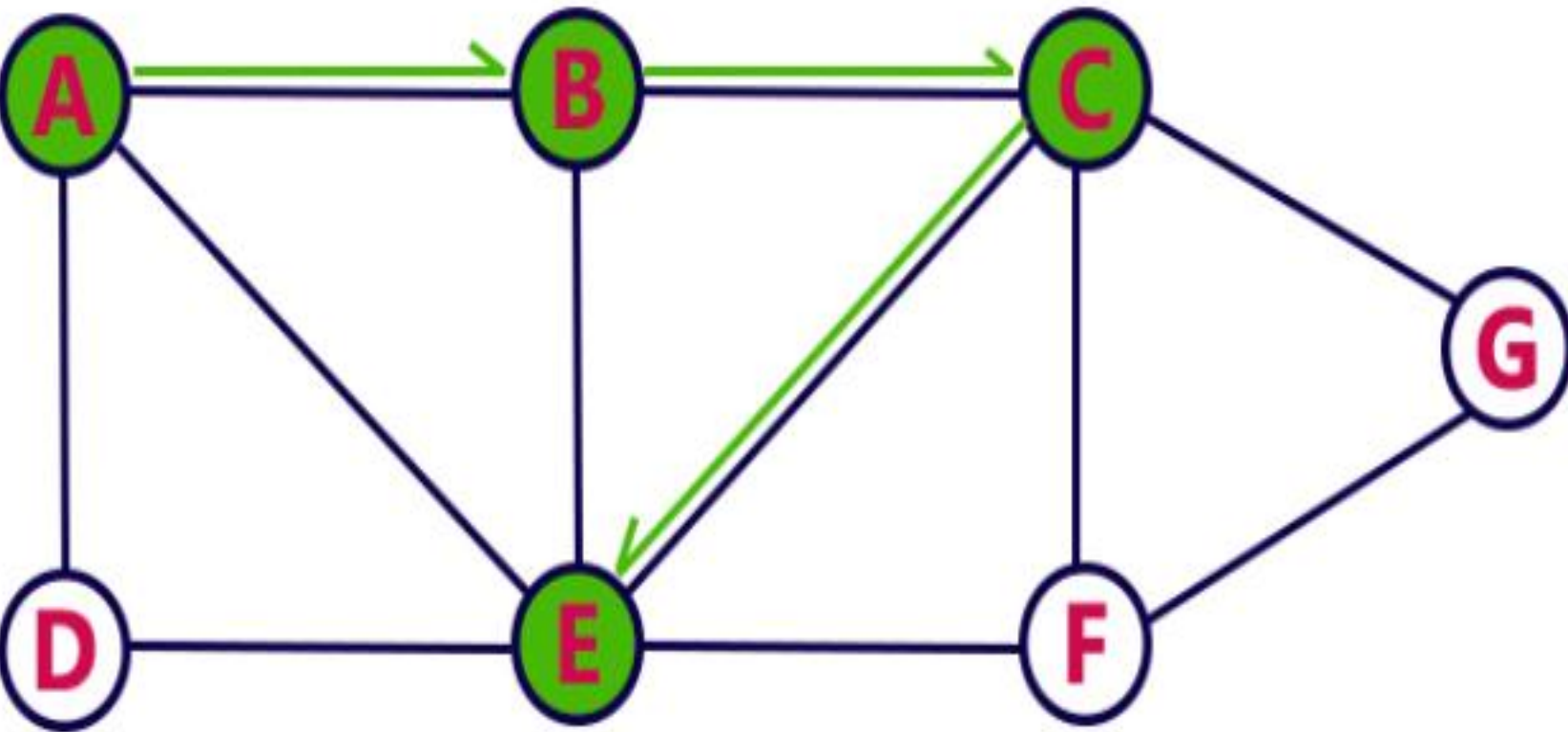
Step 3:

- Visit any adjacent vertex of **B** which is not visited (**C**).
- Push C on to the Stack.



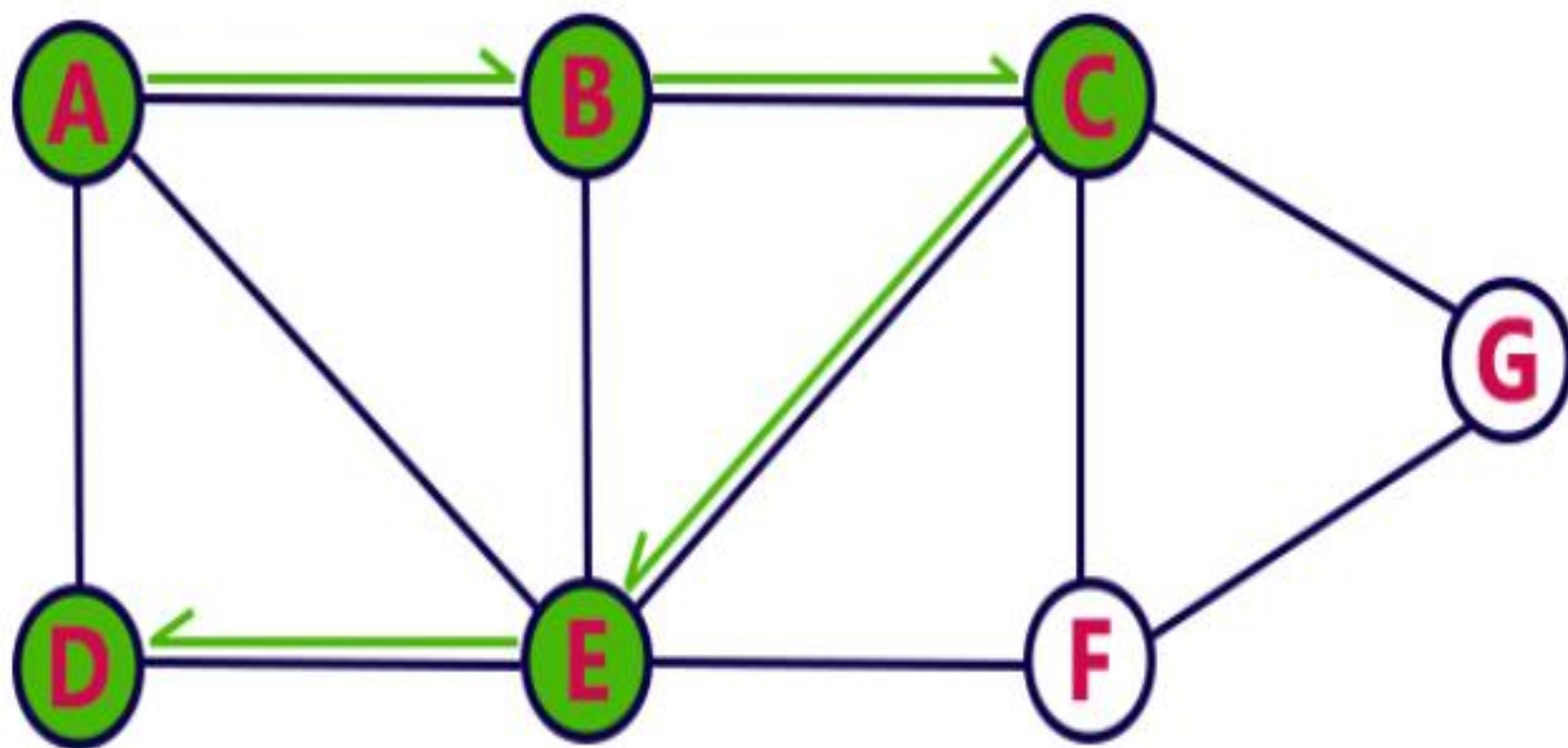
Step 4:

- Visit any adjacent vertex of **C** which is not visited (**E**).
- Push E on to the Stack



Step 5:

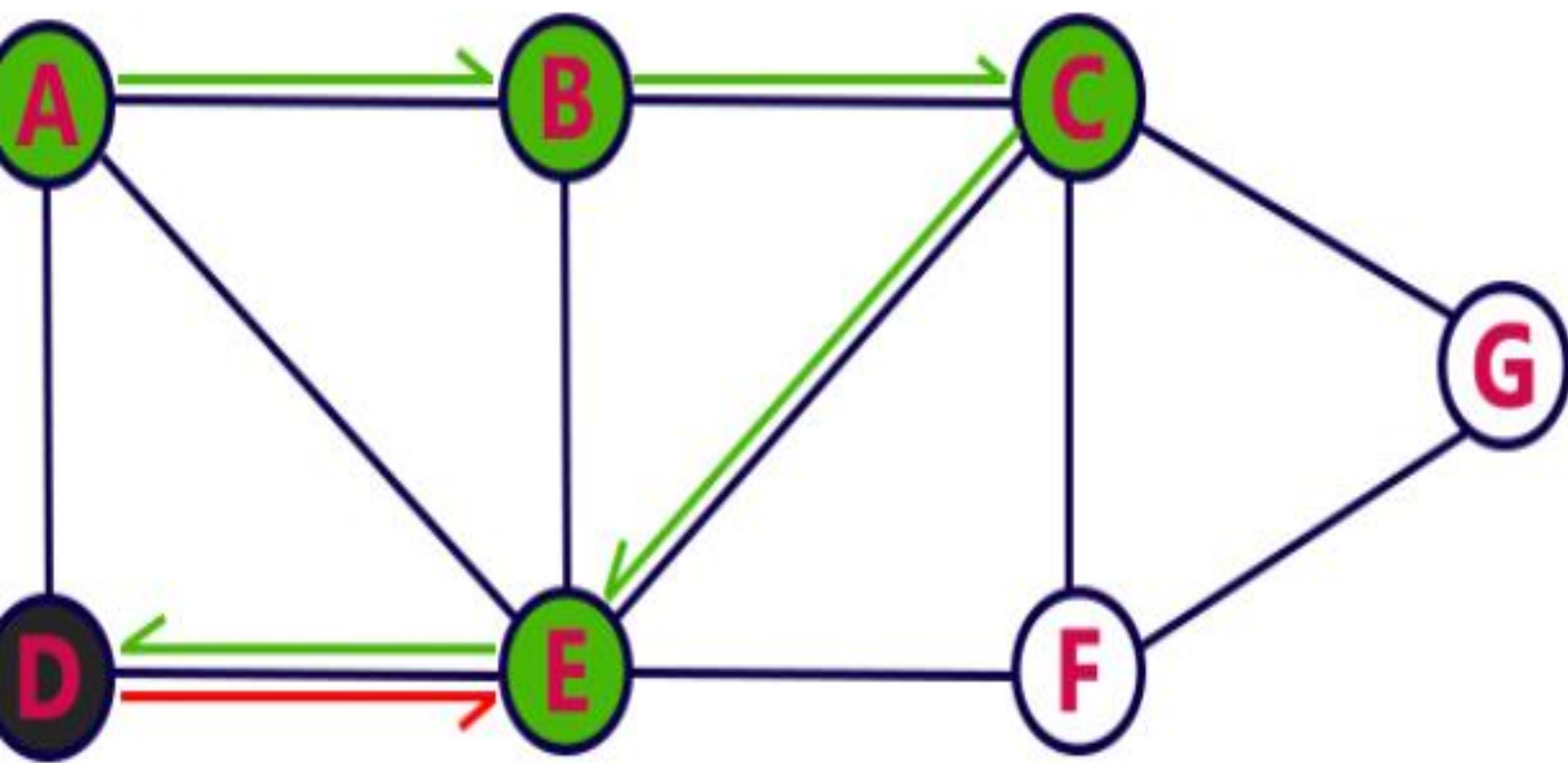
- Visit any adjacent vertex of **E** which is not visited (**D**).
- Push D on to the Stack



D
E
C
B
A
Stack

Step 6:

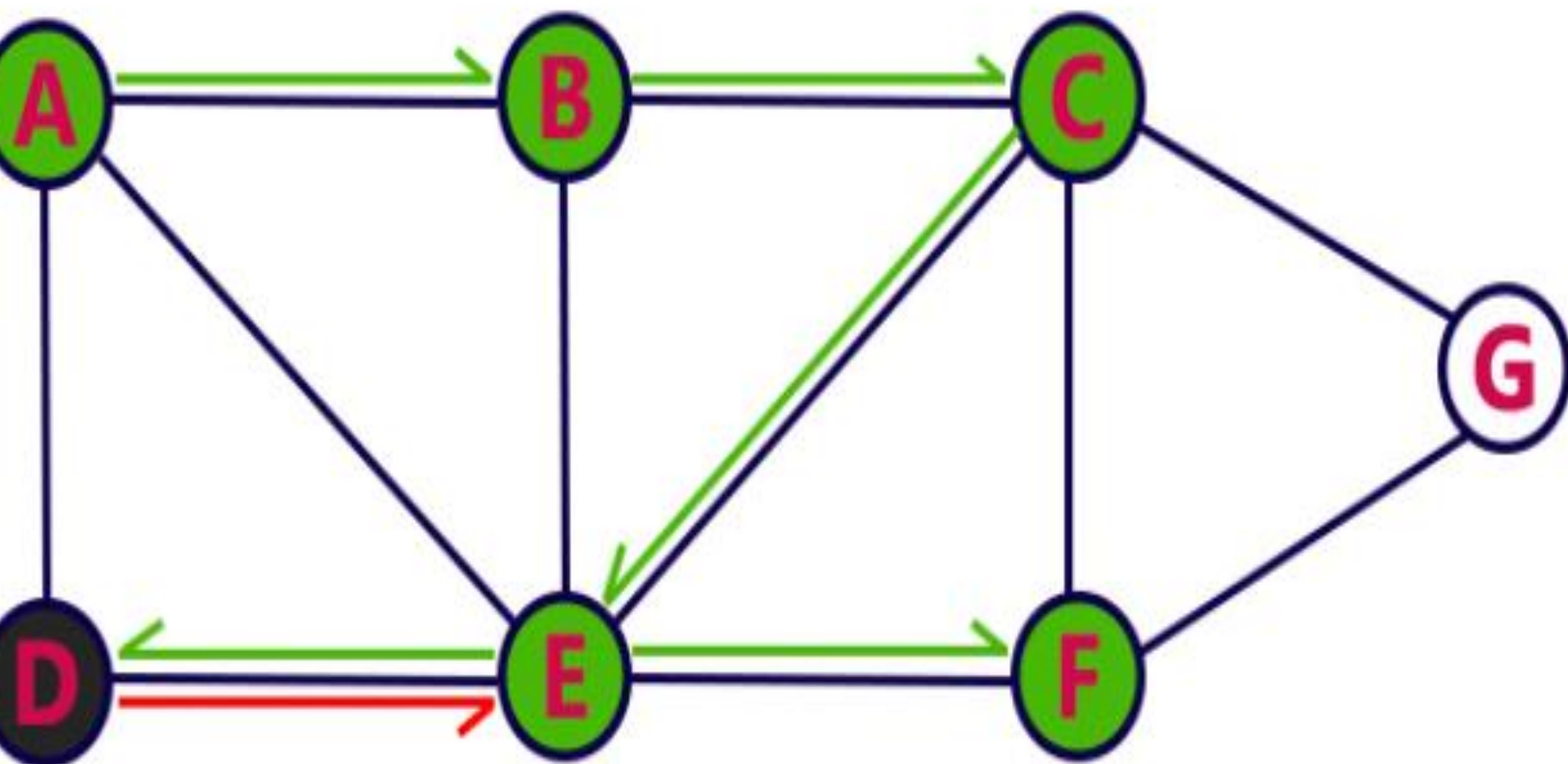
- There is no new vertex to be visited from D. So use back track.
- Pop D from the Stack.



E
C
B
A
Stack

Step 7:

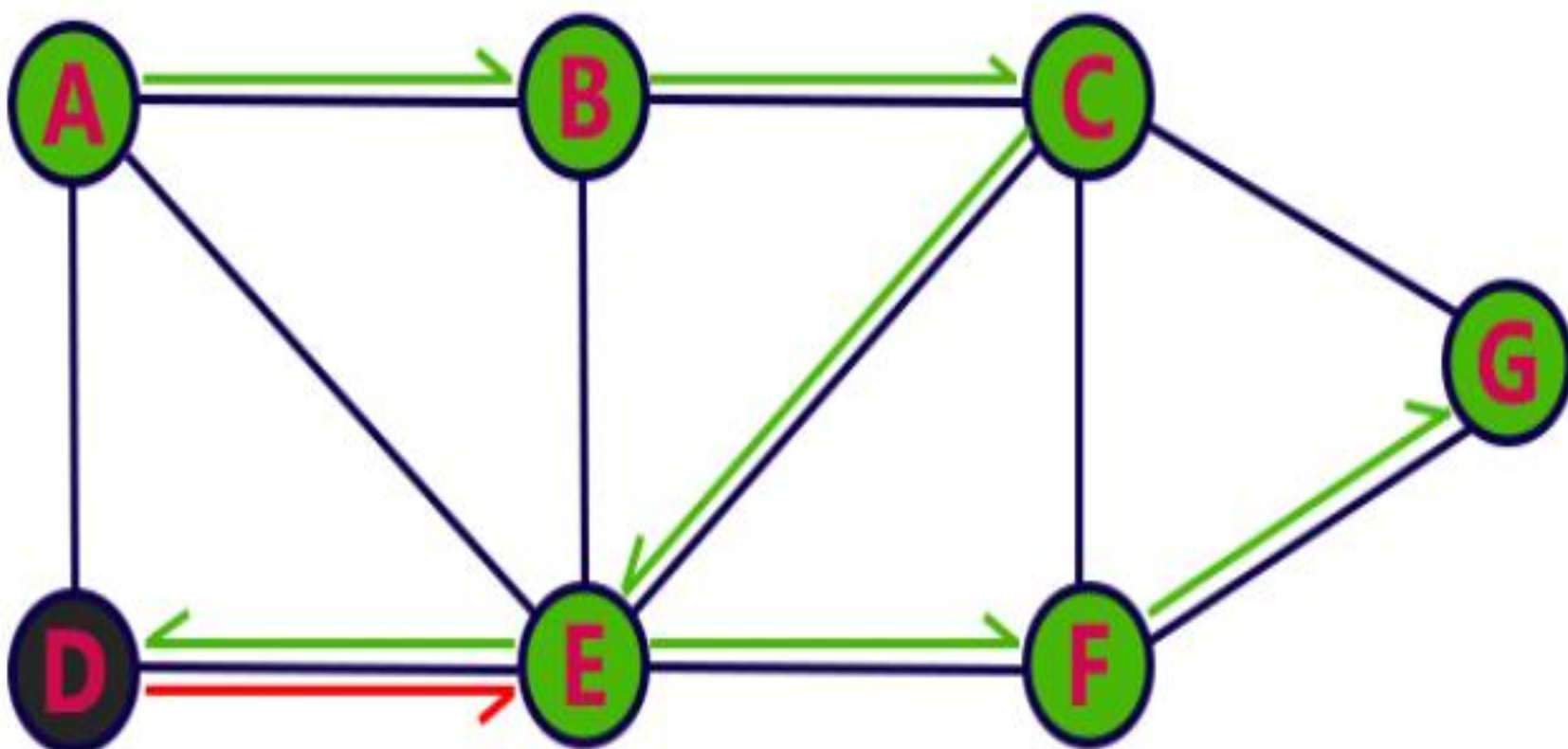
- Visit any adjacent vertex of **E** which is not visited (**F**).
- Push **F** on to the Stack.



F
E
C
B
A
Stack

Step 8:

- Visit any adjacent vertex of **F** which is not visited (**G**).
- Push **G** on to the Stack.



G

F

E

C

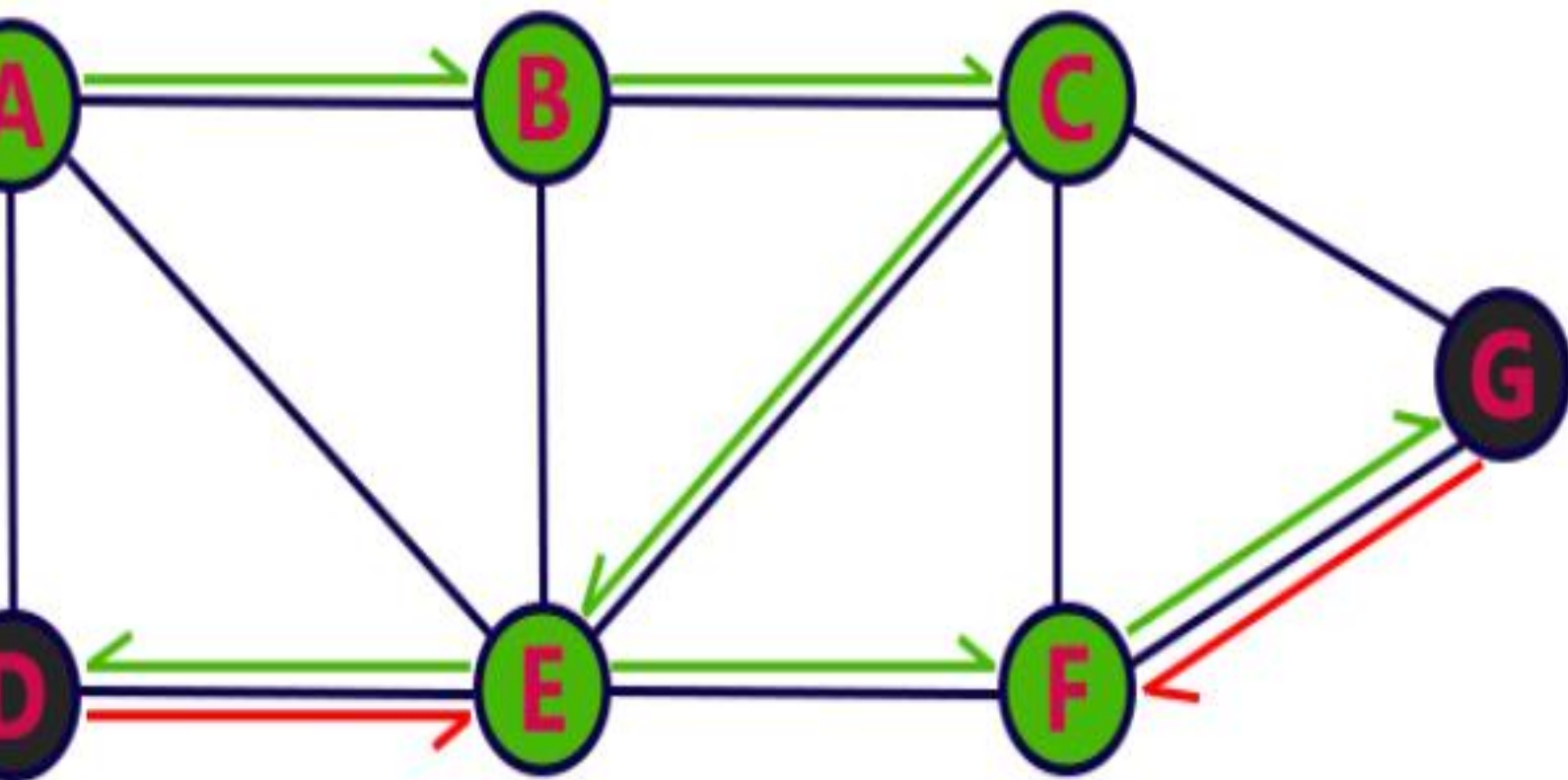
B

A

Stack

Step 9:

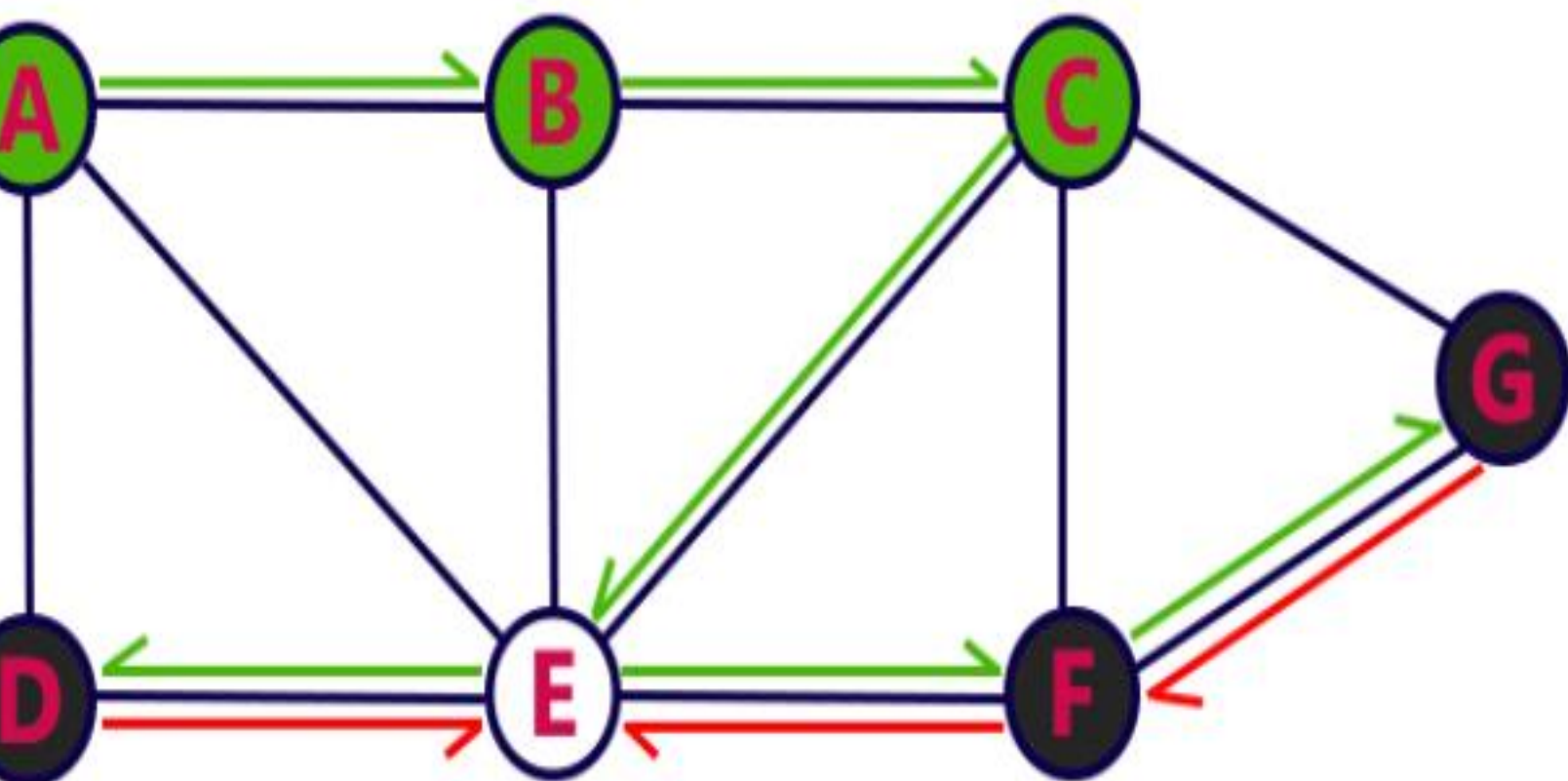
- There is no new vertex to be visited from G. So use back track.
- Pop G from the Stack.



F
E
C
B
A
Stack

Step 10:

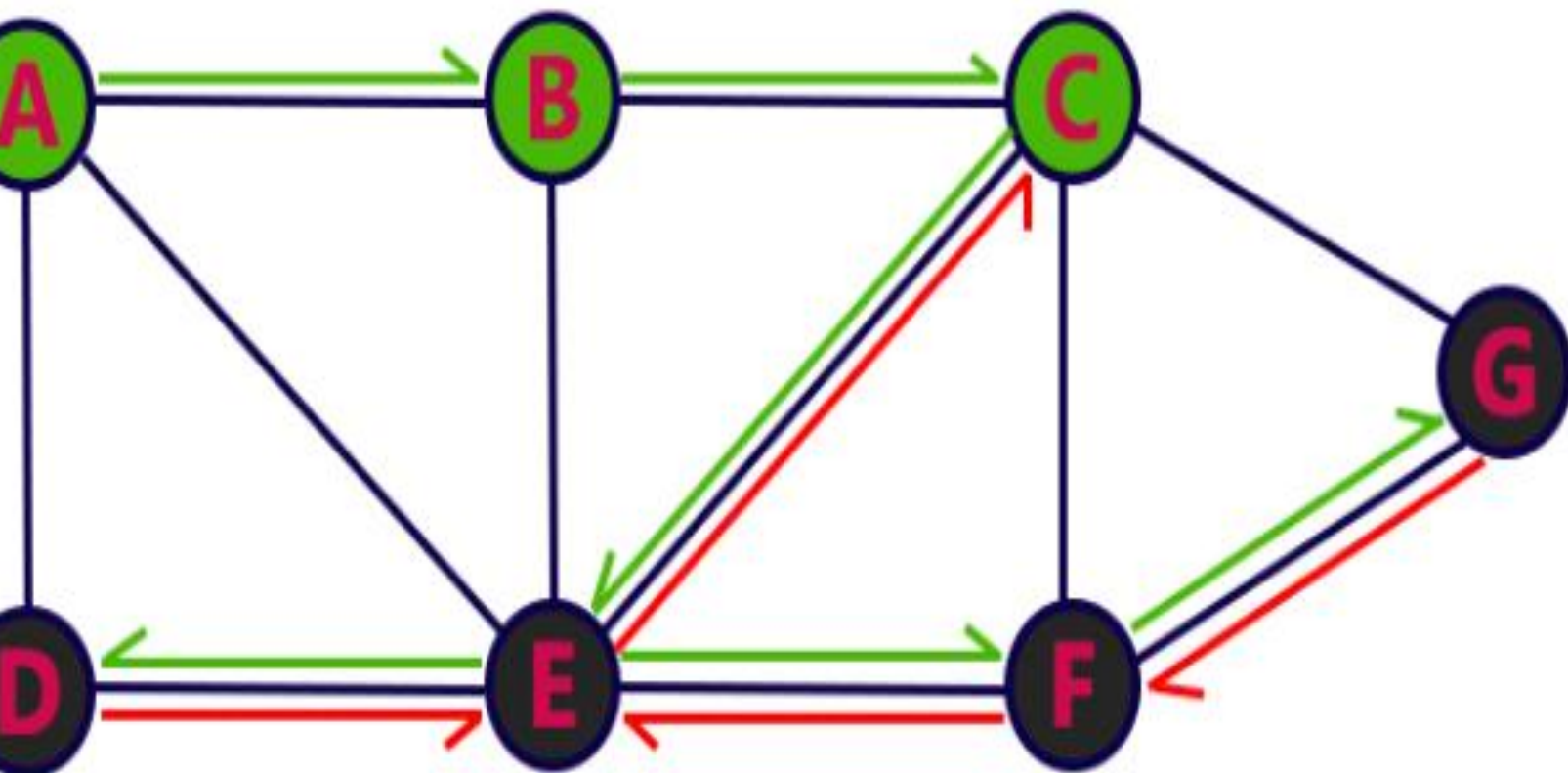
- There is no new vertex to be visited from F. So use back track.
- Pop F from the Stack.



Sta

Step 11:

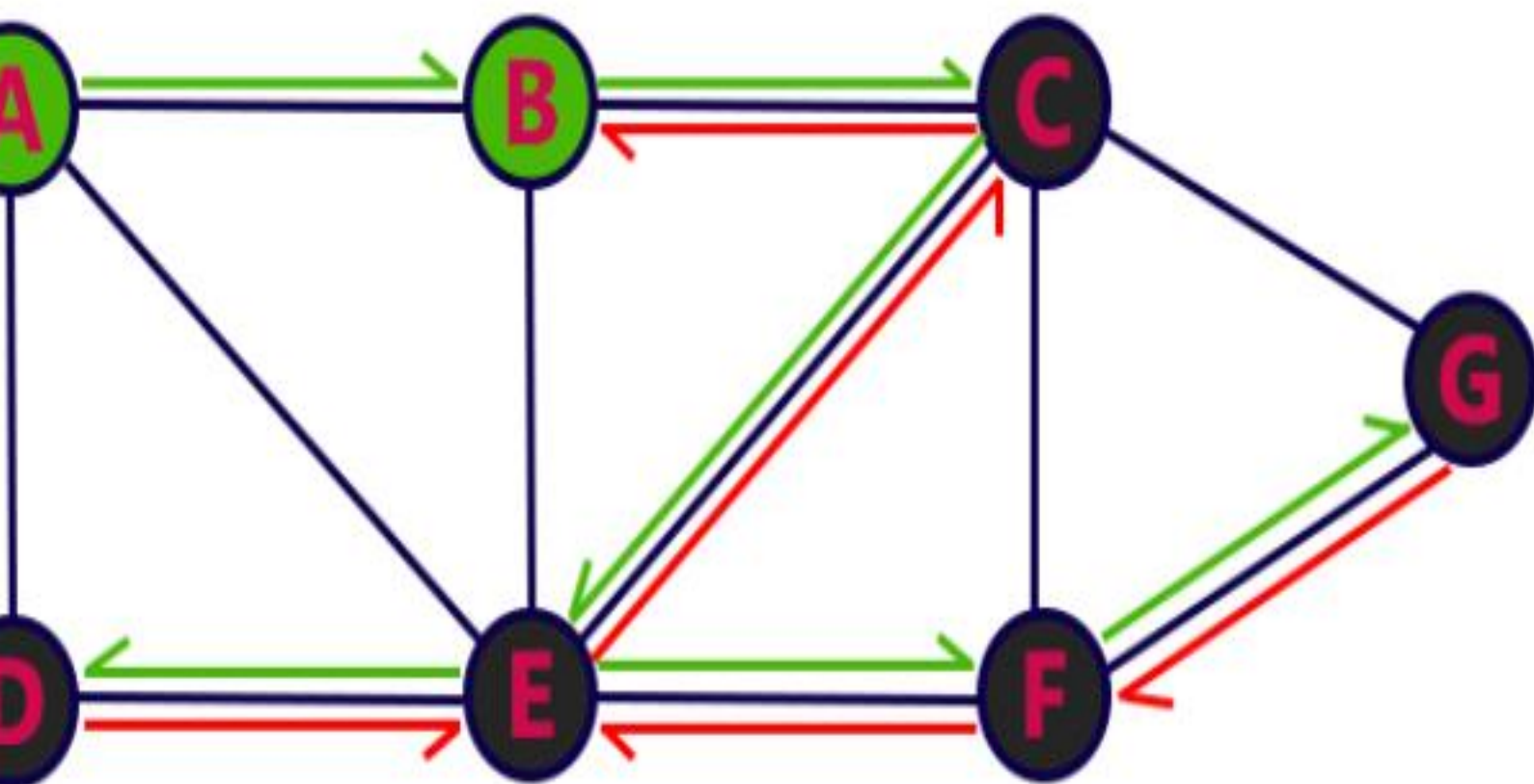
- There is no new vertex to be visited from E. So use back track.
- Pop E from the Stack.



C
B
A
Sta

12:

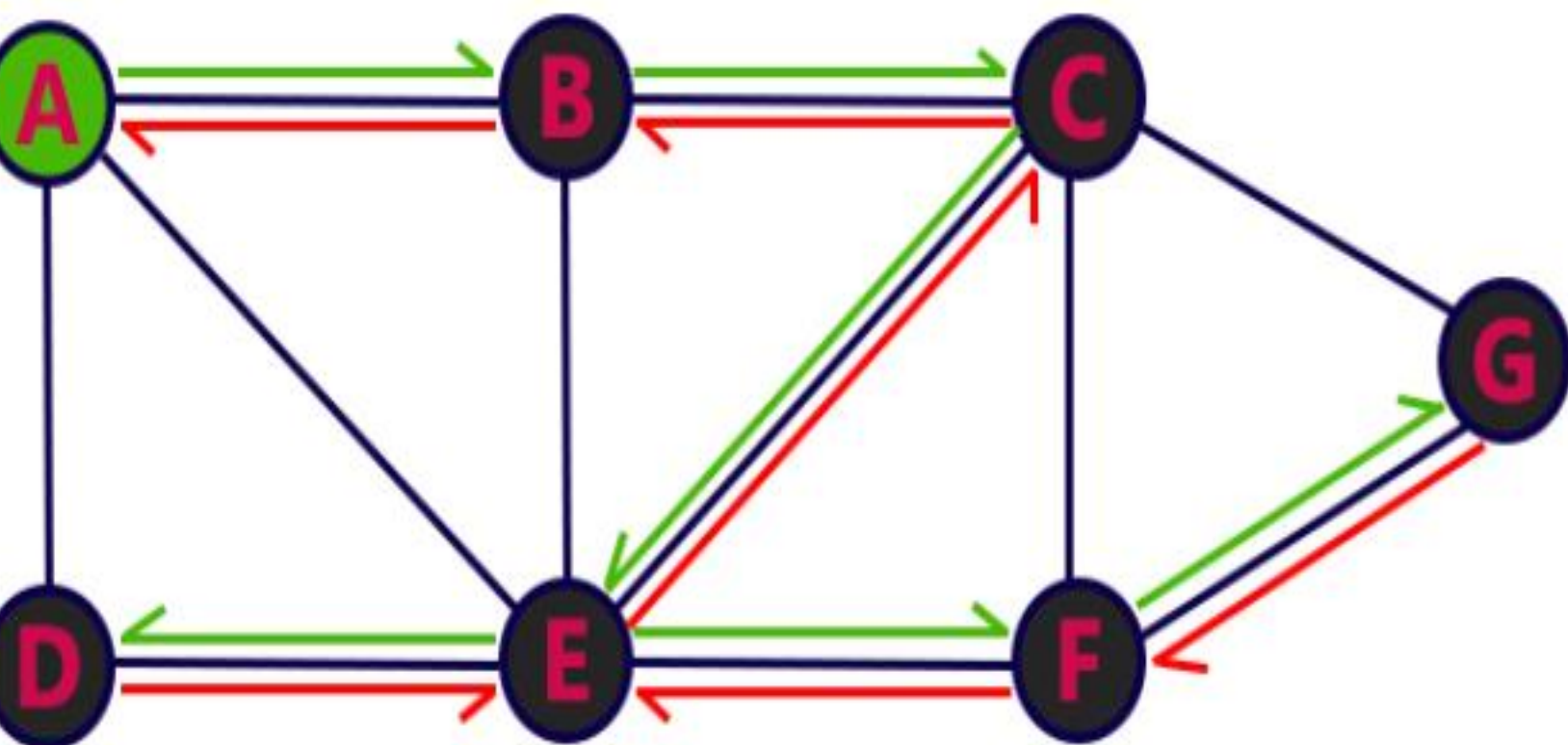
- There is no new vertex to be visited from C. So use back track.
- Pop C from the Stack.



B
A
Stack

Step 13:

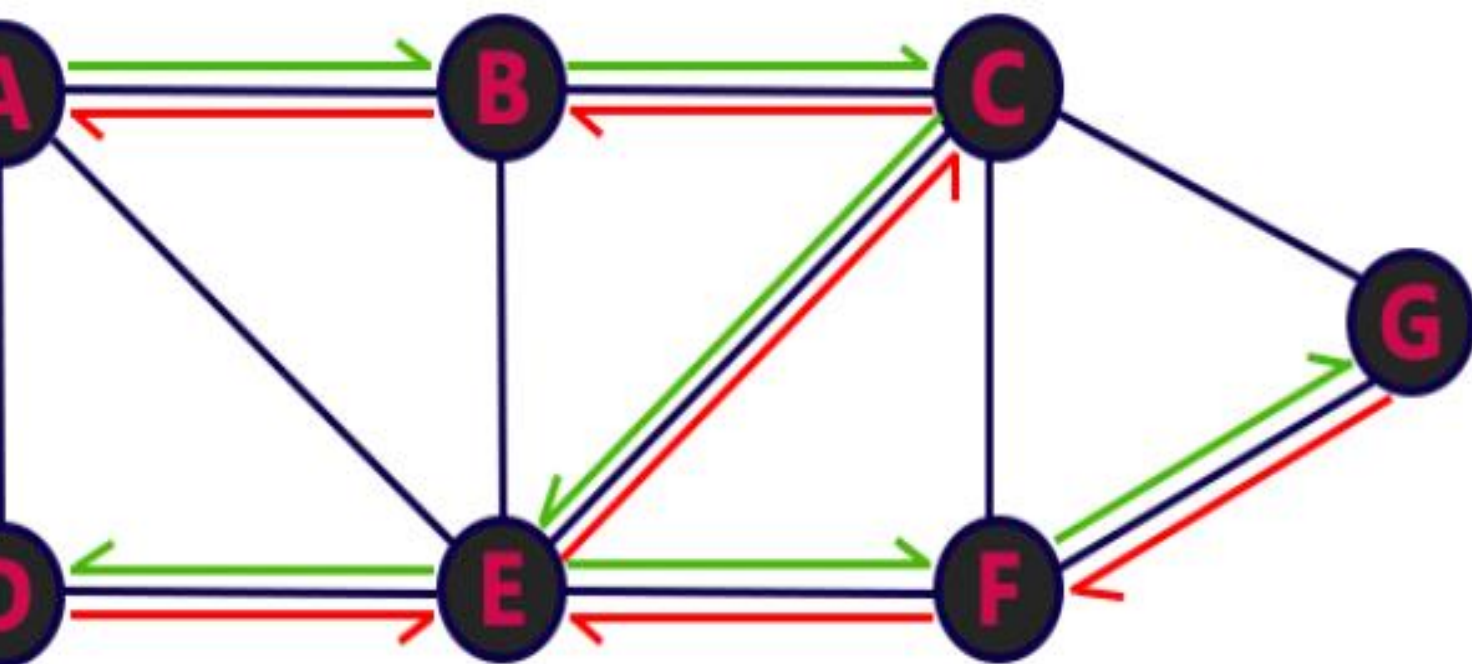
- There is no new vertex to be visited from B. So use back track.
- Pop B from the Stack.



Sta

14:

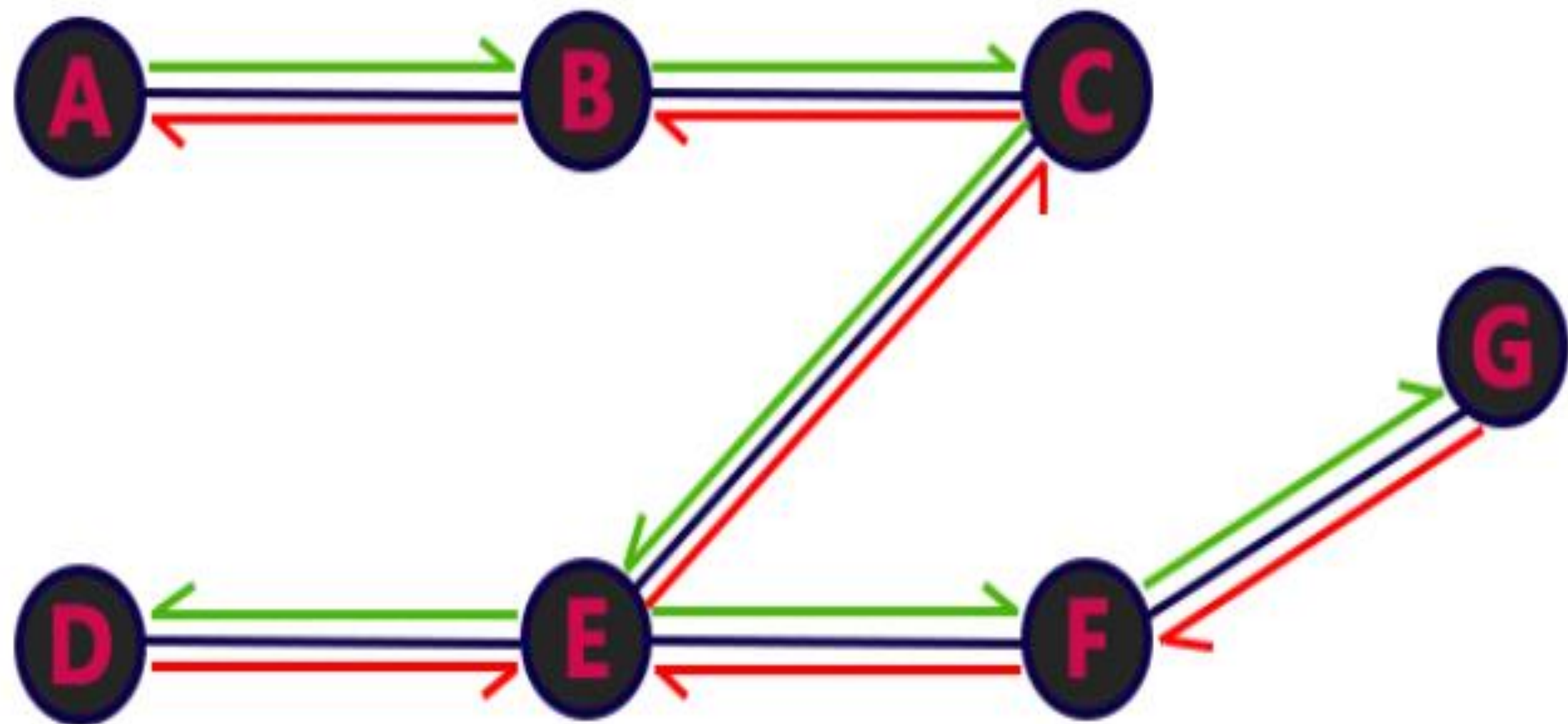
- There is no new vertex to be visited from A. So use back track.
- Pop A from the Stack.



Stack

- Stack became Empty. So stop DFS Traversal.

Final result of DFS traversal is following spanning tree.



BFS

BFS (Breadth First Search)

BFS traversal of a graph produces a **spanning tree** as final result. **Spanning Tree** is a graph without loops. We use **Queue data structure** with maximum size of total number of vertices in the graph to implement BFS traversal.

We use the following steps to implement BFS traversal...

Step 1 - Define a Queue of size total number of vertices in the graph.

Step 2 - Select any vertex as **starting point** for traversal. Visit that vertex and insert it into the Queue.

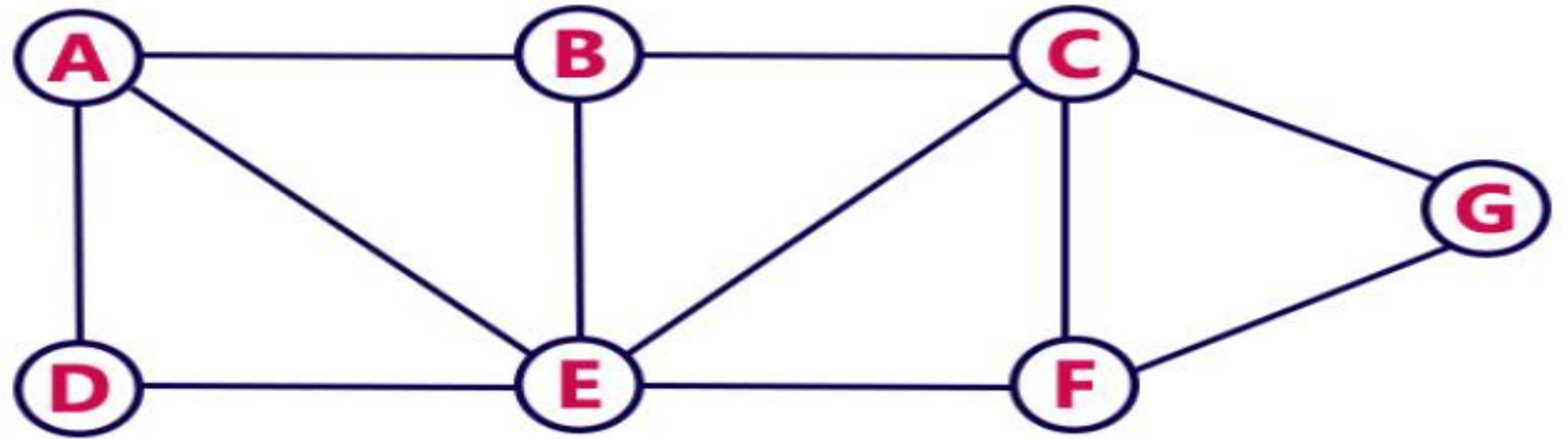
Step 3 - Visit all the non-visited **adjacent** vertices of the vertex which is at front of the Queue and insert them into the Queue.

Step 4 - When there is no new vertex to be visited from the vertex which is at front of the Queue then delete that vertex.

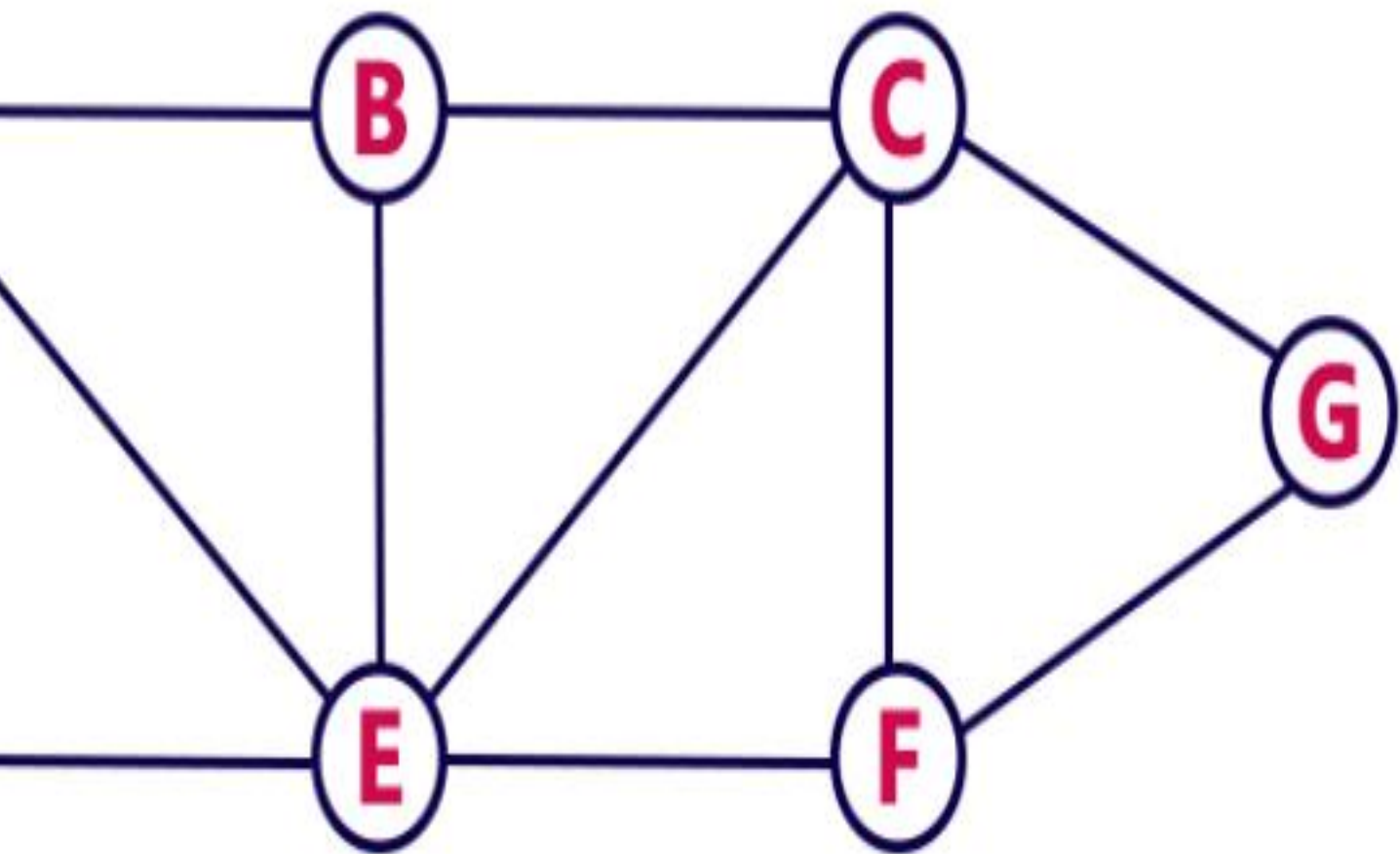
Step 5 - Repeat steps 3 and 4 until queue becomes empty.

Step 6 - When queue becomes empty, then produce final spanning tree by removing unused edges from the graph

Consider the following example graph to perform BFS traversal



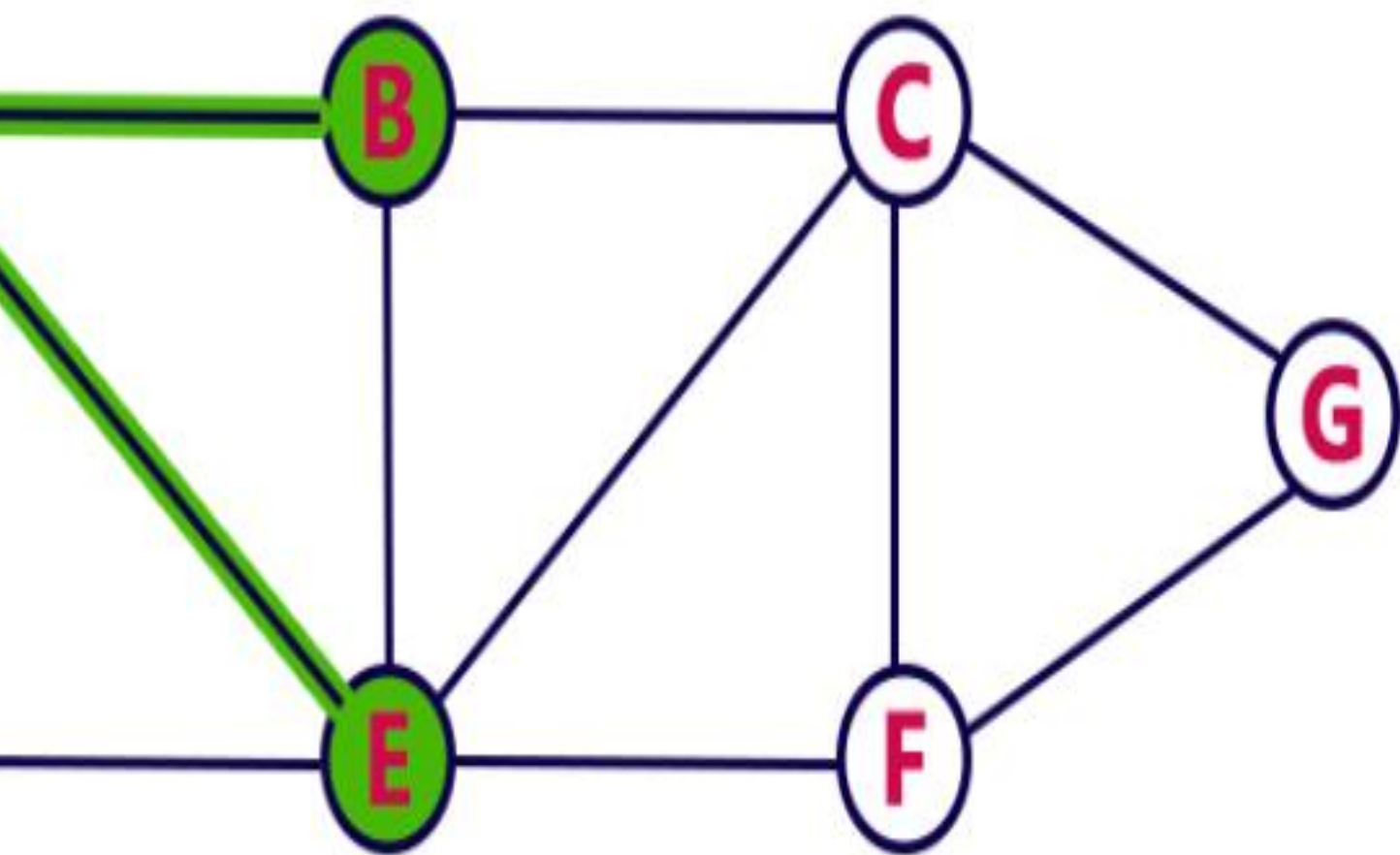
Select the vertex **A** as starting point (visit **A**).
Insert **A** into the Queue.



Queue

A					
---	--	--	--	--	--

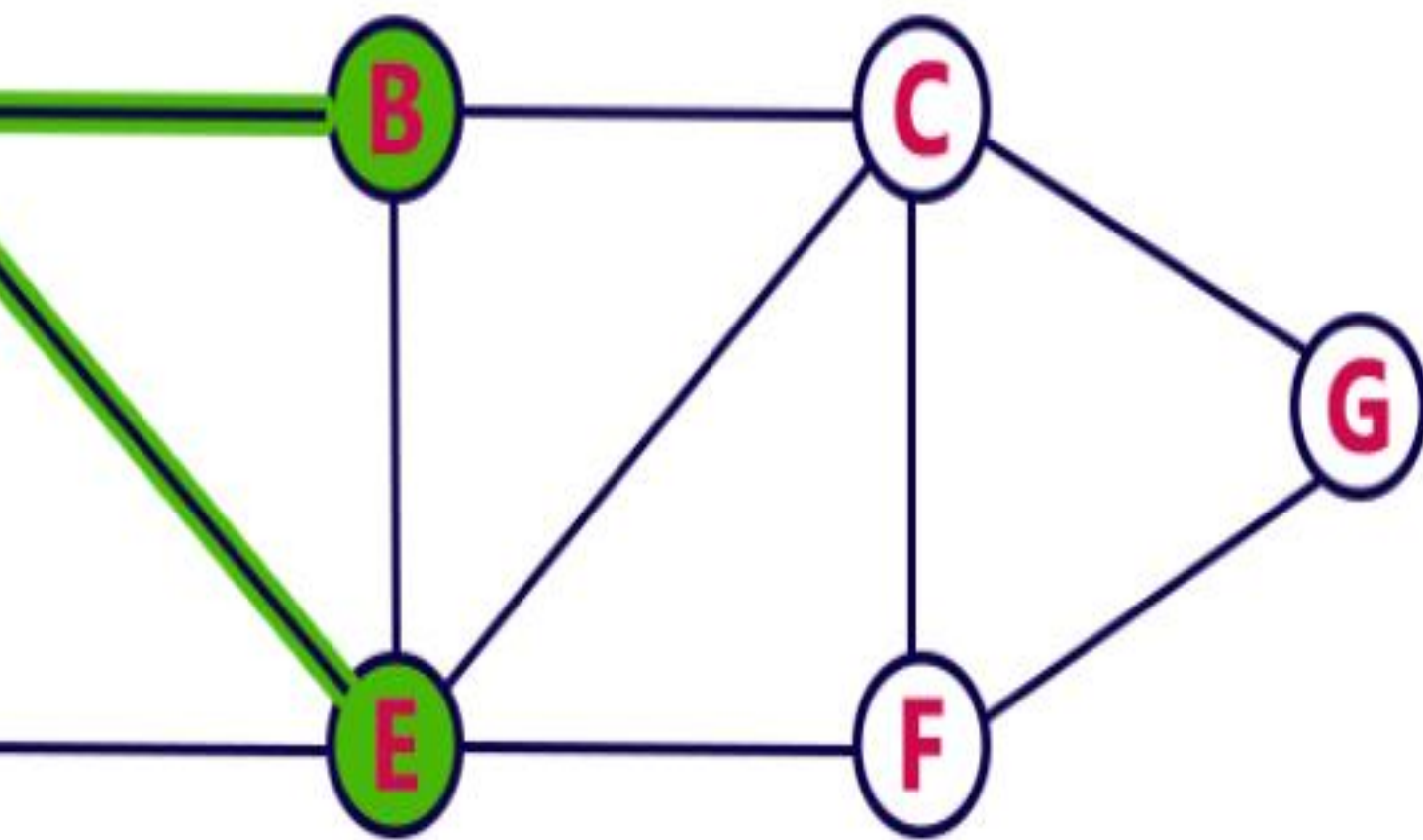
- Visit all adjacent vertices of **A** which are not visited (**D**, **E**, **B**).
- Insert newly visited vertices into the Queue and delete A from the Queue.



Queue

	D	E	B		
--	---	---	---	--	--

- Visit all adjacent vertices of **D** which are not visited (there is no vertex).
- Delete D from the Queue.

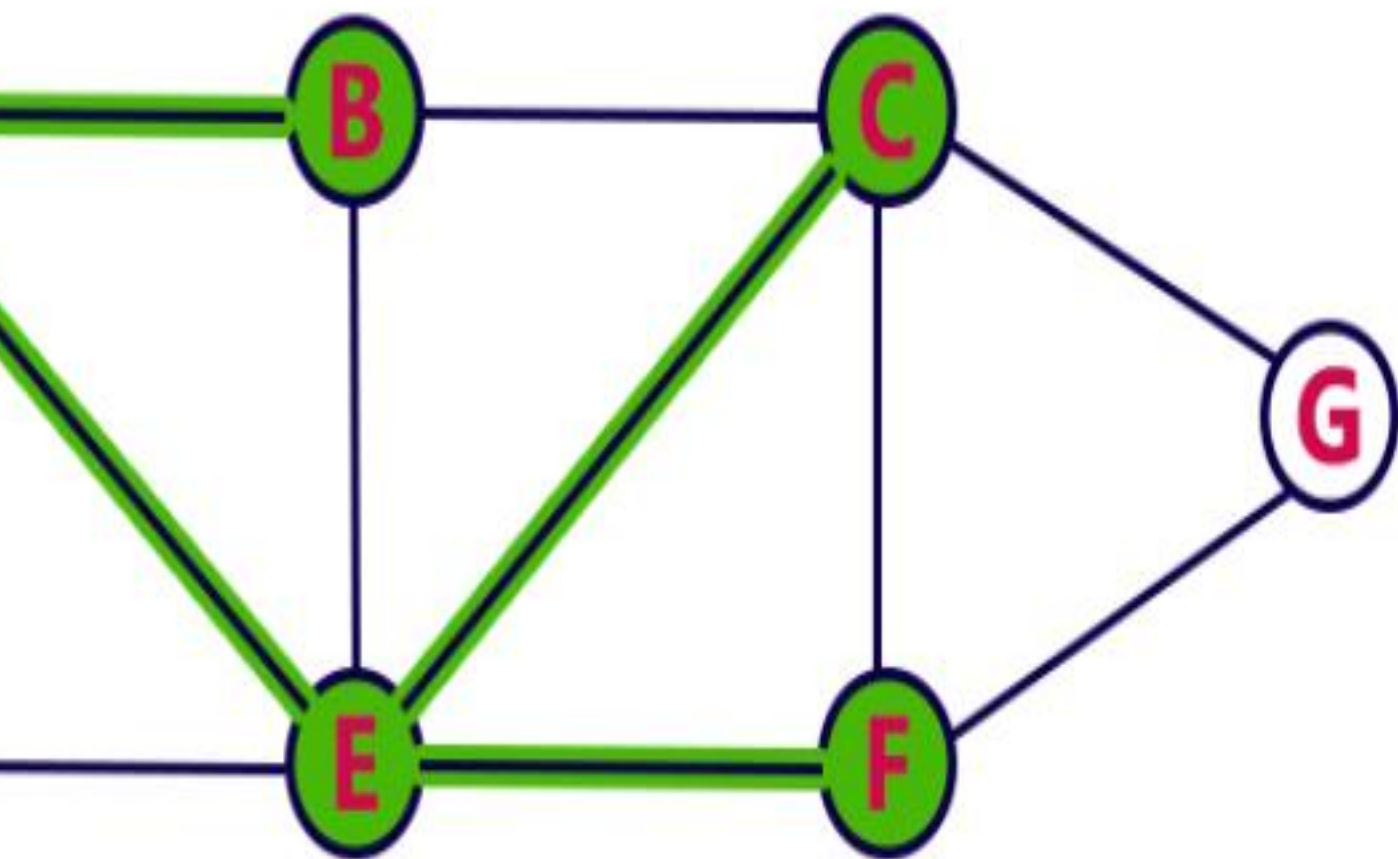


Queue

		E	B		
--	--	---	---	--	--

Visit all adjacent vertices of **E** which are not visited (**C**, **F**).

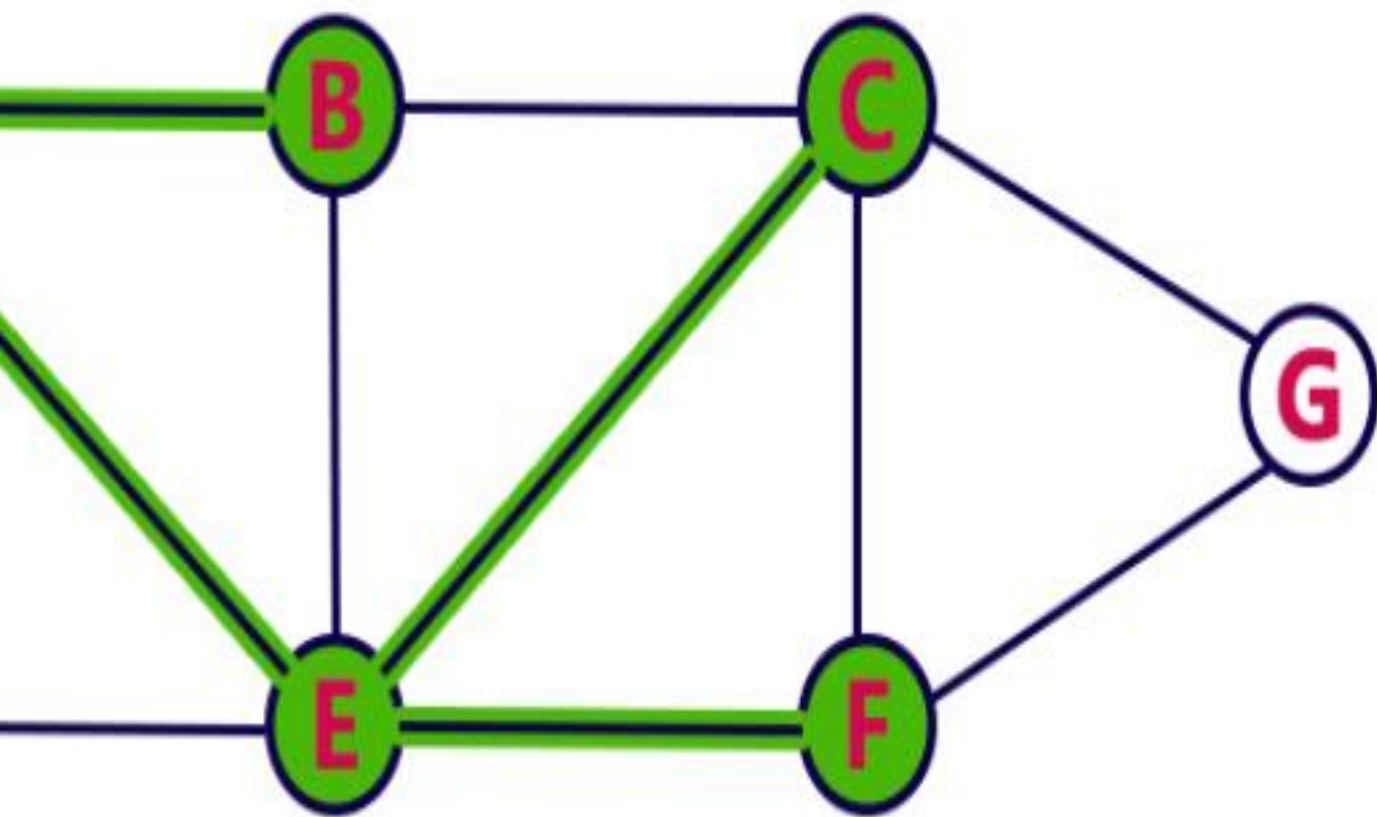
Insert newly visited vertices into the Queue and delete E from the Queue.



Queue



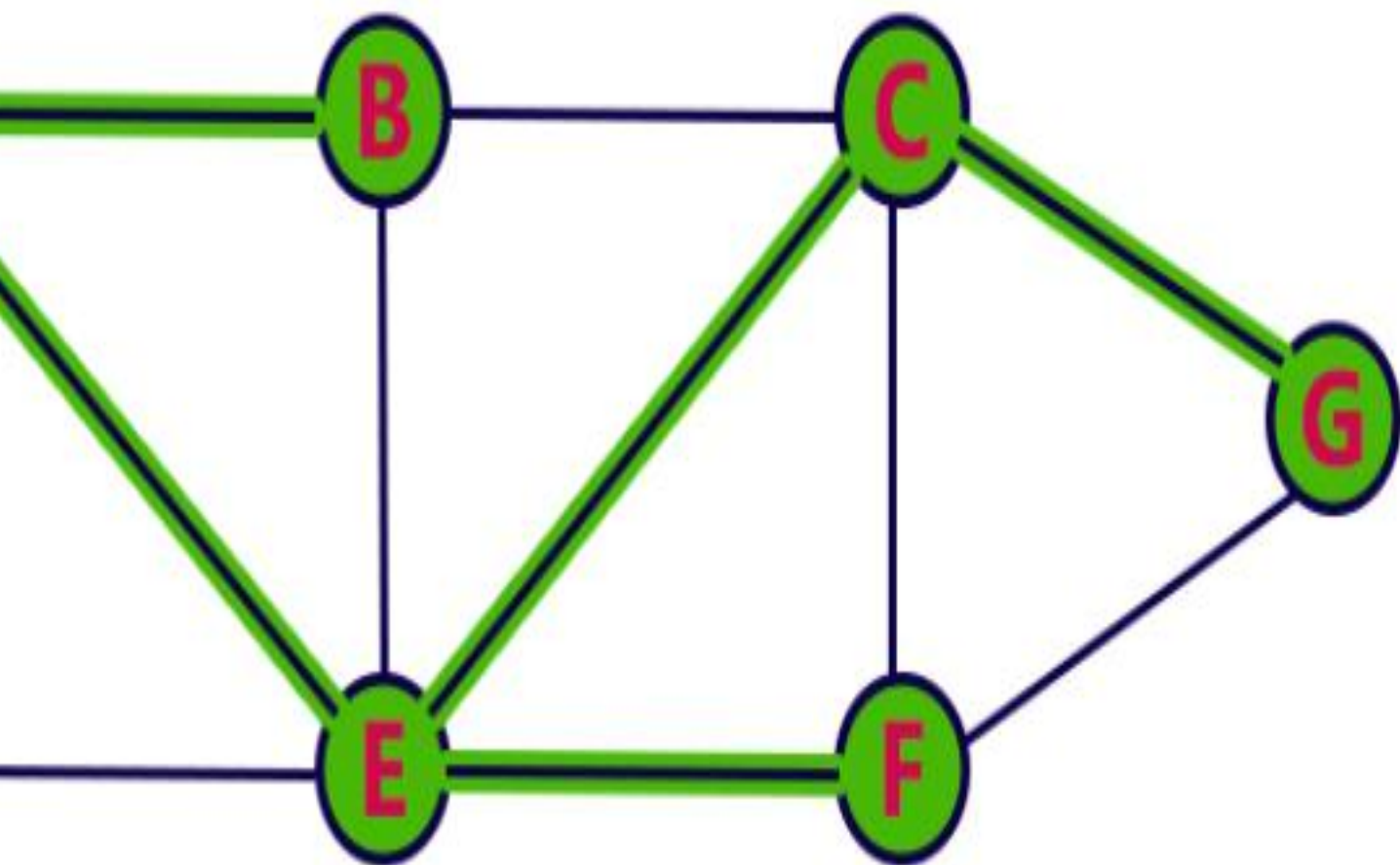
Visit all adjacent vertices of **B** which are not visited (**there is no vertex**).
Delete **B** from the Queue.



Queue



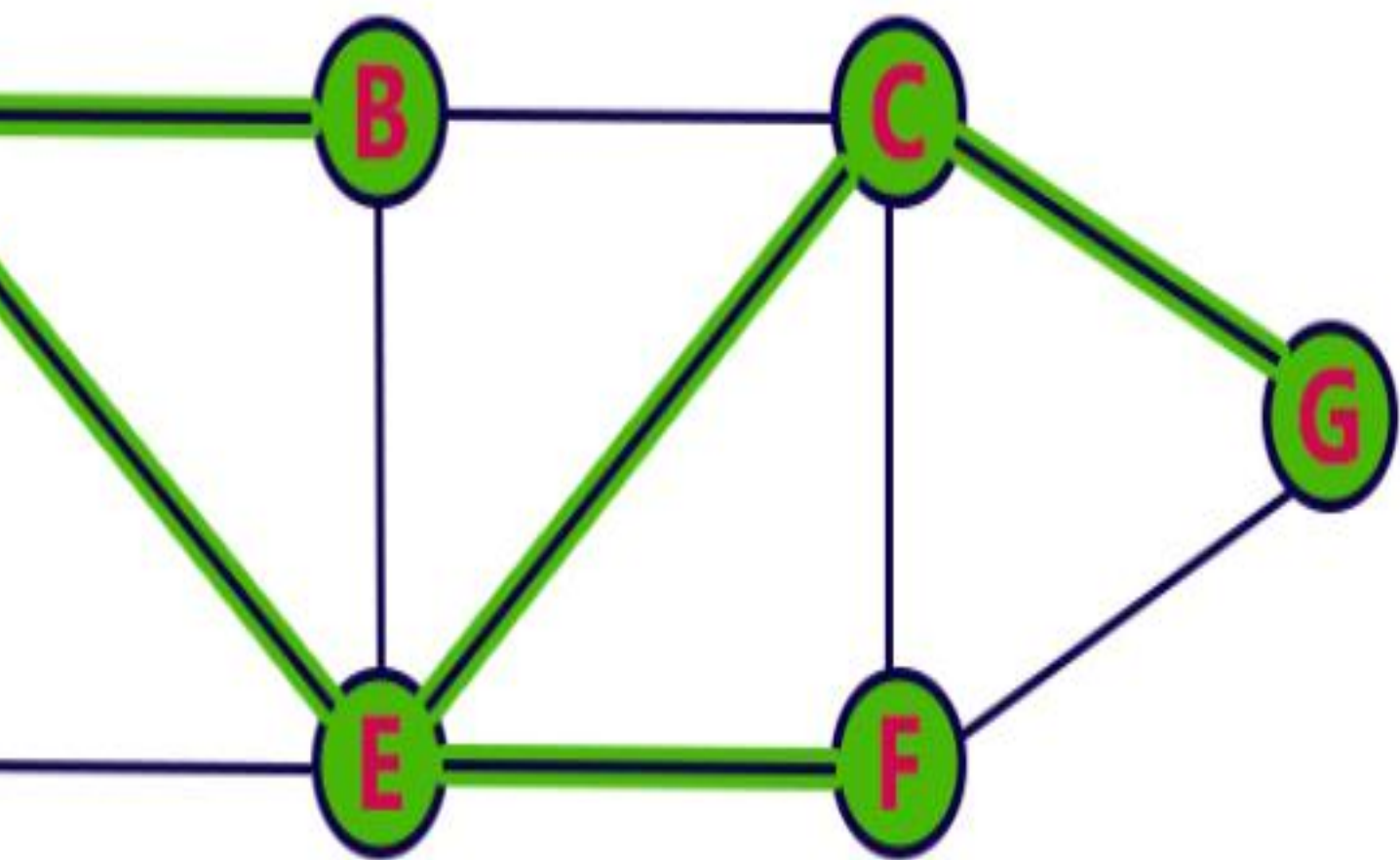
- Visit all adjacent vertices of **C** which are not visited (**G**).
- Insert newly visited vertex into the Queue and delete **C** from the Queue.



Queue



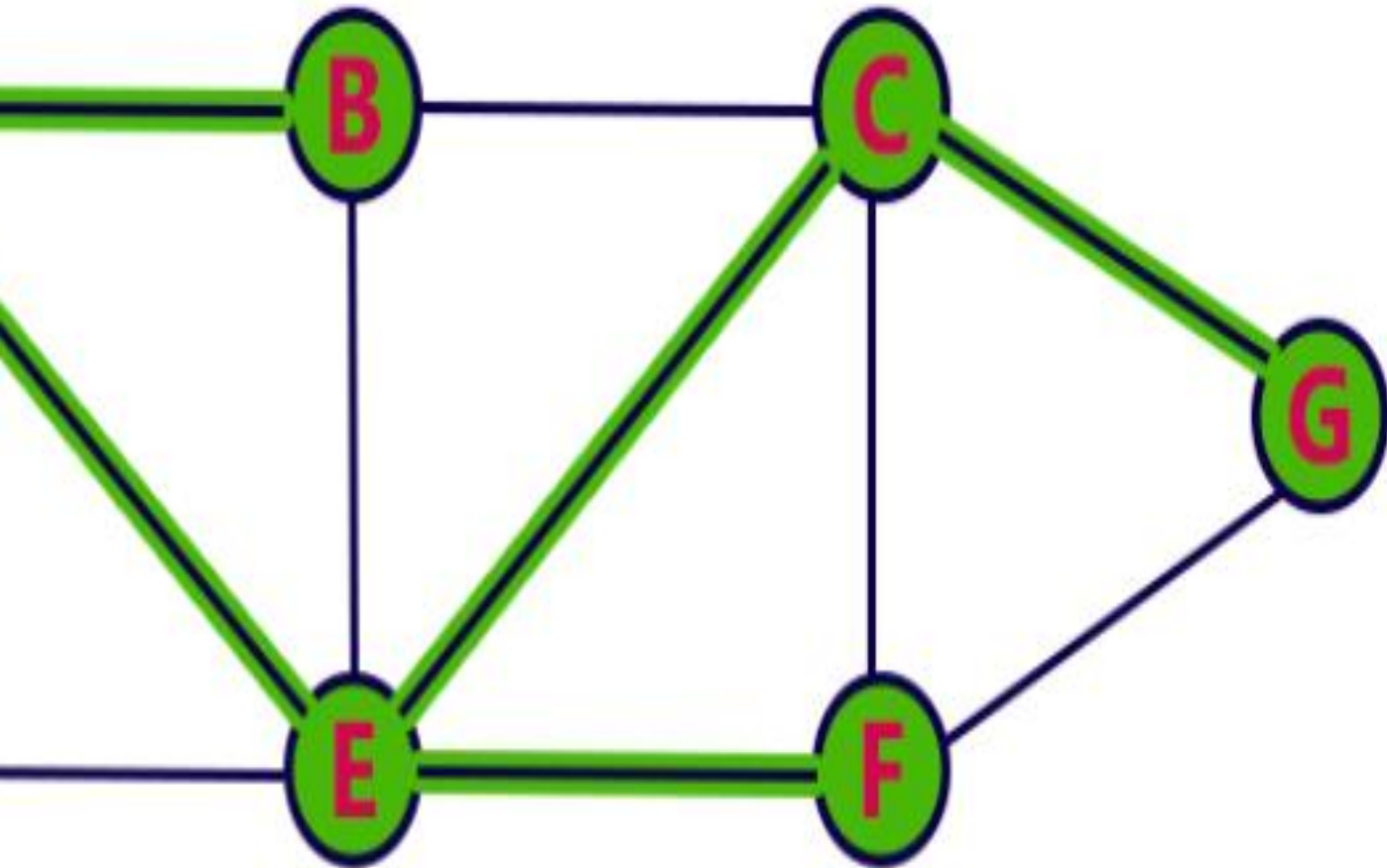
- Visit all adjacent vertices of **F** which are not visited (**there is no vertex**).
- Delete **F** from the Queue.



Queue



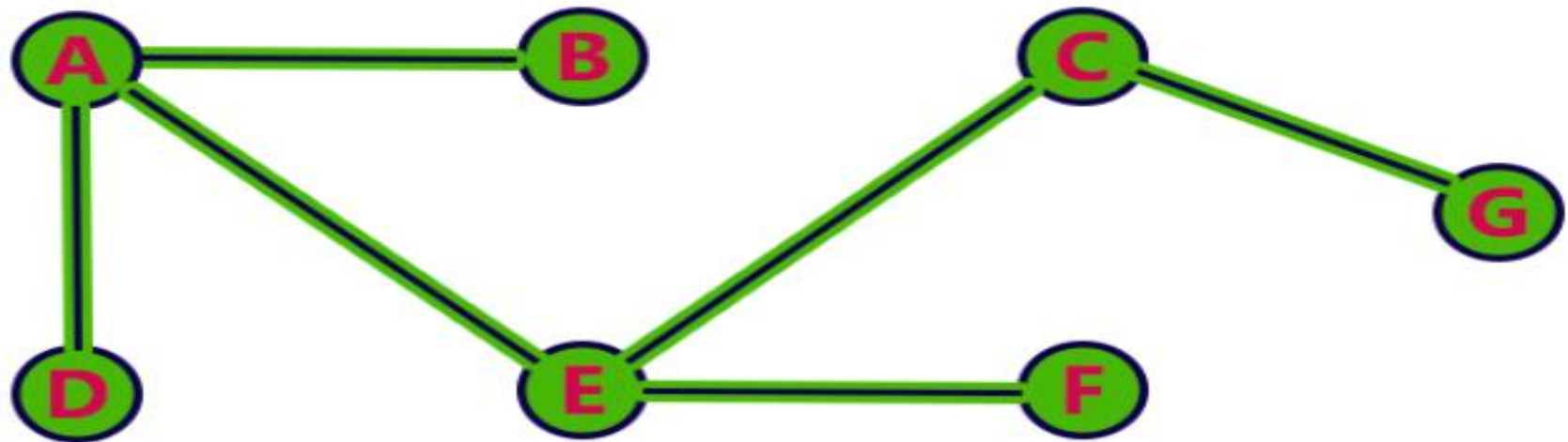
Visit all adjacent vertices of **G** which are not visited (**there is no vertex**).
Delete **G** from the Queue.



Queue



- Queue became Empty. So, stop the BFS process.
- Final result of BFS is a Spanning Tree as shown below...



Difference between BFS and DFS

Key	BFS	DFS
Definition	BFS stands for Breadth First Search.	DFS stands for Depth First Search.
Data structure	BFS uses a Queue to find the shortest path.	DFS uses a Stack to find the shortest path.
Source	BFS is better when target is closer to Source.	DFS is better when target is far from source.
Suitability for decision tree	As BFS considers all neighbor so it is not suitable for decision tree used in puzzle games.	DFS is more suitable for decision tree. As with one decision, we need to traverse further to augment the decision. If we reach the conclusion, we won.
Speed	BFS is slower than DFS.	DFS is faster than BFS.
Time Complexity	Time Complexity of BFS = $O(V+E)$ where V is vertices and E is edges.	Time Complexity of DFS is also $O(V+E)$ where V is vertices and E is edges.
Memory	BFS requires more memory space.	DFS requires less memory space.
Tapping in loops	In BFS, there is no problem of trapping into finite loops.	In DFS, we may be trapped into infinite loops.
Principle	BFS is implemented using FIFO (First In First Out) principle.	DFS is implemented using LIFO (Last In First Out) principle.

SPANNING TREE

Spanning Tree :- A spanning tree is a tree that connects all the vertices of a graph with the minimum possible number of edges. Thus, a spanning tree is always connected. Also, a spanning tree never contains a cycle. A spanning tree is always defined for a graph and it is always a subset of that graph. Thus, a disconnected graph can never have a spanning tree.

Spanning Trees Terminologies:

- 1. Connected Graph:** A connected graph is a graph in which we can reach any vertex from any vertex. Thus, in a connected graph, all vertices are connected to each other through at least one path.
- 2. Undirected Graph:** It is a graph in which the edges don't have a particular direction. In an undirected graph, the edges are bidirectional.
- 3. Directed Graph:** In this case, the edges are unidirectional. Thus, we can go from only one end to the other ends. For every edge, there is an associated direction.
- 4. Simple graph:** A simple graph is a graph that does not contain cycles and loops.

Defining a Spanning Tree:

Every undirected and connected graph has a minimum of one spanning tree.

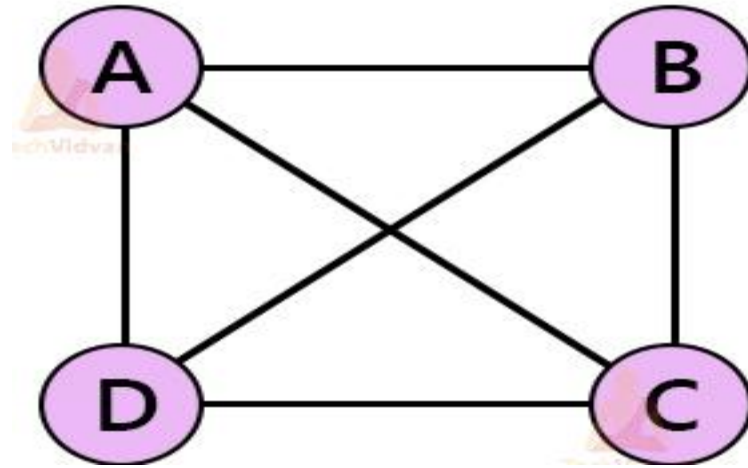
Consider a graph having V vertices and E number of edges.

Then, we will represent the graph as $G(V, E)$.

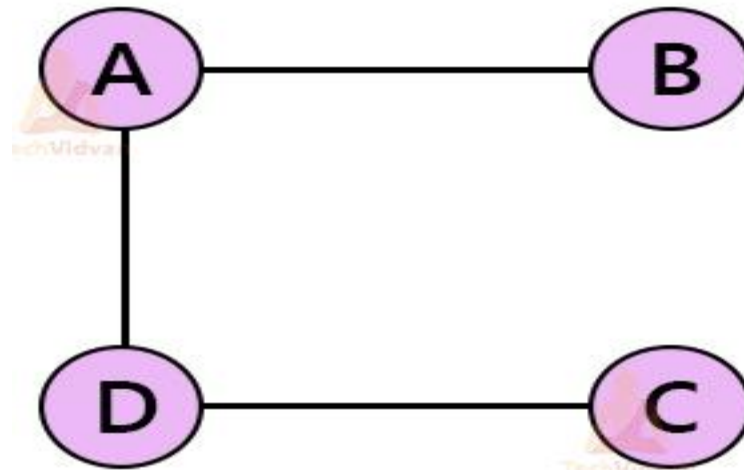
Its spanning tree will be represented as $G'(V, E')$ where $E' \subseteq E$ and the number of vertices remain the same.

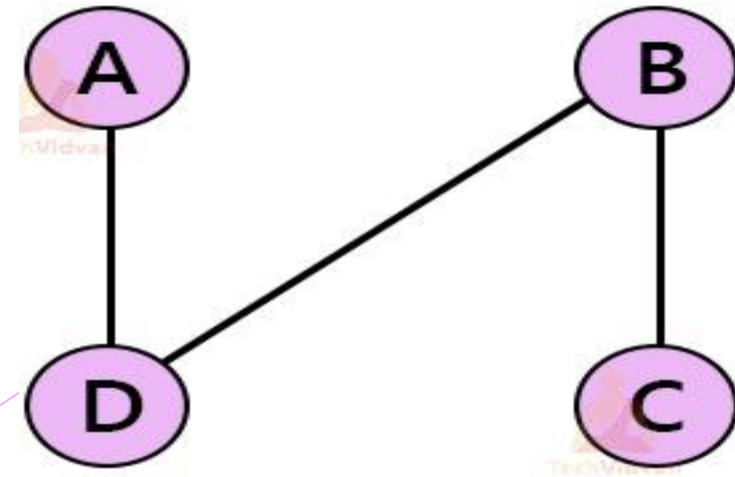
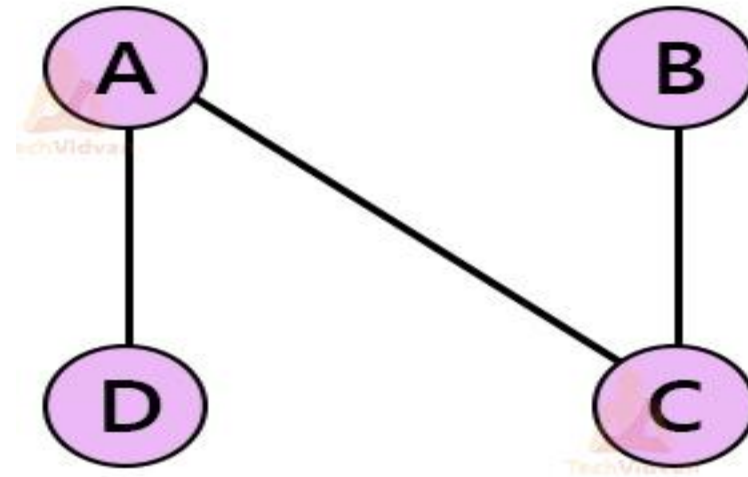
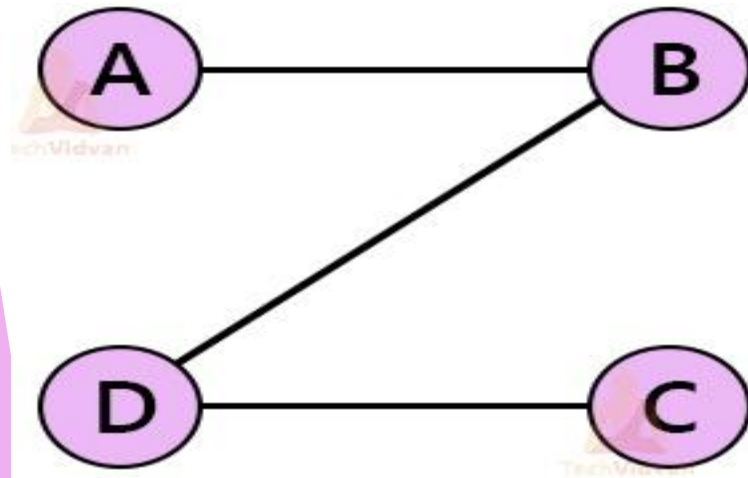
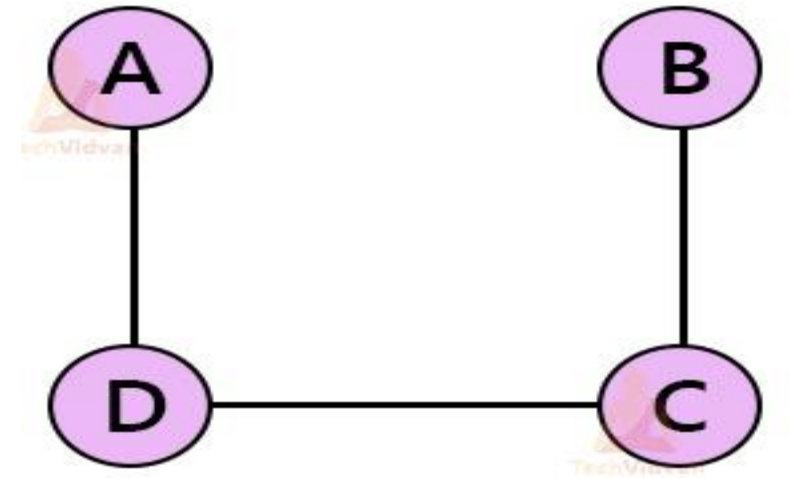
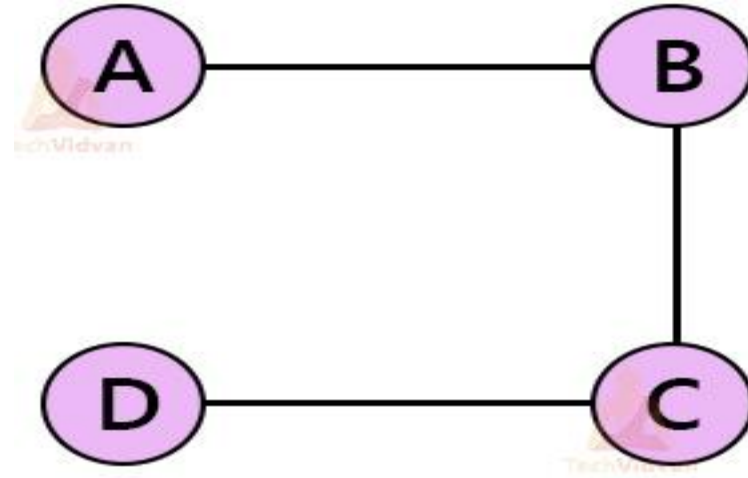
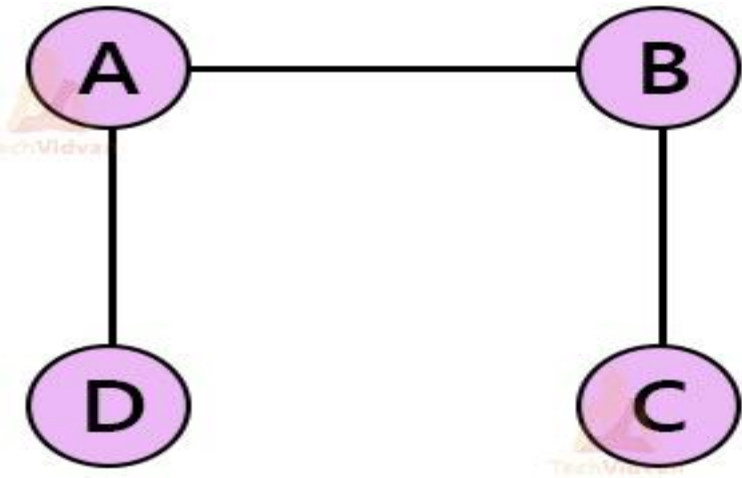
So, a spanning tree G' is a subgraph of G whose vertex set is the same but edges may be different.

Consider the following undirected, simple and connected graph G having 4 vertices: A, B, C and D.



There are many spanning trees possible for this graph.





Properties of Spanning Trees:

There can be more than one spanning tree possible for an undirected, connected graph. In the case of directed graphs, the minimum spanning tree is the one having minimum edge weight.

All the possible spanning trees of a graph have the same number of edges and vertices.

A spanning tree can never contain a cycle.

Spanning tree is always **minimally connected** i.e. if we remove one edge from the spanning tree, it will become disconnected.

A spanning tree is **maximally acyclic** i.e. if we add one edge to the spanning tree, it will create a cycle or a loop.

It is possible to have more than one minimum spanning tree if the graph weights of some edges are the same.

Any connected and undirected graph will always have at least one spanning tree.

Minimum Spanning Tree:

A minimum spanning tree is defined for a weighted graph. A spanning tree having minimum weight is defined as a minimum spanning tree. This weight depends on the weight of the edges. In real-world applications, the weight could be the distance between two points, cost associated with the edges or simply an arbitrary value associated with the edges.

Characteristics of a Minimum Spanning Tree

A minimum spanning tree connects all the vertices in the graph, ensuring that there is a path between any pair of nodes.

An MST is acyclic, i.e. it contains no cycles. This property ensures that it remains a tree and not a graph with loops.

An MST with n vertices (where n is the number of vertices in the original graph) will have exactly $n - 1$ edges.

An MST is optimal for minimizing the total edge weight, but it may not necessarily be unique.

The cut property states that if you take any cut (a partition of the vertices into two sets) in the original graph and consider the minimum-weight edge that crosses the cut, that edge is part of the MST.

Minimum Spanning Tree Algorithms:

There are various algorithms in computer science that help us find the minimum spanning tree for a weighted graph. Some of these algorithms are:

1. **Prim's Algorithm**

2. **Kruskal's Algorithm**

3. Both these algorithms follow the greedy approach to find the minimum cost spanning tree.

Kruskal's Algorithm

In Kruskal's algorithm, the spanning tree is constructed by adding the edges one by one. It falls under a class of algorithms called greedy algorithms that find the local optimum in the hopes of finding a global optimum. We start from the edges with the lowest weight and keep adding edges until we reach our goal.

Working of the Kruskal's Algorithm

Sort all the edges of the graph in the increasing order of their weight.

Pick the edge with the smallest weight.

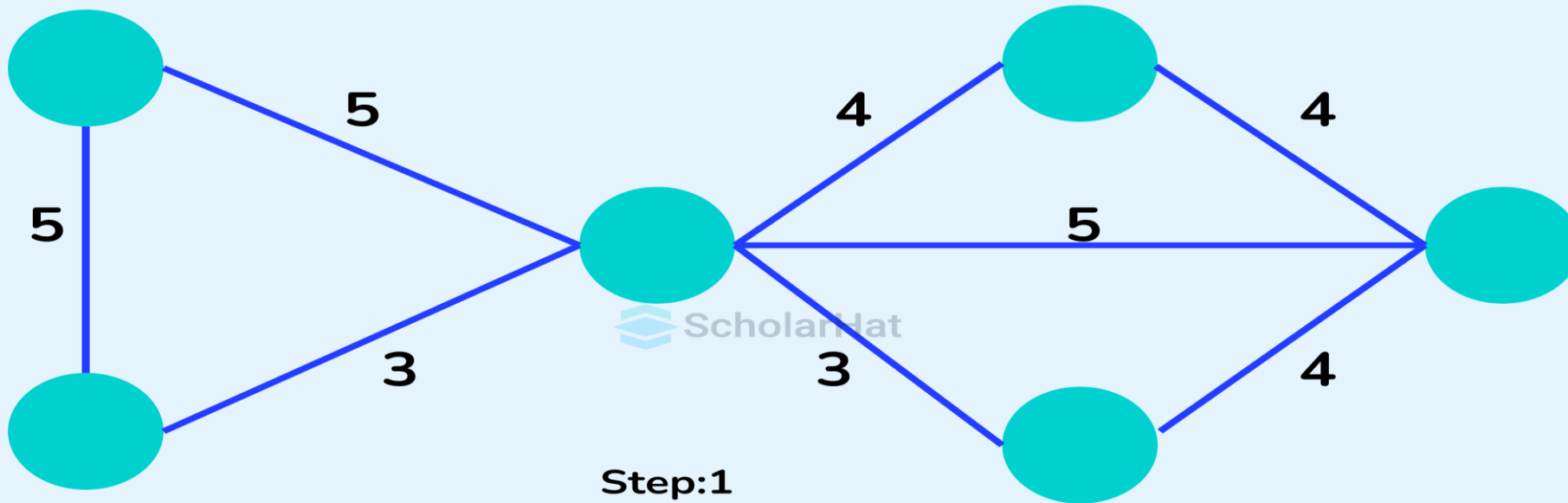
- Check if it forms a cycle with the spanning tree formed so far.

- Include the current edge if it does not form any cycle.

- Otherwise, discard it.

Repeat step 3 until there are $n-1$ edges in the spanning tree, where n is the total number of vertices in the graph.

Example of Kruskal's Algorithm



Start with a weighted graph

Prim's Algorithm

It takes a graph as input and finds the subset of the edges of that graph which form a tree that includes every vertex and has the minimum sum of weights among all the trees that can be formed from the graph.

It falls under a class of algorithms called **greedy algorithms** that find the local optimum in the hopes of finding a global optimum.

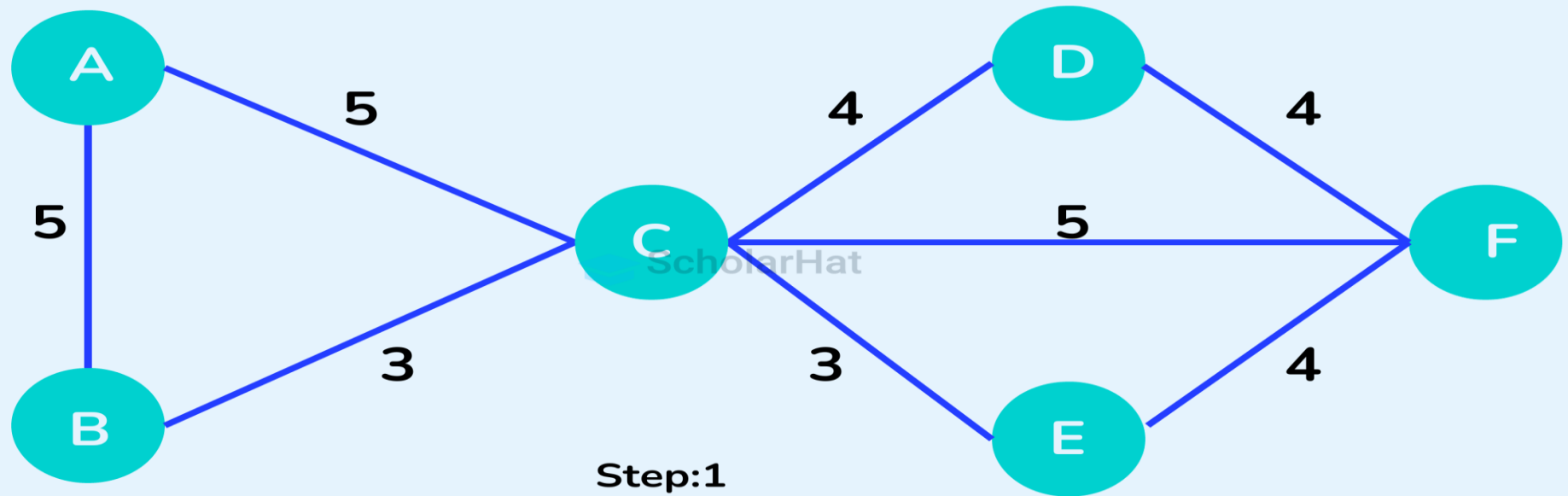
We start from one vertex and keep adding edges with the lowest weight until we reach our goal. Unlike Kruskal's algorithm, Prim's algorithm treats the nodes as a single tree and keeps on adding new nodes to the spanning tree from the given graph.

Working of the Prim's Algorithm

Initialize the minimum spanning tree with a vertex chosen at random.

Find all the edges that connect the tree to new vertices, find the minimum, and add it to the tree.

Keep repeating step 2 until we get a minimum spanning tree



Start with a weighted graph

Difference between Kruskal's and Prim's Algorithms

Prim's Algorithm	Kruskal's Algorithm
It starts to build the Minimum Spanning Tree from any vertex in the graph.	It starts to build the Minimum Spanning Tree from the vertex carrying the minimum weight in the graph.
It traverses one node more than one time to get the minimum distance.	It traverses one node only once.
It has a time complexity of $O(V^2)$, V being the number of vertices, and can be improved up to $O(E \log V)$ using Fibonacci heaps.	It has a time complexity is $O(E \log V)$, V being the number of vertices.
Prim's algorithm gives connected components as well and it works only on connected graphs.	Kruskal's algorithm can generate a forest(disconnected components) at any instant as well as it can work on disconnected components
Prim's algorithm runs faster in dense graphs.	Kruskal's algorithm runs faster in sparse graphs.
Applications are Travelling Salesman Problem, Network for roads and Rail tracks connecting all the cities, etc.	Applications are LAN connection, TV Network, etc.
Prim's algorithm prefers list data structures.	Kruskal's algorithm prefers <u>heap data structures</u> .

UNIT V

UNIT V

Searching Techniques: Linear Search, Binary Search, and Complexity.

Sorting Techniques-Bubble Sort, Insertion Sort, Selection Sort.

Divide and Conquer Techniques - Merge Sort, Quick Sort.



Searching Techniques:

Linear Search, Binary Search, and Complexity.

➤ Linear Search Algorithm (Sequential Search Algorithm)

- Linear search algorithm finds a given element in a list of elements with **$O(n)$** time complexity where **n** is total number of elements in the list.
- This search process starts comparing search element with the first element in the list.
- If both are matched then result is element found otherwise search element is compared with the next element in the list.
- Repeat the same until search element is compared with the last element in the list, if that last element also doesn't match, then the result is "Element not found in the list".

Linear search is implemented using following steps...

- Step 1** - Read the search element from the user.
- Step 2** - Compare the search element with the first element in the list.
- Step 3** - If both are matched, then display "Given element is found!!!" and terminate the function
- Step 4** - If both are not matched, then compare search element with the next element in the list.
- Step 5** - Repeat steps 3 and 4 until search element is compared with last element in the list.
- Step 6** - If last element in the list also doesn't match, then display "Element is not found!!!" and terminate the function.

list

0	1	2	3	4	5	6	7
65	20	10	55	32	12	50	99

search element **12**

Step 1:

search element (12) is compared with first element (65)

list

0	1	2	3	4	5	6	7
65	20	10	55	32	12	50	99

12

Both are not matching. So move to next element

Step 2:

search element (12) is compared with next element (20)

list

0	1	2	3	4	5	6	7
65	20	10	55	32	12	50	99

12

Both are not matching. So move to next element

Step 3:

search element (12) is compared with next element (10)

list

0	1	2	3	4	5	6	7
65	20	10	55	32	12	50	99

12

Both are not matching. So move to next element

Step 4:

search element (12) is compared with next element (55)

	0	1	2	3	4	5	6	7
list	65	20	10	55	32	12	50	99

12

Both are not matching. So move to next element

Step 5:

search element (12) is compared with next element (32)

	0	1	2	3	4	5	6	7
list	65	20	10	55	32	12	50	99

12

Both are not matching. So move to next element

Step 6:

search element (12) is compared with next element (12)

	0	1	2	3	4	5	6	7
list	65	20	10	55	32	12	50	99

12

Both are matching. So we stop comparing and display element found at index 5.

Binary Search Algorithm

- Binary search algorithm finds a given element in a list of elements with **$O(\log n)$** time complexity where **n** is total number of elements in the list.
- The binary search algorithm can be used with only a sorted list of elements.
- The binary search can not be used for a list of elements arranged in random order. This search process starts comparing the search element with the middle element in the list. If both are matched, then the result is "element found".
- Otherwise, we check whether the search element is smaller or larger than the middle element in the list.
- If the search element is smaller, then we repeat the same process for the left sublist of the middle element. If the search element is larger, then we repeat the same process for the right sublist of the middle element.
- We repeat this process until we find the search element in the list or until we left with a sublist of only one element. And if that element also doesn't match with the search element, then the result is "Element not found in the list".

Binary search is implemented using following steps...

- Step 1** - Read the search element from the user.
- Step 2** - Find the middle element in the sorted list.
- Step 3** - Compare the search element with the middle element in the sorted list.
- Step 4** - If both are matched, then display "Given element is found!!!" and terminate the function.
- Step 5** - If both are not matched, then check whether the search element is smaller or larger than the middle element.
- Step 6** - If the search element is smaller than middle element, repeat steps 2, 3, 4 and 5 for the left sublist of the middle element.
- Step 7** - If the search element is larger than middle element, repeat steps 2, 3, 4 and 5 for the right sublist of the middle element.
- Step 8** - Repeat the same process until we find the search element in the list or until sublist contains only one element.
- Step 9** - If that element also doesn't match with the search element, then display "Element is not found in the list!!!" and terminate the function.

search element **80**

Step 1:

search element (80) is compared with middle element (50)

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

80

Both are not matching. And 80 is larger than 50. So we search only in the right sublist (i.e. 55, 65, 80 & 99).

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

Step 2:

search element (80) is compared with middle element (65)

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

80

Both are not matching. And 80 is larger than 65. So we search only in the right sublist (i.e. 80 & 99).

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

Step 3:

search element (80) is compared with middle element (80)

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

80



Sorting Techniques-

Bubble Sort, Insertion Sort, Selection Sort.

✓ Selection Sort Algorithm

- ✓ Selection Sort algorithm is used to arrange a list of elements in a particular order (Ascending or Descending).
- ✓ In selection sort, the first element in the list is selected and it is compared repeatedly with all the remaining elements in the list.
- ✓ If any element is smaller than the selected element (for Ascending order), then both are swapped so that first position is filled with the smallest element in the sorted order.
- ✓ Next, we select the element at a second position in the list and it is compared with all the remaining elements in the list.
- ✓ If any element is smaller than the selected element, then both are swapped. This procedure is repeated until the entire list is sorted.

The selection sort algorithm is performed using the following steps...

- **Step 1** - Select the first element of the list (i.e., Element at first position in the list).
- **Step 2:** Compare the selected element with all the other elements in the list.
- **Step 3:** In every comparison, if any element is found smaller than the selected element (for Ascending order), then both are swapped.
- **Step 4:** Repeat the same procedure with element in the next position in the list till the entire list is sorted.

Complexity of the Selection Sort Algorithm

To sort an unsorted list with 'n' number of elements, we need to make $((n-1)+(n-2)+(n-3)+\dots+1) = (n(n-1))/2$ number of comparisons in the worst case. If the list is already sorted then it requires 'n' number of comparisons.

Worst Case : $O(n^2)$

Best Case : $\Omega(n^2)$

Average Case : $\Theta(n^2)$

Consider the following unsorted list of elements...

15	20	10	30	50	18	5	45
----	----	----	----	----	----	---	----

Iteration #1

Select the first position element in the list, compare it with all other elements in the list and whenever we found a smaller element than the element at first position then swap those two elements.

15	20	10	30	50	18	5	45
----	----	----	----	----	----	---	----

$15 > 20$
FALSE

15	20	10	30	50	18	5	45
----	----	----	----	----	----	---	----

$15 > 10$
TRUE
SWAP

10	20	15	30	50	18	5	45
----	----	----	----	----	----	---	----

$10 > 30$
FALSE

10	20	15	30	50	18	5	45
----	----	----	----	----	----	---	----

$10 > 50$
FALSE

10	20	15	30	50	18	5	45
----	----	----	----	----	----	---	----

$10 > 18$
FALSE

10	20	15	30	50	18	5	45
----	----	----	----	----	----	---	----

$10 > 5$
TRUE
SWAP

5	20	15	30	50	18	10	45
---	----	----	----	----	----	----	----

$5 > 45$
FALSE

List after 1st iteration

5	20	15	30	50	18	10	45
---	----	----	----	----	----	----	----

Iteration #2

Select the second position element in the list, compare it with all other elements in the list and whenever we found a smaller element than the element at first position then swap those two elements.

List after 2nd iteration

5	10	20	30	50	18	15	45
---	----	----	----	----	----	----	----

Iteration #3

Select the third position element in the list, compare it with all other elements in the list and whenever we found a smaller element than the element at first position then swap those two elements.

List after 3rd iteration

5	10	15	30	50	20	18	45
---	----	----	----	----	----	----	----

Iteration #4

Select the fourth position element in the list, compare it with all other elements in the list and whenever we found a smaller element than the element at first position then swap those two elements.

List after 4th iteration

5	10	15	18	50	30	20	45
---	----	----	----	----	----	----	----

Iteration #5

Select the fifth position element in the list, compare it with all other elements in the list and whenever we found a smaller element than the element at first position then swap those two elements.

List after 5th iteration

5	10	15	18	20	50	30	45
---	----	----	----	----	----	----	----

Iteration #6

Select the sixth position element in the list, compare it with all other elements in the list and whenever we found a smaller element than the element at first position then swap those two elements.

List after 6th iteration

5	10	15	18	20	30	50	45
---	----	----	----	----	----	----	----

Iteration #7

Select the seventh position element in the list, compare it with all other elements in the list and whenever we found a smaller element than the element at first position then swap those two elements.

List after 7th iteration

5	10	15	18	20	30	45	50
---	----	----	----	----	----	----	----

Final sorted list

Insertion Sort Algorithm

Sorting is the process of arranging a list of elements in a particular order(Ascending or Descending).

Insertion sort algorithm arranges a list of elements in a particular order.

In insertion sort algorithm, every iteration moves an element from unsorted portion to sorted portion until all the elements are sorted in the list.

Complexity of the Insertion Sort Algorithm

To sort an unsorted list with 'n' number of elements, we need to make $(1+2+3+.....+n-1) = (n(n-1))/2$ number of comparisons in the worst case. If the list is already sorted then it requires 'n' number of comparisons.

Worst Case : $O(n^2)$

Best Case : $\Omega(n)$

Average Case : $\Theta(n^2)$

Step by Step Process

- Step 1** - Assume that first element in the list is in sorted portion and all the remaining elements are in unsorted portion.
- Step 2:** Take first element from the unsorted portion and insert that element into the sorted portion in the order specified.
- Step 3:** Repeat the above process until all the elements from the unsorted portion are moved into the sorted portion.

Consider the following unsorted list of elements...

15	20	10	30	50	18	5	45
----	----	----	----	----	----	---	----

Assume that sorted portion of the list is empty and all elements in the list are in the unsorted portion of the list as shown in the figure below...

Sorted	Unsorted
	15 20 10 30 50 18 5 45

Move the first element 15 from unsorted portion to sorted portion of the list.

Sorted	Unsorted
15	20 10 30 50 18 5 45

To move element 20 from unsorted to sorted portion, Compare 20 with 15 and insert it at correct position

Sorted	Unsorted
15 20	10 30 50 18 5 45

To move element 10 from unsorted to sorted portion, Compare 10 with 20 and it is smaller so swap. Then compare 10 with 15 again smaller swap. And 10 is inserted at its correct position in sorted portion of the list.

Sorted	Unsorted
10 15 20	30 50 18 5 45

To move element 30 from unsorted to sorted portion, Compare 30 with 20, 15 and 10. And it is larger than all these so 30 is directly inserted at last position in sorted portion of the list.

Sorted	Unsorted
10 15 20 30	50 18 5 45

To move element 50 from unsorted to sorted portion, Compare 50 with 30, 20, 15 and 10. And it is larger than all these so 50 is directly inserted at last position in sorted portion of the list.

To move element 50 from unsorted to sorted portion, Compare 50 with 30, 20, 15 and 10. And it is larger than all these so 50 is directly inserted at last position in sorted portion of the list.

Sorted					Unsorted		
10	15	20	30	50	18	5	45

To move element 18 from unsorted to sorted portion, Compare 18 with 30, 20 and 15. Since 18 is larger than 15, move 20, 30 and 50 one position to the right in the list and insert 18 after 15 in the sorted portion.

Sorted						Unsorted	
10	15	18	20	30	50	5	45

To move element 5 from unsorted to sorted portion, Compare 5 with 50, 30, 20, 18, 15 and 10. Since 5 is smaller than all these element, move 10, 15, 18, 20, 30 and 50 one position to the right in the list and insert 5 at first position in the sorted list.

Sorted							Unsorted
5	10	15	18	20	30	50	45

To move element 45 from unsorted to sorted portion, Compare 45 with 50 and 30. Since 45 is larger than 30, move 50 one position to the right in the list and insert 45 after 30 in the sorted list.

Sorted							Unsorted
5	10	15	18	20	30	45	50

Unsorted portion of the list has become empty. So we stop the process. And the final sorted list of elements is as follows...

5	10	15	18	20	30	45	50
---	----	----	----	----	----	----	----

Bubble Sort Algorithm

Bubble Sort is a simple algorithm which is used to sort a given set of n elements provided in form of an array. Bubble Sort compares all the element one by one and sort them based on their values.

If the given array has to be sorted in ascending order, then bubble sort will start by comparing the first element of the array with the second element, if the first element is greater than the second element, it will **swap** both the elements, and then move on to compare the second and the third element, and so on.

If we have total n elements, then we need to repeat this process for $n-1$ times.

Implementing Bubble Sort Algorithm

Starting with the first element(index = 0), compare the current element with the next element of the array.

If the current element is greater than the next element of the array, swap them.

If the current element is less than the next element, move to the next element. **Repeat Step 1.**

Time and Space complexity for the Bubble Sort algorithm.

Worst Case Time Complexity [Big-O]: **$O(n^2)$**

Best Case Time Complexity [Big-omega]: **$O(n)$**

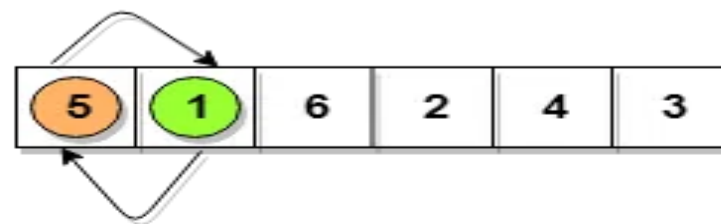
Average Time Complexity [Big-theta]: **$O(n^2)$**

Space Complexity: **$O(1)$**

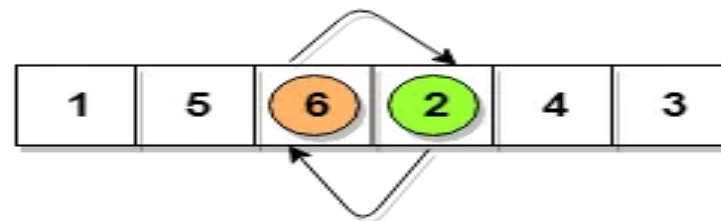
$5 > 1$
so interchange



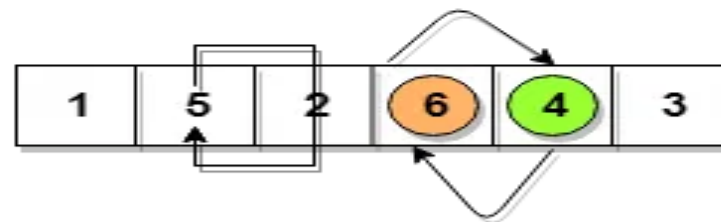
$5 < 6$
No swapping



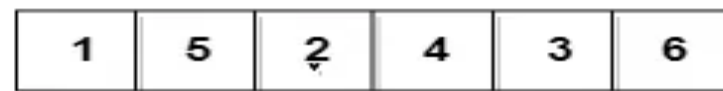
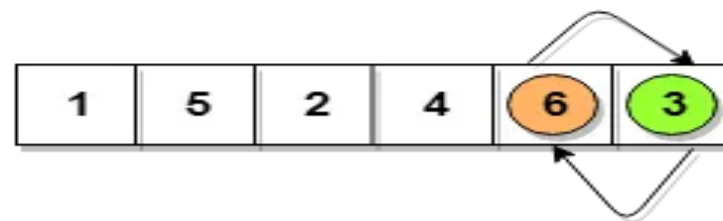
$6 > 2$
so interchange



$6 > 4$
so interchange

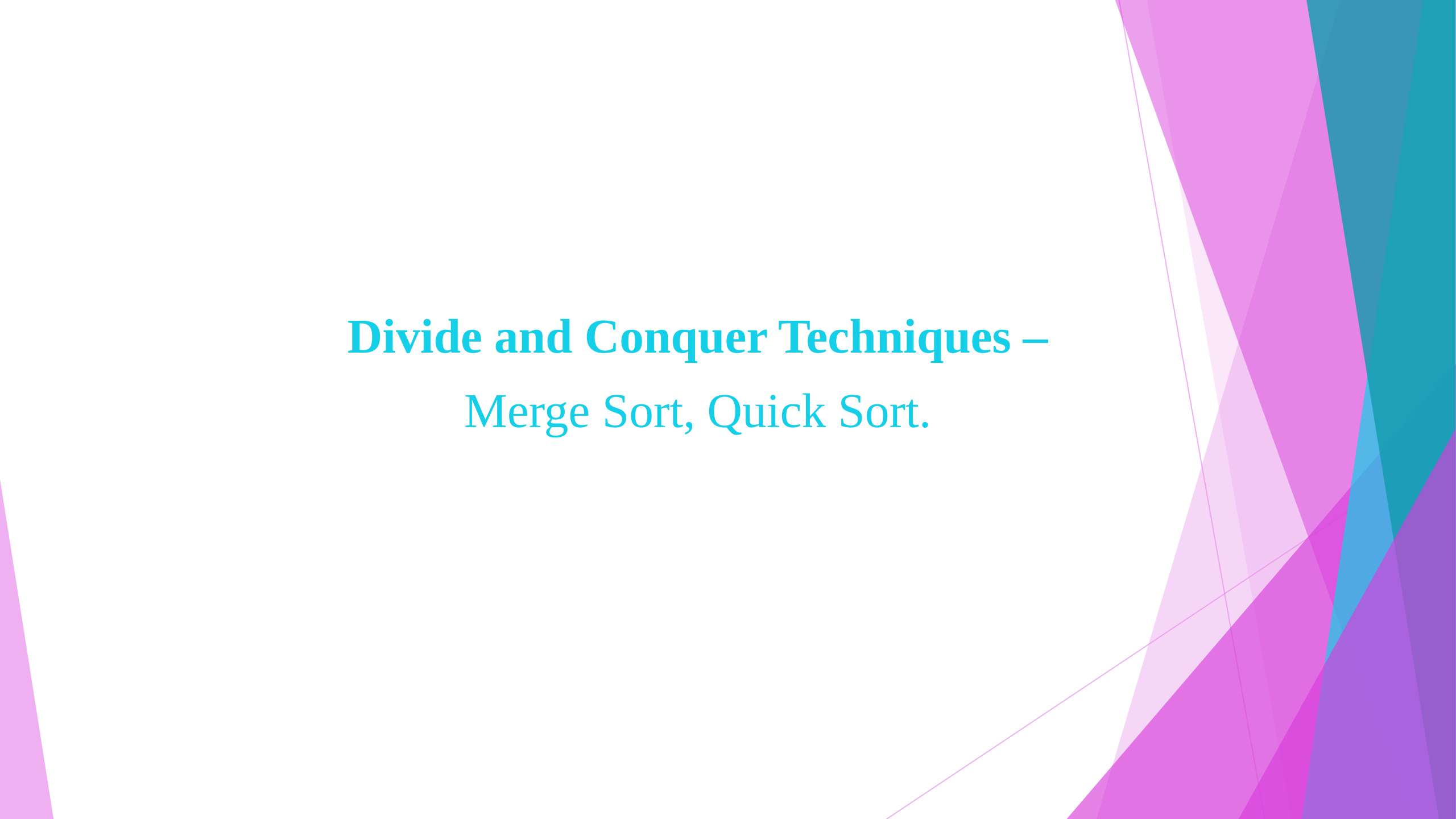


$6 > 3$
so interchange



This is first insertion

similarly, after all the
iterations, the array
gets sorted



Divide and Conquer Techniques – Merge Sort, Quick Sort.

Divide and Conquer

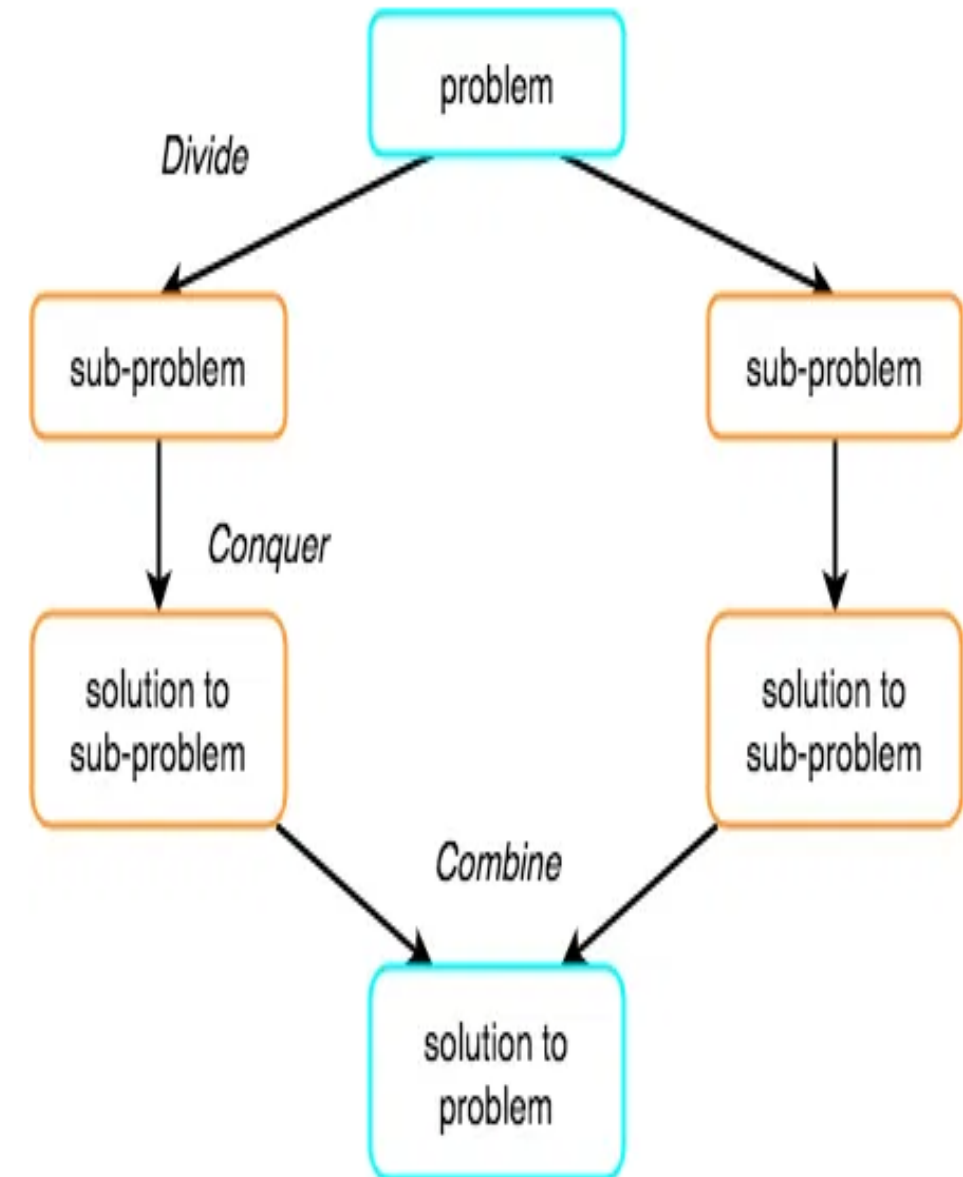
If we can break a single big problem into smaller sub-problems, solve the smaller sub-problems and combine their solutions to find the solution for the original big problem, it becomes easier to solve the whole problem.

The concept of Divide and Conquer involves three steps:

Divide the problem into multiple small problems.

Conquer the subproblems by solving them. The idea is to break down the problem into atomic subproblems, where they are actually solved.

Combine the solutions of the subproblems to find the solution of the actual problem.



Merge Sort

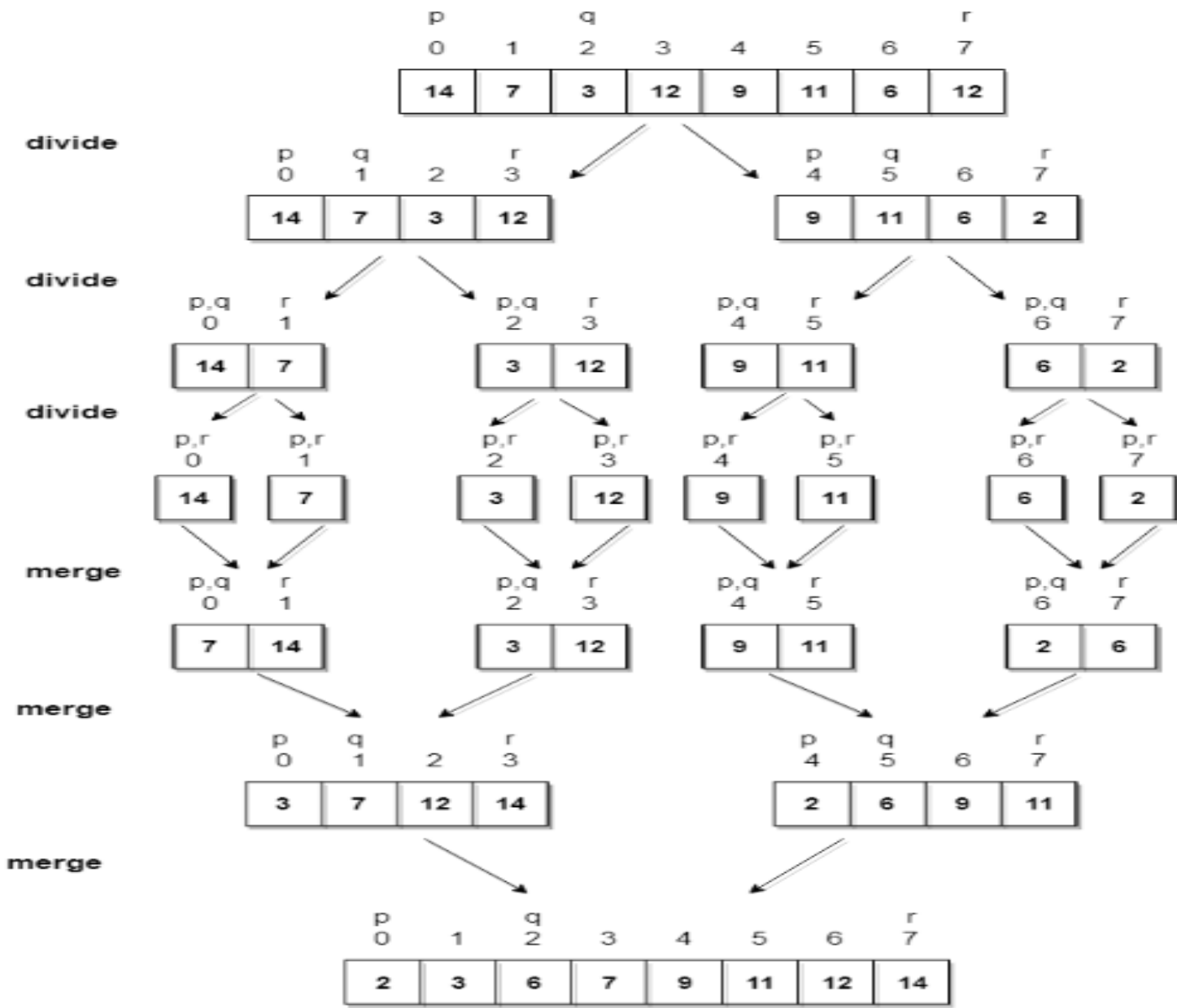
- Merge Sort follows the rule of **Divide and Conquer** to sort a given set of numbers/elements, recursively, hence consuming less time.
- In merge sort, we break the given array midway, for example if the original array had 6 elements, then merge sort will break it down into two subarrays with 3 elements each.
- But breaking the original array into 2 smaller subarrays is not helping us in sorting the array.
- So we will break these subarrays into even smaller subarrays, until we have multiple subarrays with **single element** in them.

In merge sort we follow the following steps:

- We take a variable p and store the starting index of our array in this. And we take another variable r and store the last index of array in it.
- Then we find the middle of the array using the formula $(p + r)/2$ and mark the middle index as q , and break the array into two subarrays, from p to q and from $q + 1$ to r index.
- Then we divide these 2 subarrays again, just like we divided our main array and this continues.
- Once we have divided the main array into subarrays with single elements, then we start merging the subarrays.

Let's consider an array with values {14, 7, 3, 12, 9, 11, 6, 12}

Below, we have a pictorial representation of how merge sort will sort the given array.



Quick sort algorithm

The quick sort algorithm attempts to separate the list of elements into two parts and then sort each part recursively. That means it uses **divide and conquer** strategy. In quick sort, the partition of the list is performed based on the element called ***pivot***. Here pivot element is one of the elements in the list. The list is divided into two partitions such that "**all elements to the left of pivot are smaller than the pivot and all elements to the right of pivot are greater than or equal to the pivot**".

Step by Step Process

In Quick sort algorithm, partitioning of the list is performed using following steps...

- **Step 1** - Consider the first element of the list as **pivot** (i.e., Element at first position in the list).
- **Step 2** - Define two variables *i* and *j*. Set *i* and *j* to first and last elements of the list respectively.
- **Step 3** - Increment *i* until $\text{list}[i] > \text{pivot}$ then stop.
- **Step 4** - Decrement *j* until $\text{list}[j] < \text{pivot}$ then stop.
- **Step 5** - If $i < j$ then exchange $\text{list}[i]$ and $\text{list}[j]$.
- **Step 6** - Repeat steps 3, 4 & 5 until $i > j$.
- **Step 7** - Exchange the pivot element with $\text{list}[j]$ element.

Consider the following unsorted list of elements...

List

5	3	8	1	4	6	2	7
---	---	---	---	---	---	---	---

Define pivot, left & right. Set pivot = 0, left = 1 & right = 7. Here '7' indicates 'size-1'.

List

5	3	8	1	4	6	2	7
---	---	---	---	---	---	---	---

left right
pivot

Compare List[left] with List[pivot]. If **List[left]** is greater than **List[pivot]** then stop left otherwise move left to the next.

Compare List[right] with List[pivot]. If List[right] is smaller than List[pivot] then stop right otherwise move right to the previous.

Repeat the same until **left >= right**.

If both left & right are stopped but left < right then swap List[left] with List[right] and continue the process.
If left >= right then swap List[pivot] with List[right].

List

5	3	8	1	4	6	2	7
---	---	---	---	---	---	---	---

left right
pivot

Compare List[left] < List[pivot] as it is true increment left by one and repeat the same, left will stop at 8.

Compare List[right] > List[pivot] as it is true decrement right by one and repeat the same, right will stop at 2.

List

5	3	8	1	4	6	2	7
---	---	---	---	---	---	---	---

left right
pivot

Here left & right both are stopped and left is not greater than right so we need to swap List[left] and List[right]

List

5	3	2	1	4	6	8	7
---	---	---	---	---	---	---	---

left right
pivot

Compare List[left] < List[pivot] as it is true increment left by one and repeat the same, left will stop at 6.

Compare List[right] > List[pivot] as it is true decrement right by one and repeat the same, right will stop at 4.



Here left & right both are stopped and left is greater than right so we need to swap List[pivot] and List[right]



Here we can observe that all the numbers to the left side of 5 are smaller and right side are greater. That means 5 is placed in its correct position.

Repeat the same process on the left sublist and right sublist to the number 5.



In the left sublist as there are no smaller number than the pivot left will keep on moving to the next and stops at last number. As the List[right] is smaller, right stops at same position. Now left and right both are equal so we swap pivot with right.



In the right sublist left is greater than the pivot, left will stop at same position.

As the List[right] is greater than List[pivot], right moves towards left and stops at pivot number position.

Now left > right so we swap pivot with right. (6 is swap by itself).



Repeat the same recursively on both left and right sublists until all the numbers are sorted.

The final sorted list will be as follows...



Complexity of the Quick Sort Algorithm

To sort an unsorted list with 'n' number of elements, we need to make $((n-1)+(n-2)+(n-3)+\dots+1) = (n(n-1))/2$ number of comparisons in the worst case. If the list is already sorted, then it requires 'n' number of comparisons.

Worst Case : $O(n^2)$

Best Case : $O(n \log n)$

Average Case : $O(n \log n)$